In this course, we will build a complete Freelancer Invoice Tracker while learning a practical, review-driven workflow with Codex. The finished full-stack application will organize clients, projects, and invoices, then turn that data into useful business insights and exports.

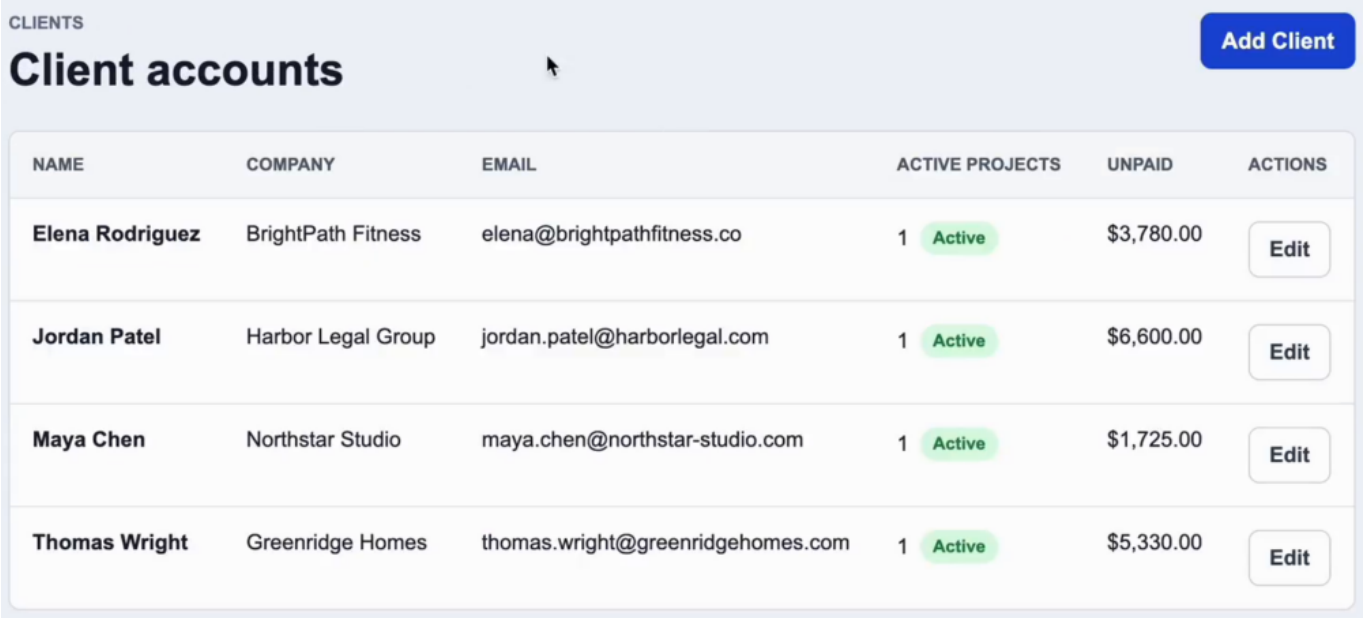
The application we will build

The dashboard gives us an immediate overview of the business. It summarizes outstanding revenue, payments received during the current month, overdue invoices, and recent revenue trends.

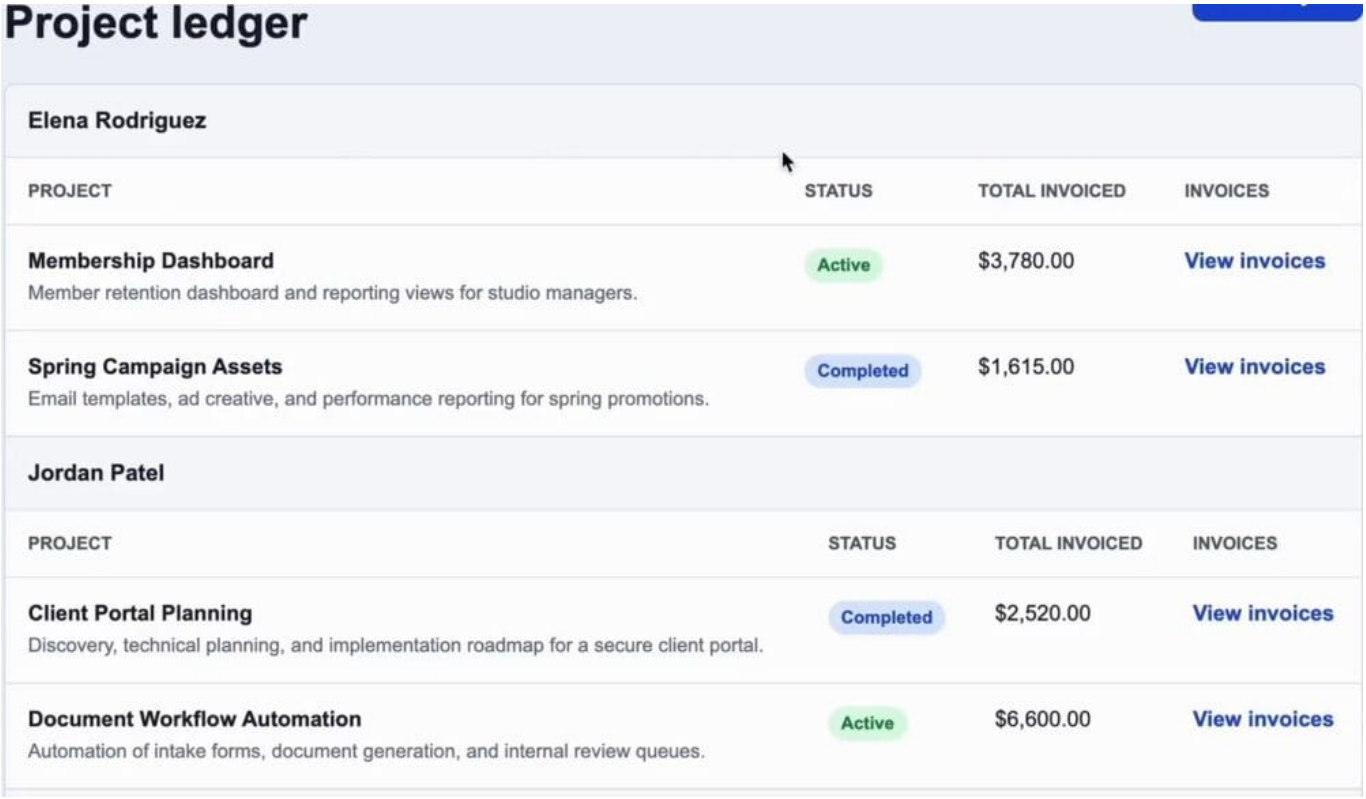


The Clients page keeps important account information in one place. We can review contact details, project counts, and unpaid balances, then add a client or edit an existing record through an inline form.

© 2026 Zenva Pty Ltd. All rights reserved.

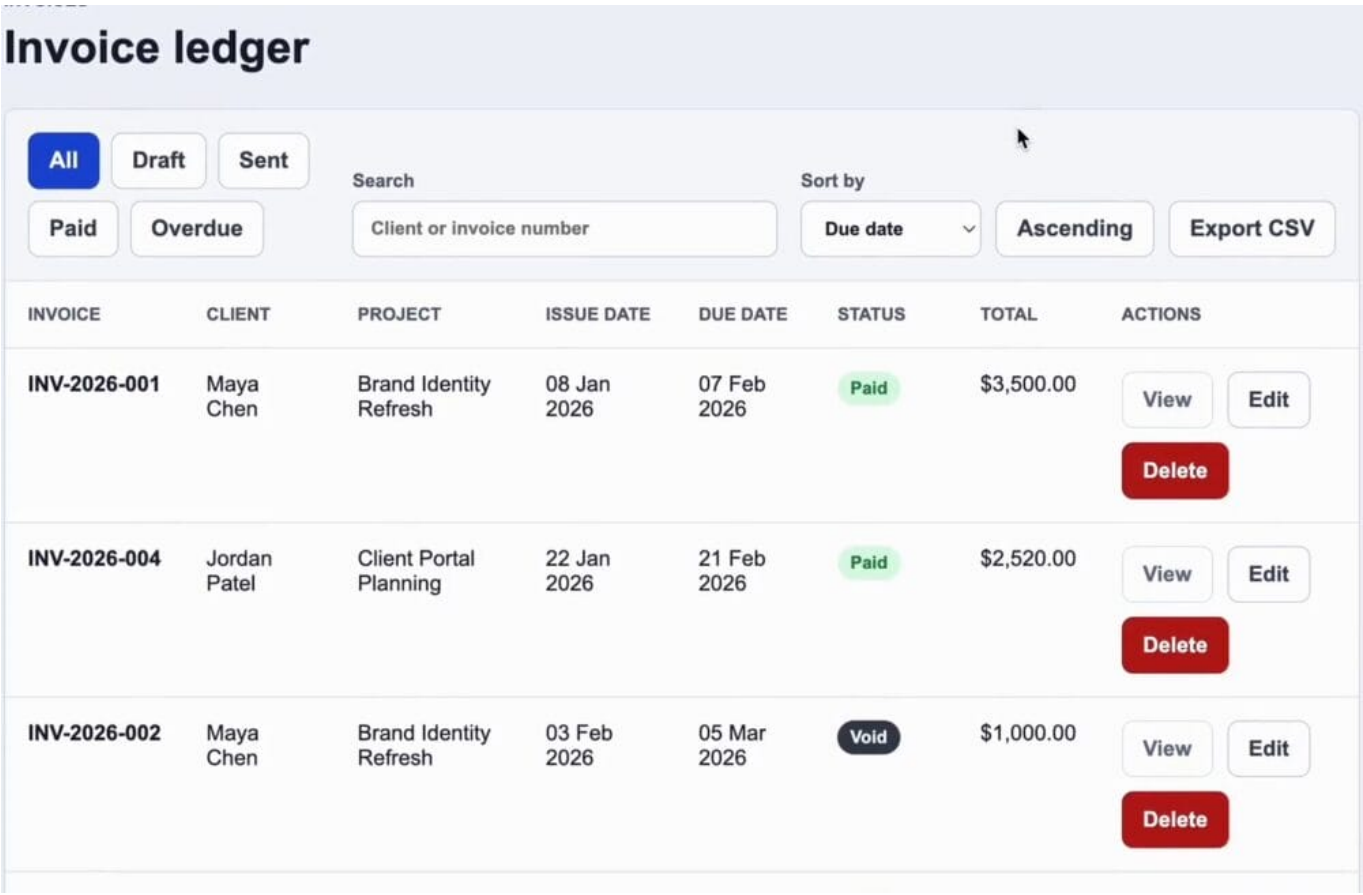


The Projects page groups work by client. Each project includes its current status and total invoiced amount, with direct access to the related invoices.



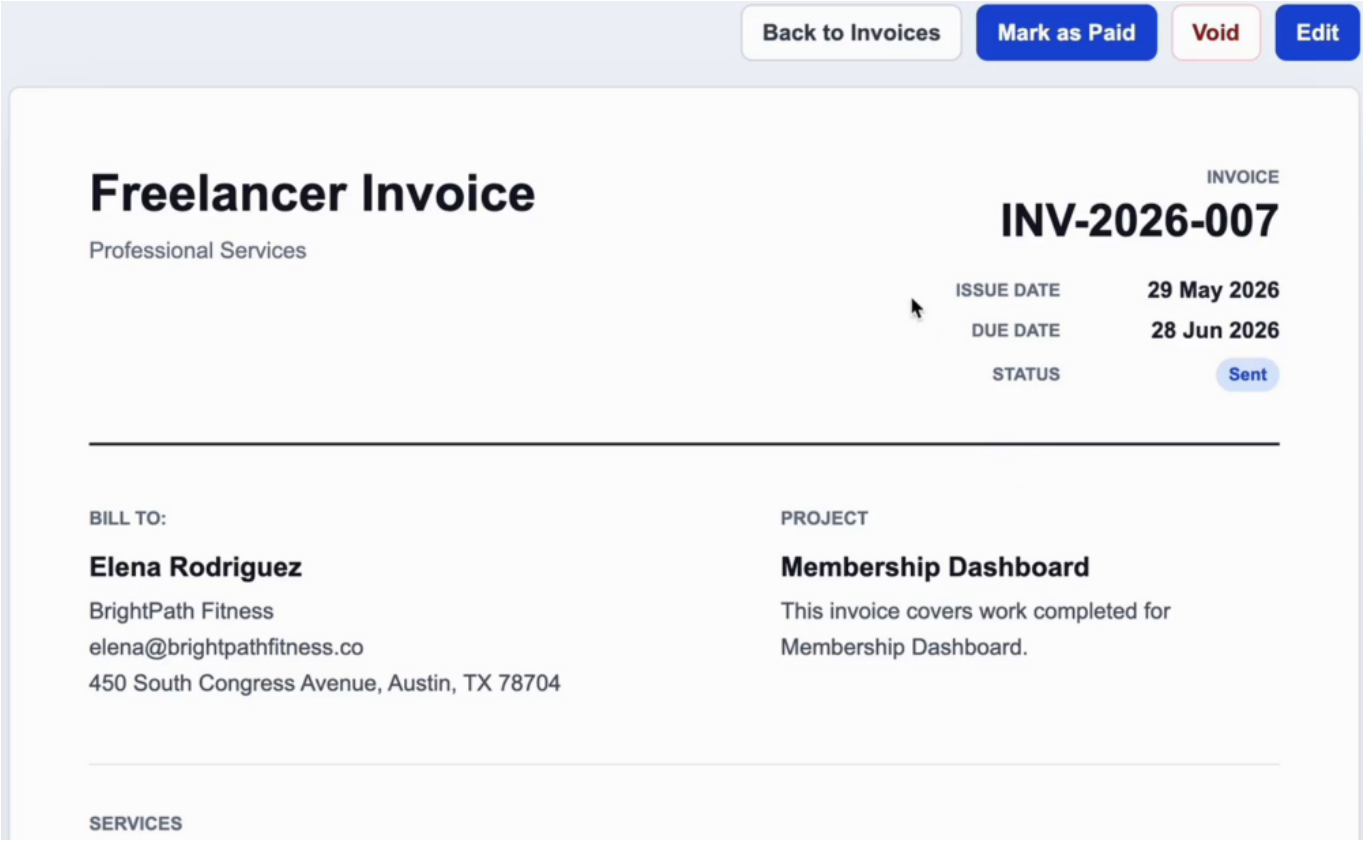
The Invoices page is the application’s main working view. We can filter invoices by status, search by client or invoice number, change the sort order, mark invoices as paid, and access the available record actions.

© 2026 Zenva Pty Ltd. All rights reserved.

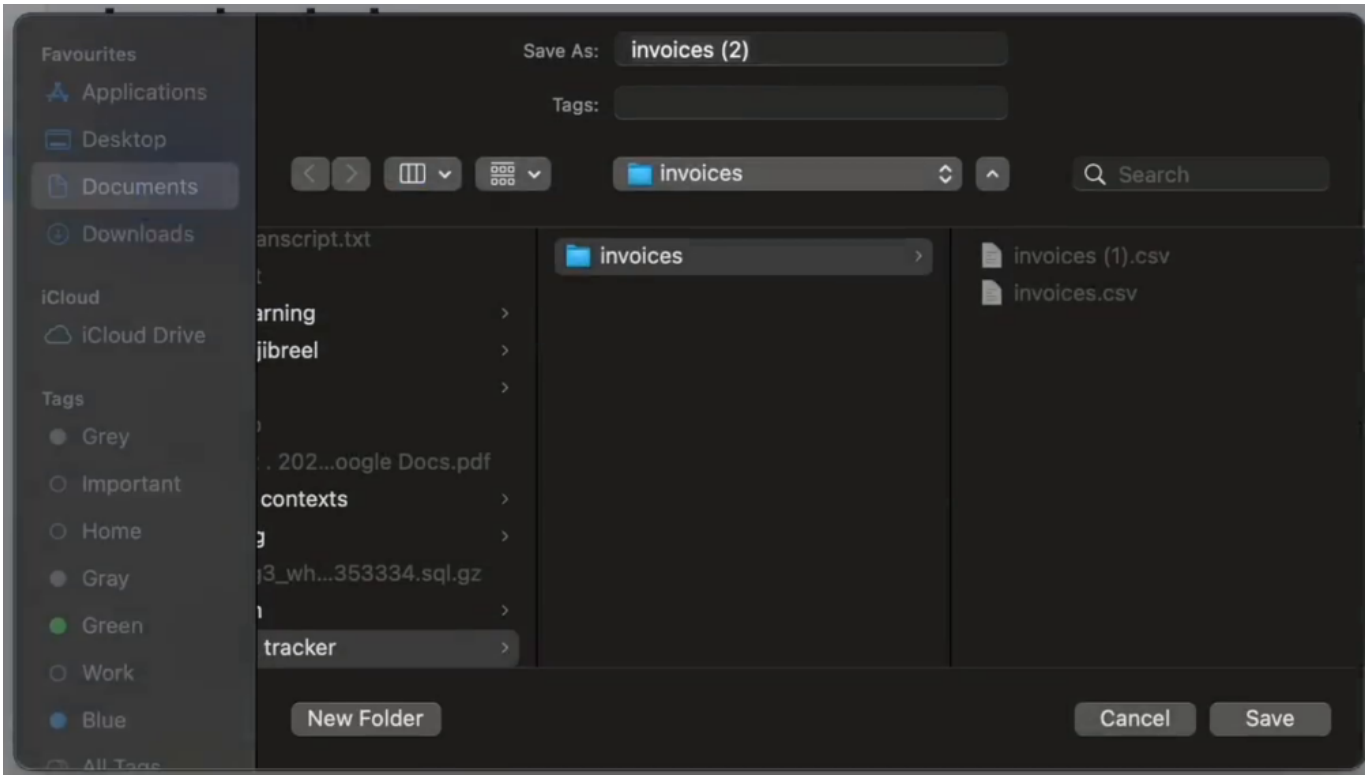


Every invoice also has a clean, document-style detail page. This view presents the issue date, due date, billing information, project details, line items, totals, and the actions needed to move the invoice through its workflow.

© 2026 Zenva Pty Ltd. All rights reserved.



When financial data needs to be shared with an accountant or bookkeeper, the application can export the invoice records as a CSV file.



Planning and building with Codex

© 2026 Zenva Pty Ltd. All rights reserved.

Before writing code, we will use Codex plan mode to define the database schema, page structure, and build order. This up-front planning gives the implementation a clear direction and reduces avoidable rework.

We will then create a full-stack Next.js application that combines several connected layers:

- A SQLite database for persistent application data
- A structured service layer for database access
- Server actions for mutations
- Interconnected pages for clients, projects, invoices, and reporting

As the project grows, we will use subagents to investigate focused technical decisions without filling the main session with every detail. We will also use parallel sessions to develop independent features at the same time.

Reviewing and improving generated code

AI-generated code still requires careful human review. We will use /review and git diff to inspect changes with the same scrutiny we would apply to a pull request from a junior developer.

When we find a recurring mistake, we will convert it into a durable rule in AGENTS.md. This lets the project carry lessons from earlier sessions into future work. By the end, we will also address empty states, error boundaries, print support, and automated tests so the result is a polished application rather than a rough prototype.

Course requirements

Some familiarity with Codex is useful, but we will cover the workflow as we build. To follow the project comfortably, we should understand basic React concepts such as components, props, and state. We should also be comfortable working in a terminal and using common Git commands such as git status, git diff, git commit, and git restore.

Before starting, make sure the following tools and access are ready:

- Node.js 18 or later
- A correctly configured Codex installation
- An OpenAI API key or a compatible premium ChatGPT subscription for Codex access

© 2026 Zenva Pty Ltd. All rights reserved.

Careful planning gives Codex the context needed to produce a coherent application instead of a collection of disconnected features. In this lesson, we will define the Invoice Tracker's scope, ask Codex for a structured implementation plan, and begin reviewing its proposed database schema.

Define the project before writing code

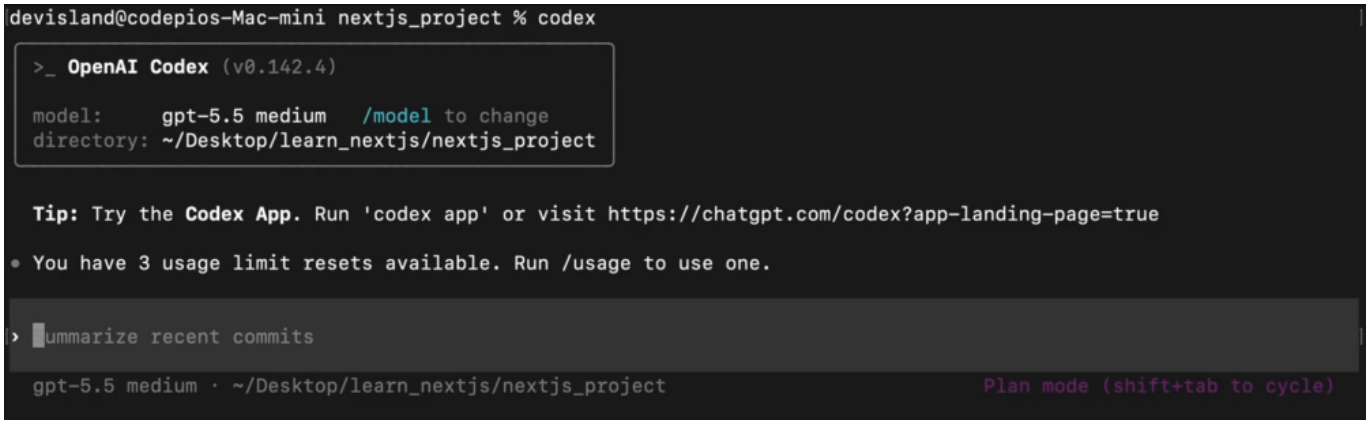
Our application will manage the full lifecycle of freelance work. Clients own projects, projects have invoices, and invoices contain line items. The finished product will include a financial dashboard, searchable management pages, printable invoice details, and CSV export.

We will build the application with Next.js 16 and its App Router, using a local SQLite database through better-sqlite3. The project will remain self-contained so we can focus on application logic and Codex-assisted planning without introducing third-party services.

Before making implementation changes, initialize a Git repository, stage the current files, and create an initial commit. This establishes a clean baseline that can be reviewed or restored later.

Start a Codex planning session

Open a terminal in the project root and launch Codex. Switch the session into Plan mode before entering the project brief. The mode indicator confirms that Codex is ready to reason about the architecture without immediately editing files.

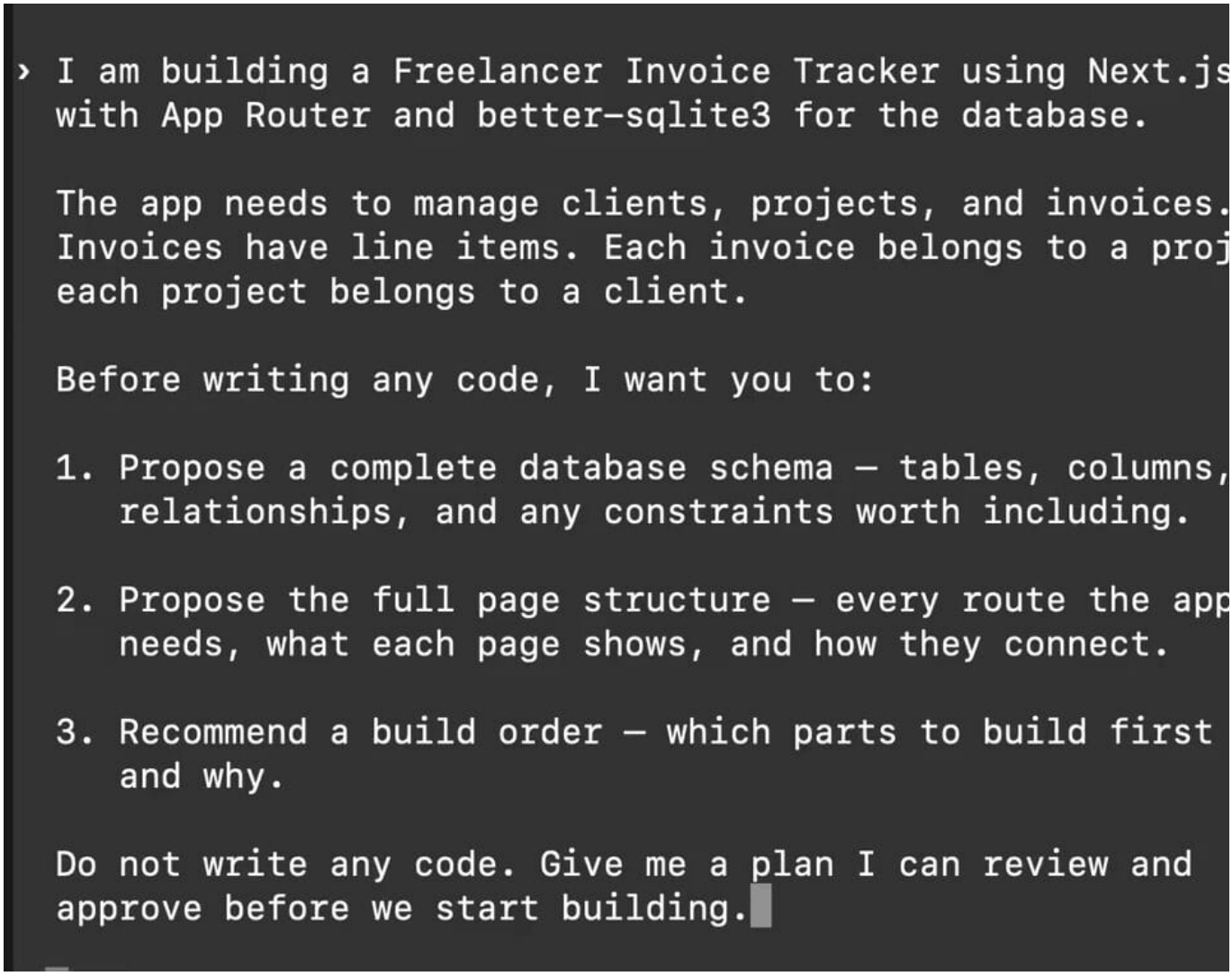


Use the following prompt to give Codex the technology stack, data relationships, and three planning outputs we need:

> I am building a Freelancer Invoice Tracker using Next.js 16 with App Router and better-sqlite3 for the database.
>
> The app needs to manage clients, projects, and invoices. Invoices have line items. Each invoice belongs to a project, each project belongs to a client.
>
> Before writing any code, I want you to:
>
> 1. Propose a complete database schema — tables, columns, relationships, and any constraints worth including.
> 2. Propose the full page structure — every route the app needs, what each page shows, and how they connect.
> 3. Recommend a build order — which parts to build first and why.
>
> Do not write any code. Give me a plan I can review and approve before we start building.

© 2026 Zenva Pty Ltd. All rights reserved.

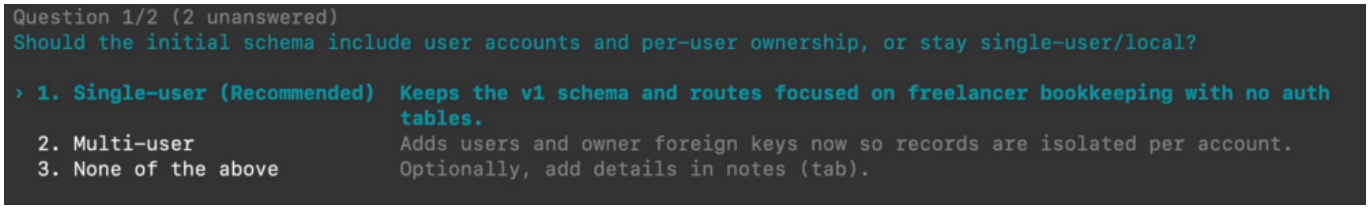
The final instruction keeps the session focused on a reviewable plan instead of premature implementation.



Resolve architectural questions

Codex first examines the existing App Router project and relevant documentation. It then asks questions when a product decision would materially change the architecture.

For the first version, choose a single-user, local application. This keeps the schema focused on freelancer bookkeeping and avoids authentication tables or per-user ownership fields.



For invoice calculations, choose simple totals. The application will track line items, subtotal, total, paid amount, and payment status without adding tax or discount modeling to the first version.

© 2026 Zenva Pty Ltd. All rights reserved.

Review the proposed database plan

After the clarifications are submitted, Codex produces a structured plan covering the data model, routes, build order, tests, and implementation assumptions. The schema begins with four connected entities: clients, projects, invoices, and invoice line items.

```
- hourly_rate_cents integer
- created_at text not null default current timestamp
- updated_at text not null default current timestamp
- Constraints:
    - status in ('active', 'paused', 'completed', 'archived'
    - hourly_rate_cents is null or hourly_rate_cents >= 0

- Indexes:
    - idx_projects_client_id
    - idx_projects_status

### invoices

- id integer primary key
- project_id integer not null references projects(id) on del
- invoice_number text not null unique
- status text not null default 'draft'
- issue_date text not null
- due_date text
- currency text not null default 'USD'
- subtotal_cents integer not null default 0
- total_cents integer not null default 0
- paid_cents integer not null default 0
- sent_at text
- paid_at text
- notes text
```

The Clients table forms the root of the model. A client name is required, while details such as email, phone number, company, address, and notes can remain optional. Creation and update timestamps default to the current time so the application does not need to set them manually.

Every project belongs to one client through a required client identifier. The relationship uses a delete restriction, which prevents a client from being removed while projects still reference that client. This protects the database from orphaned project records.

Project names are required and descriptions are optional. A status constraint limits values to active, paused, completed, or archived, while an optional hourly rate must be non-negative. These checks

© 2026 Zenva Pty Ltd. All rights reserved.

enforce valid data at the database level. We will continue reviewing the remaining schema and planning details in the next lesson.

© 2026 Zenva Pty Ltd. All rights reserved.

With the first part of the data model established, we can finish reviewing the schema, map the application's routes, and choose an implementation order that respects the dependencies between features.

Design the invoices table

Every invoice belongs to a project. A restricted delete relationship prevents us from removing a project while invoices still reference it. Each invoice number is required and unique, which prevents duplicate identifiers at the database level.

Invoice status is limited to draft, sent, paid, overdue, or void. Financial fields such as subtotal, total, and paid amount are stored as integer cents instead of decimal currency values. This avoids floating-point rounding errors. A further constraint prevents the paid amount from exceeding the invoice total.

The table also records issue and due dates, uses USD as its default currency, and supports optional sent and paid timestamps and notes. Creation and update timestamps are managed automatically.

Design invoice line items

Line items use a cascading delete relationship. When an invoice is deleted, its dependent line items are removed as well because they have no independent meaning.

Each line item has a position that is unique within its invoice, preserving a consistent display order. It also stores a description, quantity, unit price in cents, and calculated line total in cents, along with automatic timestamps.

Map the main application pages

The root route will host a dashboard with summary cards for unpaid amounts, overdue invoices, amounts paid this month, and active projects. It will also provide quick actions for common tasks such as creating an invoice, adding a client, or adding a project.

Three core list pages will manage the main entities:

- /clients will support search and show project counts and financial summaries for each client.
- /projects will filter projects by status or client and show related invoice counts.
- /invoices will support filtering by status, client, project, and due date, invoice-number search, payment actions, and deletion.

These list pages will follow a consistent pattern: search and filtering, useful summary data, links to individual records, and access to creation forms. This predictable structure makes it easier for Codex to reproduce reliable behavior across resource types.

Plan detail and form routes

Clients and projects will each have detail, create, and edit routes. A client detail page will combine contact information with related projects and invoices. A project detail page will combine project metadata with its associated invoices.

Invoice creation and editing are more complex because users must select a project, set dates, and dynamically add, edit, or remove line items. Totals will be calculated on the server, not trusted from the browser, which prevents client-side manipulation and keeps financial data consistent.

The invoice detail page will use a print-friendly layout containing client and project information, line

© 2026 Zenva Pty Ltd. All rights reserved.

items, status, totals, paid amount, and remaining balance. It will also provide actions for editing, marking the invoice as sent or paid, and voiding it.

Choose a dependency-driven build order

1. Build the database schema and service functions that every page depends on.
2. Create shared status badges and date and currency formatting utilities.
3. Implement client pages, followed by project pages.
4. Add the invoice list, invoice forms, and invoice detail view.
5. Build the dashboard once meaningful application data exists.
6. Add reports and CSV export after the core workflow is stable.

This sequence lets each Codex session focus on one layer at a time. In the next lesson, we will turn the agreed plan into an AGENTS.md file that provides durable project context.

© 2026 Zenva Pty Ltd. All rights reserved.

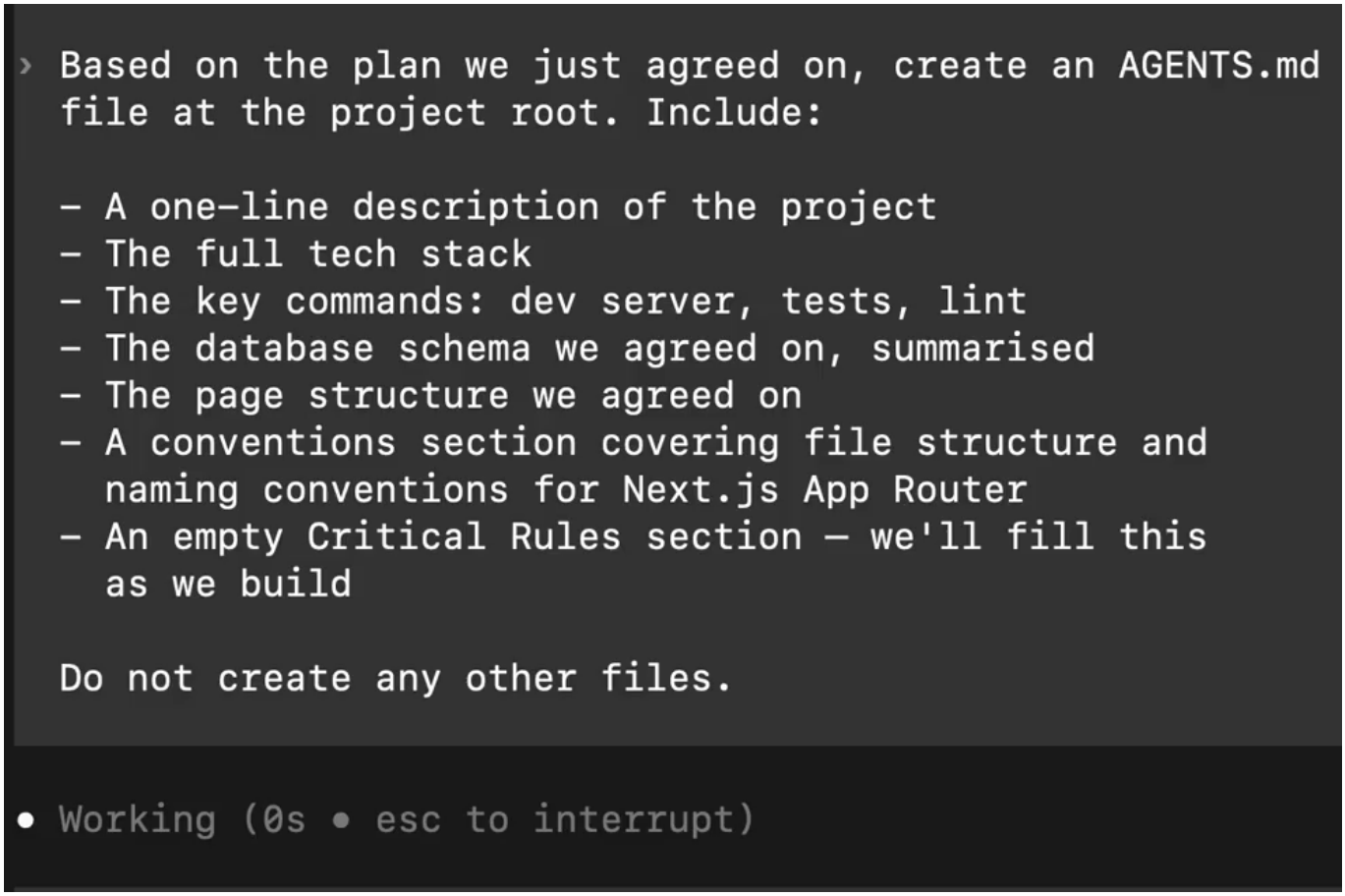
After approving the project plan, we can turn its decisions into durable guidance that Codex will read in future sessions. In this lesson, we will create, review, and commit the project's AGENTS.md file.

Create the project guide

First, leave Plan mode by pressing Shift-Tab. Once Codex has returned to its default instruction mode, ask it to create AGENTS.md from the plan we just approved. The prompt names every required section and limits the task to this single file.

The exact prompt used is:

> Based on the plan we just agreed on, create an AGENTS.md file at the project root. Include:
>
> - A one-line description of the project
> - The full tech stack
> - The key commands: dev server, tests, lint
> - The database schema we agreed on, summarised
> - The page structure we agreed on
> - A conventions section covering file structure and naming conventions for Next.js App Router
> - An empty Critical Rules section – we'll fill this as we build
>
> Do not create any other files.

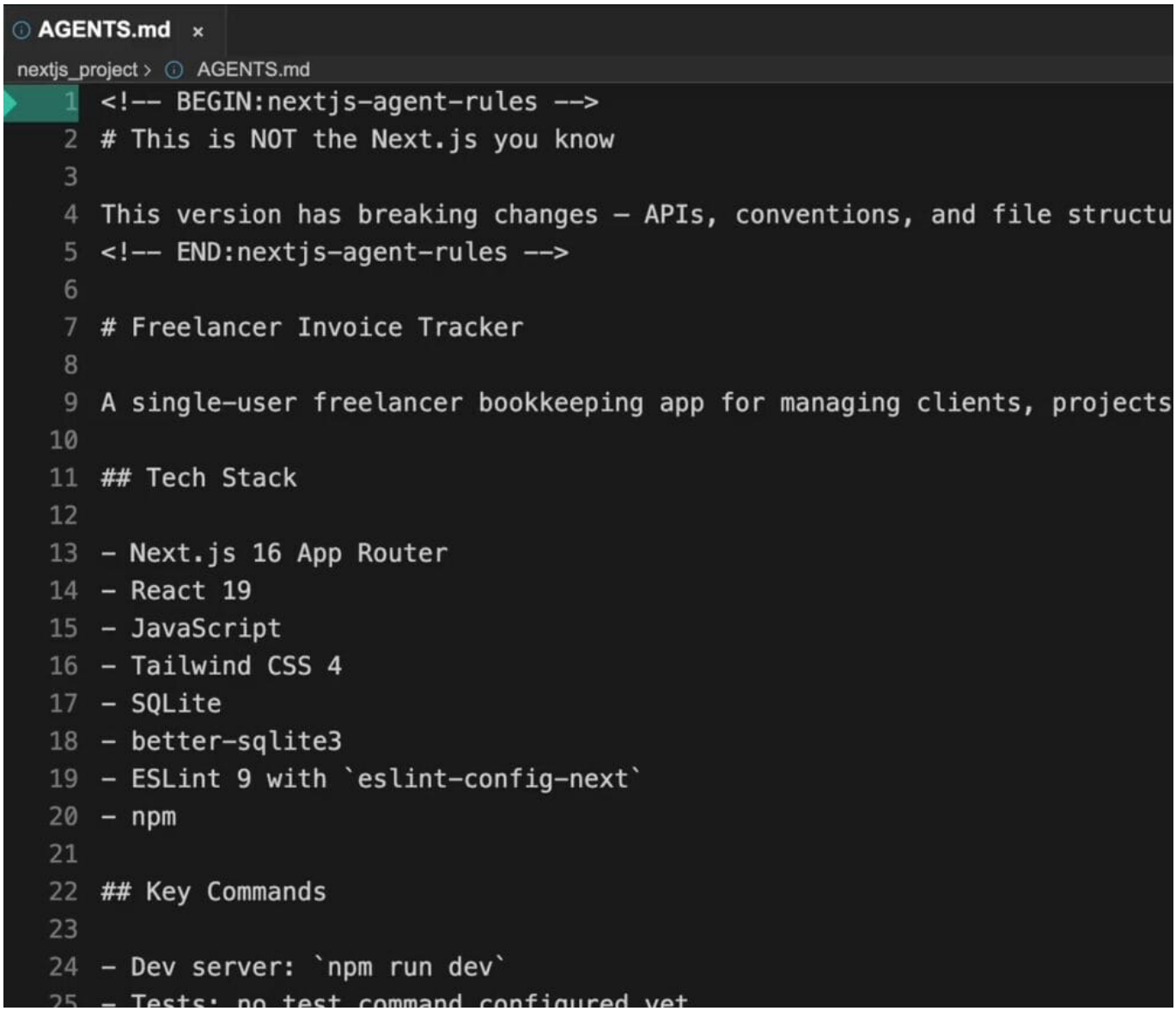


© 2026 Zenva Pty Ltd. All rights reserved.

Codex begins by inspecting the project root and reading the existing AGENTS.md. It preserves the existing Next.js warning while adding the project-specific guidance around it. This keeps useful instructions intact and respects the prompt's requirement not to create unrelated files.

Review the generated file

Open AGENTS.md in the editor and carefully verify every section. The generated guide should contain the project description, complete technology stack, development commands, summarized database schema, page structure, and conventions for the Next.js App Router.



The conventions section captures the file structure, route organization, architectural choices, and naming rules agreed during planning. The Critical Rules section starts empty so we can add project-specific requirements and anti-patterns as we discover them during development.

© 2026 Zenva Pty Ltd. All rights reserved.

```
## Conventions

- Read the relevant guide in `node_modules/next/dist/docs/` before changing App Router code.
- Keep routes under `src/app`.
- Use `page.js` for route UI and `layout.js` for shared route shells.
- Use dynamic route segment folders with bracket names, such as `[clientId]`, `[projectId]`, and `[i
- Use lowercase kebab-case folder names for static route segments.
- Keep shared UI components outside route folders when reused across multiple sections.
- Keep route-specific components close to the route that owns them.
- Use Server Components by default.
- Use Client Components only for interactive browser state.
- Put server-only database access behind a dedicated data layer instead of querying SQLite directly
- Use Server Functions for form mutations.
- Recalculate derived invoice totals on the server, not from trusted client input.
- Use integer cents for all money values.
- Use ISO text values for dates and timestamps.
- Name database columns in snake_case.
- Name JavaScript variables and functions in camelCase.
- Name React components in PascalCase.

## Critical Rules
```

This review is essential because AGENTS.md is more than passive documentation. Codex reads it at the beginning of each session and uses it as ongoing project context. If the file contains an inaccurate rule, Codex can repeatedly base later decisions on that mistake.

Commit the shared context

Once the guide accurately reflects the approved plan and conventions, ask Codex to commit the file. Version control provides a clear starting point for future development and makes later changes to the shared guidance easy to review.

Complete the planning workflow

This commit completes the planning workflow. We have defined the data model and its relationships, mapped the application’s pages and navigation, established a logical build order from the data layer to the finished interface, and captured those decisions in AGENTS.md. Codex now has consistent project context to guide every coding session that follows.

© 2026 Zenva Pty Ltd. All rights reserved.

The Complete Codex Development Course - Plan Before You Build

1. Codex is ready to edit, but the architecture is not agreed yet. What should you do first?

a. Ask it to scaffold every page
b. Use Plan mode to define the schema, routes, and build order
c. Add more packages before planning
d. Start with the dashboard UI

2. Why build the database and service functions before the dashboard?

a. The dashboard depends on meaningful application data
b. Dashboards cannot use Server Components
c. SQLite must be added after the UI
d. Git requires database files first

3. A recurring Codex mistake should be prevented in future sessions. What is the best response?

a. Mention it only in the current chat
b. Add an accurate rule to AGENTS.md and commit it
c. Remove the affected feature
d. Ignore it after fixing the current code

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will establish the Invoice Tracker's database foundation. We will install SQLite support, create a shared database connection, generate a rerunnable migration, and verify the resulting schema before adding application data.

Prepare the repository and dependencies

Begin by running git status and confirming that the working tree is clean. This keeps the database work separate from the previously committed AGENTS.md instructions.

Open Codex from the project root. Because better-sqlite3 is a native dependency that can encounter sandbox restrictions inside an agent session, install it directly in the terminal together with its TypeScript definitions:

```
npm install better-sqlite3
npm install --save-dev @types/better-sqlite3
```

Create the database connection

Use the following prompt to ask Codex for the shared database module:

```
Create the database module at src/lib/db.js that:
- Opens a SQLite database file at the project root called
  invoice-tracker.db
- Enables WAL mode for better performance
- Enables foreign key enforcement - this is critical,
  do not skip it
- Exports the database instance as the default export

Do not create any other files yet.
```

Codex reviews the project conventions and relevant Next.js guidance before creating src/lib/db.js. At this point in the project, the module is:

```
import Database from 'better-sqlite3'
import path from 'node:path'

const dbPath = path.join(process.cwd(), 'invoice-tracker.db')
const db = new Database(dbPath)

db.pragma('journal_mode = WAL')
db.pragma('foreign_keys = ON')

export default db
```

process.cwd() resolves the database path from the project root. Write-ahead logging, enabled through journal_mode = WAL, improves how SQLite handles reads and writes. Foreign-key enforcement must be enabled explicitly in SQLite. Without foreign_keys = ON, the restricted and cascading delete relationships described in AGENTS.md would not be enforced.

© 2026 Zenva Pty Ltd. All rights reserved.

Generate an idempotent migration

Next, give Codex the exact migration request:

```
Create a migration script at src/lib/migrate.js that:
- Imports the database from src/lib/db.js
- Creates the four tables from our agreed schema in
  AGENTS.md: clients, projects, invoices, invoice_line_items
- Uses CREATE TABLE IF NOT EXISTS so it is safe to run
  multiple times
- Creates all indexes defined in the schema
- Runs the migration immediately when the script is executed

Then run the migration script to confirm it executes without errors
```

Codex creates src/lib/migrate.js as a single database transaction. The script imports the shared connection, executes the schema statements, and runs the transaction immediately:

```
import db from './db.js'

const migrate = db.transaction(() => {
  db.exec(`
    CREATE TABLE IF NOT EXISTS clients (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      name TEXT NOT NULL,
      email TEXT,
      phone TEXT,
      company TEXT,
      address TEXT,
      notes TEXT,
      created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
      updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
    );

    CREATE INDEX IF NOT EXISTS idx_clients_name
      ON clients (name);
  `)
})

migrate()
```

The complete migration applies the same rerunnable pattern to four tables:

- clients stores contact details and requires a name.
- projects references a client with ON DELETE RESTRICT, limits status values, and prevents negative hourly rates.
- invoices references a project with ON DELETE RESTRICT, requires unique invoice numbers, validates statuses and three-letter uppercase currency codes, and protects monetary totals with non-negative checks.
- invoice_line_items references an invoice with ON DELETE CASCADE, preserves unique item positions within each invoice, and validates quantities and prices.

© 2026 Zenva Pty Ltd. All rights reserved.

The migration also creates eight named indexes for the client name, project client and status, invoice project, status, due date and issue date, and line-item invoice fields. Using CREATE TABLE IF NOT EXISTS and CREATE INDEX IF NOT EXISTS makes the migration safe to run again.

Run and verify the migration

Execute the migration from the project root:

```
node src/lib/migrate.js
```

A Node.js module-type warning may appear because the package does not declare "type": "module". In this run, the warning is non-fatal and the migration still creates invoice-tracker.db.

Inspect SQLite metadata to confirm that the four user tables and eight named indexes exist. Then query the active connection and verify that write-ahead logging and foreign-key enforcement are enabled. The expected values are wal and 1:

```
node --input-type=module -e "import db from './src/lib/db.js'; console.log({ journalM
ode: db.pragma('journal_mode', { simple: true }), foreignKeys: db.pragma('foreign_key
s', { simple: true }) })"
```

Finally, use this prompt to inspect every table definition:

```
Run a SQLite query to list all tables in the database
and show the schema for each one.
```

Codex queries the SQLite catalog and prints the creation statement for clients, projects, invoices, and invoice_line_items. Compare the fields, foreign keys, uniqueness rules, and check constraints with the schema in AGENTS.md. This final review confirms that the database is structurally correct before realistic seed data and the user interface are added.

© 2026 Zenva Pty Ltd. All rights reserved.

With our schema and migration in place, we can populate the Invoice Tracker with realistic development data. In this lesson, we will ask Codex to create a rerunnable seed script, verify the resulting relationships and totals, document the database commands, and commit the completed database foundation.

Generate realistic seed data

We begin with a precise prompt that defines the required records, their relationships, and the safety behavior of the seed script:

```
Create a seed script at src/lib/seed.js that inserts
realistic starting data:

- 4 clients with realistic names, companies, and contact details
- 8 projects spread across the 4 clients, with a mix of
  active and completed statuses and realistic hourly rates
- 12 invoices spread across the 8 projects, covering all
  5 statuses: draft, sent, paid, overdue, and void
- 2–4 line items per invoice with realistic descriptions,
  quantities, and unit prices in cents
- Correct subtotal_cents and total_cents calculated from
  the line items
- Realistic invoice numbers, issue dates, and due dates
  spread across the last 6 months

The seed script should clear existing data before inserting
so it is safe to run multiple times.

Then run the seed script after creating it.
```

Codex creates src/lib/seed.js and runs it with the following command:

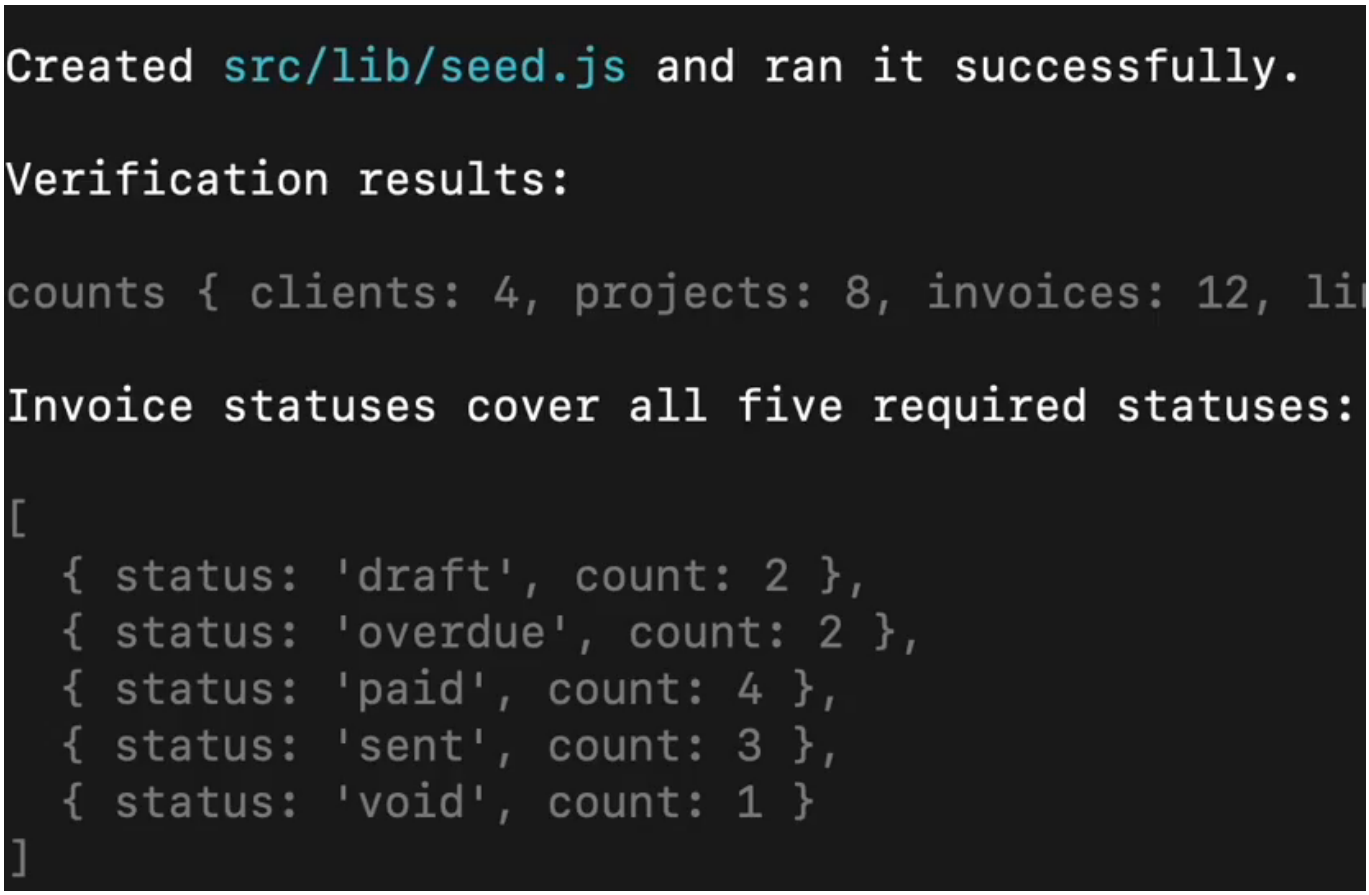
```
node src/lib/seed.js
```

The script clears the existing dependent records before inserting the new dataset, which makes it safe to run again. It completes successfully. Node reports an existing module-type warning because package.json does not declare "type": "module", but that warning does not prevent the seed from running.

Verify the seed results

The first verification confirms that the database contains 4 clients, 8 projects, 12 invoices, and 35 line items. The invoices cover every required status: 2 draft, 3 sent, 4 paid, 2 overdue, and 1 void. A totals check also finds zero mismatches between each stored invoice total and the sum of its line items.

© 2026 Zenva Pty Ltd. All rights reserved.

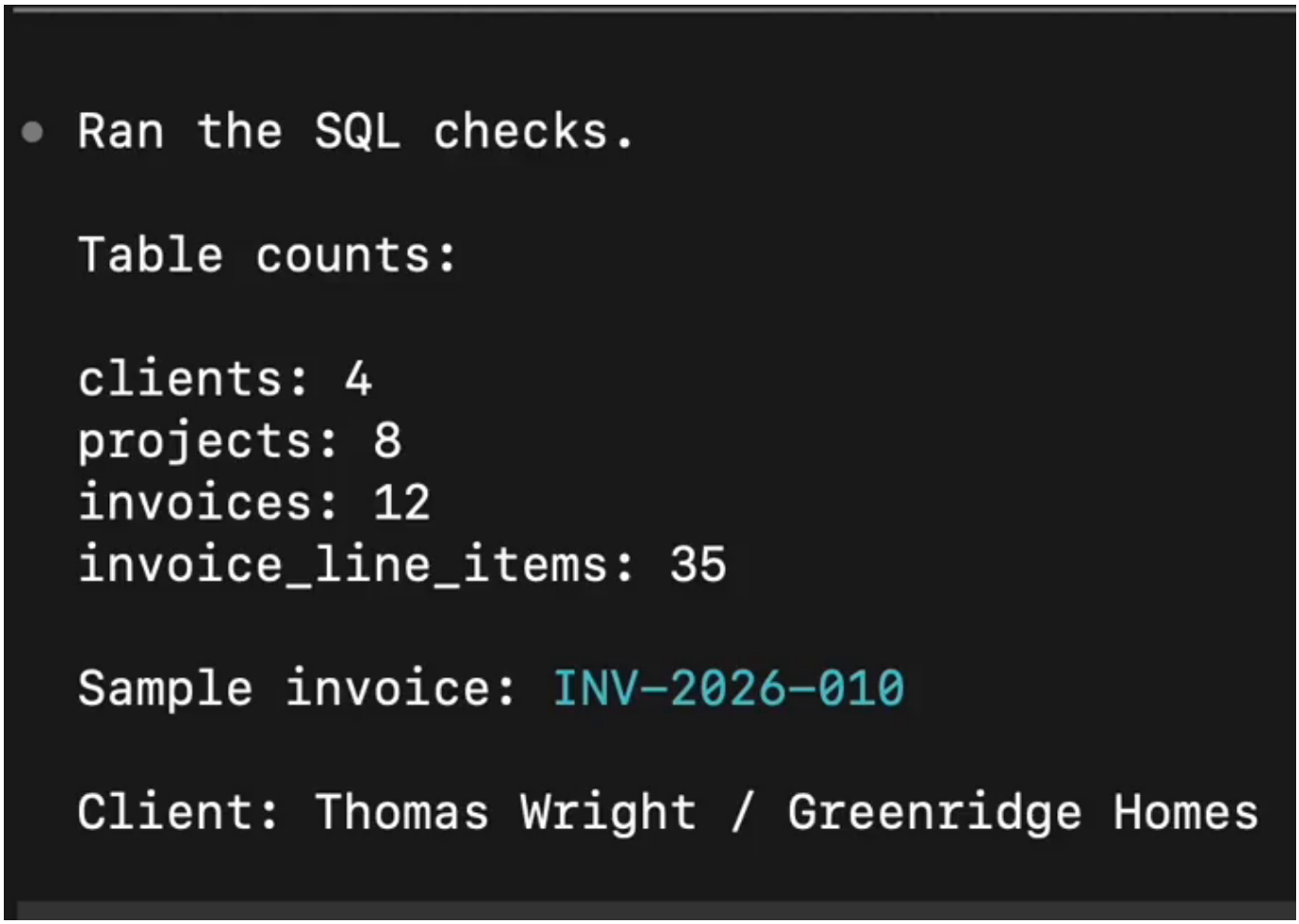


We then ask Codex to perform a second set of read-only checks with this exact prompt:

```
Run SQL queries to confirm:
- How many rows are in each table
- One sample invoice with its line items and calculated total
- The breakdown of invoices by status
```

Codex uses better-sqlite3 to query invoice-tracker.db. The table counts match the seed specification, and a sample invoice is joined to its client, project, and line items. Its stored total matches the calculated line-item total, confirming that the relational data and monetary calculations are consistent.

© 2026 Zenva Pty Ltd. All rights reserved.



The status breakdown again confirms that the data spans draft, sent, paid, overdue, and void invoices. This range gives the later dashboard and invoice-list features meaningful states to display and test.

Document the database commands

Next, we update only the Key Commands section of AGENTS.md with the following prompt:

```
Update the Commands section in AGENTS.md to add:
- node src/lib/migrate.js — run the database migration
- node src/lib/seed.js — reseed the database with starting data
```

Codex preserves the existing document and adds the commands in the established bullet style:

```
- Database migration: `node src/lib/migrate.js`
- Database seed: `node src/lib/seed.js`
```

Keeping these commands in the project guide gives future Codex sessions a reliable way to recreate and repopulate the database.

Commit the database foundation

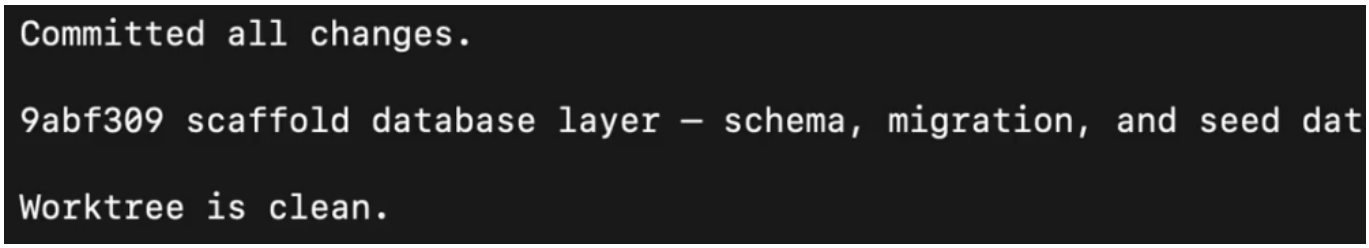
© 2026 Zenva Pty Ltd. All rights reserved.

We finish by asking Codex to commit the complete database scaffold with an explicit message:

```
Commit all changes with the message "scaffold database
layer — schema, migration, and seed data"
```

Before staging, Codex reviews the working tree, the relevant diffs, and the diff summary. When the sandbox prevents direct access to the Git index, Codex requests narrowly scoped permission for the staging command. After approval, it stages the database connection, schema, migration, seed script, generated database, dependency changes, and AGENTS.md update.

The commit completes successfully with the requested message. A final git status --short produces no output, confirming that the working tree is clean and the database layer is safely version controlled.



Review the completed scaffold

Across both scaffolding lessons, we installed and configured better-sqlite3, enabled WAL mode and foreign-key constraints, created the schema and indexes through a migration, generated realistic seed data, verified the stored relationships and totals, documented the key commands, and committed the result. We now have a stable database foundation that is ready for the Invoice Tracker user interface.

© 2026 Zenva Pty Ltd. All rights reserved.

The Complete Codex Development Course - Build a Reliable Database Foundation

1. You ask Codex to build the database foundation. Which scope is clearest?

a. Build the database and dashboard together
b. Create a shared connection and rerunnable migration, then verify the schema
c. Refactor the full app while adding SQLite
d. Let Codex decide which features to include

2. Codex is creating a seed script that will be rerun often. Which requirement should you set?

a. Append another copy of the sample data each time
b. Reset records safely and wrap the operation in a transaction
c. Change the schema whenever the seed runs
d. Store the sample data in UI components

3. Codex requests permission to stage the database work. What should you do first?

a. Approve any Git command it requests
b. Let it stage the entire repository
c. Review the status and diff, then approve only the expected scope
d. Skip version control for database work

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will create the shared application shell for our Invoice Tracker. The shell gives every page the same responsive sidebar, route-aware navigation, header, and main content area. This work forms the first part of a broader shared-interface phase that will also introduce reusable status and formatting utilities and record the resulting conventions in AGENTS.md.

Confirm a clean starting point

Before asking Codex to change the project, run git status. We should see that the repository is on the expected branch and that there is nothing to commit. A clean working tree makes the new changes easy to isolate and review.

From the project root, run codex to open the Codex command-line interface. Keeping Codex in the project root allows it to read the repository instructions and inspect the relevant Next.js files.

Give Codex the complete app-shell prompt

Use the following prompt exactly:

```
Build the app shell for the invoice tracker. Create:

1. A sidebar navigation component at src/components/Sidebar.jsx
   with links to: Dashboard (/), Clients (/clients),
   Projects (/projects), and Invoices (/invoices).
   Highlight the active link based on the current route.

2. A header component at src/components/Header.jsx that shows
   the current page title and a "New Invoice" button that
   links to /invoices/new.

3. Update src/app/layout.js to wrap every page with the
   sidebar and header. The layout should use a two-column
   structure: sidebar on the left, main content on the right.

Use plain CSS modules for all styling. Match the existing
globals.css approach – no Tailwind, no external component
libraries.

Do not touch the database or any existing lib files.
```

This prompt defines the requested files, expected navigation behavior, styling approach, layout structure, and boundaries. The final sentence is especially important because it prevents an interface task from expanding into the database or existing library code.

Let Codex inspect the project and documentation

Codex first examines the existing App Router structure, configuration, package information, and local Next.js documentation. This follows the repository rule in AGENTS.md to consult the installed Next.js documentation before changing App Router code.

The implementation keeps the root layout as a Server Component. The sidebar and header become Client Components because they read the current route with usePathname. CSS modules remain next to the components they style, while globals.css supplies the small global reset.

© 2026 Zenva Pty Ltd. All rights reserved.

Build the route-aware shell

The sidebar defines the four primary destinations and treats nested pages as part of their parent section. The root route needs an exact match so it does not appear active on every page:

```
const navigationItems = [
  { href: "/", label: "Dashboard" },
  { href: "/clients", label: "Clients" },
  { href: "/projects", label: "Projects" },
  { href: "/invoices", label: "Invoices" },
];

function isActiveRoute(pathname, href) {
  if (href === "/") {
    return pathname === "/";
  }

  return pathname === href || pathname.startsWith(`${href}/`);
}
```

The header also uses the pathname to select a useful page title and provides a consistent New Invoice link. The root layout then places both components around the page content:

```
<div className={styles.shell}>
  <Sidebar />
  <div className={styles.content}>
    <Header />
    <main className={styles.main}>{children}</main>
  </div>
</div>
```

The associated CSS modules create a two-column desktop layout and collapse it to one column on narrower screens. Plain global styles replace the Tailwind import so the project uses the styling approach requested in the prompt.

Run lint and resolve the existing script warning

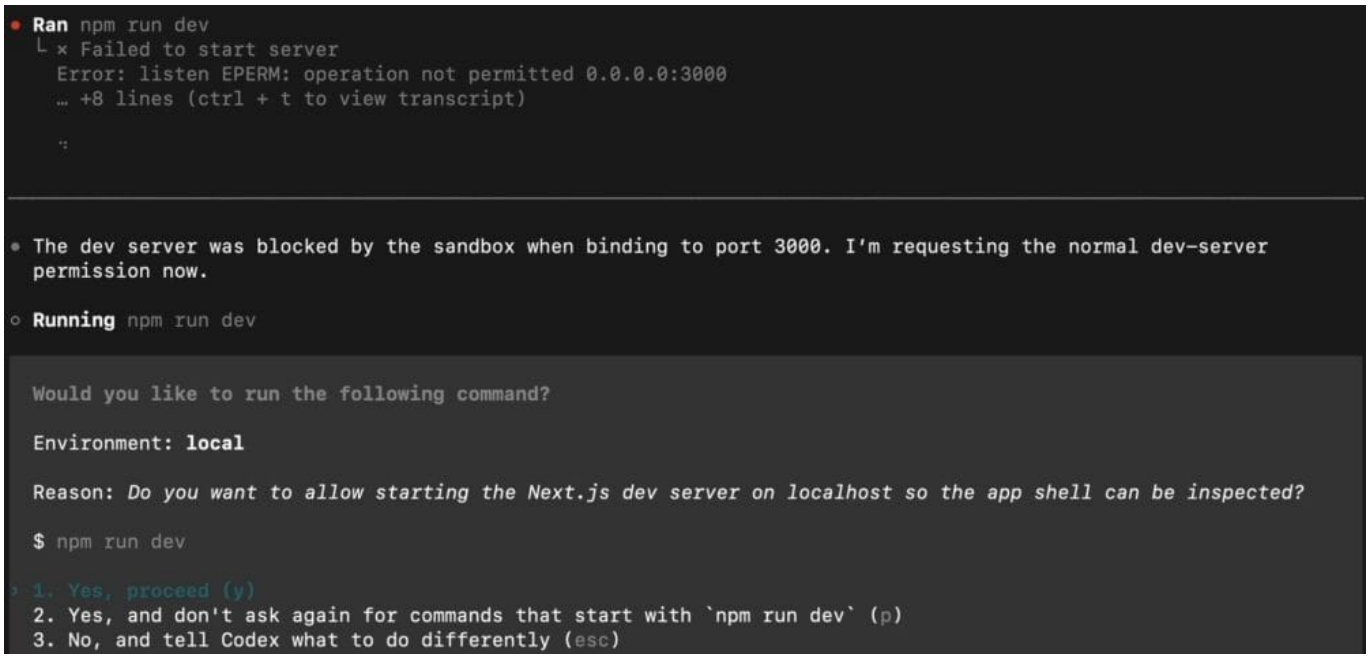
After the first implementation, run npm run lint. The check finds a pre-existing raw <script> tag in src/app/layout.js. Codex reviews the Next.js Script documentation and replaces the raw tag with the framework’s Script component while preserving its original behavior.

Run lint again and confirm that it passes. Then inspect both git diff and git status --short. A normal diff covers tracked files, while the short status also reveals newly created, untracked component and CSS module files.

Handle the development-server permission

Codex attempts to run npm run dev so we can inspect the shell. The sandbox initially blocks the server from binding to port 3000 and asks for permission to run the command locally. Review the command and reason before approving it.

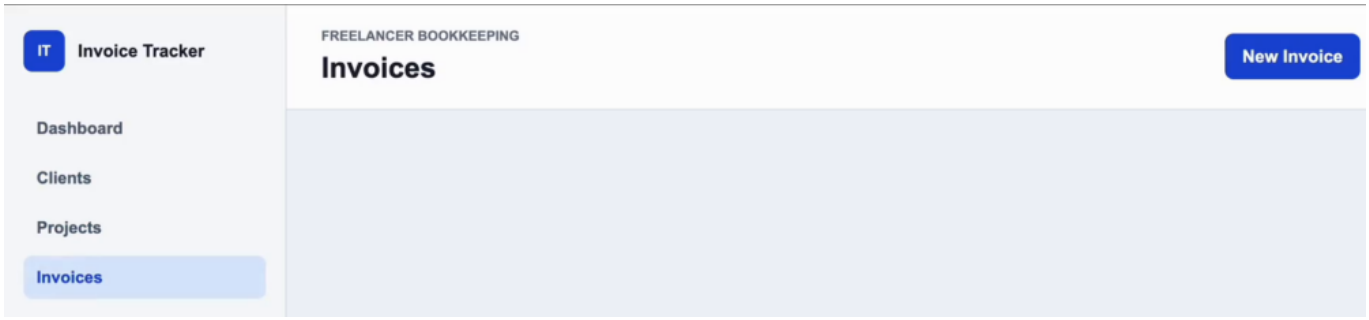
© 2026 Zenva Pty Ltd. All rights reserved.



After approval, Next.js may select port 3001 if port 3000 is already occupied. A request made from inside the sandbox can still fail even when the server reports that it is ready. In that situation, inspect the server log and verify the page directly in the browser instead of assuming the implementation failed.

Verify the rendered shell

Open the application in the browser and visit one of the resource routes. The shared shell should display the Invoice Tracker brand, Dashboard, Clients, Projects, and Invoices navigation links, the title for the current route, and the New Invoice action. The active section should be visually highlighted. The resource page itself may still be empty or return a not-found state because this lesson only builds the surrounding shell.



Review every changed file

Before committing, ask Codex to perform one final scope review with this exact prompt:

```
Run git diff and summarise every file you created or
modified and what changed in each one.
```

Confirm that the changes are limited to the root layout, global styles, layout CSS module, Sidebar and Header components, and their CSS modules. Lint should pass, and no database or src/lib files

© 2026 Zenva Pty Ltd. All rights reserved.

should be modified. With this scope verified, the app shell is ready for the shared utilities and remaining pages that follow.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will add reusable status and formatting utilities to the Invoice Tracker application. We will use Codex to create a status badge, centralize currency, date, and invoice-number formatting, verify the generated code, document the conventions in AGENTS.md, and commit the completed application shell.

Prompt Codex to create the shared utilities

Start by giving Codex the complete requirements for both the status badge and the formatter module. The exact prompt used in the lesson is:

```
Create the following shared components and utilities:

1. src/components/StatusBadge.jsx — a badge component that
   accepts a status prop and renders it with an appropriate
   colour. Support these statuses:
   - Invoice statuses: draft (grey), sent (blue), paid (green),
     overdue (red), void (dark grey)
   - Project statuses: active (green), paused (yellow),
     completed (blue), archived (grey)

2. src/lib/formatters.js — utility functions:
   - formatCurrency(cents, currency = 'USD') — converts
     integer cents to a formatted currency string like $1,234.56
   - formatDate(isoString) — converts an ISO date string to
     a readable format like 12 Jan 2025
   - formatInvoiceNumber(number) — returns the invoice number
     as-is, padded to at least 4 digits if numeric

Use plain CSS modules for StatusBadge styling.
No external libraries for formatting — use built-in
Intl APIs only.
```

Codex plans three new files: StatusBadge.jsx, its adjacent CSS module, and src/lib/formatters.js. The badge normalizes its status prop, maps the known invoice and project statuses to color tones, and falls back to gray for an unknown value. Keeping these rules in one component prevents individual pages from defining inconsistent inline status colors.

Centralize display formatting

The formatter module exposes three named functions. formatCurrency converts integer cents into a localized amount, preserving the database convention of storing money as whole numbers:

```
export function formatCurrency(cents, currency = "USD") {
  const amount = Number(cents || 0) / 100;

  return new Intl.NumberFormat("en-US", {
    style: "currency",
    currency,
  }).format(amount);
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

formatDate uses the built-in Intl.DateTimeFormat API and explicitly selects UTC. This makes date-only values deterministic and prevents a date from shifting when the application runs in a different local time zone:

```
export function formatDate(isoString) {
  if (!isoString) {
    return "";
  }

  return new Intl.DateTimeFormat("en-GB", {
    day: "2-digit",
    month: "short",
    year: "numeric",
    timeZone: "UTC",
  }).format(new Date(isoString));
}
```

The invoice-number formatter pads numeric values to four digits and leaves nonnumeric values unchanged:

```
export function formatInvoiceNumber(number) {
  const value = String(number);

  if (/^d+$/.test(value)) {
    return value.padStart(4, "0");
  }

  return value;
}
```

Verify the formatter output

After Codex creates the files and runs npm run lint, ask it to exercise the formatter functions with known values. The exact prompt is:

```
Run a quick node script to test the formatters — pass in
sample values and confirm the output looks correct:
- formatCurrency(125000) should return $1,250.00
- formatDate('2025-01-15') should return a readable date
- formatInvoiceNumber(42) should return INV-0042 or similar
```

The initial import check confirms that the module loads, its functions execute, and linting completes without errors.

© 2026 Zenva Pty Ltd. All rights reserved.

```
• The component and utility are in place. I'm running lint and a small Node import ch

• Ran git status --short
  └  M src/app/globals.css
     M src/app/layout.js
    ?? src/app/layout.module.css
    ?? src/components/
    ?? src/lib/formatters.js

• Ran node --input-type=module -e "import { formatCurrency, formatDate, formatInvoice
  │ formatters.js'; console.log(formatCurrency(123456)); console.log(formatDate('2025-
  │ console.log(formatInvoiceNumber(7)); console.log(formatInvoiceNumber('INV-7'));"
  └ $1,234.56
    12 Jan 2025
    … +7 lines (ctrl + t to view transcript)
    package.json.
    (Use `node --trace-warnings ...` to show where the warning was created)

• Ran npm run lint
  └ (no output)
```

The focused examples return $1,250.00, 15 Jan 2025, and 0042. The invoice number intentionally has no INV- prefix because the original implementation requirement says to preserve the value and only pad it when it is numeric.

```
Formatter smoke test output:

formatCurrency(125000): $1,250.00
formatDate('2025-01-15'): 15 Jan 2025
formatInvoiceNumber(42): 0042

This matches the current implementation. formatInvoiceNumber(42) pads numeric values to 4 digits but does not
an INV- prefix, following the earlier requirement to return the invoice number as-is, padded if numeric.
```

Record the shared-component conventions

Reusable components are most effective when later AI-assisted tasks know that they exist. Add durable guidance to AGENTS.md with the following exact prompt:

```
Add a Shared Components section to AGENTS.md documenting:
- StatusBadge at src/components/StatusBadge.jsx — use this
  for all status display, never hardcode status colours inline
- formatCurrency, formatDate, formatInvoiceNumber at
  src/lib/formatters.js — use these for all formatting,
  never format inline
```

The resulting Markdown establishes the shared project rules:

```
## Shared Components

- Use `StatusBadge` at `src/components/StatusBadge.jsx` for all status display. Never
 hardcode status colours inline.
- Use `formatCurrency`, `formatDate`, and `formatInvoiceNumber` at `src/lib/formatter
s.js` for all currency, date, and invoice number formatting. Never format these value
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
s inline.
```

This section acts as a living style guide. Future Codex sessions can reuse the same badge and formatting behavior instead of duplicating presentation logic.

Review and commit the milestone

Before committing, review git status, the complete diff, and the diff summary. This confirms that the staged set contains only the application shell, shared components, formatter utilities, and AGENTS.md update. The exact commit prompt is:

```
Commit all changes with the message "add app shell
and shared components"
```

Codex may need explicit permission to run git add because staging changes repository metadata. After approval, verify the staged files, create the commit, and run a final status check to confirm that the working tree is clean.

Review the completed foundation

This milestone completes the core application shell with its sidebar, dynamic header, and root layout. It also adds the reusable status badge and formatter functions, verifies the interface and utility output, records the shared conventions for future Codex work, and commits the changes to Git. The project now has a consistent, documented foundation for the database service layer and later application features.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will build and test a dedicated database service layer for the Invoice Tracker application. This layer centralizes data access, validation, calculations, and transactions so that future pages can work with predictable JavaScript objects instead of issuing raw SQL queries.

Start from a clean repository

Run git status before making changes and confirm that the working tree is clean. A known starting point makes the upcoming service, test, configuration, and documentation changes easier to review.

Research the database integration pattern

Before implementing the service layer, identify how better-sqlite3 should be initialized in a Next.js App Router project, how server-only modules should be organized, and how development hot reloads can affect database connections. Delegate this focused research to a Codex subagent so the investigation runs in a separate context and the main implementation session stays concise.

Use this prompt to delegate the research:

```
Use a subagent to research the correct patterns for using
better-sqlite3 with Next.js 16 App Router. Specifically:

1. How to safely initialise a single database connection
   that works across server components and API routes
   without being bundled on the client side
2. The recommended file structure for a service layer
   in a Next.js App Router project
3. Any known gotchas with better-sqlite3 and Next.js
   hot reload in development

The subagent should report back a concise summary of
findings. Do not write any code yet.
```

The resulting architecture keeps the SQLite connection in src/lib/db.js and marks src/lib/services.js as server-only:

```
import 'server-only'

import db from './db.js'
```

This guard prevents the database module from being bundled into client-side code. The service file then becomes the single entry point for all client, project, and invoice data operations.

Build the database service layer

Ask Codex to implement read and write functions for clients, projects, and invoices. Each function should return camelCase objects that are ready for the UI, use descriptive errors, and keep all money values as integer cents.

Use this implementation prompt:

```
Based on the subagent's findings, create the database
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
service layer at src/lib/services.js. Include these
functions:

Clients:
- getClients() – returns all clients ordered by name
- getClientById(id) – returns a single client with their
  project count and total unpaid amount
- createClient(data) – inserts a new client, returns the
  created record
- updateClient(id, data) – updates a client, returns the
  updated record
- deleteClient(id) – deletes a client, throws if projects exist

Projects:
- getProjects(filters) – returns projects with optional
  filters for clientId and status, includes client name
- getProjectById(id) – returns a single project with client
  details and invoice summary
- createProject(data) – inserts a new project
- updateProject(id, data) – updates a project
- deleteProject(id) – deletes a project, throws if invoices exist

Invoices:
- getInvoices(filters) – returns invoices with optional
  filters for status, clientId, projectId; includes client
  and project name
- getInvoiceById(id) – returns a single invoice with its
  line items, client, and project details
- createInvoice(data) – inserts an invoice and its line items
  in a single transaction, calculates totals
- updateInvoice(id, data) – updates invoice and replaces
  line items in a single transaction, recalculates totals
- deleteInvoice(id) – deletes an invoice and its line items

All money values are in cents. All functions should throw
descriptive errors for invalid operations. Do not run npm run build to verify – run n
pm run lint
instead and show the results.
```

Shared normalization helpers enforce these rules before values reach SQLite. For example, monetary inputs must be nonnegative integers:

```
function normalizeMoney(value, label, { allowNull = false } = {}) {
  if (value === undefined || value === null || value === '') {
    if (allowNull) return null
    throw new Error(`${label} is required`)
  }

  const cents = Number(value)

  if (!Number.isInteger(cents) || cents < 0) {
    throw new Error(`${label} must be a non-negative integer number of cents`)
  }
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  return cents
}
```

The client functions list, retrieve, create, update, and delete clients. Their read results include useful aggregates such as project counts and unpaid totals. The project functions provide the equivalent operations with filters, client information, invoice counts, and nonvoid invoiced totals. Delete guards turn foreign-key failures into clear business messages when a client still owns projects or a project still owns invoices.

The invoice functions handle richer data. They return project, client, and line-item details, calculate every line-item total on the server, and derive invoice subtotal and total values instead of trusting totals supplied by the UI.

Keep invoice writes atomic

Creating or updating an invoice affects both the parent invoice and its line items. Wrap these operations in a better-sqlite3 transaction so they either succeed together or leave the database unchanged:

```
export function createInvoice(data) {
  const invoice = normalizeInvoiceData(data)

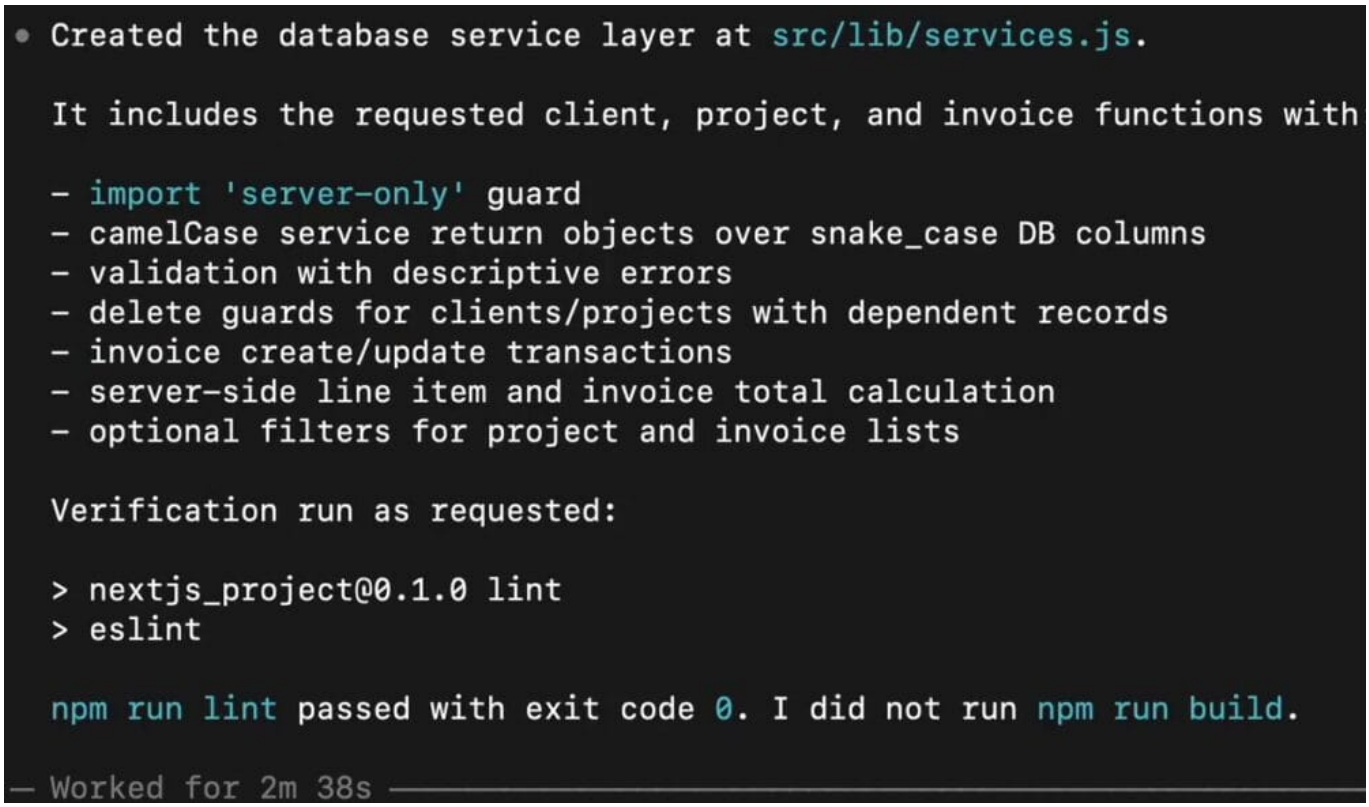
  try {
    const invoiceId = insertInvoiceWithItems(invoice)
    return getInvoiceById(invoiceId)
  } catch (error) {
    if (error.code === 'SQLITE_CONSTRAINT_UNIQUE') {
      throw new Error(`Invoice number ${invoice.invoiceNumber} already exists`)
    }

    throw error
  }
}
```

The update transaction replaces the invoice metadata and line items atomically. The service also catches unique-constraint failures and reports a descriptive error when an invoice number already exists.

Run npm run lint and review the generated file. The completed layer should contain the requested client, project, and invoice operations, server-only protection, validation, delete guards, filters, calculations, and transactional invoice writes.

© 2026 Zenva Pty Ltd. All rights reserved.



Configure Vitest with an isolated database

Install Vitest as a development dependency and add an npm run test script. Configure the suite for a Node environment and alias the server-only import so the service module can load during tests.

Use this prompt to create and verify the test suite:

```
Create a test file at src/lib/services.test.js that tests
the core service functions.

For setup:
- Create a test database setup that uses an in-memory
  SQLite database so tests don't touch the real database
- Run the migration on the in-memory database before tests
- Seed minimal test data before each test

Write tests for:
- getClients() returns all clients
- getClientById() returns the correct client
- createClient() inserts and returns a new client
- deleteClient() throws when projects exist
- getInvoices() returns all invoices
- getInvoiceById() returns the invoice with line items
- createInvoice() calculates totals correctly from line items
- deleteInvoice() cascades to line items

Do not run npm run build to verify — run npm run test
instead and show the results.
```

© 2026 Zenva Pty Ltd. All rights reserved.

Modify src/lib/db.js to select an in-memory SQLite database only when NODE_ENV is test. Normal application runs continue using the file-based database:

```
const dbPath = process.env.NODE_ENV === 'test'
  ? ':memory:'
  : path.join(process.cwd(), 'invoice-tracker.db')

const db = new Database(dbPath)

db.pragma('journal_mode = WAL')
db.pragma('foreign_keys = ON')
```

This makes the tests fast and isolated while preserving the same schema, constraints, and foreign-key behavior as the application database.

Seed predictable test data

Create src/lib/services.test.js, import the migration and service functions, and reset the database before each test. Seed two clients, one project, and one invoice containing two line items. Every test then begins from the same controlled state.

```
beforeEach(() => {
  fixture = seedTestData()
})
```

The tests should exercise business behavior rather than only checking that queries execute. Verify that client reads calculate unpaid totals, invoice reads include related records and line items, and invoice creation calculates totals from quantities and unit prices:

```
expect(invoice.subtotalCents).toBe(55000)
expect(invoice.totalCents).toBe(55000)
expect(invoice.paidCents).toBe(0)
expect(invoice.lineItems.map((item) => item.lineTotalCents))
  .toEqual([25000, 30000])
```

Also delete an invoice and query both tables to confirm that its line items are removed by the schema’s cascade rule. This test proves that the service and database constraints work together correctly.

Run the verification checks

Run npm run test and then npm run lint. The completed suite contains eight Vitest tests covering client, project, and invoice services, with all eight tests and the lint check passing. A production build is intentionally unnecessary for this task.

© 2026 Zenva Pty Ltd. All rights reserved.

```
npm run test result:

Test Files  1 passed (1)
Tests       8 passed (8)

I did not run npm run build. I also ran npm run lint afterward; it passed.
```

Review and document the changes

Ask Codex for a plain-English summary of git status, git diff, and the diff statistics, including untracked files. Confirm that the changes are limited to the service layer, test suite, Vitest configuration, test-aware database initialization, package metadata, and project instructions.

The review prompt shown in the lesson is:

```
Run git status and git diff and summarise every file
you created or modified and what changed in each one.
```

Update the Data Layer section in AGENTS.md with four durable conventions:

```
Add a Data Layer section to AGENTS.md documenting:
- All database access goes through src/lib/services.js
- Never write raw SQL queries inside components or pages
- All money values are stored and passed as integer cents
- Run npm run test before completing any task that
  touches the service layer
```

- All database access goes through src/lib/services.js.
- Components and pages never contain raw SQL queries.
- Money values are stored and passed as integer cents.
- npm run test must run after any service-layer change.

Correct any outdated test command while editing the file. These instructions ensure that later Codex tasks use the tested data layer instead of bypassing it.

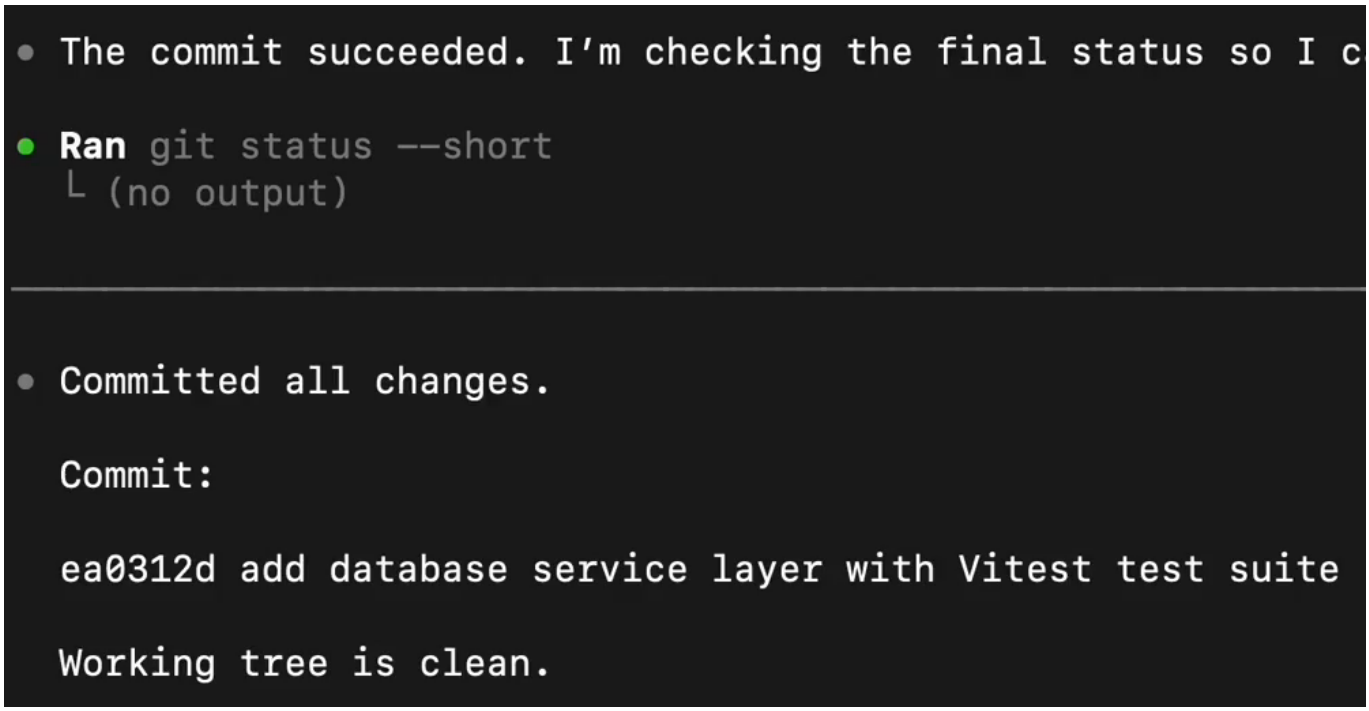
Commit the database milestone

Review the final status, approve repository-write permission if Codex cannot stage files inside its sandbox, and commit the changes with a descriptive message such as add database service layer and tests. Finish with another status check to confirm that the commit succeeded and the working tree is clean.

Use the exact commit prompt from the lesson:

```
Commit all changes with the message "add database
service layer with Vitest test suite"
```

© 2026 Zenva Pty Ltd. All rights reserved.



The application now has a centralized, validated, transactional, and tested data foundation. Future Server Components and Server Actions can call these functions without duplicating SQL or business rules.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will create Next.js Server Actions for the client, project, and invoice mutations in our Invoice Tracker. These functions form a server-only boundary between interactive components and the existing service layer, keeping database access and validation out of the browser.

Understand the Server Action pattern

Without Server Actions, an application may create internal REST endpoints, call them with fetch(), choose HTTP methods, serialize request bodies, and parse JSON responses. Next.js Server Actions remove much of this internal networking boilerplate. A component can invoke a server-side function directly, while Next.js manages the request boundary.

We will use actions for mutations such as creating, updating, and deleting records. Server Components can continue calling the service layer directly for reads, so the application does not need to fetch its own internal data.

Start from a clean project

Before generating code, run git status and confirm that the working tree is clean. This gives us a known baseline and makes the resulting diff easier to review. Start Codex from the project directory once the repository state is confirmed.

Generate the action modules

Use the following prompt to create the three action files. This is the exact prompt presented in the lesson:

```
Create Next.js Server Actions for the invoice tracker.
Organise them into three files under src/actions/.

All actions must:
- Start with 'use server' at the top of the file
- Import from src/lib/services.js only — no raw SQL
- Return a consistent shape: { data, error } where error
  is a string message or null

src/actions/clientActions.js
- createClientAction(formData or plain object) — validates
  name is present, calls createClient()
- updateClientAction(id, data) — calls updateClient()
- deleteClientAction(id) — calls deleteClient(), returns
  a descriptive error if projects exist

src/actions/projectActions.js
- createProjectAction(data) — validates clientId and name
  are present, calls createProject()
- updateProjectAction(id, data) — calls updateProject()
- deleteProjectAction(id) — calls deleteProject(), returns
  a descriptive error if invoices exist

src/actions/invoiceActions.js
- createInvoiceAction(data) — validates projectId,
  invoiceNumber, issueDate, and at least one line item
  are present, calls createInvoice()
- updateInvoiceAction(id, data) — calls updateInvoice()
- deleteInvoiceAction(id) — calls deleteInvoice()
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
- updateInvoiceStatusAction(id, status) — validates status
  is one of draft, sent, paid, overdue, void, calls
  updateInvoice() with only the status field

Do not run npm run build — verify by reading the created
files and confirming the structure is correct.
```

Before editing, Codex inspects the service exports, relevant Next.js documentation, and existing tests. It then creates src/actions/clientActions.js, src/actions/projectActions.js, and src/actions/invoiceActions.js.

Use a consistent response shape

Every action file begins with 'use server'. This directive marks the module as server-only code. The files import database operations from src/lib/services.js instead of querying SQLite directly.

Small helper functions give every action the same response contract:

```
function result(data = null, error = null) {
  return { data, error }
}

function errorMessage(error) {
  return error instanceof Error ? error.message : 'Something went wrong'
}
```

A successful action returns its value in data and a null error. A failed action returns null data and a readable error string. Components can therefore process every mutation result in a predictable way.

Create client actions

The client module provides create, update, and delete actions. It accepts either a plain object or FormData, converts form entries when necessary, and validates the required client name before calling createClient().

The delete action catches the relationship error raised when projects still belong to a client and replaces it with a clearer message for the interface. The service layer continues to enforce the database rule, while the action layer makes the result easier for users to understand.

Create project actions

The project module follows the same pattern. Project creation requires both clientId and name. Update and delete operations call their matching service functions inside try/catch blocks. If invoices still belong to a project, the delete action returns a specific relationship error.

Create invoice actions

Invoice creation validates all required values before touching the service layer. It requires a project, invoice number, issue date, and at least one line item:

```
if (!data?.projectId || String(data.projectId).trim() === '') {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  return result(null, 'Project is required')
}

if (!data?.invoiceNumber || String(data.invoiceNumber).trim() === '') {
  return result(null, 'Invoice number is required')
}

if (!data?.issueDate || String(data.issueDate).trim() === '') {
  return result(null, 'Issue date is required')
}

if (!Array.isArray(data.lineItems) || data.lineItems.length === 0) {
  return result(null, 'At least one invoice line item is required')
}
```

The module also wraps general invoice updates and deletions. These actions keep error handling consistent without duplicating database logic.

Handle focused status updates safely

The requested status action accepts only draft, sent, paid, overdue, or void. During planning, Codex identifies an important mismatch: the service function updateInvoice() expects complete invoice data, so a call such as updateInvoice(id, { status }) does not provide enough information.

The completed implementation retrieves the current invoice, applies the new status to a complete object, and then calls the service. When an invoice becomes paid, it also records the total as paid and adds a payment timestamp if one is not already present:

```
const invoice = getInvoiceById(id)

if (!invoice) {
  return result(null, `Invoice ${id} not found`)
}

const nextInvoice = {
  ...invoice,
  status,
  paidCents: status === 'paid' ? invoice.totalCents : invoice.paidCents,
  paidAt: status === 'paid'
    ? (invoice.paidAt || new Date().toISOString())
    : invoice.paidAt,
}

return result(updateInvoice(id, nextInvoice), null)
```

This focused action can support controls that mark an invoice as sent, paid, overdue, or void without submitting the full invoice form from the browser.

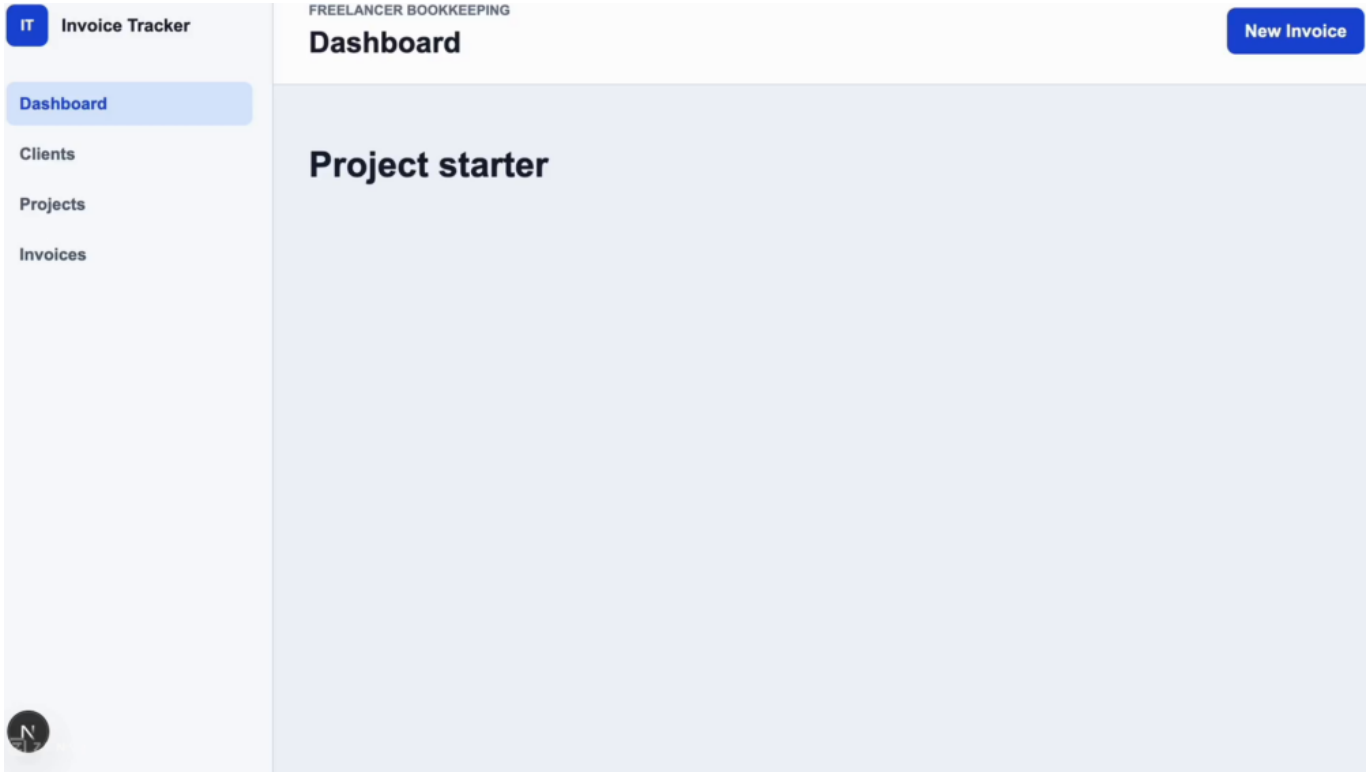
Verify the generated files

Read all three action modules and inspect git status --short. Confirm that each file starts with 'use

© 2026 Zenva Pty Ltd. All rights reserved.

server', imports only service functions, catches errors, and returns { data, error }. The project instructions specifically prohibit npm run build, so do not use it for verification.

The development server continuously compiles file changes. Open the application in the browser and confirm that it renders without a red Next.js error overlay or related console warning. This is an initial structural check. The first end-to-end mutation test will occur when the Clients page begins using these actions.



Document the conventions

Use a second prompt to record the Server Action boundaries in AGENTS.md. This is the exact prompt shown in the lesson:

```
Add a Server Actions section to AGENTS.md documenting:
- All data mutations go through Server Actions in src/actions/
- Actions import from src/lib/services.js only — never
  query the database directly
- Every action returns { data, error } — error is a string
  or null
- Action files must start with 'use server'
- For reads, use service functions directly in Server
  Components — no action needed
- Never use fetch() to call our own data — use actions
  or service functions directly

Then commit all changes with the message "add Server Actions
for clients, projects, and invoices"
```

The resulting AGENTS.md section ensures that future Codex tasks preserve the same architecture:

© 2026 Zenva Pty Ltd. All rights reserved.

mutations use actions, reads use service functions in Server Components, action modules never query the database directly, and internal application data is not accessed through fetch().

Review and commit the milestone

Review the complete diff before staging. The expected scope is the three new action modules and the updated AGENTS.md file. Stage those files, commit them with the requested message, and run git status --short one final time. No output confirms a clean working tree and completes the Server Action milestone.

© 2026 Zenva Pty Ltd. All rights reserved.

The Complete Codex Development Course - Keep Application Layers Predictable

1. You need focused research without crowding the main Codex session. What should you do?

a. Ask the main session to investigate every option
b. Delegate a narrow research task to a subagent
c. Skip research and accept the first approach
d. Open two sessions that edit the same files

2. Codex generated a large service layer. What should you do before continuing?

a. Review only the filenames
b. Commit it as soon as it compiles
c. Run behavior tests and lint, then inspect the complete diff
d. Test only the first function

3. Codex proposes code that conflicts with existing project conventions. How should you respond?

a. Allow a new pattern for this feature
b. Remove the conventions from AGENTS.md
c. Restate the relevant rules, then review the revised changes
d. Accept the change if the page loads

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will build and verify a complete Clients page with Codex. The feature combines a Server Component for data fetching and table rendering, a Client Component for inline add and edit forms, Server Actions for mutations, and service-layer aggregates for active projects and unpaid balances.

Begin from a clean baseline

Start by running git status. The repository should report that there is nothing to commit and the working tree is clean. This gives us a known baseline before Codex begins planning or changing files.

Launch Codex in the project directory and switch to Plan mode. Planning first lets Codex inspect the existing service layer, shared components, styling conventions, and Server Actions before it proposes an architecture.

Give Codex the complete planning prompt

Use the following prompt to define the feature and require a plan before any code is written:

```
I want to build the Clients page at src/app/clients/page.js.

The page should:
- List all clients using getClients() from the service layer
- Show each client's name, company, email, number of active
  projects, and total unpaid amount in cents formatted as currency
- Have an Add Client button that opens an inline form
- Allow editing a client inline by clicking an Edit button
  on each row
- Use createClientAction and updateClientAction from
  src/actions/clientActions.js for mutations
- Use the StatusBadge and formatCurrency from our shared
  components and formatters
- Use plain CSS modules for styling

Before writing any code, give me:
1. The list of files you will create or modify
2. The component structure - which components you will
   build and how they nest
3. How you will handle the inline form state
4. Any questions or decisions that need my input before
   you start
```

Codex inspects the repository and identifies two product decisions that are not encoded in the existing project. We need to decide how StatusBadge should appear on a client row and how many fields the inline form should expose. Use an active badge next to the active-project count, because clients do not have their own status field. For the form, select all client fields: name, company, email, phone, address, and notes. This preserves the complete client record when editing.

Review the resulting plan before continuing. The accepted design keeps src/app/clients/page.js as the Server Component, places interactive state in ClientInlineForm.jsx, uses a route-local CSS module, extends getClients(), adds service tests, and reuses the existing client Server Actions.

Protect the project workflow

© 2026 Zenva Pty Ltd. All rights reserved.

Before implementing the page, add the environment-specific build restriction to AGENTS.md and commit it separately. Enter the following prompt:

```
Add this to the Critical Rules section of AGENTS.md:
- NEVER run npm run build - it hangs in this environment.
  Use npm run test for the test suite only.

Then commit with the message "add build restriction to AGENTS.md"
```

Review the diff to confirm that only the new critical rule was added. After the commit, verify that the working tree is clean again. Keeping this documentation change separate gives the Clients feature another clear starting point.

Implement the approved architecture

Once the plan and repository rules are correct, give Codex the implementation instruction exactly as follows:

```
The plan looks good. Go ahead and implement it.
Use plain CSS modules throughout - no Tailwind,
no inline styles. Make sure the page is a Server
Component that fetches data directly, and only the
inline form is a Client Component.
```

This boundary keeps the data query and list rendering on the server while sending only the form interaction code to the browser. It also prevents styling from drifting away from the project's existing CSS Module conventions.

Add client aggregates to the service layer

The original client list contains raw client fields, so getClients() must also return the values required by the new table. The grouped query counts only active projects, calculates each non-void invoice's remaining balance, and preserves clients without related projects or invoices through left joins:

```
export function getClients() {
  const rows = db.prepare(`
    SELECT
      c.*,
      COUNT(DISTINCT CASE WHEN p.status = 'active' THEN p.id END) AS active_project_c
ount,
      COALESCE(SUM(
        CASE
          WHEN i.status != 'void' AND i.paid_cents < i.total_cents
            THEN i.total_cents - i.paid_cents
          ELSE 0
        END
      ), 0) AS total_unpaid_cents
    FROM clients c
    LEFT JOIN projects p ON p.client_id = c.id
    LEFT JOIN invoices i ON i.project_id = p.id
    GROUP BY c.id
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    ORDER BY c.name COLLATE NOCASE ASC
  `).all()

  return rows.map((row) => ({
    ...mapClient(row),
    activeProjectCount: row.active_project_count,
    totalUnpaidCents: row.total_unpaid_cents,
  }))
}
```

Add focused service tests before relying on these values in the UI. The tests should cover active and paused projects, partially paid invoices, void invoices, and clients with no projects. This protects the business calculations independently of the page.

Render the Clients page on the server

The Server Component calls getClients() directly. When the returned array is empty, it renders an invitation to add the first client. Otherwise, it renders a table containing each client's name, company, email, active-project count, unpaid amount, and editing control.

Reuse StatusBadge to label the active-project count and formatCurrency to convert integer cents into a readable currency value. Mount ClientInlineForm once in create mode near the page heading and once in edit mode for each client row.

Keep form interaction in the Client Component

ClientInlineForm.jsx begins with "use client". It tracks whether the form is open, its current values, an action error, and the pending state. The same component supports creation and editing by selecting the appropriate Server Action:

```
startTransition(async () => {
  const result = isEdit
    ? await updateClientAction(client.id, formData)
    : await createClientAction(formData);

  if (result.error) {
    setError(result.error);
    return;
  }

  setIsOpen(false);
  setFormData(emptyFormData);
  router.refresh();
});
```

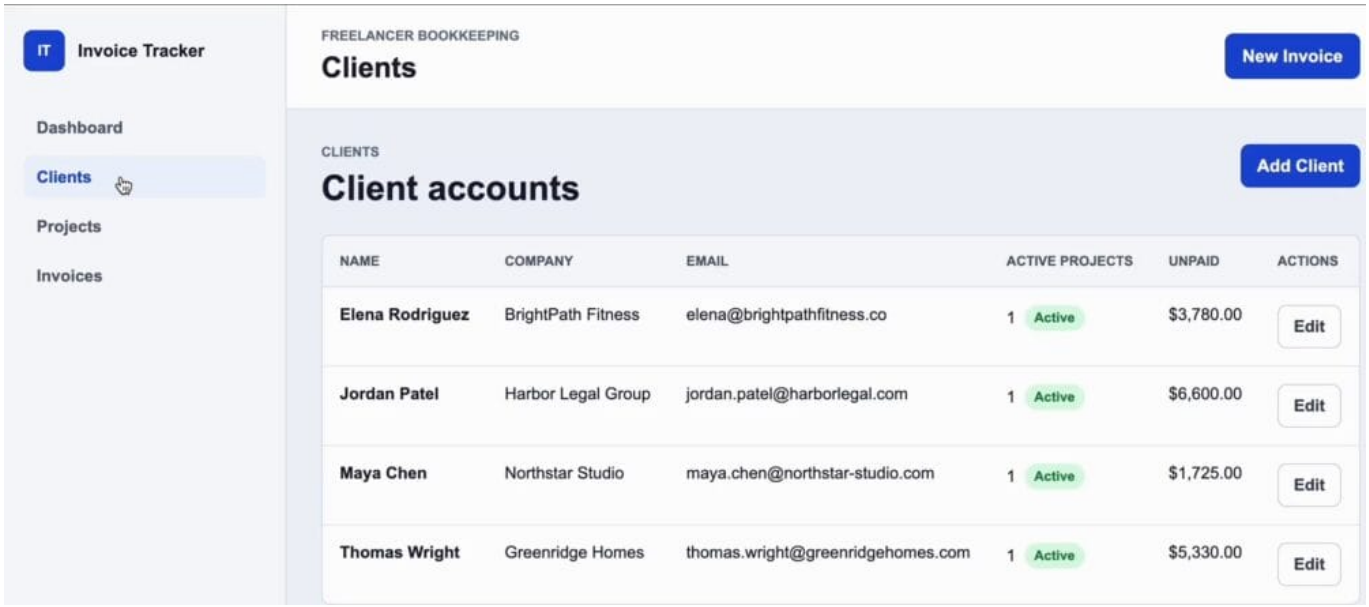
On success, the form closes, resets its values, and calls router.refresh(). The refresh requests current server-rendered data without forcing a full browser reload. On failure, the form stays open and displays the returned error.

Verify the generated implementation

© 2026 Zenva Pty Ltd. All rights reserved.

Run npm run test and npm run lint. Both commands should pass. Do not run npm run build in this environment. Start the development server and request /clients; the route should return HTTP 200.

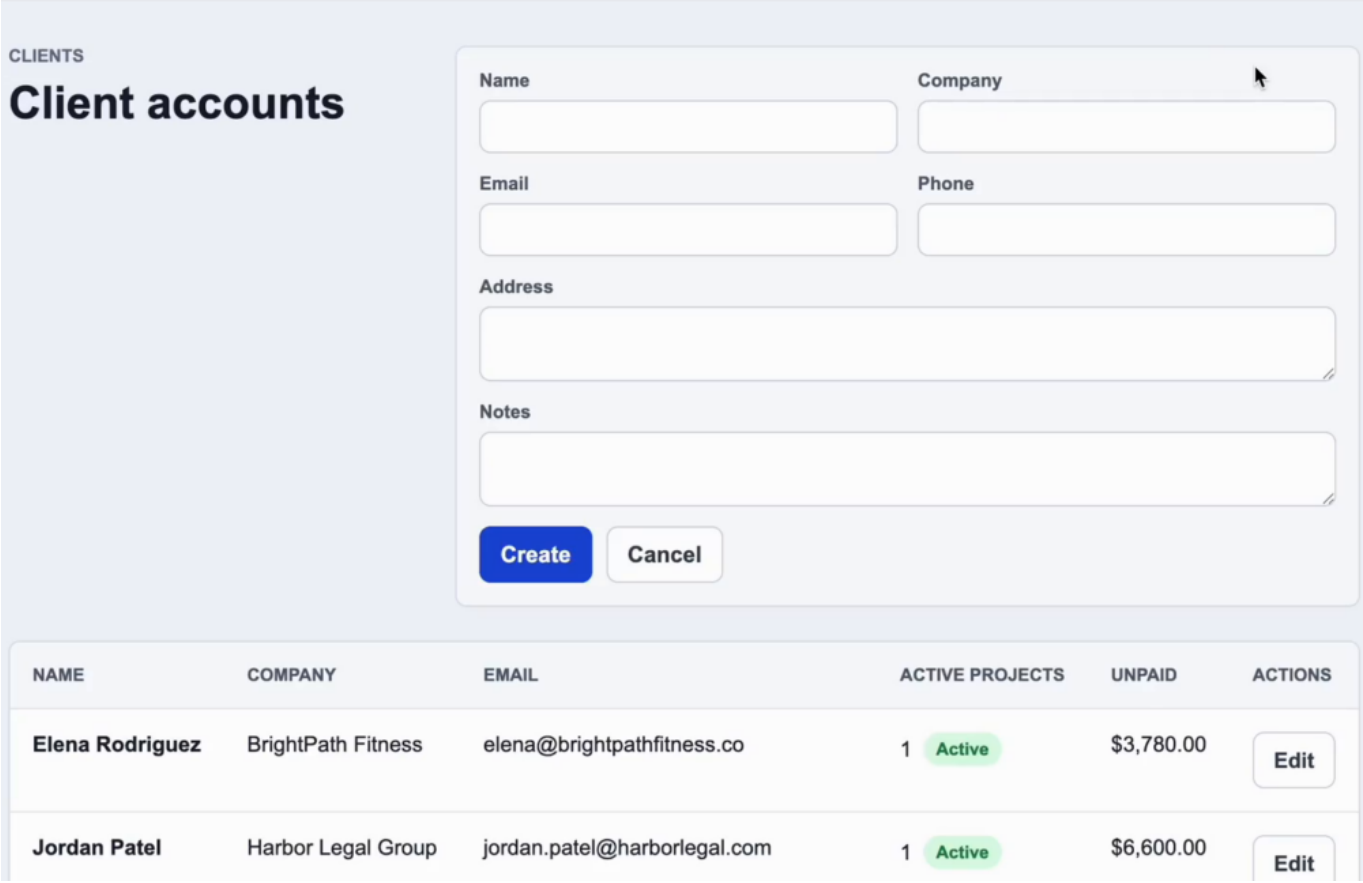
The completed table displays the seeded accounts, active-project counts, unpaid balances, and Edit controls. This confirms that the Server Component can read the new aggregate fields and render them with the shared formatting utilities.



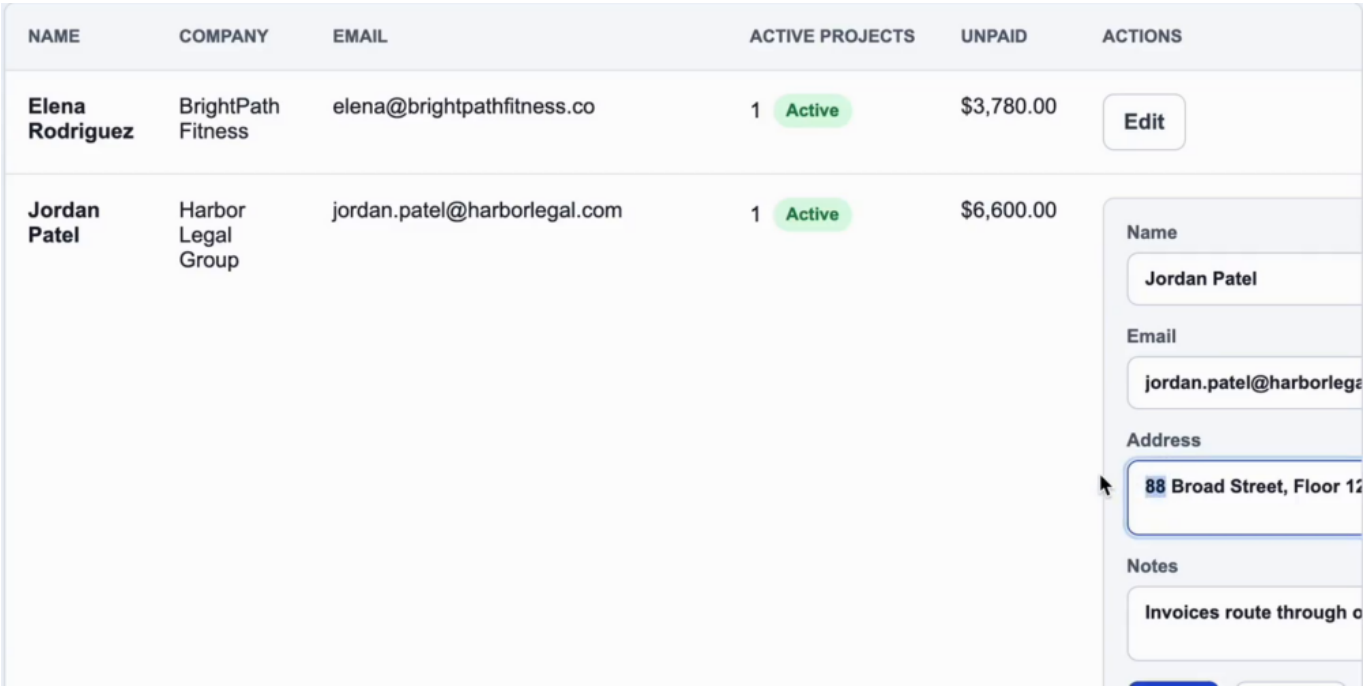
Test inline creation and editing

Select Add Client and confirm that the inline form contains name, company, email, phone, address, and notes, along with Create and Cancel controls. Cancel the form after verifying its layout.

© 2026 Zenva Pty Ltd. All rights reserved.



Next, open Jordan Patel's edit form. Change the address from 88 Broad Street, Floor 12 to 99 Broad Street, Floor 12, then save. The form closes and the updated client data appears immediately after the router refresh, which verifies the connection between the Client Component, Server Action, service layer, and database.



Review every changed file

© 2026 Zenva Pty Ltd. All rights reserved.

Before committing, ask Codex to inspect both tracked and untracked work with this exact prompt:

```
Run git status and git diff and summarise every file
you created or modified and what changed in each one.
```

Review the resulting inventory file by file. Confirm that src/lib/services.js contains the aggregate query, src/lib/services.test.js covers the new calculations, src/app/clients/page.js remains a Server Component, ClientInlineForm.jsx owns only the interactive form behavior, and ClientsPage.module.css contains the route styling.

git diff shows only tracked changes, so Codex must also inspect untracked files. The generated invoice-tracker.db-shm and invoice-tracker.db-wal files are SQLite runtime sidecars, not source code. Leave them untracked and do not include them in the feature commit.

Commit the completed feature

After the review is complete, use the following final prompt:

```
Commit all changes with the message "add Clients page
with inline add and edit"
```

Approve the scoped Git commands after verifying that they stage only the five implementation and test files. When the commit succeeds, run one final status check. The SQLite sidecars can remain untracked, while the Clients page work now provides a clean project milestone.

Apply the workflow to future pages

This lesson establishes a reusable workflow for complex UI features: begin from a clean repository, plan before coding, resolve product decisions, preserve the Server and Client Component boundary, test service logic, verify behavior in the browser, review every file, and commit only intended source changes.

The next Projects page can build on the same list-view and inline-editing pattern while adding client relationships and status filters. Continuing to use Plan mode and deliberate review will help us add that complexity without losing architectural clarity.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will use Codex to plan, implement, test, review, and commit a Projects page for the Invoice Tracker. The finished page will group projects by client, show useful project details, support inline project creation, and link each project to its filtered invoices.

Clean up generated database files

Before starting the feature, run git status and inspect every unexpected file. SQLite in WAL mode has generated two runtime sidecar files, invoice-tracker.db-shm and invoice-tracker.db-wal. These are not source files and should not be committed.

Add wildcard rules to .gitignore so any matching SQLite sidecar files are ignored:

```
*.db-shm
*.db-wal
```

Give Codex an exact planning prompt

Launch Codex from the project directory and switch to Plan mode. A detailed prompt gives the agent a clear feature boundary, points it to the existing services and shared utilities, and asks it to resolve architecture before changing code. The exact prompt used in the lesson is:

```
I want to build the Projects page at src/app/projects/page.js.

The page should:
- List all projects using getProjects() from the service layer
- Group projects by client — each client is a section header
  with their projects listed beneath
- Show for each project: name, status as a StatusBadge,
  total invoiced formatted as currency, and a link to
  /invoices?projectId=[id] that filters invoices for that project
- Have an Add Project button that opens an inline form
- The inline form should have: a client dropdown populated
  from getClients(), a project name field, a description
  field, and an hourly rate field
- Use createProjectAction from src/actions/projectActions.js
  for the mutation
- Use StatusBadge and formatCurrency from shared components
- Use plain CSS modules for styling

Before writing any code, give me:
1. The list of files you will create or modify
2. The component structure
3. How you will handle the client dropdown data in the
   inline form
4. Any questions that need my input before you start
```

Let Codex inspect the project

Codex first reads the repository instructions, the existing Clients page and form, the service layer, the project Server Action, shared components, and the relevant Next.js documentation. This context gathering reveals two useful local patterns: createProjectAction expects an object with camelCase fields, and the existing inline form uses useTransition() with router.refresh().

© 2026 Zenva Pty Ltd. All rights reserved.

Codex then asks how to calculate total invoiced amounts and how users should enter hourly rates. Extend getProjects() so the service layer supplies each total, excluding void invoices. Let users type familiar currency amounts such as 75.50, then convert the value to integer cents before sending it to the action.

Review the implementation plan

The plan keeps src/app/projects/page.js as a Server Component. It will fetch projects and clients directly from the service layer, group projects by client, format totals with formatCurrency(), render statuses through StatusBadge, and pass the client array into the inline form.

Only ProjectInlineForm.jsx needs to be a Client Component because it owns interactive state. It will open and close inline, collect the client, name, description, and hourly rate, call createProjectAction, report errors, and refresh the route after a successful creation. A route-specific CSS Module will style the grouped tables, form controls, responsive layout, and empty state.

The service change will calculate totalInvoicedCents for every project while retaining projects that have no invoices. A focused service test will confirm the calculation and verify that void invoices do not contribute to the total.

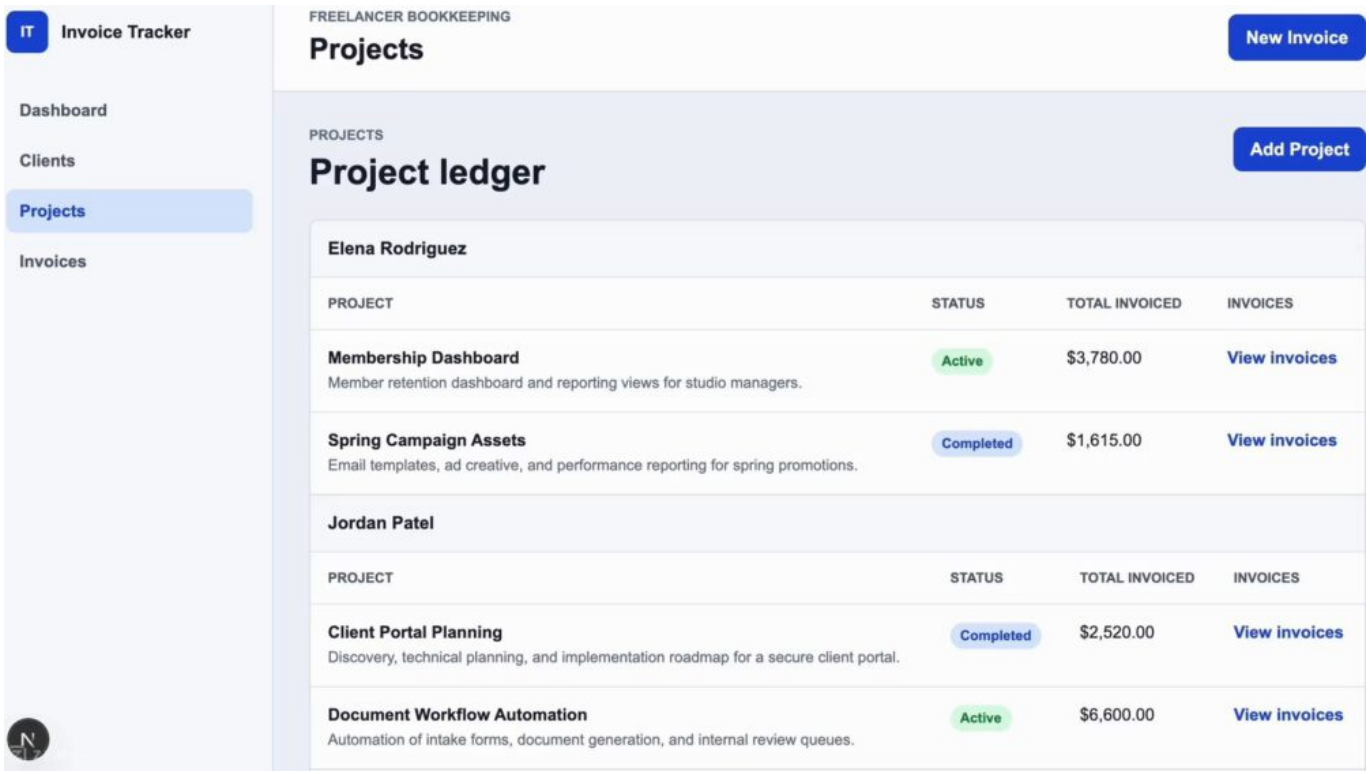
Implement and test the feature

After approving the plan, let Codex make the scoped changes. When implementation is complete, run npm run test and npm run lint. In the lesson, all ten tests pass and lint completes cleanly. Codex then requests approval to start the Next.js development server with npm run dev so we can verify the route in a browser.

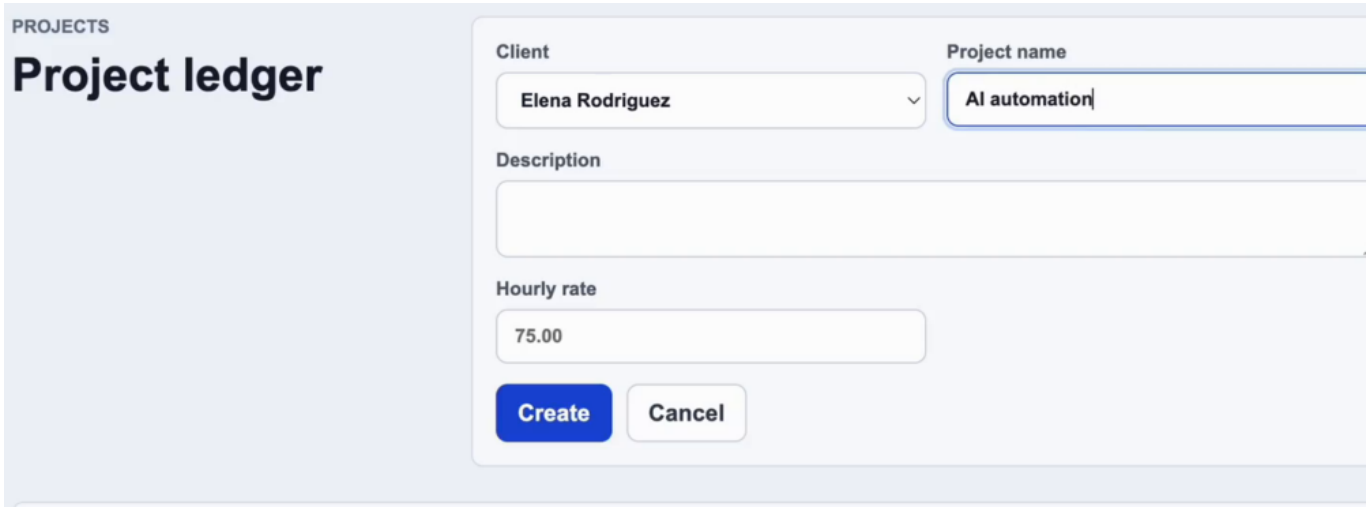
Verify the grouped project ledger

Open /projects and confirm that projects appear beneath the correct client headings. Each row should include the project name and description, a status badge, the formatted total invoiced amount, and a View invoices link. The page header should also provide an Add Project control.

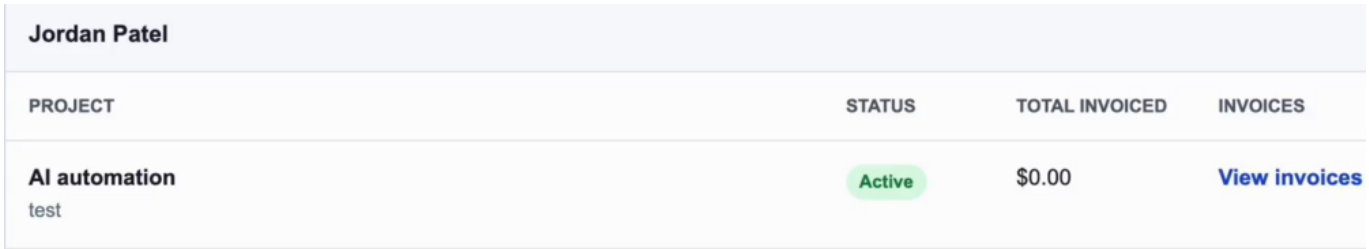
© 2026 Zenva Pty Ltd. All rights reserved.



Open the inline form and verify that it contains a populated client dropdown, project name, description, and hourly-rate fields, along with Create and Cancel controls.



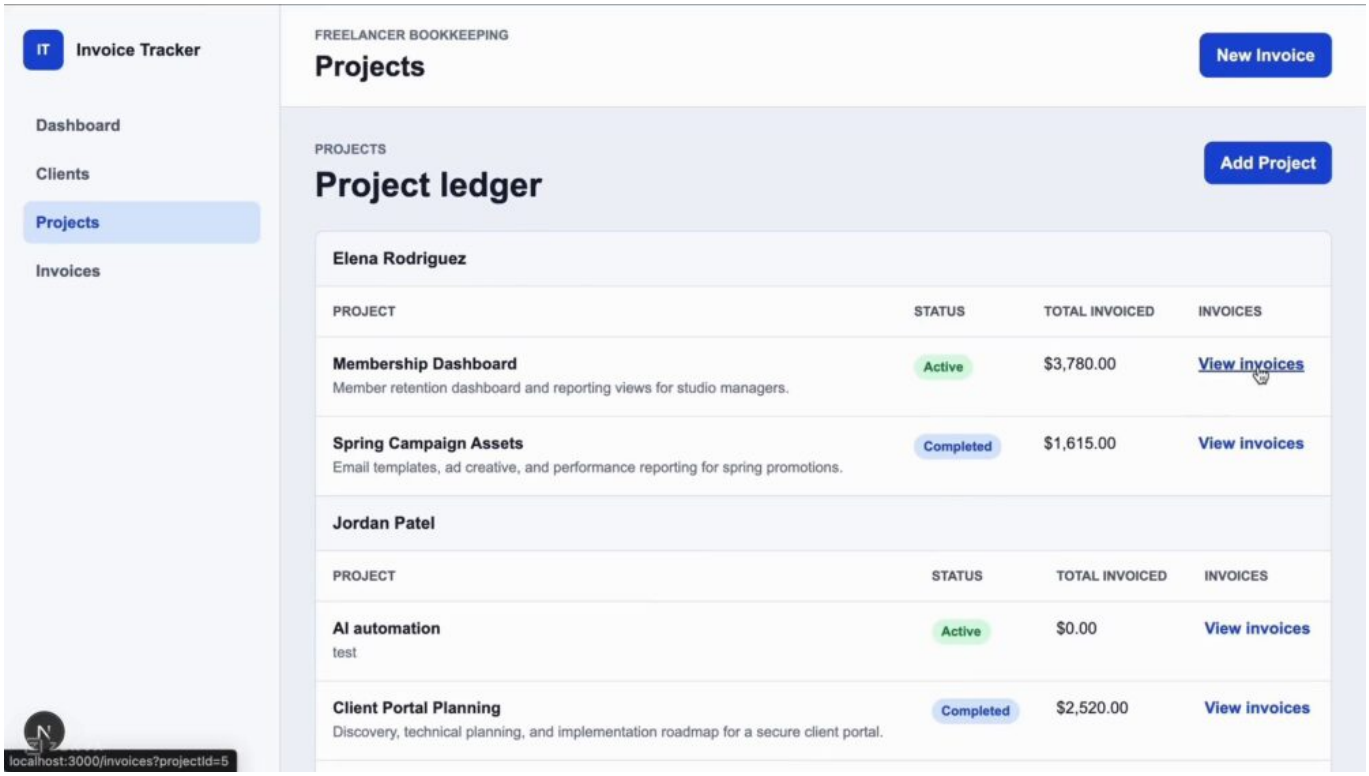
Create a project and confirm that it appears immediately beneath the selected client. A new project begins with an active status and a zero invoiced total.



Verify filtered invoice navigation

© 2026 Zenva Pty Ltd. All rights reserved.

Hover over or follow a project's View invoices link. The invoices page is still a placeholder at this point, but the destination must include the selected project ID in the query string, such as /invoices?projectId=5. This establishes the filtering contract that the Invoices page can use later.



Review and commit the changes

Before committing, ask Codex to run git status and inspect the complete diff, including untracked files. The expected scope includes the .gitignore rules, the project service aggregation and its test, the Projects Server Component, the inline Client Component, and the route CSS Module.

During the review, confirm that the service query preserves projects without invoices, excludes void invoice totals, and returns data in the shape the page expects. Also confirm that client data flows from the Server Component into the form through props, with no unnecessary client-side fetch.

Stage the intended files and create a descriptive commit. Codex may request approval before writing to Git's index and again before creating the commit. Finish by checking that git status --short returns no output, confirming a clean working tree.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will build the invoices page for our application and use parallel Codex sessions to develop separate parts of the feature at the same time. The finished page will support status filters, real-time search, configurable sorting, inline payment updates, and invoice deletion with confirmation.

Prepare a clean working state

Before starting a new feature, we should confirm that the repository has no unrelated changes. Run git status and check that the working tree is clean. This gives both Codex sessions a reliable baseline and makes the changes from this lesson easier to inspect later.

Open two terminal windows at the project root, then launch Codex in each terminal. These are independent sessions, so each one can inspect the project and complete a different task concurrently.

Divide the feature between parallel sessions

We will give the first session responsibility for the main invoices page UI. Its task includes fetching invoice data, rendering the table, and adding the status filters, search field, and sorting controls.

The second session will create the interactive action components. These include the invoice row actions and the delete confirmation modal used to mark an invoice as paid or remove it safely.

Each prompt must define strict file ownership boundaries. We should state exactly which files or directories a session may modify and which areas are outside its scope. This separation prevents the two sessions from overwriting each other’s work while they operate in the same repository.

The exact prompt for the first session is:

> Build the Invoices page UI at src/app/invoices/page.js.
>
> The page should:
>
> - Fetch all invoices using getInvoices() from the service layer
> - Display them in a table showing: invoice number, client name, project name, issue date, due date, status as a StatusBadge, and total formatted as currency
> - Include a filter bar above the table with buttons for: All, Draft, Sent, Paid, Overdue – clicking a filter shows only invoices with that status
> - Include a search input that filters invoices by client name or invoice number as the user types
> - Include sort controls for sorting by due date or total amount
> - Each invoice row should have placeholder buttons for Mark as Paid and Delete – these will be wired up by a parallel session, leave them as props or slots for now
> - Use StatusBadge and formatCurrency and formatDate from shared components
> - Use plain CSS modules for all styling
>
> You are only allowed to create or modify files in src/app/invoices/. Do not touch src/actions/, src/lib/, or any other directory.

The exact prompt for the second session is:

> Build the invoice action components for the Invoices page.

© 2026 Zenva Pty Ltd. All rights reserved.

Create two components in src/app/invoices/:

1. InvoiceActions.jsx – a Client Component that receives an invoice object as a prop and renders two buttons:
    - Mark as Paid: calls updateInvoiceStatusAction(id, 'paid') from src/actions/invoiceActions.js, only shows if invoice status is not already paid or void
    - Delete: opens a confirmation modal before calling deleteInvoiceAction(id)
2. DeleteConfirmModal.jsx – a simple modal that asks “Are you sure you want to delete invoice [number]?” with Confirm and Cancel buttons. On confirm, calls deleteInvoiceAction(id) and refreshes the page.

Use plain CSS modules for styling.

You are only allowed to create files in src/app/invoices/. Do not touch src/lib/, src/actions/, or any other directory.

```
Implemented the invoices UI under src/app/invoices/ only.

Added:

- src/app/invoices/page.js
- src/app/invoices/InvoicesTable.jsx
- src/app/invoices/InvoicesPage.module.css

The page fetches via getInvoices(), renders the table with StatusBadge, formatCurrency, formatDate,
formatInvoiceNumber, and includes client-side status filters, search, and sort controls. Row actior
placeholders for Mark as Paid and Delete.

Verification: npm run lint passed.

Dev server is running at: http://localhost:3001/invoices
```

Monitor both Codex sessions

While the sessions work, monitor their tool-use notifications in both terminals. These notifications provide an early warning if a session attempts to access a file outside its assigned boundary.

If a boundary breach occurs, press Escape to interrupt the session. Then restate the permitted file boundary clearly and direct Codex back to its assigned task. This active supervision keeps the parallel workflow predictable and avoids conflicts.

The first session creates the invoices page, client-side table component, and CSS module. It also connects the page to the invoice service and prepares the filters, search, and sorting behavior. At the same time, the second session builds the separately owned action components.

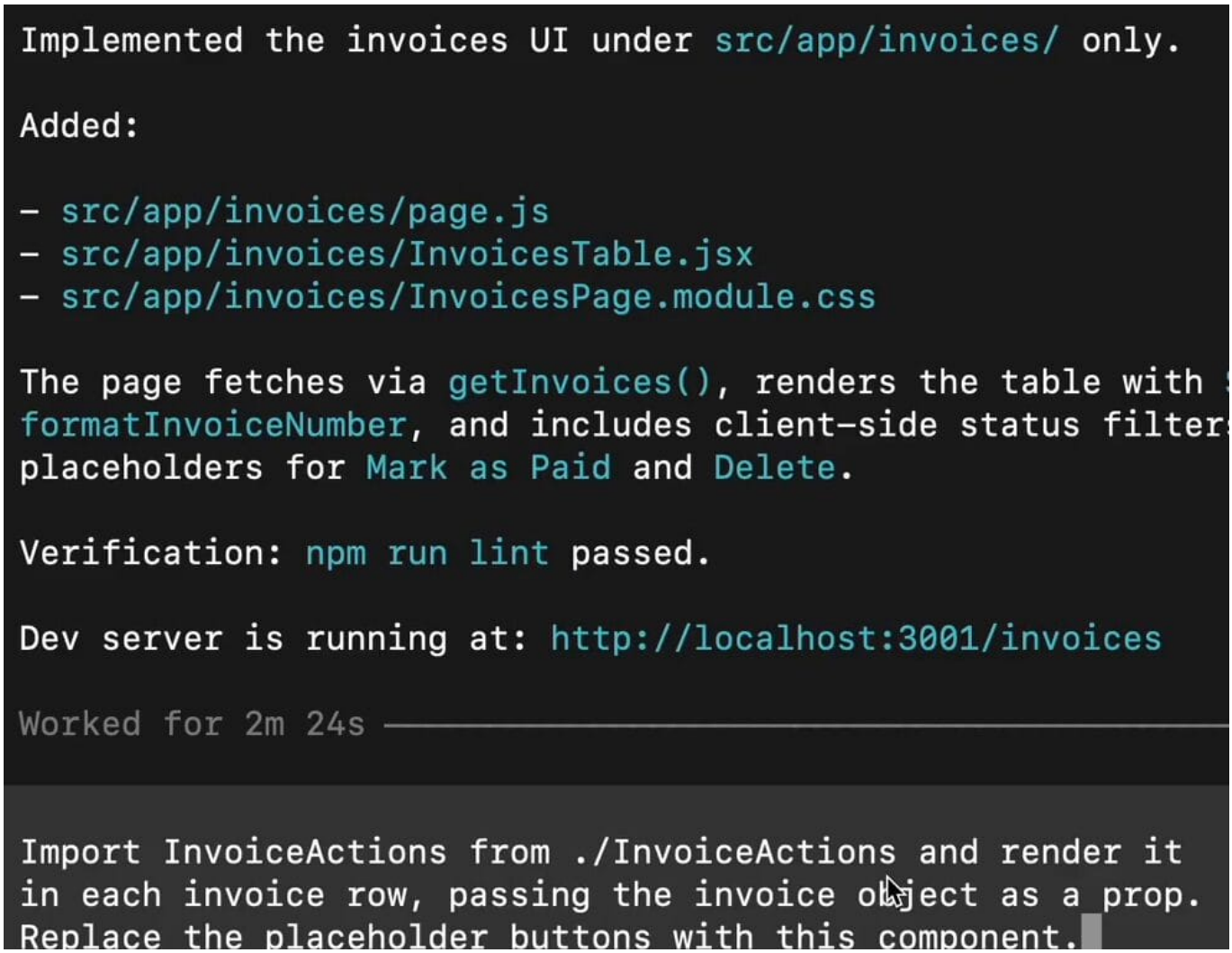
Integrate the independently developed components

After both sessions complete their initial tasks, we can join their work. In the first terminal, ask Codex to import the invoice actions component created by the second session and render it in every invoice row. The temporary placeholder buttons can then be removed.

The exact integration prompt is:

© 2026 Zenva Pty Ltd. All rights reserved.

Import InvoiceActions from ./InvoiceActions and render it in each invoice row, passing the invoice object as a prop. Replace the placeholder buttons with this component.

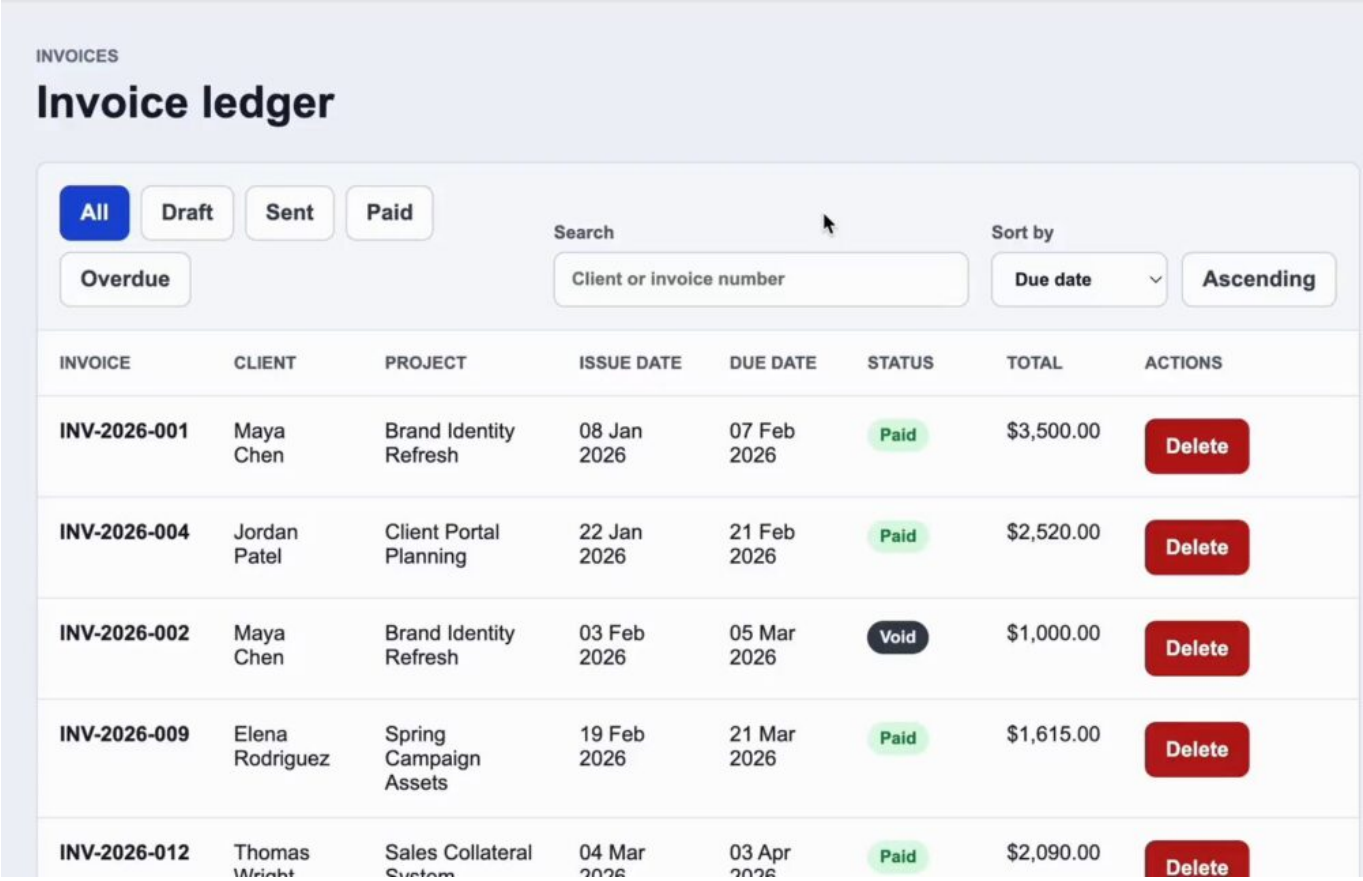


Codex inspects the invoices directory, confirms that the action component is available locally, and updates the table component. It then runs the linter to verify the integration. This demonstrates the main benefit of clear ownership boundaries: separate pieces can be developed concurrently and combined without conflicting edits.

Test the invoices page

Open the invoices route in the browser and review the full invoice ledger. Each row contains the invoice number, client, project, issue date, due date, status, total, and available actions.

© 2026 Zenva Pty Ltd. All rights reserved.

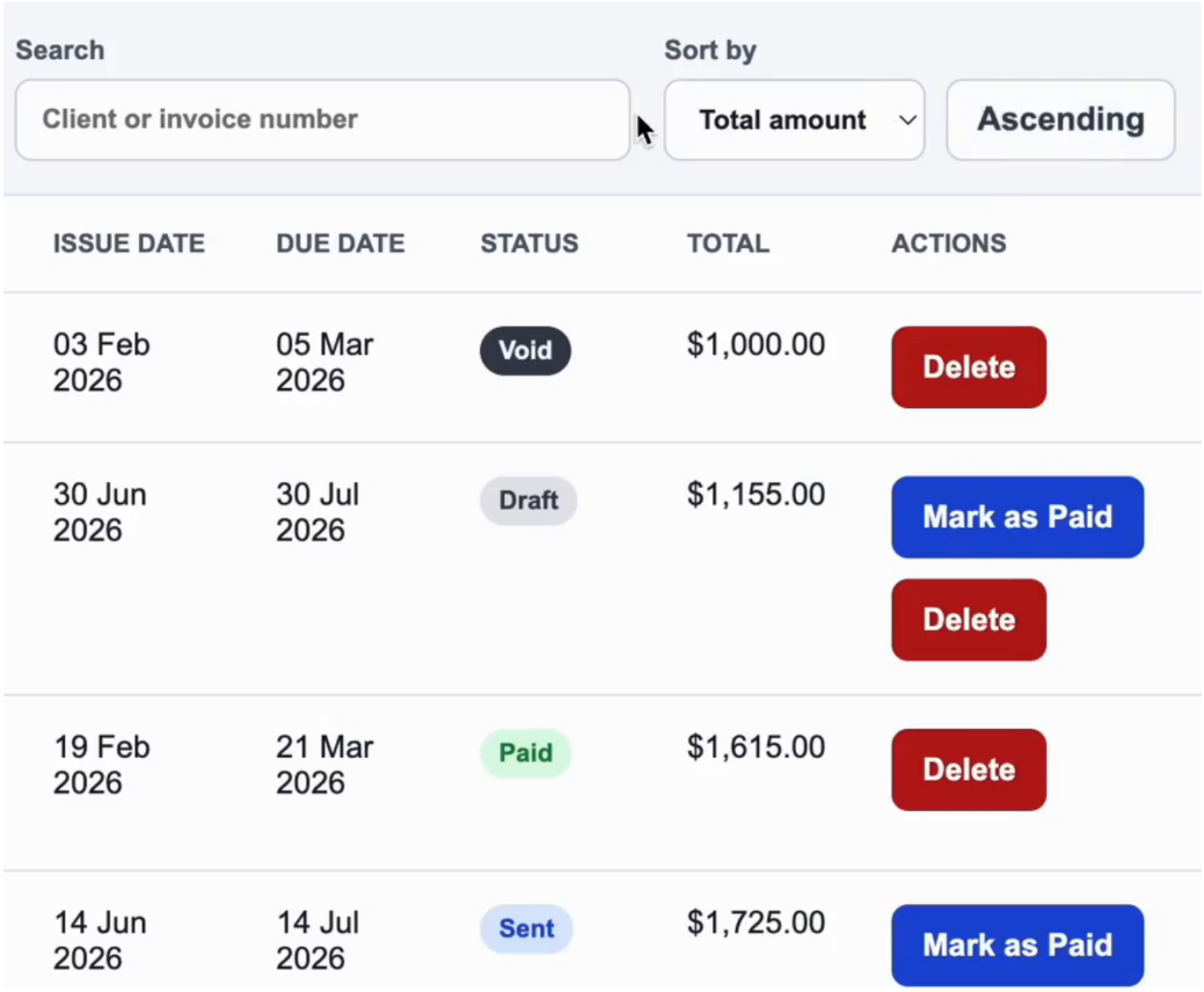


Test each status filter to confirm that the list updates correctly:

- **Draft** displays invoices that have not been sent.
- **Sent** displays invoices that have been issued to clients.
- **Paid** displays invoices that have been settled.
- **Overdue** displays invoices whose payment is outstanding.
- **All** restores the complete invoice list.

Next, change the sorting field to total amount and toggle between ascending and descending order. The table should reorder immediately, allowing users to compare invoices from lowest to highest value or from highest to lowest value.

© 2026 Zenva Pty Ltd. All rights reserved.

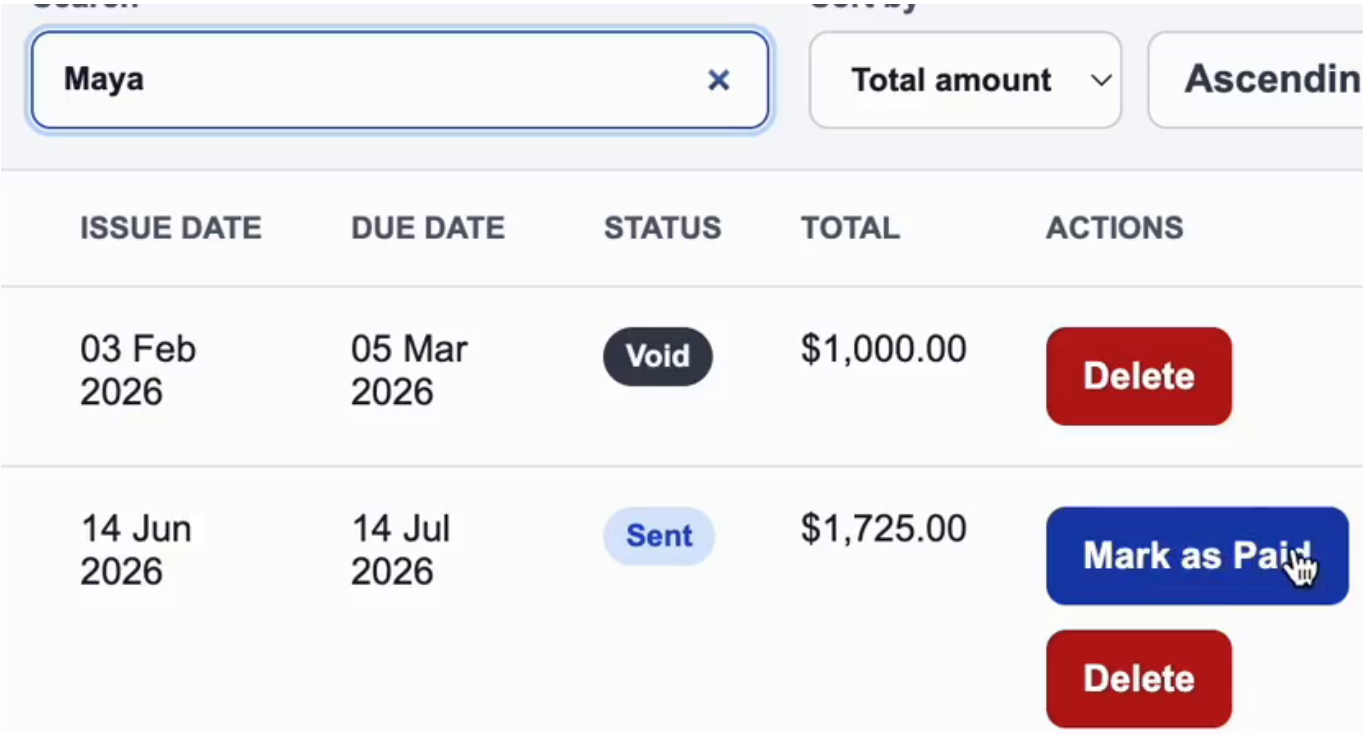


Enter a client name or invoice number in the search field. The results should filter in real time while preserving the active sorting option.

Reproduce the payment-status bug

Search for a client with a sent invoice and select **Mark as Paid**. At this stage, the application reports that at least one invoice line item is required. The status-only update is passing incomplete invoice data into validation, so the action cannot complete.

© 2026 Zenva Pty Ltd. All rights reserved.



Copy the exact error into a new Codex prompt. Providing the original error message gives Codex the context it needs to trace the update path and identify why changing only the status triggers full invoice validation.

The exact bug-fix prompt is:

> When I press “Mark as Paid”, I get an error: “At least one invoice line item is required”

Preserve the complete invoice during status updates

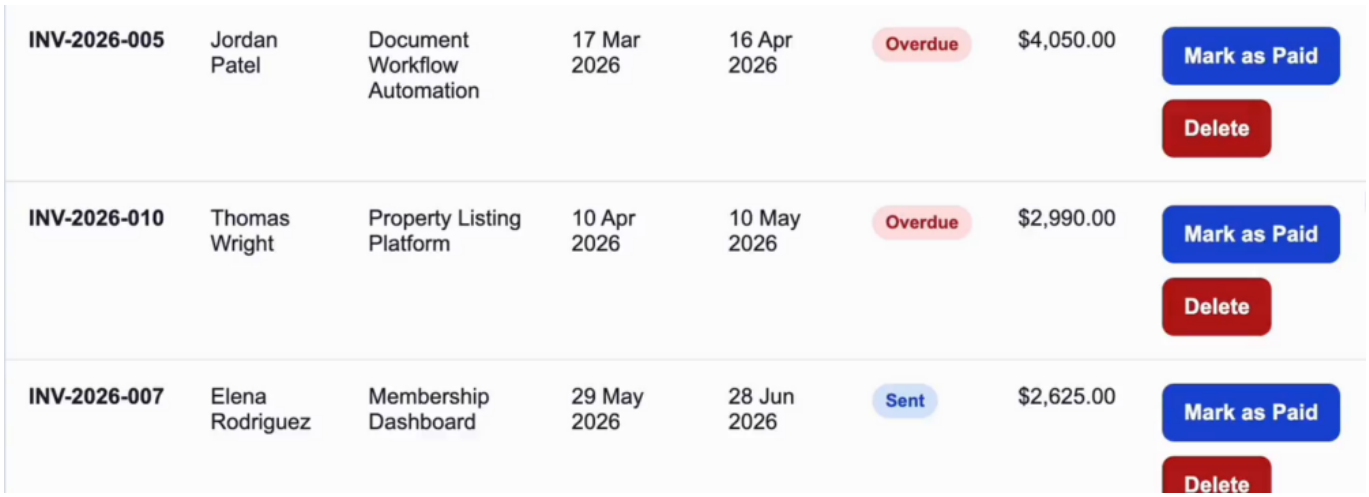
Codex fixes the action by retrieving the existing invoice before applying the status change. The update can then preserve the original fields and line items while changing only the values related to payment status.

When an invoice becomes paid, the action updates its status, paid amount, and paid date. The complete invoice payload is then sent through the existing update path, satisfying validation without discarding any data.

A test run initially fails because a native dependency does not match the current environment. Codex rebuilds that dependency and reruns the checks. Both npm run lint and npm run test pass after the rebuild.

Return to the browser and test the actions again. Marking an invoice as paid now succeeds, and deleting an invoice completes without errors after confirmation.

© 2026 Zenva Pty Ltd. All rights reserved.



Review and commit the completed feature

Run git status to review the repository state. The output identifies the modified invoice action file and the new invoices page directory created by the parallel sessions.

Use git diff to inspect the service-action change in detail. Confirm that the status update now retrieves the existing invoice and preserves its data before updating the paid fields.

Once the changes have been reviewed, stage them with git add and create a descriptive commit for the invoices page, including its filters, search, payment action, and deletion workflow. The commit records the UI, action components, integration, and bug fix as one complete feature.

Summary

We built a complete invoices page by assigning the UI and action layers to two parallel Codex sessions. Strict ownership boundaries kept their work independent, active monitoring prevented conflicts, and a final integration step combined both results. We then verified filtering, search, sorting, payment updates, and deletion, corrected a validation bug, ran linting and tests, and committed the finished feature.

© 2026 Zenva Pty Ltd. All rights reserved.

The Complete Codex Development Course - Coordinate Interactive Features

1. Two Codex sessions will build different parts of the invoices feature. What reduces merge conflicts?

a. Give both sessions the whole invoices directory
b. Define strict, separate file ownership for each session
c. Let each session rewrite shared files freely
d. Skip monitoring until both sessions finish

2. Both parallel Codex sessions finished their separate tasks. What should happen next?

a. Commit both results without reviewing them
b. Integrate the components, remove placeholders, and run the checks
c. Restart both tasks in one session
d. Let both sessions edit the integration files

3. A generated status update fails with a validation error. What makes the follow-up prompt most useful?

a. Ask Codex to try again without details
b. Request a complete feature rewrite
c. Provide the exact error and ask Codex to trace the update path
d. Describe only what the page should look like

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will use Codex to build a shared form for adding and editing invoices. The form will manage invoice details and dynamic line items, calculate totals, persist data through server actions, and return users to the invoice list after a successful save. We will also identify an AI-generated input bug, correct it, and record the lesson in AGENTS.md so future Codex sessions follow the same rule.

Start from a clean repository

Run git status before beginning the feature. The command should report that there is nothing to commit and that the working tree is clean. This gives us a reliable baseline for reviewing every file that Codex creates or changes.

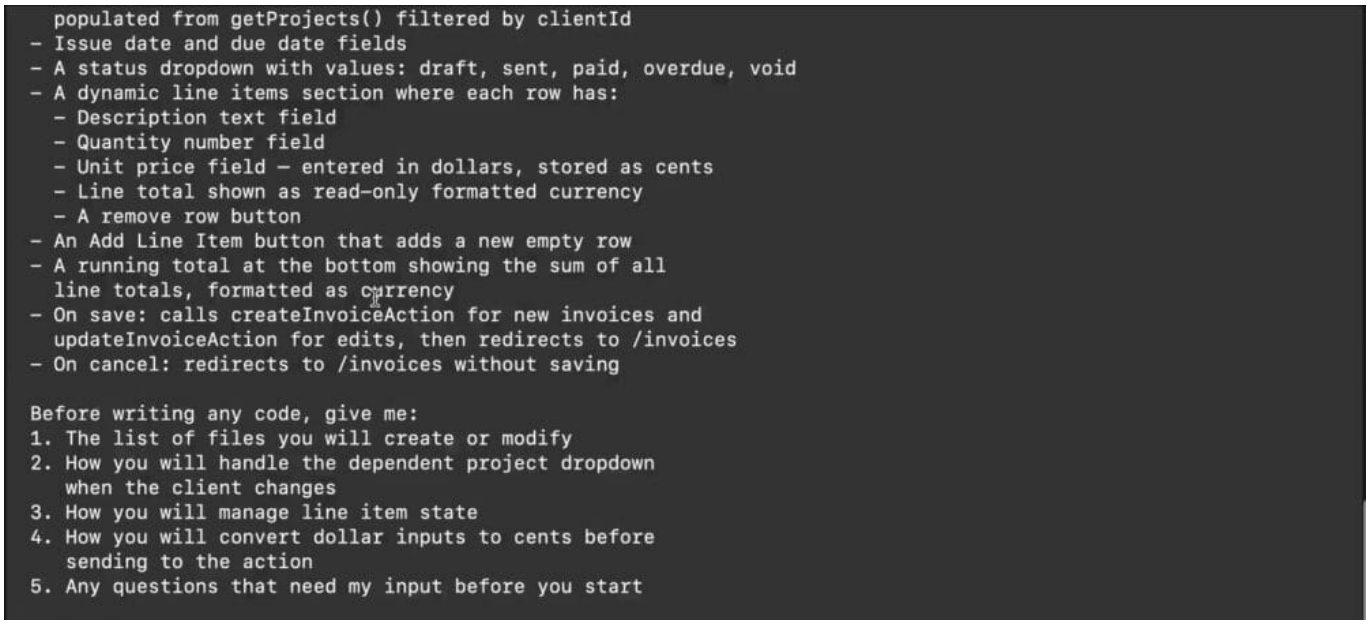
Launch Codex, then switch to plan mode. The add and edit workflow has several connected responsibilities, so planning helps us resolve route conventions, form state, data conversion, and server behavior before implementation begins.

Describe the invoice workflow

The planning prompt requests separate routes for creating and editing invoices, client and project selectors, invoice metadata, issue and due dates, a status selector, and dynamic line-item rows. Each row needs a description, quantity, unit price, calculated line total, and remove action. The form also needs an Add Line Item control, a running total, save behavior through the existing actions, and cancel behavior that returns to the invoice list.

The final visible portion of the planning prompt reads:

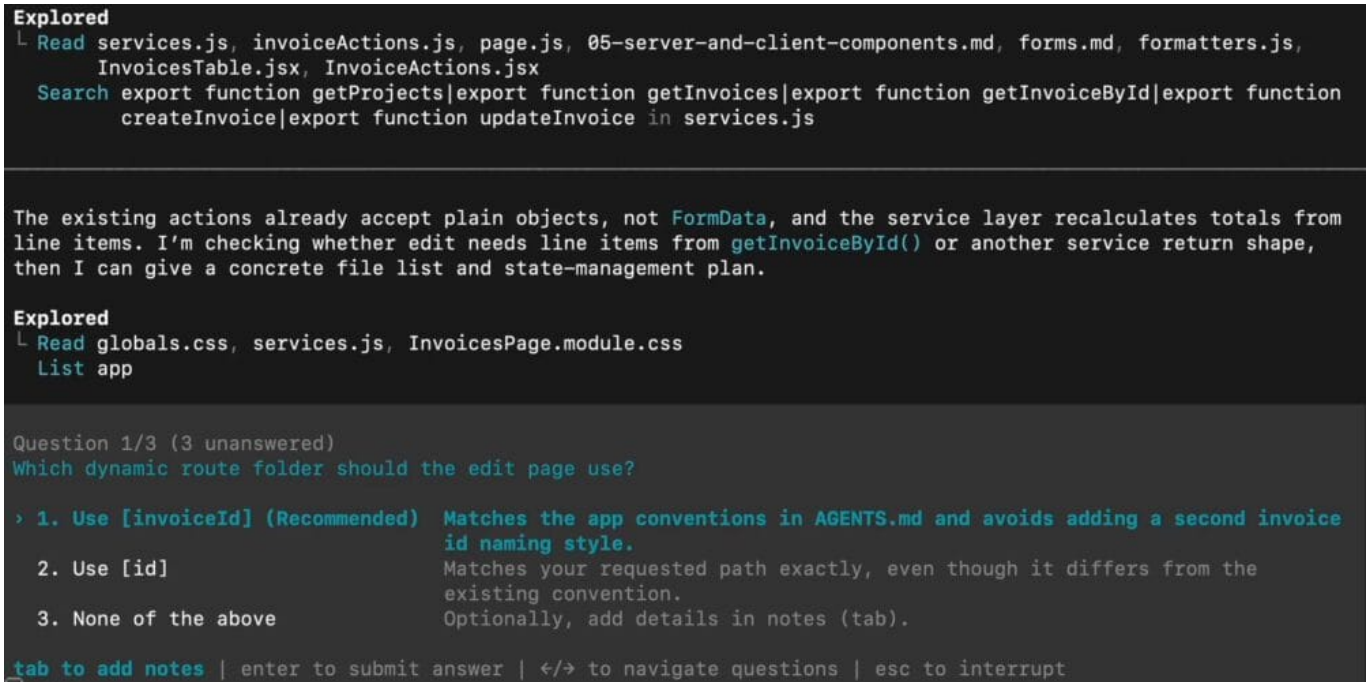
> Before writing any code, give me:
>
> 1. The list of files you will create or modify
> 2. How you will handle the dependent project dropdown when the client changes
> 3. How you will manage line item state
> 4. How you will convert dollar inputs to cents before sending to the action
> 5. Any questions that need my input before you start



© 2026 Zenva Pty Ltd. All rights reserved.

Clarify the requirements before implementation

Codex inspects the current routes, actions, service functions, styles, and relevant Next.js guidance before proposing changes. It then asks focused questions about the dynamic edit-route name, whether invoice numbers remain editable, and how the paid amount should be handled. Resolve these questions before approving the plan so the generated feature matches the project conventions.



Review the resulting plan carefully. In particular, verify how Codex will filter projects when the selected client changes, manage line-item state in the browser, and convert editable dollar values into integer cents before calling the server action. The plan should also account for validation, navigation, linting, and tests.

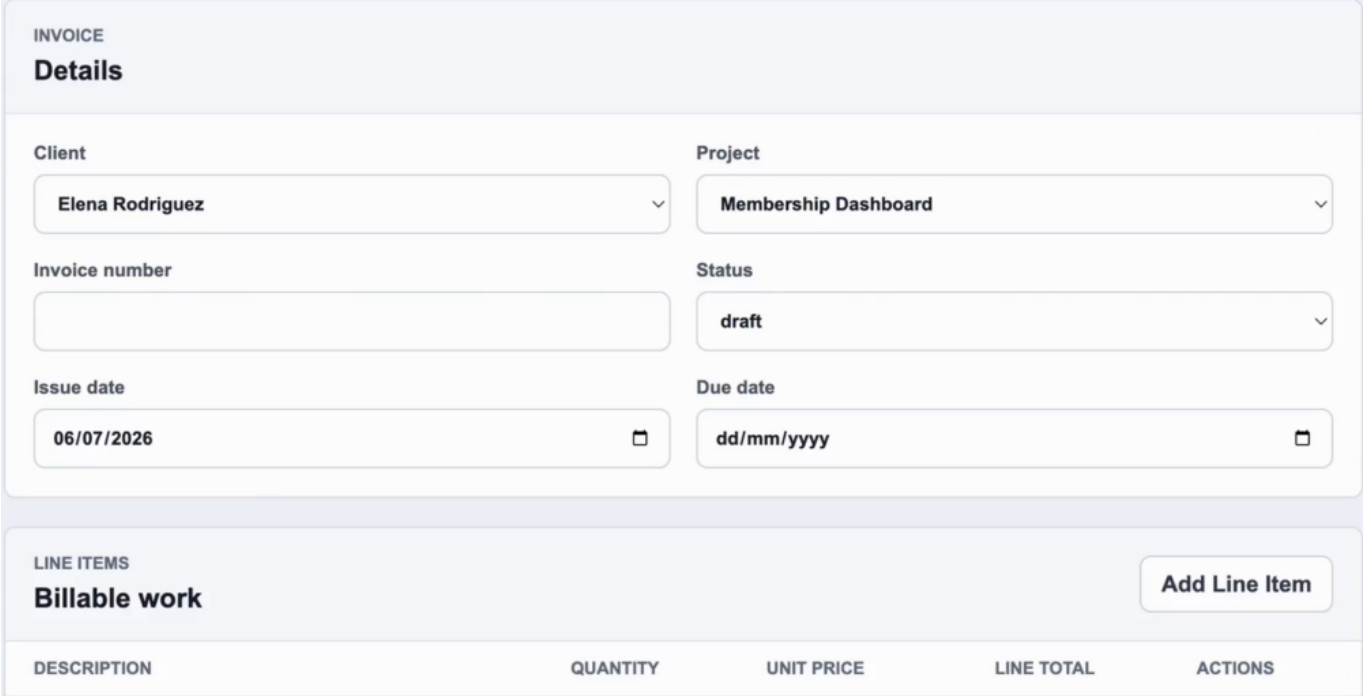
Generate the shared form and routes

After the plan is approved, Codex creates a shared client-side InvoiceForm component and its CSS module. It also creates server-rendered pages for new and existing invoices. These pages load clients, projects, and existing invoice data before passing them into the shared form.

The form manages the invoice number, client, project, status, issue date, due date, and line items. Users can add or remove line-item rows and edit each description, quantity, and unit price. Dollar values remain convenient strings while the user edits them, then the save payload converts unit prices to integer cents.

Codex also adds an Edit link to the invoice actions. Creating or updating an invoice uses the existing server actions, persists the data, and navigates back to /invoices. The initial implementation passes lint and the existing test suite.

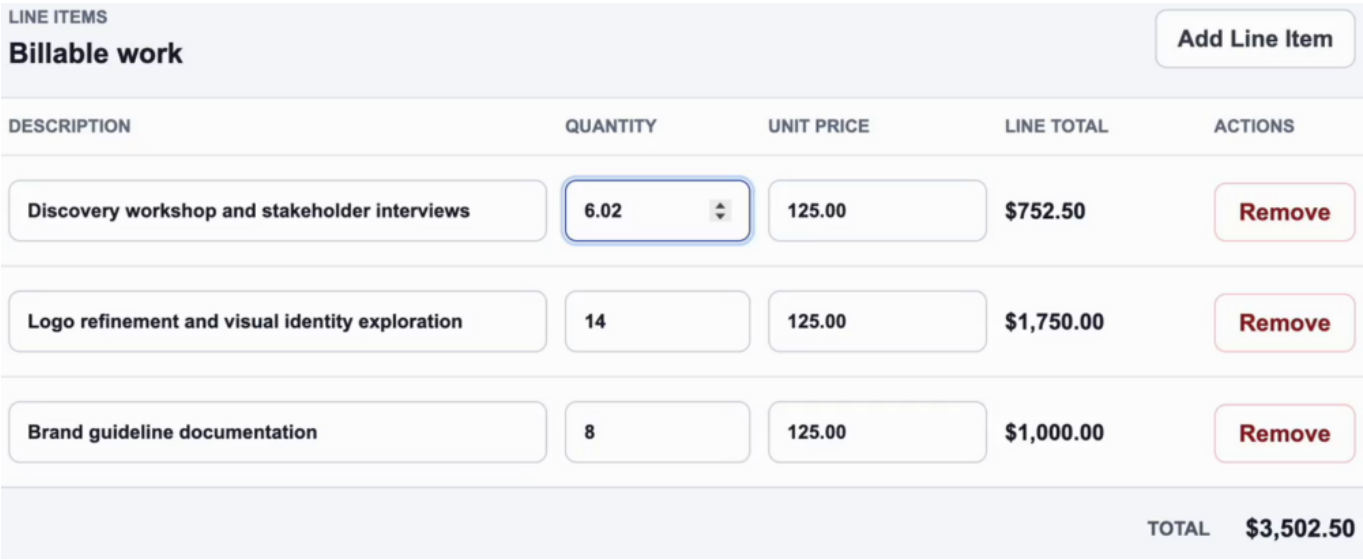
© 2026 Zenva Pty Ltd. All rights reserved.



Test the form through real interaction

Open the new-invoice route and interact with the quantity input. The arrow controls reveal that the quantity changes by 0.01 rather than by one whole unit. Return to the invoice ledger and edit an existing invoice to confirm the same behavior there. Because both pages use the shared form, the correction belongs in InvoiceForm.jsx.

The faulty step also affects the calculated values. A quantity of 6.02 at a unit price of 125.00 produces a line total of $752.50 and raises the invoice total to $3,502.50.



Correct the numeric input constraints

Use the following prompt to describe both the observed failure and the expected behavior precisely:

> The quantity field in the invoice line items form is stepping by 0.01 instead of 1 when using the arrow keys. Fix it so

© 2026 Zenva Pty Ltd. All rights reserved.

the quantity field always steps by whole numbers.

The correct behaviour:
– quantity input: step=”1″, min=”1″
– unit price input: verify it also has appropriate step and min values

Fix both inputs in InvoiceForm.jsx.

```
› The quantity field in the invoice line items form is stepping
  by 0.01 instead of 1 when using the arrow keys. Fix it so
  the quantity field always steps by whole numbers.

  The correct behaviour:
  - quantity input: step="1", min="1"
  - unit price input: verify it also has appropriate step
    and min values

  Fix both inputs in InvoiceForm.jsx.
```

Codex changes the quantity input to use a minimum of 1 and a step of 1. It configures the unit-price input separately because prices may include cents, using a numeric input with a minimum of 0 and a step of 0.01. It then runs npm run lint, which passes.

Turn the correction into a durable rule

Fixing the current component solves the immediate bug, but recording the constraint in the repository instructions reduces the chance that a later AI session will recreate it. Use the exact prompt shown in the lesson:

Update AGENTS.md to add this rule to the Critical Rules section:
– In InvoiceForm, quantity inputs must have step=”1″ and min=”1″ – never leave number inputs without explicit step and min attributes

```
  Quantity now uses min="1" and step="1". Unit price is now a numeric input with type="number", min="0", and
  step="0.01".

  Verification: npm run lint passed.

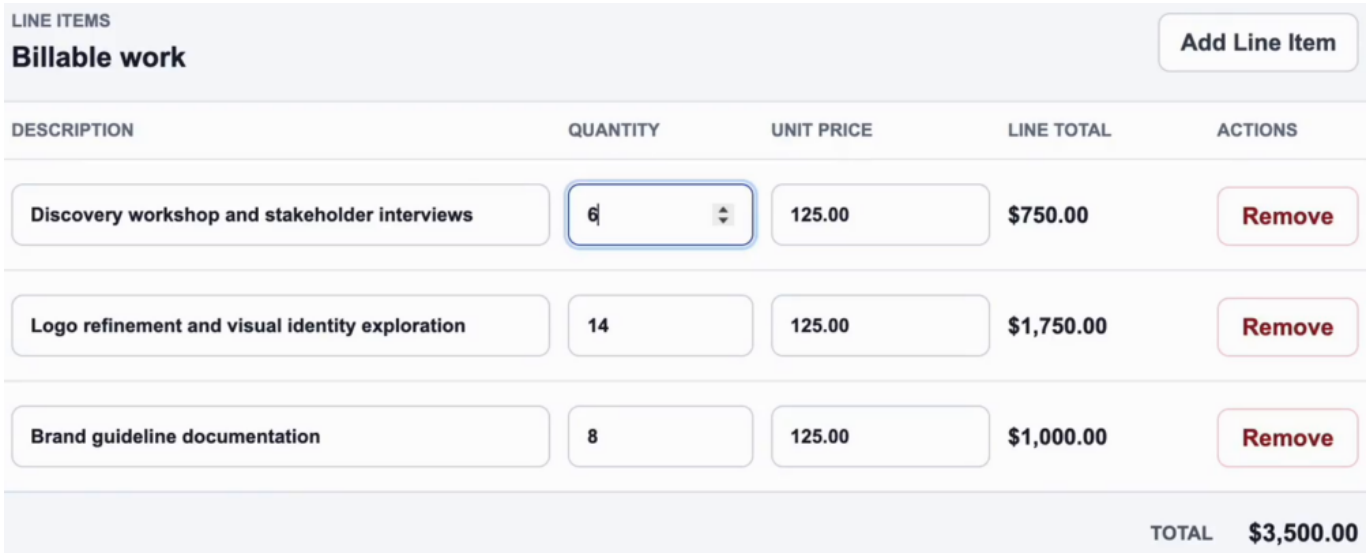
› Update AGENTS.md to add this rule to the Critical Rules section:
  - In InvoiceForm, quantity inputs must have step="1" and
    min="1" — never leave number inputs without explicit
    step and min attributes
```

Codex locates the Critical Rules section and adds the new requirement without changing the surrounding instructions. The final AGENTS.md rule requires every quantity input in InvoiceForm to define step="1" and min="1".

Verify creation, editing, and totals

© 2026 Zenva Pty Ltd. All rights reserved.

Return to the create-invoice form and test the quantity arrows again. The value should now change by whole units. Then edit an existing invoice and adjust a line-item quantity. Confirm that the line total and overall invoice total update immediately and remain mathematically consistent.



Also test adding and removing line-item rows, saving a new invoice, and updating an existing invoice. A successful submission should persist the invoice through the server action and return the user to the invoice ledger.

Review and commit the work

Review both tracked and untracked files before committing. The completed change includes the shared invoice form, its CSS module, the new and edit pages, the Edit action and styling, and the new rule in AGENTS.md.

The final commit prompt is:

> Commit all changes with the message “add Add and Edit Invoice form with dynamic line items”

After the commit succeeds, the repository contains a complete add-and-edit workflow with dependent selections, dynamic line items, dollar-to-cent conversion, live totals, server persistence, navigation, and a documented safeguard against fractional quantity steps.

Summary

We planned and generated a shared invoice form, connected it to create and edit routes, tested the feature through the browser, and corrected an AI-generated numeric-input mistake. We also converted that correction into a permanent repository rule, verified live total calculations and save behavior, reviewed the complete diff, and committed the finished workflow.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will build a polished, client-facing invoice detail page with Codex. We will plan the page before changing files, implement a document-style server component, review the generated work, correct the review findings, inspect the result in a browser, and connect the invoice ledger to the new detail route.

Prepare the repository

Run git status before starting and confirm that the working tree is clean. This gives us a reliable baseline and makes the final diff easier to review. Launch Codex from the project directory, then enter plan mode so we can evaluate the approach before any code is written.

Plan the invoice detail page

Enter the following prompt in Codex:

```
I want to build the Invoice Detail page at
src/app/invoices/[id]/page.js.

The page should be a Server Component that fetches the
invoice using getInvoiceById(id) from the service layer.

The layout should look like a real invoice document:

Header section:
- Left side: business branding – "Freelancer Invoice" as
  the title, and a placeholder tagline "Professional Services"
- Right side: invoice number prominently displayed,
  issue date, due date, and status as a StatusBadge

Client section:
- "Bill To:" label followed by the client's name, company,
  email, and address

Project section:
- Project name and a brief note showing which project
  this invoice covers

Line items table:
- Columns: Description, Quantity, Unit Price, Total
- One row per line item, all amounts formatted as currency
- A subtotal row at the bottom
- A total row in bold

Payment section:
- Total amount due prominently displayed
- Payment due date
- A note area showing the invoice notes if present

Action buttons (not printed):
- Edit button linking to /invoices/[id]/edit
- Back to Invoices link

This page will be printed by clients – the layout must be
clean, professional, and readable.
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
Use plain CSS modules. This is a Server Component – no
client-side interactivity needed on this page.

Before writing any code, give me:
1. The list of files you will create
2. The proposed layout structure in plain English
3. Any design decisions you are making that I should review
```

When Codex asks which dynamic route naming convention to use, choose the recommended invoice ID option. The resulting route uses src/app/invoices/[invoiceId]/page.js, which makes the parameter's purpose explicit.

Review the proposed files, information hierarchy, data access, formatting helpers, and print behavior. The invoice should prioritize its number, dates, status, client and project information, line items, totals, payment details, and notes. Navigation and action controls must remain outside the printable document.

```
- Main invoice document:
    - Header with "Freelancer Invoice" and "Professional Services" on the left.
    - Invoice number, issue date, due date, and StatusBadge on the right.
    - "Bill To" section with client name, company, email, and address.
    - Project section with project name and note that the invoice covers that project.
    - Line items table with description, quantity, unit price, and total.
    - Totals footer with subtotal and bold total.
    - Payment section with prominent total due, due date, and notes when present.

## Design Decisions

- Use CSS modules only, with print-specific rules via @media print.
- Style the invoice like a clean document: white page, constrained width, subtle borders, readable spa
  decorative cards.

- Hide navigation/action buttons when printing.
- Use existing StatusBadge, formatCurrency, formatDate, and formatInvoiceNumber.
- Preserve server-only data access through src/lib/services.js; no actions or client components needed

## Test Plan

- Run npm run lint.
```

Implement the approved plan

Once the plan is satisfactory, submit the implementation prompt shown in the lesson:

```
The plan looks good. Go ahead and implement it.
Use plain CSS modules – no Tailwind, no inline styles.
This is a Server Component – no 'use client' directive.
Make the layout clean and document-like.
Generous whitespace, clear typography hierarchy,
professional appearance.
```

Codex creates the detail page and InvoiceDetailPage.module.css. The server component retrieves the selected invoice, and the CSS module supplies the document hierarchy, responsive layout, and print-ready presentation. Codex then runs npm run lint and confirms that the initial implementation passes.

© 2026 Zenva Pty Ltd. All rights reserved.

Review the generated changes

Client-facing work needs more than a successful lint check. Run /review, select the uncommitted changes, and let Codex inspect the new page, styles, invoice lookup, and diff. Read the tool output carefully, including failed checks. During this review, one lint attempt uses an invalid option, so it should not be mistaken for a successful verification.

The review finds two concrete issues:

- The invoice detail page does not expose the required status transitions.
- A non-positive or non-numeric route parameter can cause an error instead of returning a 404 page.

```
- [P2] Add the missing invoice status actions — /Users/devisland/Desktop/learn_nextjs/nextjs_project/src/app/
  invoices/[invoiceId]/page.js:46-55
  On the invoice detail route, the action bar only renders Back and Edit, so users viewing /invoices/:invoiceId
  cannot mark draft invoices as sent, mark unpaid invoices as paid, or void invoices from the detail page as
  required. The status update action already supports these transitions, but this page does not expose them.

- [P2] Return 404 for invalid invoice route params — /Users/devisland/Desktop/learn_nextjs/nextjs_project/src/
  app/invoices/[invoiceId]/page.js:31-32
  When the route is requested with a non-positive or non-numeric invoiceId such as /invoices/abc,
  getInvoiceById() throws during ID normalization before the notFound() check can run, causing an error page
  instead of a 404. Validate the param or catch the service error before calling notFound() for invalid IDs.
```

Fix the review findings

Use the following targeted prompt to resolve both issues:

```
Fix these two issues flagged by /review:

1. Add invoice status action buttons to the detail page.
   On src/app/invoices/[invoiceId]/page.js, add action
   buttons that allow:
   - Marking a draft invoice as sent
   - Marking a sent or overdue invoice as paid
   - Voiding an invoice that is not already void or paid
   Use updateInvoiceStatusAction from invoiceActions.js.
   Only show each button when the current status makes
   the action relevant.

2. Return a 404 for invalid invoice route params.
   Before calling getInvoiceById(), validate that the
   invoiceId param is a positive integer. If it is not,
   call notFound() immediately rather than letting the
   service throw an error.
```

Codex adds the positive-integer guard before the invoice lookup. It also adds server-action forms for the permitted status changes, revalidates the invoice list and current detail route, updates the action bar styles, and keeps all controls hidden in print output. A fresh npm run lint confirms that the corrections pass verification.

© 2026 Zenva Pty Ltd. All rights reserved.

```
• The CSS now covers links and submit buttons consistently. I'm running lint and then I'll do a short diff/status
  check.

• Ran npm run lint
  └ (no output)

──────────────────────────────────────────────────────────────────────────────────────────────────────────────────

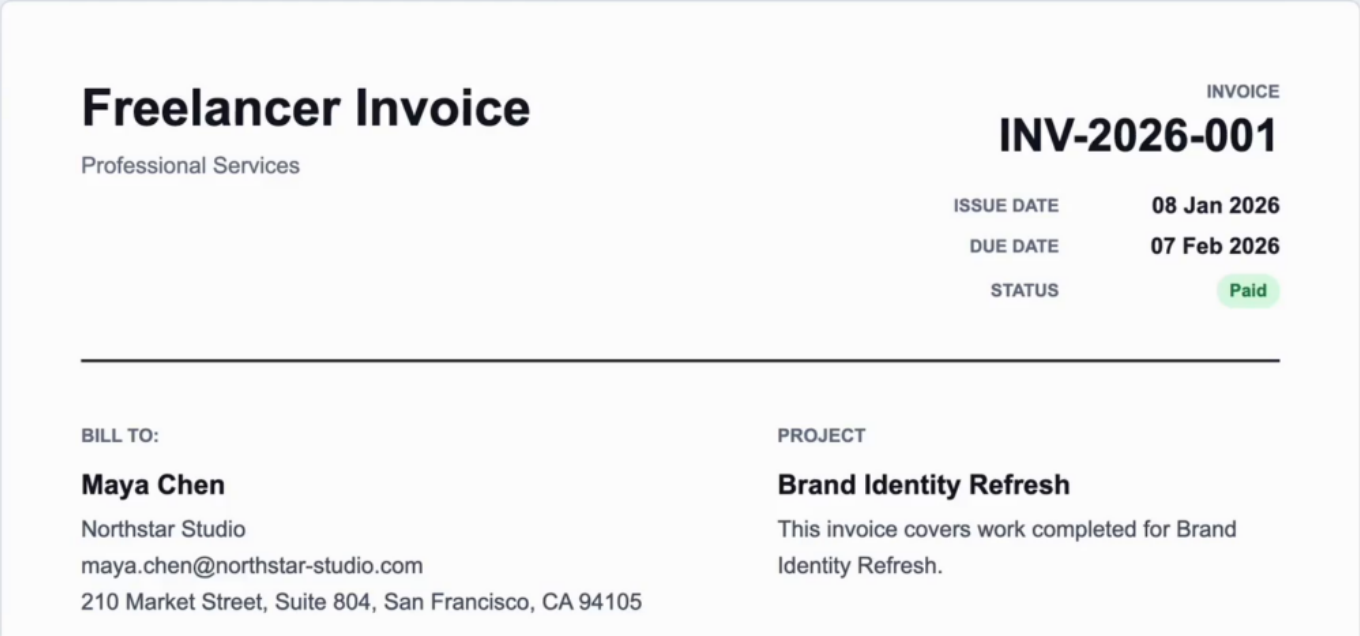
• Lint passed. I'm reviewing the final changed files to make sure the status conditions match the review request
  exactly.

• Explored
  └ Read page.js, InvoiceDetailPage.module.css

• Ran git status --short
  └ ?? src/app/invoices/[invoiceId]/InvoiceDetailPage.module.css
    ?? src/app/invoices/[invoiceId]/page.js
```

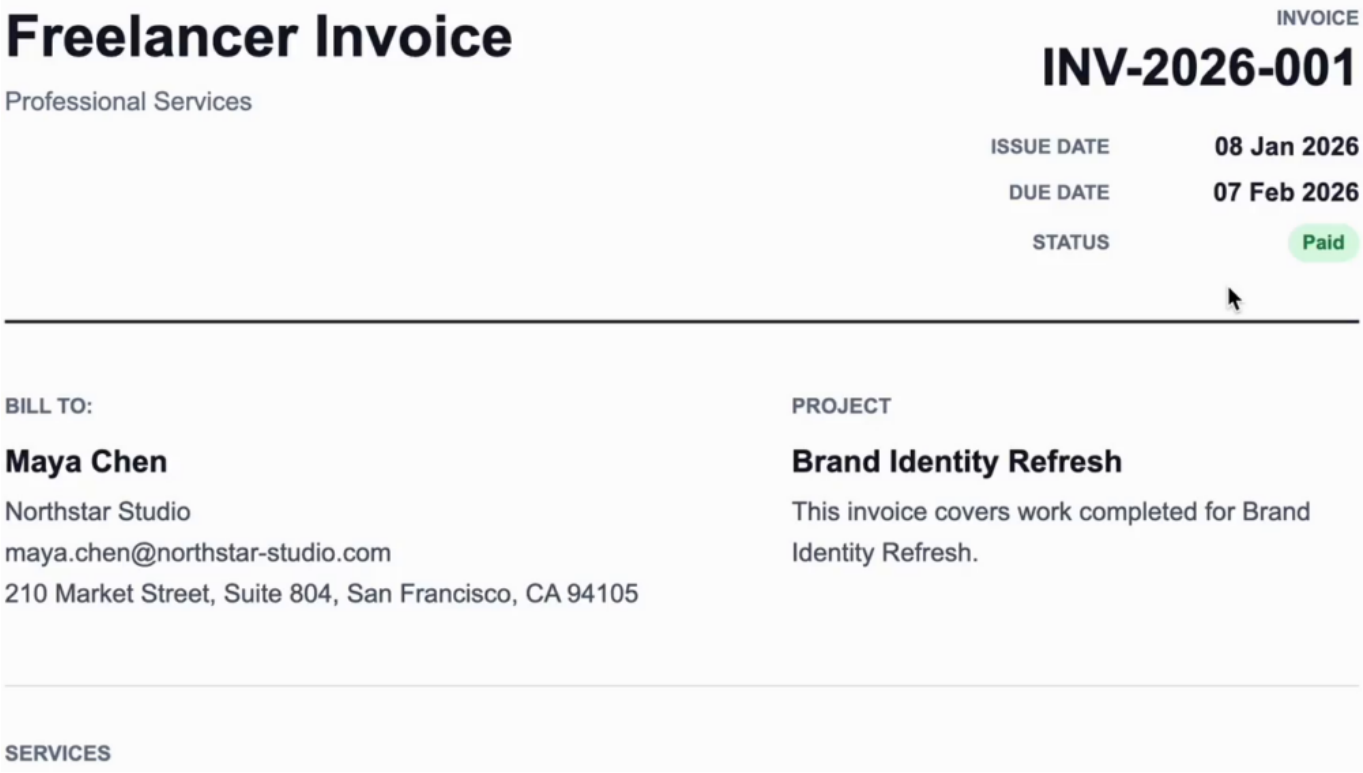
Inspect the result as a client

Open one of the seeded invoices in the browser and review it as a client would. Confirm that the invoice number is obvious, the branding is clear, the total is prominent, the due date is easy to find, and the line items are readable.



Scroll through the complete document and inspect the type sizes, spacing, alignment, itemized charges, totals, payment information, and notes. Browser verification can reveal layout problems that static review and linting cannot detect. If anything looks unbalanced or incomplete, describe the specific problem to Codex and request a focused correction.

© 2026 Zenva Pty Ltd. All rights reserved.



Add navigation from the invoice ledger

The invoice ledger already contains Edit and Delete actions, but it still needs a direct link to the client-facing document. Enter this prompt:

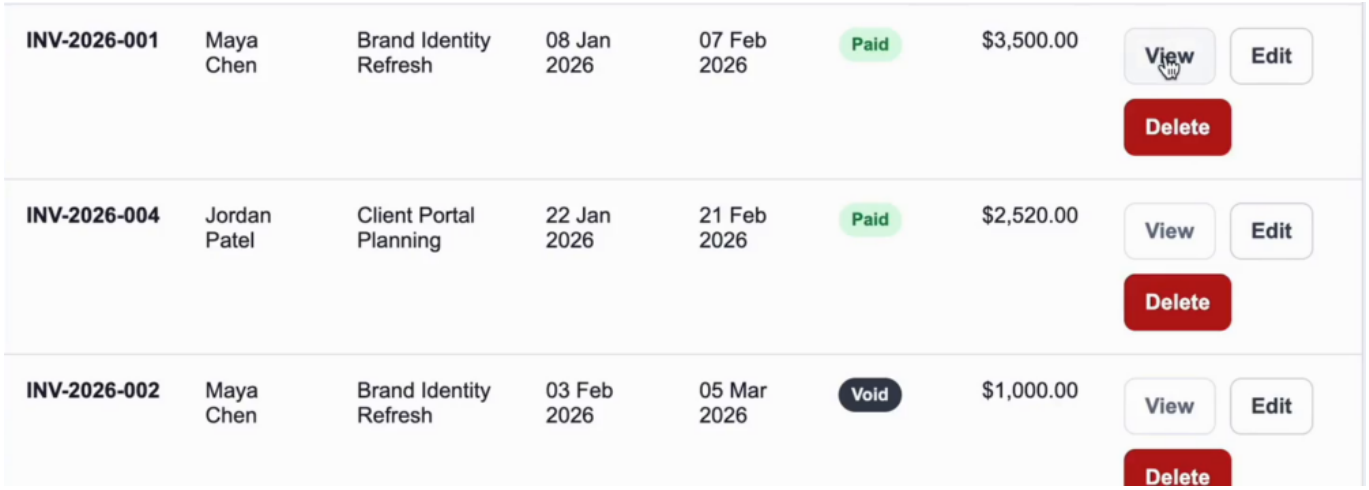
```
Add a View button to each invoice row in the invoices list page.

The View button should:
- Link to /invoices/[id] for each invoice
- Appear alongside the existing Edit and Delete buttons
- Be styled as a secondary button – less prominent than
  Edit and Delete
- Use plain CSS modules for styling

Update whichever component renders the invoice row action
buttons – likely InvoiceActions.jsx.
```

Codex updates InvoiceActions.jsx and its CSS module, then runs linting again. Return to the ledger and confirm that View appears with the existing actions.

© 2026 Zenva Pty Ltd. All rights reserved.



Select View and confirm that it opens the correct /invoices/[invoiceId] page. This verifies the connection between the ledger and the new route.

Review and commit the final changes

Run git status and git diff to inspect every created or modified file. Confirm that the changes are focused on the invoice detail page, its styles, and invoice-row navigation.

Once the diff is clean, stage the changes and commit them with the message **Add Invoice Detail Page**. Review and approve the git add and git commit commands when Codex requests permission.

Summary

We planned and implemented a professional invoice detail page with client information, itemized charges, totals, status actions, route validation, responsive styles, and print behavior. We used Codex review to find missing functionality, corrected the findings with a precise prompt, verified the visual result in the browser, added View navigation from the ledger, and confirmed that the final diff remained contained.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will create a dedicated print stylesheet for the invoice detail page. The goal is to turn the browser view into a professional paper or PDF document while removing navigation, headers, action buttons, and other controls that only belong on screen.

Define the print requirements

Start by running git status and confirming that the working tree is clean. This gives us a reliable baseline before Codex changes the project. Launch Codex, then provide the complete requirements:

```
Add print stylesheet support to the invoice detail page
at src/app/invoices/[id]/page.js.

The print stylesheet should:

1. Hide all navigation elements when printing:
   - The sidebar navigation
   - The header
   - The Edit button and Back to Invoices link
   - Any action buttons

2. Adjust the layout for A4 paper:
   - Remove the main content margin and padding used
     for the screen layout
   - Set the invoice content to use the full page width
   - Add appropriate page margins using @page rule:
     margin of 15mm on all sides

3. Ensure correct rendering:
   - Force black text on white background for all elements
   - Ensure the line items table renders with visible borders
   - Prevent line items from breaking across page boundaries
     where possible using page-break-inside: avoid
   - Ensure the total section stays together on the same page

4. Keep the invoice document itself unchanged — only
   hide the UI chrome and adjust spacing
```

Codex inspects the global layout and invoice-specific styles, then limits the implementation to CSS. This keeps the invoice markup unchanged and reduces the risk of affecting the normal application view.

Remove application chrome from print output

The shared layout stylesheet adds a print media block. It changes the grid-based shell to a simple block layout, hides the sidebar and header, removes the main content padding, and uses a white background:

```
@media print {
  .shell {
    display: block;
    min-height: auto;
    background: #ffffff;
  }
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  .shell > aside,
  .content > header {
    display: none;
  }

  .content {
    display: block;
  }

  .main {
    padding: 0;
  }
}
```

Because these rules are inside @media print, the navigation and spacing remain available in the normal browser interface.

Format the invoice for A4 paper

The invoice-specific stylesheet defines 15 mm page margins and hides the invoice action controls. It also expands the document to the full printable width, removes screen-only decoration, and forces reliable black-on-white output:

```
@media print {
  @page {
    margin: 15mm;
  }

  .actions {
    display: none;
  }

  .page {
    display: block;
  }

  .document {
    width: 100%;
    max-width: none;
    border: 0;
    border-radius: 0;
    background: #ffffff;
    color: #000000;
    box-shadow: none;
    padding: 0;
  }

  .document *,
  .document *::before,
  .document *::after {
    background: #ffffff;
    color: #000000;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    box-shadow: none;
  }
}
```

The table receives visible black borders and comfortable cell padding. Break rules help keep each row and the payment section together when the invoice spans multiple pages:

```
.table {
  border: 1px solid #000000;
  page-break-inside: auto;
}

.table th,
.table td {
  border: 1px solid #000000;
  padding: 10px;
}

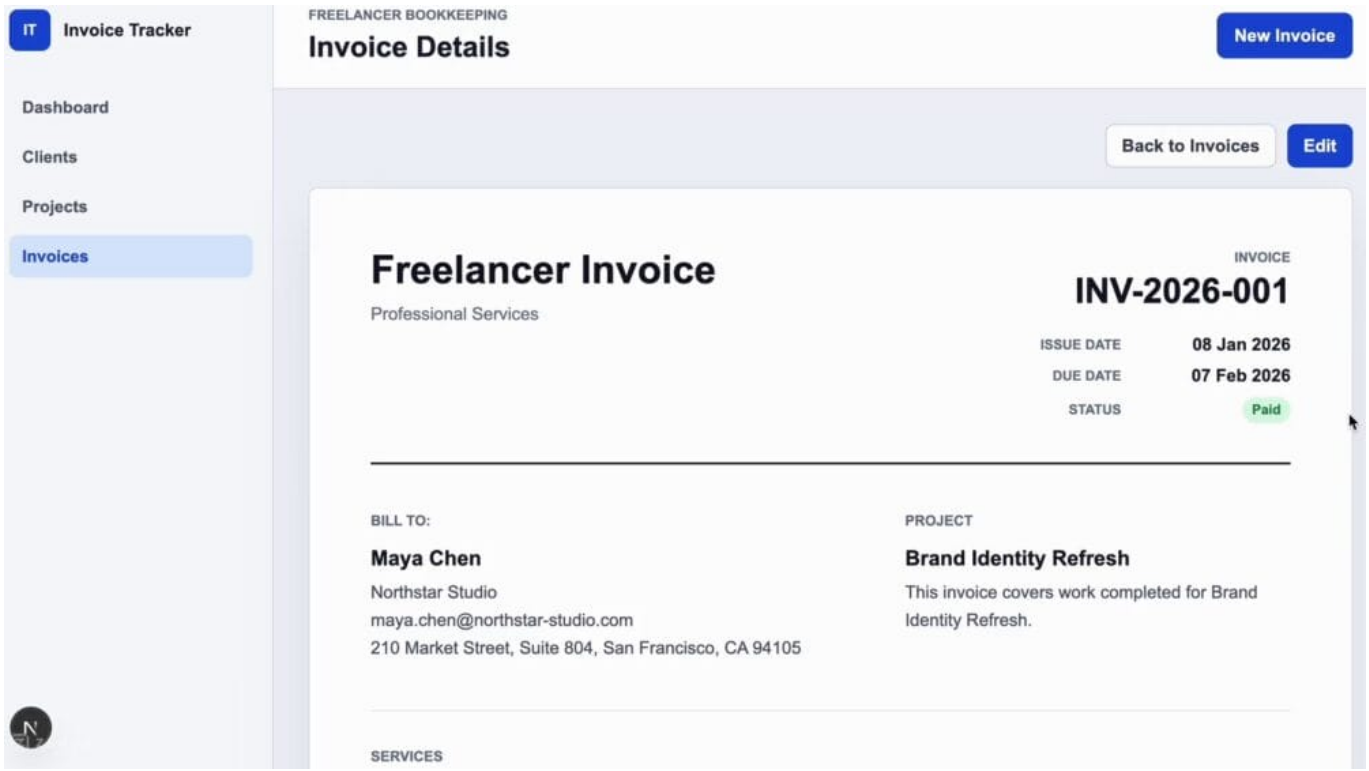
.table tr,
.table tbody,
.table tfoot,
.payment {
  break-inside: avoid;
  page-break-inside: avoid;
}
```

After applying the CSS, run npm run lint. A passing lint check confirms that the stylesheets contain no detected syntax or module issues, but the final physical layout still requires visual testing.

Compare the screen and print layouts

Open an invoice detail page in the browser. The screen layout should still include the sidebar, header, navigation links, and invoice action controls because these elements remain useful while working in the application.

© 2026 Zenva Pty Ltd. All rights reserved.



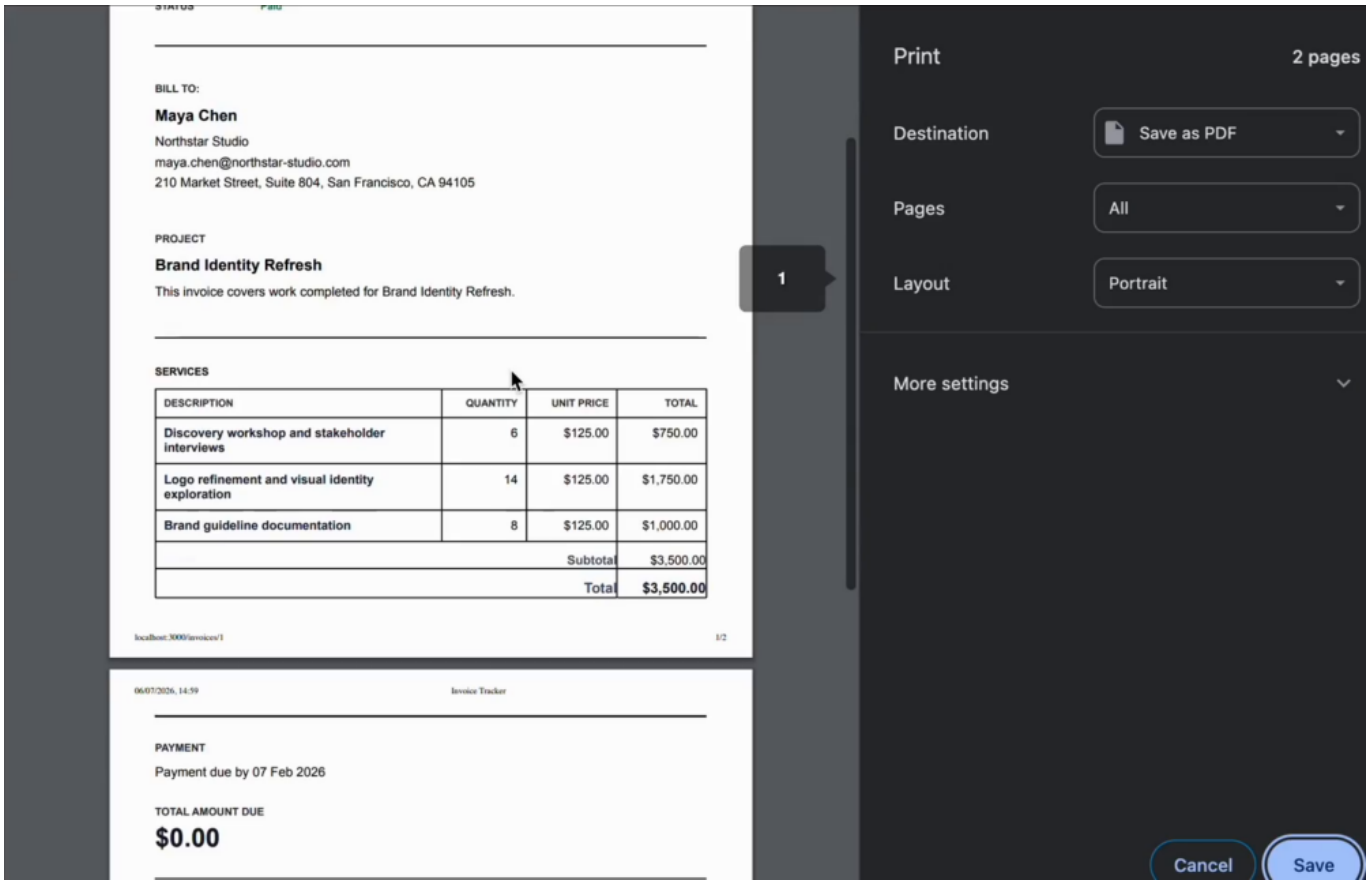
Open the browser print preview with Command-P on macOS or Control-P on Windows. The print preview is the most reliable way to evaluate CSS that controls paper and PDF output.

Verify the print preview

Check that the preview meets every requirement:

- The sidebar, header, navigation links, and action buttons are absent.
- The invoice uses the available printable width with 15 mm A4 margins.
- Text is black on a white background.
- The line-item table has clear borders.
- Rows, totals, and the payment section avoid awkward page splits where possible.

© 2026 Zenva Pty Ltd. All rights reserved.



The destination can be set to Save as PDF, so the same print stylesheet supports both physical documents and digital exports. If content is clipped, navigation remains visible, or a section breaks poorly, return to the print CSS and make a focused adjustment to the margins, width, or break behavior.

Review and commit the CSS changes

Ask Codex to summarize every modified or created file, then review the print-specific blocks in src/app/layout.module.css and src/app/invoices/[invoiceId]/InvoiceDetailPage.module.css. Confirm that the work is limited to the intended CSS changes.

Stage and commit the completed styles. If Codex requests permission to write to the repository's .git directory, approve the expected git add and git commit operations. Run git status afterward and confirm that the working tree is clean.

Summary

We created print-only rules that remove application chrome, apply A4 margins, preserve readable colors and table borders, and control page breaks. We then verified the result in the browser print preview, reviewed the contained CSS changes, and committed the finished print stylesheet.

© 2026 Zenva Pty Ltd. All rights reserved.

The Complete Codex Development Course - Review Forms and Client-Facing Output

1. The same generated form bug appears on both create and edit pages. What is the best workflow?

a. Fix only the page where it was first noticed
b. Fix the shared component, verify both pages, and add a durable rule
c. Create a separate form for each page
d. Leave the bug for a later review

2. The invoice detail page passes linting. What is still needed before treating it as client-ready?

a. Inspect the rendered invoice in the browser
b. Replace all CSS with inline styles
c. Remove route validation
d. Hide the invoice total

3. Automated checks pass, but print preview shows visual problems. How should you continue?

a. Accept the result because lint passed
b. Ask Codex to rewrite the entire page
c. Describe the exact issues, request a focused fix, and recheck preview
d. Remove print support from the feature

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will build an interactive business dashboard for our invoice tracker. The finished page will combine four financial summary cards, a six-month revenue chart, and an overdue-invoices alert list. Along the way, we will use a research sub-agent, extend and test the server-side service layer, preserve the boundary between Server and Client Components, verify the route, and commit the completed feature.

Start from a clean repository

Before making changes, run git status and confirm that the working tree is clean. This gives us a reliable baseline and makes the final diff easier to review. Once the repository is ready, launch Codex from the project directory.

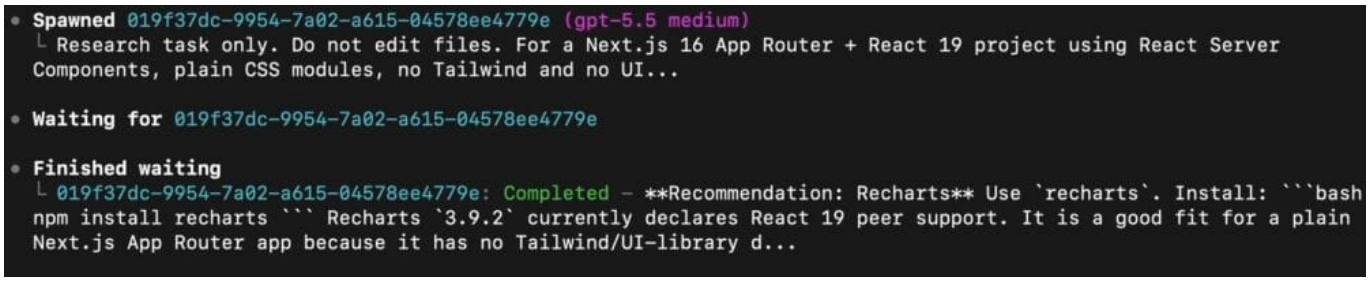
Delegate chart-library research

Choosing a charting library is a focused research task that can consume valuable context in the main Codex session. We can delegate it to a sub-agent before writing any dashboard code. The exact prompt used in the lesson is:

> Use a subagent to research the best charting approach for a Next.js 16 App Router project. Specifically:
>
> 1. Which charting libraries work well with Next.js App Router and React Server Components – and which ones have known compatibility issues
> 2. Whether the recommended library requires a Client Component wrapper or can be used in Server Components
> 3. How to install and configure the recommended library with minimal setup
> 4. What kind of chart is most appropriate for showing monthly revenue trends as a bar or line chart
>
> The project uses plain CSS modules and no Tailwind.
> No external UI libraries are installed.
>
> The subagent should report back a concise recommendation with the library name, install command, and any important usage notes. Do not write any code yet.

The first delegation attempt uses a shape that is too broad for the environment, so Codex retries with a narrower request and successfully starts the sub-agent. The sub-agent checks current package versions, peer dependencies, React support, and the practical requirements of several charting libraries.

The research recommends Recharts. It fits naturally into a React project, supports the required React version, and can be isolated in a Client Component while the page remains server-rendered.



© 2026 Zenva Pty Ltd. All rights reserved.

Install the package from a regular terminal with npm install recharts. Running the installation outside the Codex session avoids unnecessary sandbox restrictions during package installation.

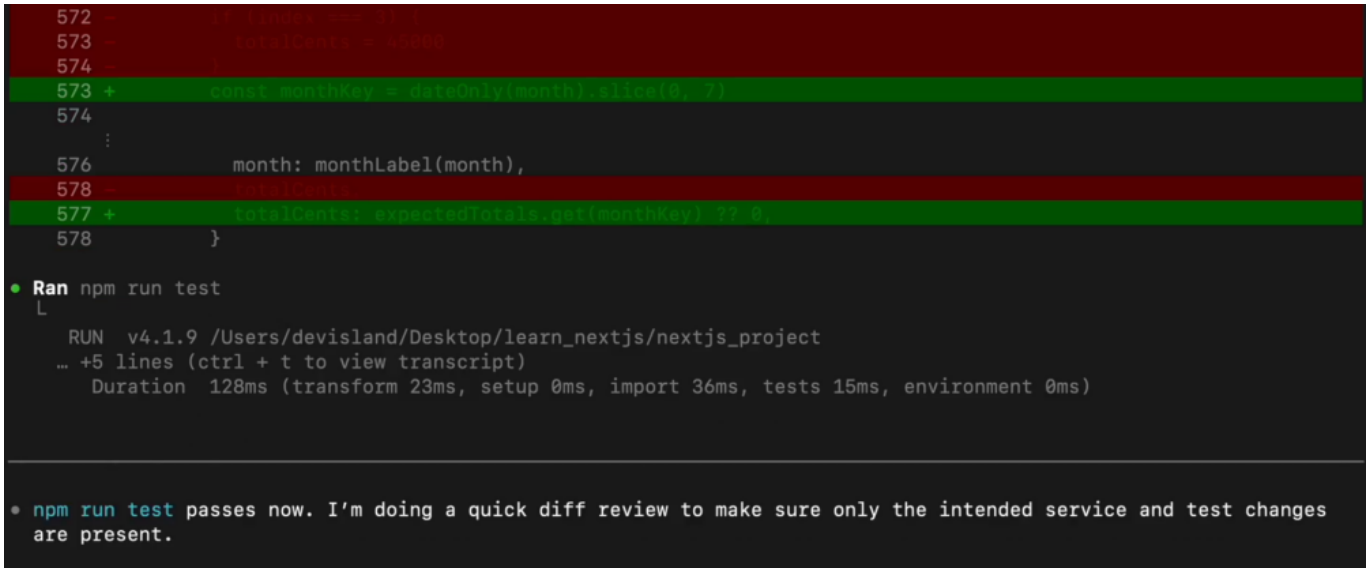
Build the dashboard data services first

The existing service layer does not yet provide the aggregates needed by the dashboard. Before building the interface, direct Codex to add focused, read-only service functions for:

- The total outstanding balance, revenue paid during the current month, overdue invoice count, and highest-revenue client.
- Monthly revenue totals for the latest six months, including zero-filled months.
- Overdue invoices ordered by due date for follow-up.

Codex first inspects the existing service and test files so the new work follows the repository's SQL patterns and fixture conventions. It generates month boundaries in JavaScript and passes them into SQLite queries. This avoids dependence on SQLite local-time behavior and keeps the tests deterministic.

The tests cover outstanding balances, paid-this-month filtering, top-client revenue, six-month revenue buckets, and overdue-invoice ordering. The first test run exposes an incorrect expectation: a baseline fixture invoice legitimately falls inside the current six-month window. Codex updates the expected total to include that invoice and reruns the suite successfully.



After the tests pass, Codex reviews the diff to confirm that the service and test changes are focused and do not alter unrelated behavior.

Give Codex the dashboard specification

With the data layer verified, we can describe the page and its component boundary precisely. The exact dashboard prompt used in the lesson is:

> Build the dashboard at src/app/page.js.
>
> The page is a Server Component that calls getDashboardSummary(), getMonthlyRevenue(), and getOverdueInvoices() from the service layer.
>
> Build these sections:

© 2026 Zenva Pty Ltd. All rights reserved.

1. Summary cards row – four cards side by side:
    - Total Outstanding: totalOutstandingCents formatted as currency
    - Paid This Month: totalPaidThisMonthCents formatted as currency
    - Overdue Invoices: overdueCount as a number, red if > 0
    - Top Client: topClientByRevenue as a name
2. Revenue trend chart – a bar chart showing monthly revenue for the last 6 months using the charting library the subagent recommended. This must be a Client Component because charting libraries require browser APIs. Pass the getMonthlyRevenue() data as props from the Server Component page.
3. Overdue invoices alert list – a list of overdue invoices showing client name, invoice number, due date, and amount due. Each row links to the invoice detail page. If there are no overdue invoices, show a positive empty state message.

Use plain CSS modules for all styling.
Name the chart component src/app/RevenueChart.jsx.

Separate server and client responsibilities

The dashboard page remains a Server Component. It calls the three service functions directly and renders the summary cards and overdue-invoice list. This keeps database access on the server.

The Recharts wrapper is a Client Component because the charting library depends on browser APIs. The Server Component passes serializable monthly revenue data into that small client boundary as props. Keeping the boundary narrow preserves server rendering for the rest of the page.

Before creating the files, Codex reads the local Next.js App Router and use client guidance, then checks the existing app structure, CSS modules, shared layout styles, and formatter helpers. It creates the server-rendered dashboard page, the RevenueChart.jsx client component, and a dashboard CSS module.

Verify the implementation

Codex runs both the test suite and lint checks after creating the dashboard. It also checks the live route. A second development server cannot start because one is already running on port 3000, so Codex verifies the existing server instead. The dashboard route returns a successful response.

© 2026 Zenva Pty Ltd. All rights reserved.

```
There is already a Next dev server running for this project on port 3000, so the se
checking the existing server instead of stopping it.

Ran curl -I http://localhost:3000
└  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                  Dload  Upload   Total   Spent    Left  Speed
0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
  0      0 --:--:-- --:--:-- --:--:--     0
  curl: (7) Failed to connect to localhost port 3000 after 0 ms: Couldn't connect t

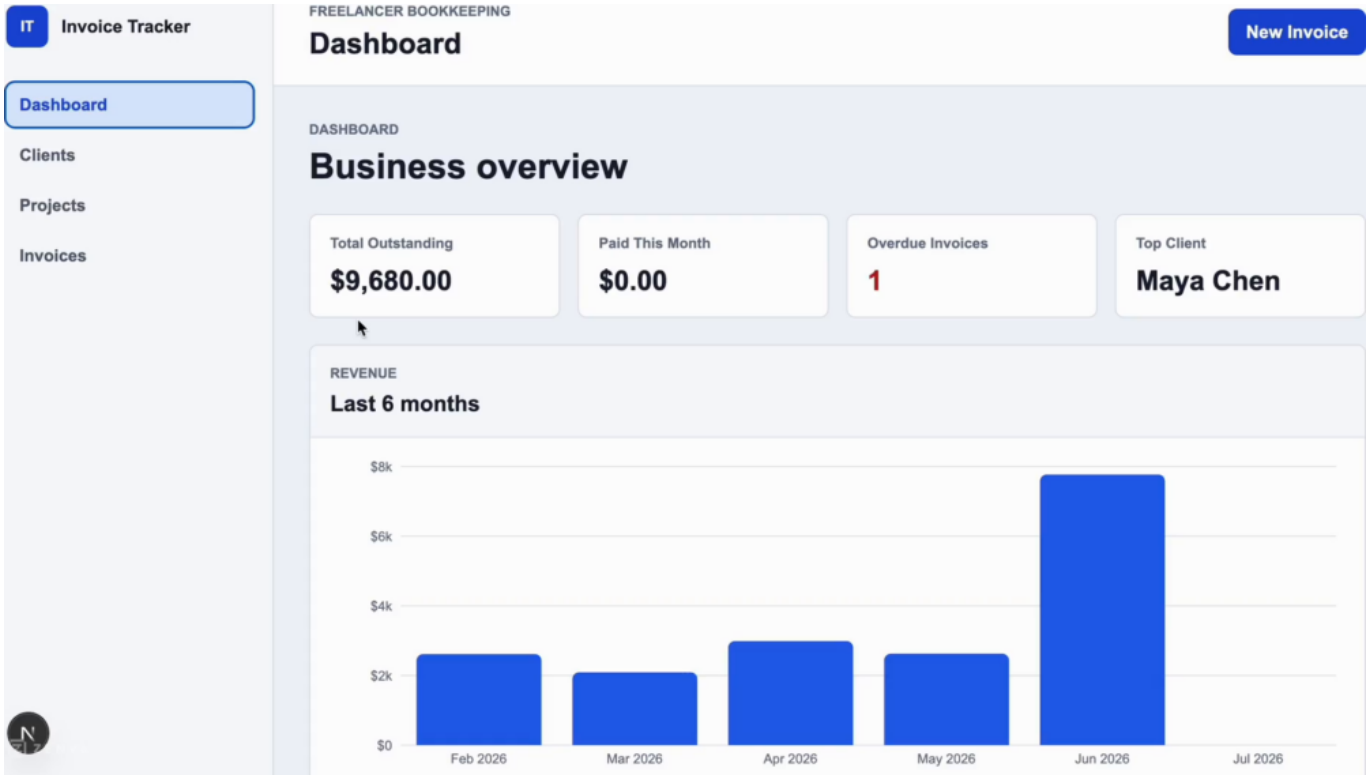
You approved codex to run curl -I http://localhost:3000 this time

Ran curl -I http://localhost:3000
└  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                  Dload  Upload   Total   Spent    Left  Speed
  … +10 lines (ctrl + t to view transcript)
  Keep-Alive: timeout=5
```

This verification sequence checks the data layer, code quality, and runtime route before we review the interface.

Review the completed dashboard

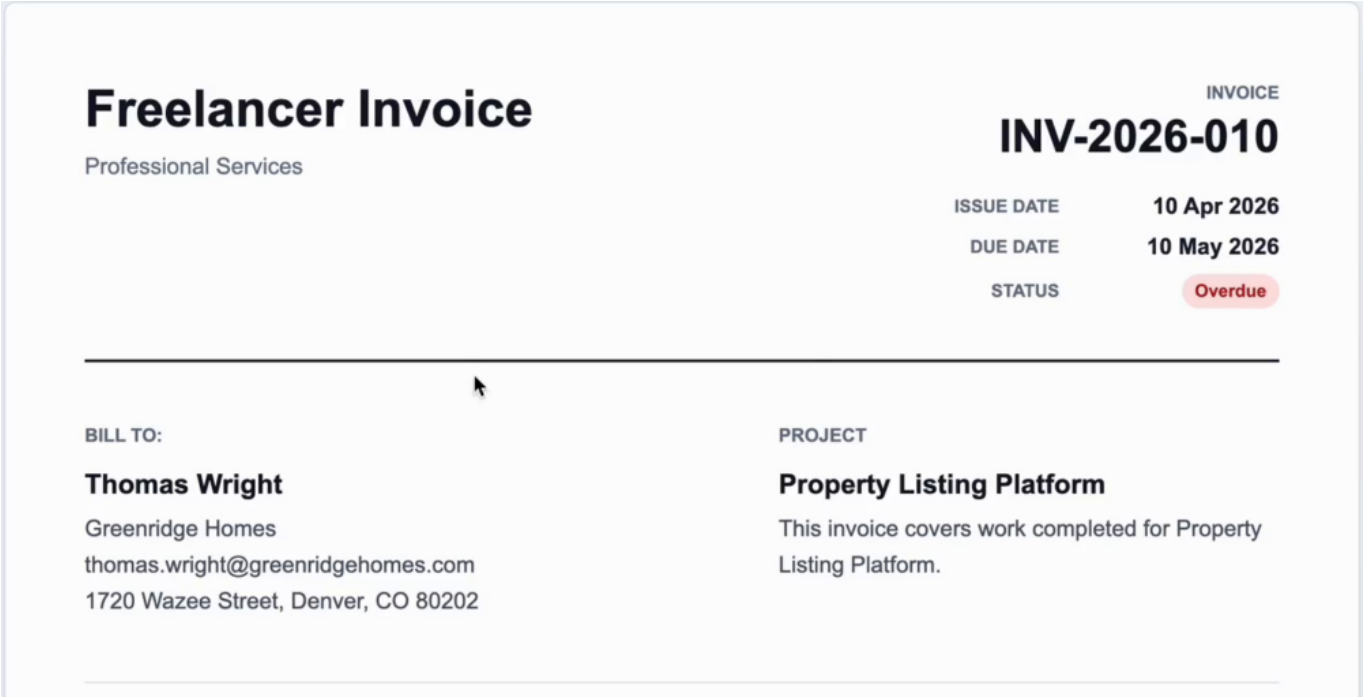
The completed dashboard presents four cards at the top: total outstanding, paid this month, overdue invoices, and top client. The overdue count appears in red when action is required. Beneath the cards, the Recharts bar chart visualizes revenue across the latest six months while retaining months with no revenue.



The overdue-invoices panel turns the dashboard into an actionable workspace. Each row includes the relevant client, invoice number, due date, and outstanding amount, and links directly to the invoice

© 2026 Zenva Pty Ltd. All rights reserved.

detail page.

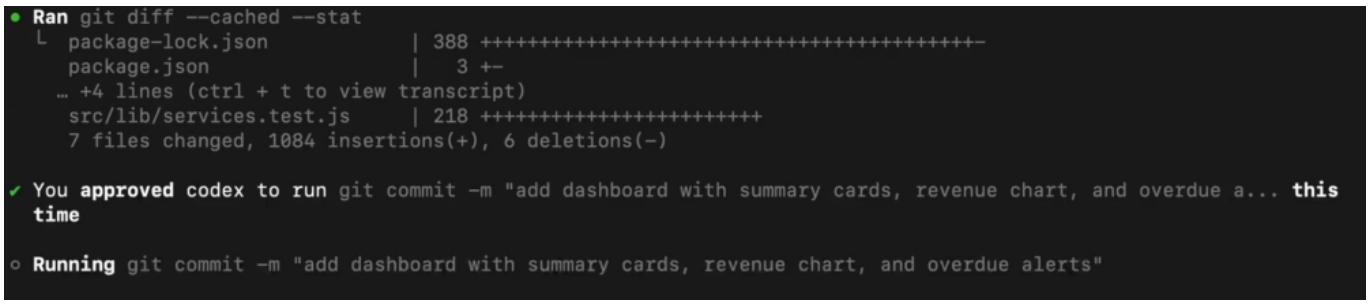


Following one of these links confirms that the invoice details are connected correctly. We can review the invoice, its overdue status, client information, project, dates, and available actions, then navigate back to the business overview.

Review and commit the feature

Run git status and git diff to review the complete scope of the work. The expected changes include the Recharts dependency, dashboard service functions and tests, the main dashboard page, the client chart component, and the CSS module.

Once the diff matches the intended feature, stage the files and commit them with a descriptive message. Codex then runs a final status check to confirm that the working directory is clean.



Summary

We used delegated research to select Recharts, built and tested the required server-side aggregates before creating the interface, and kept browser-only chart behavior inside a focused Client Component. We then verified the tests, lint checks, and dashboard route, reviewed the finished financial overview and invoice links, and committed the completed feature.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will add CSV export to the invoices page. Users will be able to download all invoices or preserve the page's active filters in the exported file.

Prepare Codex and define the task

Run git status and confirm that the working tree is clean. Launch Codex from the project directory with codex, then submit the following prompt:

```
Add a CSV export feature to the invoices page.

Part 1 – API route for CSV generation:
Create src/app/api/invoices/export/route.js that:
- Accepts the same optional query parameters as the
  invoices page: status, clientId, projectId
- Calls getInvoices() with those filters from the
  service layer
- Generates a CSV string with these columns in this order:
  Invoice Number, Client, Project, Issue Date, Due Date,
  Status, Total
- Total should be formatted as a dollar amount e.g. 1250.00
  not as cents
- Dates should be formatted as YYYY-MM-DD
- Returns the CSV as a file download with these headers:
  Content-Type: text/csv
  Content-Disposition: attachment; filename="invoices.csv"

Part 2 – Export button on the invoices page:
Add an Export CSV button to src/app/invoices/page.js
or its relevant Client Component that:
- Sits alongside the existing filter controls
- When clicked, constructs the correct URL with the
  current active filters as query parameters
- Triggers a file download by navigating to
  /api/invoices/export with those params
- No libraries needed – use a plain anchor tag or
  window.location approach

Use plain CSS modules for the button styling.
Match the existing button style in the invoices page.
```

A traditional API route is appropriate because a file download needs explicit response headers, including its content type and attachment filename. Codex first inspects the invoices route, service API, and local Next.js route-handler documentation so the implementation follows the project's conventions.

Implement and verify the feature

Codex creates src/app/api/invoices/export/route.js. The route reads status, clientId, and projectId from the query string, passes those filters to getInvoices(), serializes the matching records, and returns the CSV with Content-Type and Content-Disposition headers.

The invoices page reads the same URL filters, while InvoicesTable.jsx builds the export URL from the active filter state. Codex also adds the button styling and removes a duplicate function declaration

© 2026 Zenva Pty Ltd. All rights reserved.

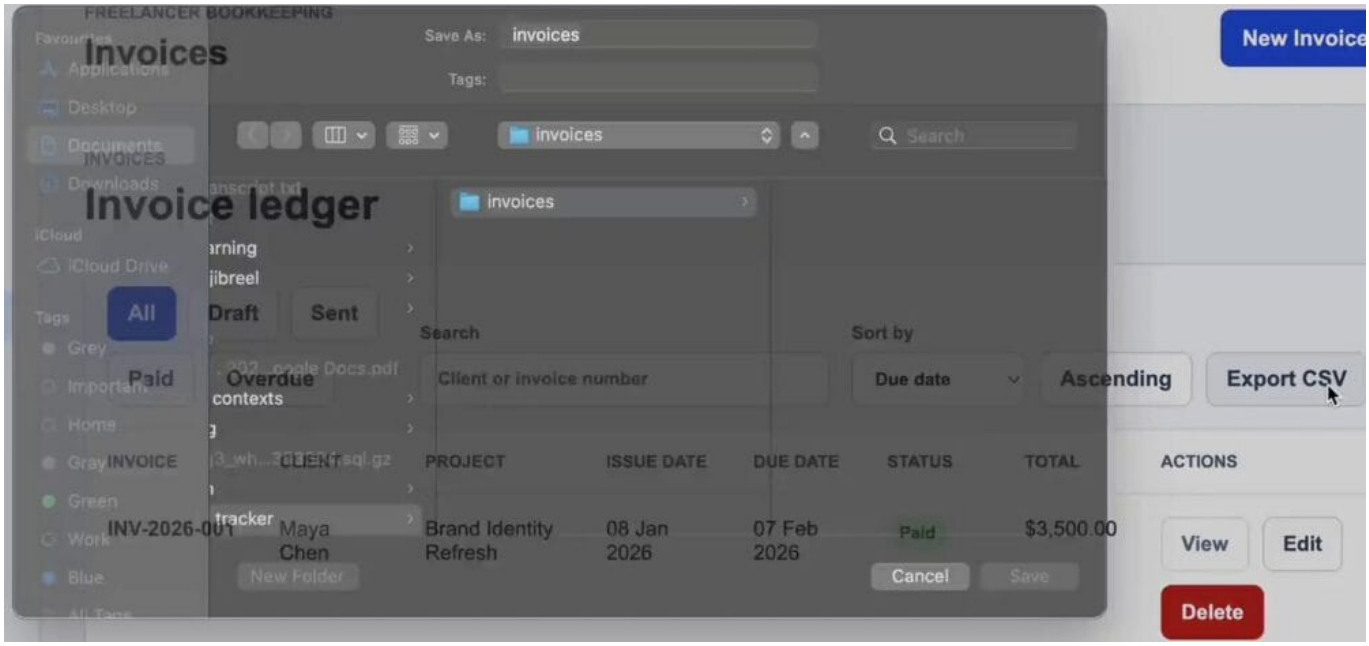
found during implementation. It then runs the test suite and linter successfully.

```
• The implementation is in place. I'm running the project checks now and will adjus

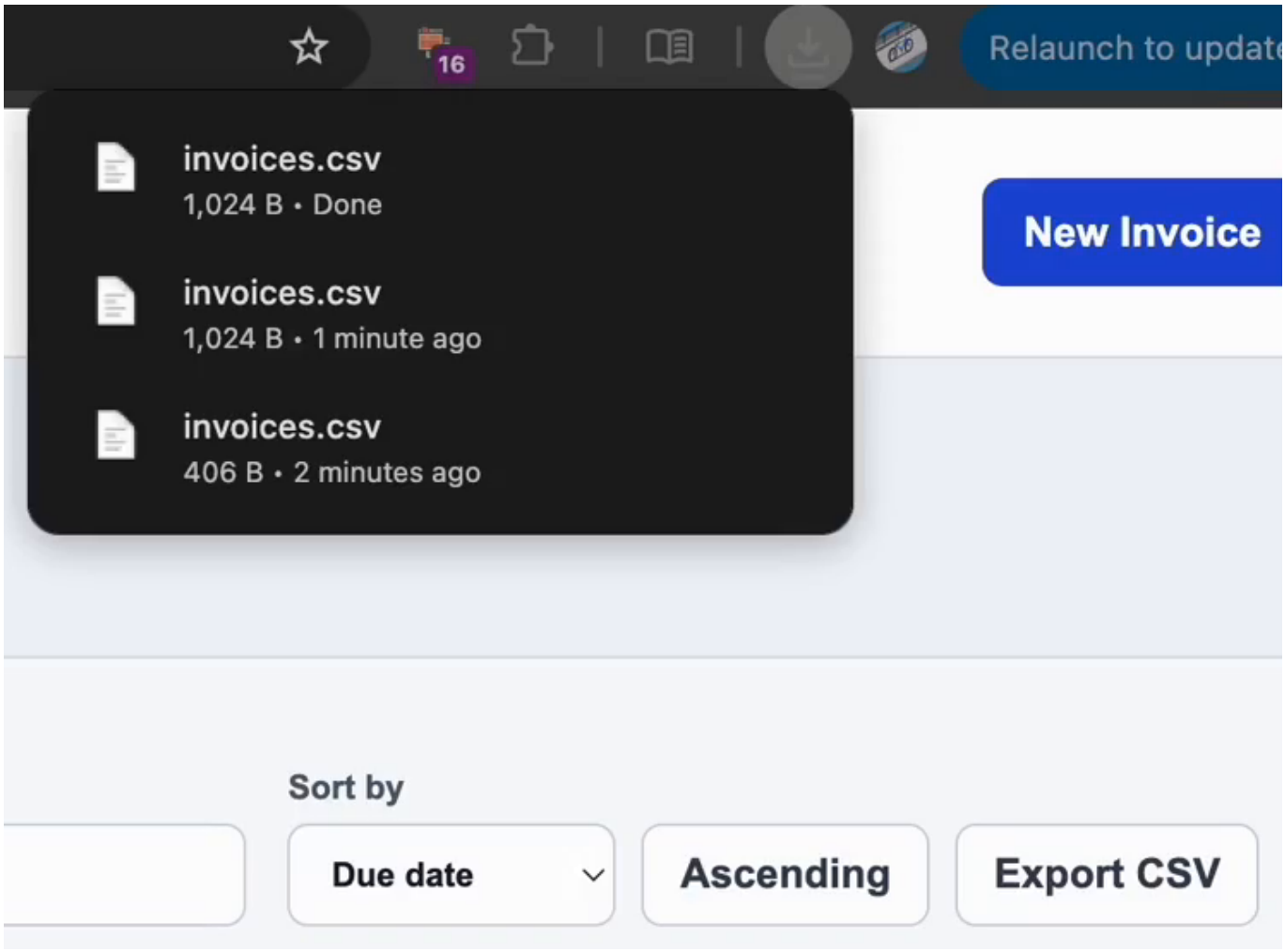
• Ran npm run test
  └
    RUN  v4.1.9 /Users/devisland/Desktop/learn_nextjs/nextjs_project
   … +5 lines (ctrl + t to view transcript)
      Duration  158ms (transform 41ms, setup 0ms, import 63ms, tests 18ms, environ
```

Export the complete invoice list

Open the invoices page without filters and select Export CSV. Save the generated invoices.csv file to the local system.



© 2026 Zenva Pty Ltd. All rights reserved.

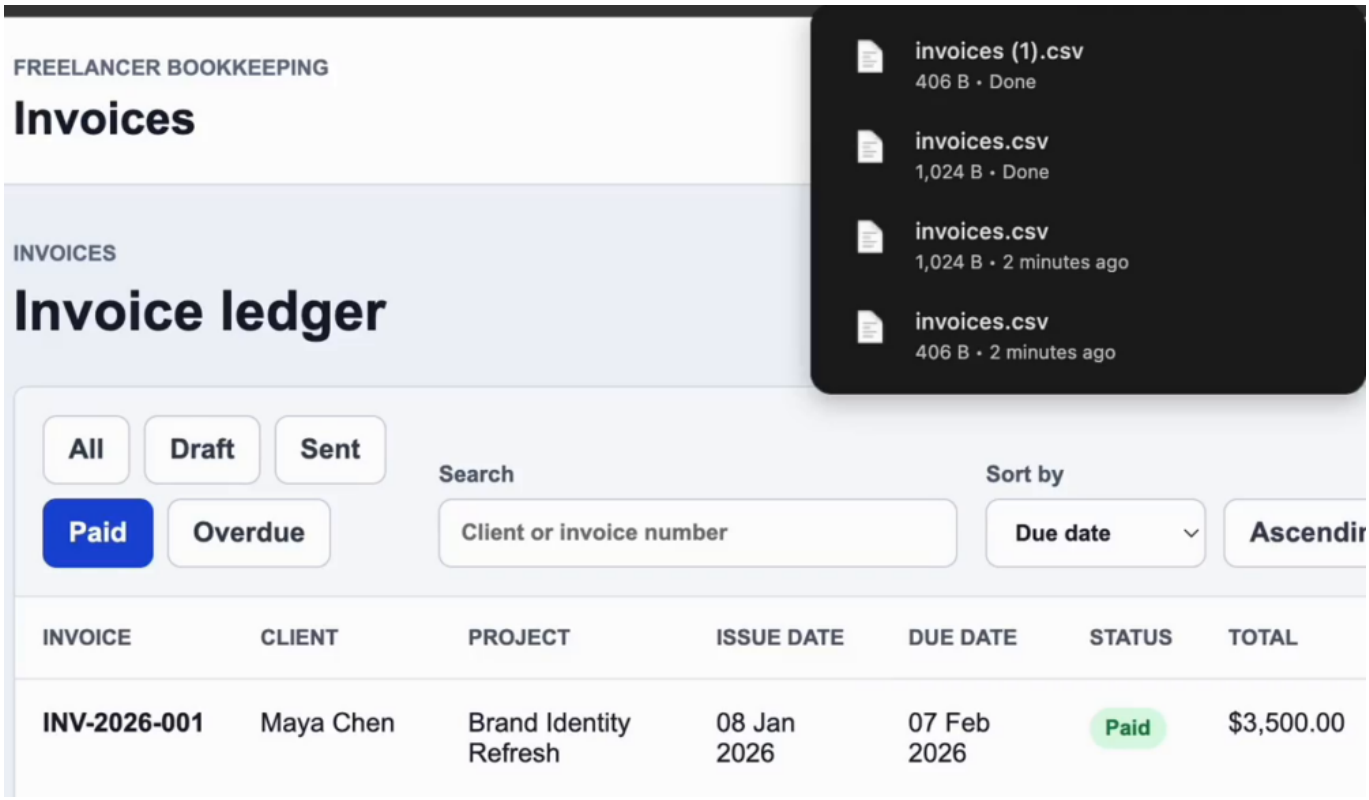


Open the CSV in a spreadsheet application and verify the column headers, invoice rows, dates in year-month-day format, statuses, and currency totals. Direct inspection confirms the serialization format rather than only proving that a file downloaded.

Verify filtered export

Return to the invoices page and apply the Paid filter. Export another CSV and confirm that the filter is included in the download request.

© 2026 Zenva Pty Ltd. All rights reserved.



Open the second file and confirm that it contains only paid invoices while preserving the same headers, date format, and dollar totals.

A	B	C	D	E	F	G	H	I	J
Invoice	Number	Client	Project	Issue	Date	Due	Date	Status	Total
INV-2026-012	Thomas	Wright	Sales	Collateral	System	2026-03-04	2026-04-03	paid	2090
INV-2026-009	Elena	Rodriguez	Spring	Campaign	Assets	2026-02-19	2026-03-21	paid	1615
INV-2026-004	Jordan	Patel	Client	Portal	Planning	2026-01-22	2026-02-21	paid	2520
INV-2026-001	Maya	Chen	Brand	Identity	Refresh	2026-01-08	2026-02-07	paid	3500

Review and commit

Review git status and git diff. The expected changes are the export API route, URL-filter handling on the invoices page, the table export link and initial filters, and the export-button styles. Commit the feature with the message Add CSV export for invoices with filter support, then confirm that the working tree is clean.

Summary

We built a filtered CSV download through an API route, connected it to the invoice page’s URL state, and verified both complete and paid-only exports. This demonstrates why an API route with explicit HTTP headers is often preferable to a server action for downloadable files.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we make our Invoice Tracker more resilient and helpful when data is missing or an unexpected problem occurs. We use Codex to implement empty states and Next.js error boundaries, verify the changes with automated tests, and inspect important edge cases in the browser.

Preparing a clean baseline

Before changing the project, we run git status and confirm that the working tree is clean. This gives us a reliable baseline for the session and makes the final review easier. We then launch Codex from the project directory.

Adding useful empty states

An empty page should explain what is missing and give the user a clear next step. We give Codex the following prompt:

```
Add empty states to every page that shows a list of records.

For each page, when there are no records to display, show a helpful empty state inste
ad of an empty table or blank area.

Clients page (src/app/clients/):
- Empty state: "No clients yet. Add your first client to get started." with the Add C
lient button visible

Projects page (src/app/projects/):
- Empty state: "No projects yet. Add your first project to get started." with the Add
 Project button visible

Invoices page (src/app/invoices/):
- Empty state when no invoices exist: "No invoices yet. Create your first invoice to
get started." with a Create Invoice button linking to /invoices/new
- Empty state when a filter returns no results: "No invoices match the current filter
." with a Clear Filter button

Dashboard overdue list:
- Already handled – verify it shows a positive message when there are no overdue invo
ices

Keep empty states simple – a short message and a clear call to action. Use plain CSS
modules for styling.
```

Codex explores the project structure, reads the relevant Next.js App Router guidance, and reviews the existing page components and CSS modules before planning its changes. It then adds conditional rendering so pages with no records display friendly guidance and an appropriate action instead of a blank area. It checks for stale references and runs linting and tests after the changes.

The empty-state work covers the main data areas of the Invoice Tracker:

- The dashboard presents zero-valued summaries and an empty revenue chart when the database contains no business data.
- The Clients page explains that no clients exist yet and offers an Add Client action.
- The Projects page explains that no projects exist yet and offers an Add Project action.
- The Invoices page explains that no invoices exist yet and offers a Create Invoice action while

© 2026 Zenva Pty Ltd. All rights reserved.

preserving its filters, search, sorting, and CSV export controls.

Adding error boundaries

Next, we ask Codex to add error boundaries for unexpected failures:

```
Add error boundary files for the main sections of the app.

Create these error boundary files:

src/app/error.js – root error boundary
- Catches any unhandled error in the app
- Shows: "Something went wrong. Please try refreshing the page."
- Includes a Refresh button that calls router.refresh()

src/app/clients/error.js – clients section
- Shows: "Unable to load clients. Please try again."
- Includes a Try Again button

src/app/projects/error.js – projects section
- Shows: "Unable to load projects. Please try again."
- Includes a Try Again button

src/app/invoices/error.js – invoices section
- Shows: "Unable to load invoices. Please try again."
- Includes a Try Again button

All error boundary files must be Client Components – Next.js requires this. Use plain
 CSS modules for styling.
Keep the error UI simple and reassuring, not alarming.
```

Codex reviews the current Next.js guidance and creates a root boundary along with boundaries for the Clients, Projects, and Invoices sections. Each boundary has a matching CSS module and a simple fallback interface that reassures the user and provides a way to refresh the page.

This structure allows an affected section to fail gracefully instead of leaving the user with a broken interface. Codex finishes by running linting and tests to verify that the new client-side boundary components integrate correctly.

Running the full test suite

We ask Codex to run the complete test suite and make any necessary corrections:

```
Run npm run test and show the full output. If any tests fail, identify the cause and
fix them before we commit.
```

In this run, the test file passes and all 13 tests succeed, confirming that the changes have not introduced a regression.

© 2026 Zenva Pty Ltd. All rights reserved.

```
Test Files  1 passed (1)
     Tests  13 passed (13)
  Start at  20:03:35
  Duration  142ms (transform 35ms, setup 0ms, import 53ms, tests 15ms, environment 0ms)
```

Creating a controlled empty database

To test the empty states visually, we temporarily clear the application data with this prompt:

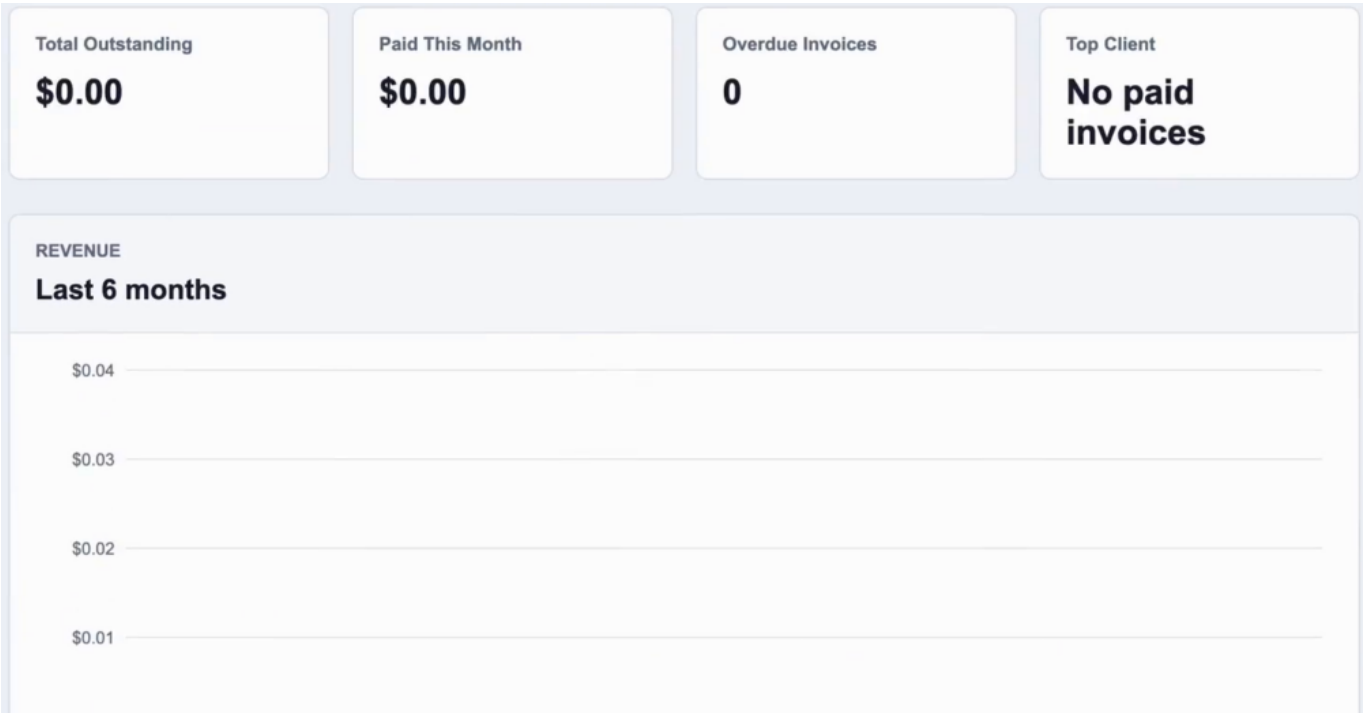
```
Run a quick node script that clears all data from the database – delete all rows from
 invoice_line_items, invoices, projects, and clients in that order.
Do not drop the tables.
```

Codex uses the same SQLite database helper and removes rows inside one transaction. The deletion order respects the relationships between the tables: invoice line items first, followed by invoices, projects, and clients. The tables themselves remain in place.

The first count verification fails because shell-sensitive backticks interfere with the inline command. Codex reruns the check using plain string concatenation, and the corrected query confirms that all four tables contain zero rows. A Node.js module-type warning also appears, but it is expected and does not indicate a failure.

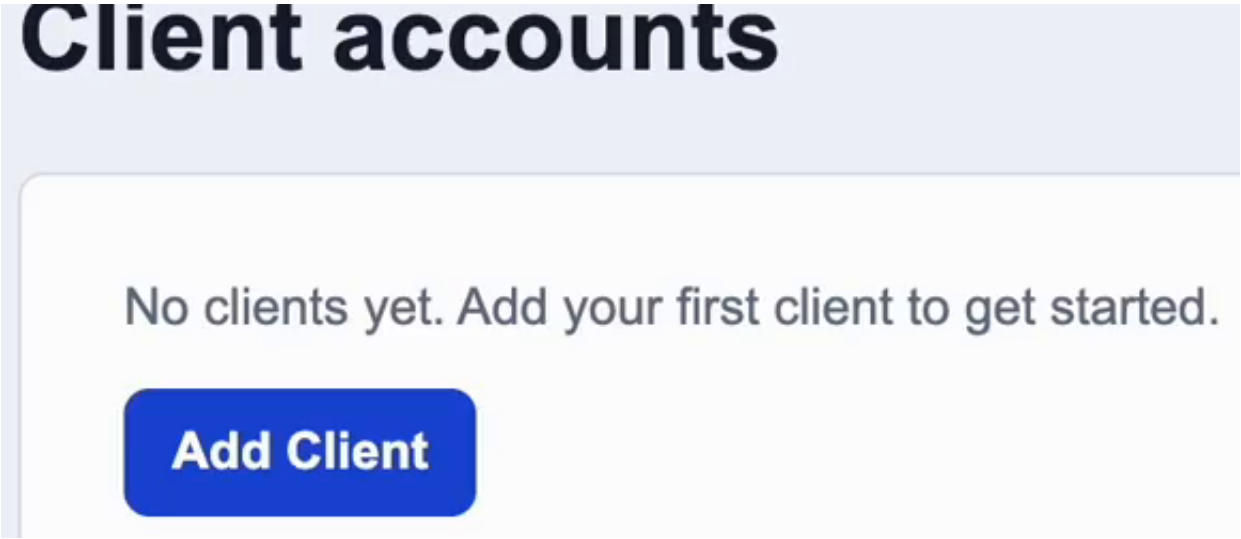
Verifying empty states in the browser

After refreshing the browser, the dashboard reports no outstanding balance, no payments for the month, no overdue invoices, and no top client. Its six-month revenue chart contains no data, which confirms that the dashboard responds correctly to an empty database.

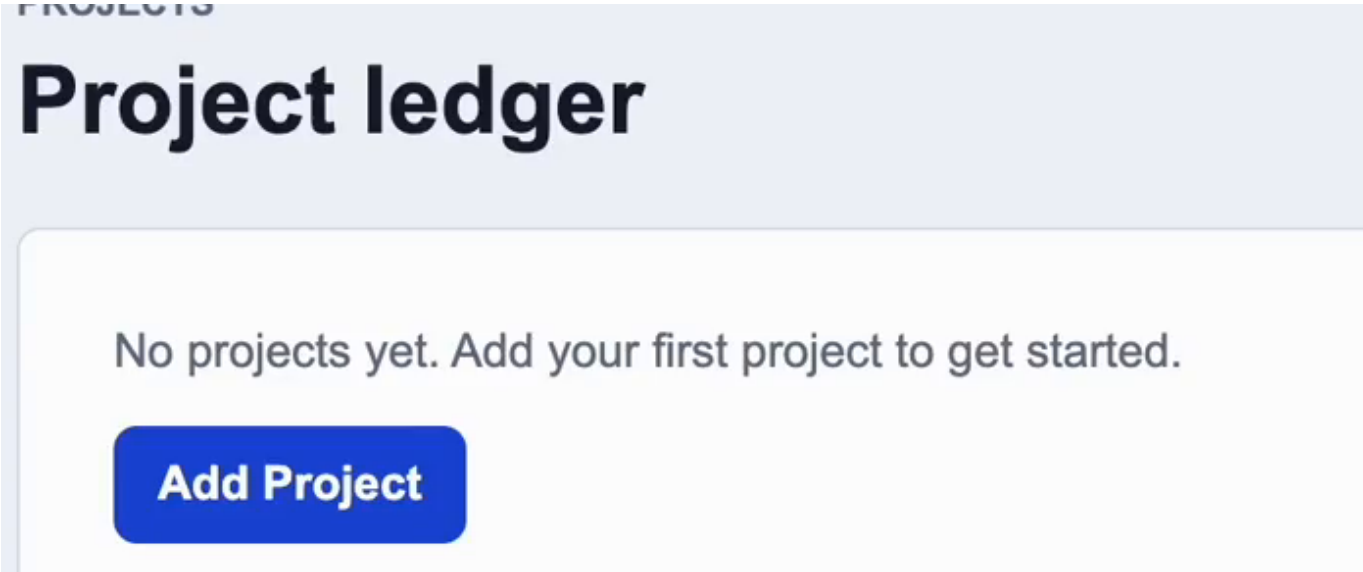


The Clients page provides direct guidance for creating the first client.

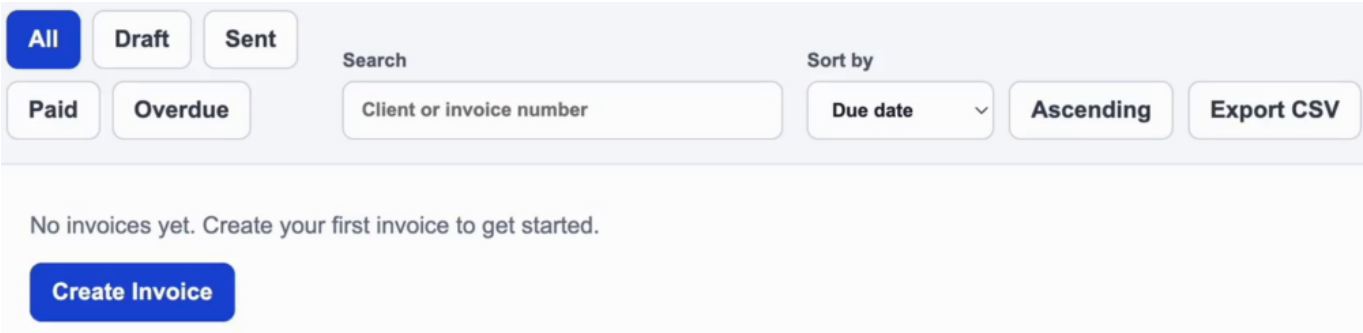
© 2026 Zenva Pty Ltd. All rights reserved.



The Projects page uses the same clear pattern and directs the user toward creating the first project.



The Invoices page keeps its management controls available while explaining how to create the first invoice.



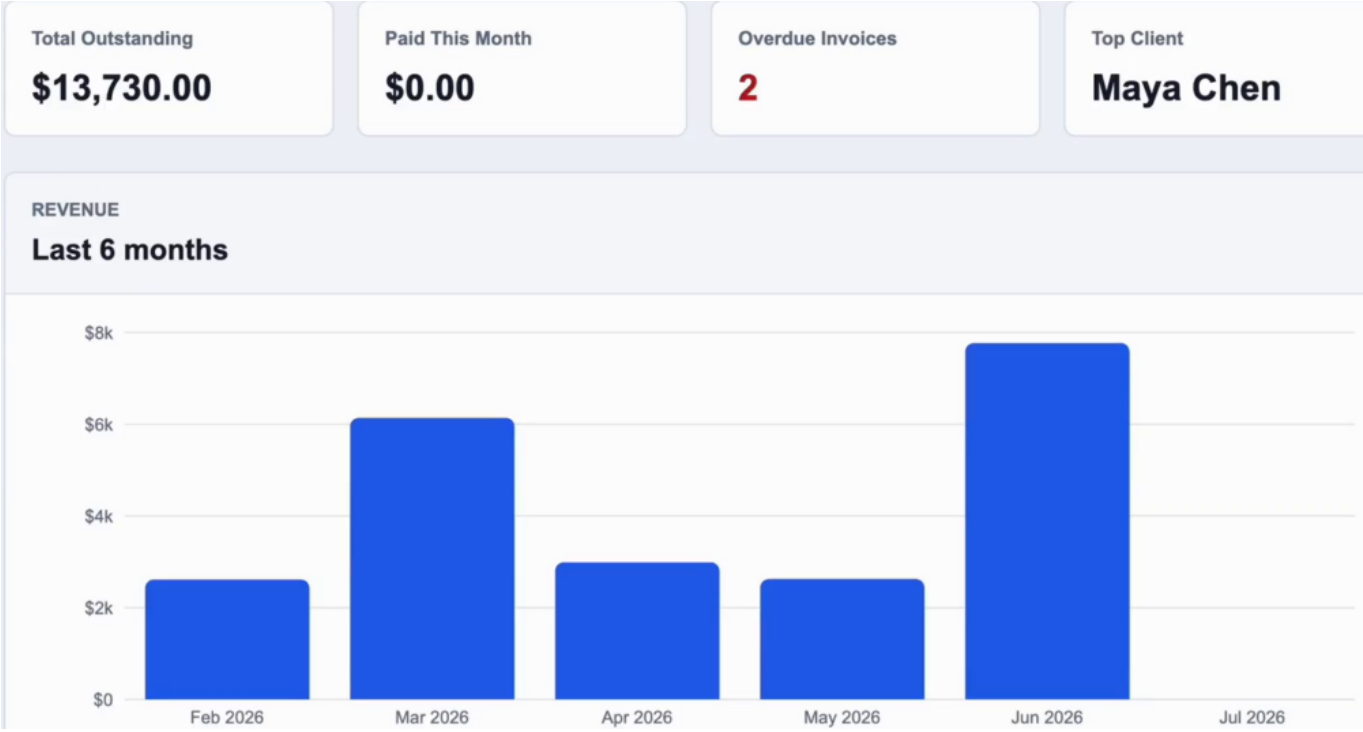
Restoring the sample data

Once the edge cases are verified, we restore the sample database with the following prompt:

© 2026 Zenva Pty Ltd. All rights reserved.

```
Run node src/lib/seed.js to reseed the database.
```

The expected module-type warning appears again, but the script exits successfully with a zero status. A final browser review confirms that the dashboard totals, overdue invoices, top client, and revenue history have returned. The Clients, Projects, and Invoices pages also contain their original records.



Reviewing and committing the work

We ask Codex to summarize every created or modified file:

```
Run git status and git diff and summarise every file you created or modified and what
 changed in each one.
```

The review includes the empty-state component and styling changes, the new error boundary files, and their CSS modules. Because a standard diff does not include untracked files, Codex reads the new boundary files separately so the summary remains complete.

After reviewing the changes, we provide the exact commit instruction:

```
Commit all changes with the message "add empty states, error boundaries, and verify f
ull test suite"
```

Codex commits the work and runs one final status check. The clean working tree confirms that the error handling and edge-case work is recorded. The application now guides users through empty states, handles unexpected errors with clear fallback interfaces, passes its automated tests, and

© 2026 Zenva Pty Ltd. All rights reserved.

behaves correctly both with and without sample data.

© 2026 Zenva Pty Ltd. All rights reserved.

The Complete Codex Development Course - Make the App Useful and Resilient

1. A charting library needs browser behavior, but dashboard data is loaded on the server. What is the best boundary?

a. Keep the page server-rendered and isolate the chart in a Client Component
b. Move all database queries into the chart
c. Render the entire application on the client
d. Remove the service layer

2. A dashboard library choice has uncertain compatibility. How can Codex help efficiently?

a. Install the first library it finds
b. Use a focused subagent to compare compatibility and dependencies
c. Rewrite the dashboard while researching libraries
d. Ignore version and peer-dependency checks

3. Codex added empty states across the app. How should you verify them before committing?

a. Read the generated code without running it
b. Permanently remove the sample data
c. Test with controlled empty data, inspect the pages, then restore the seed
d. Wait for a real user to encounter an empty page

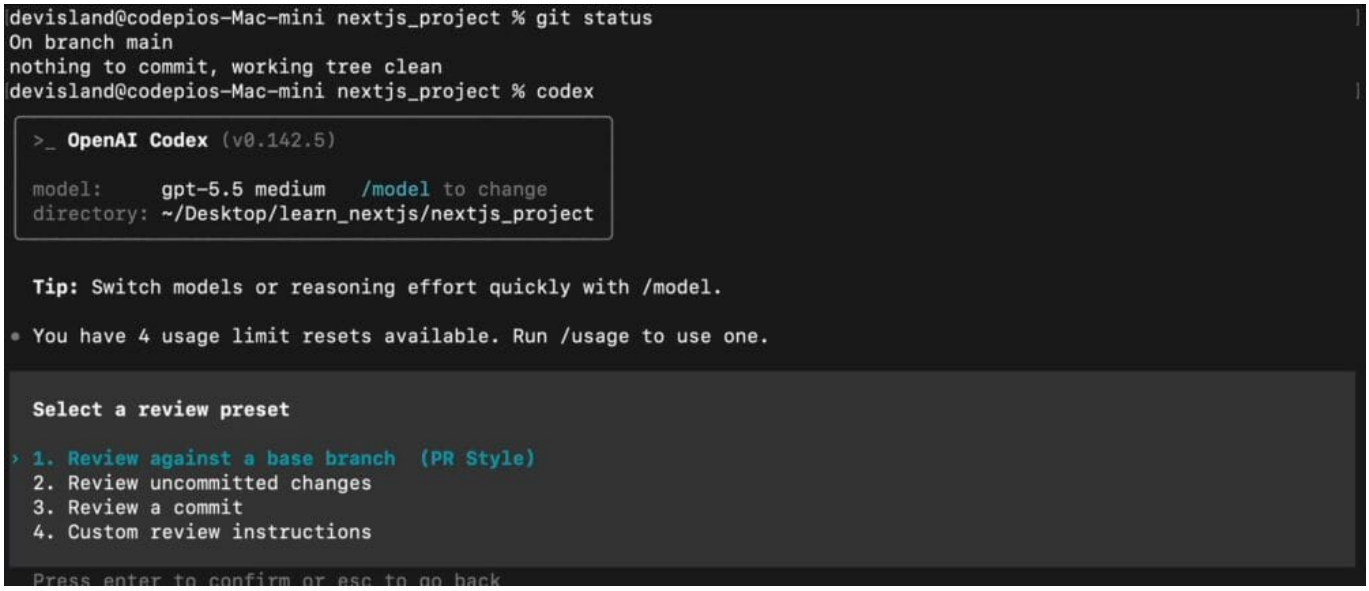
© 2026 Zenva Pty Ltd. All rights reserved.

In this final review, we bring the Freelancer Invoice Tracker project together and verify that it is complete. We use Git and Codex to inspect the repository, review its development history, summarize the finished modules, reflect on the workflow, and run the final checks.

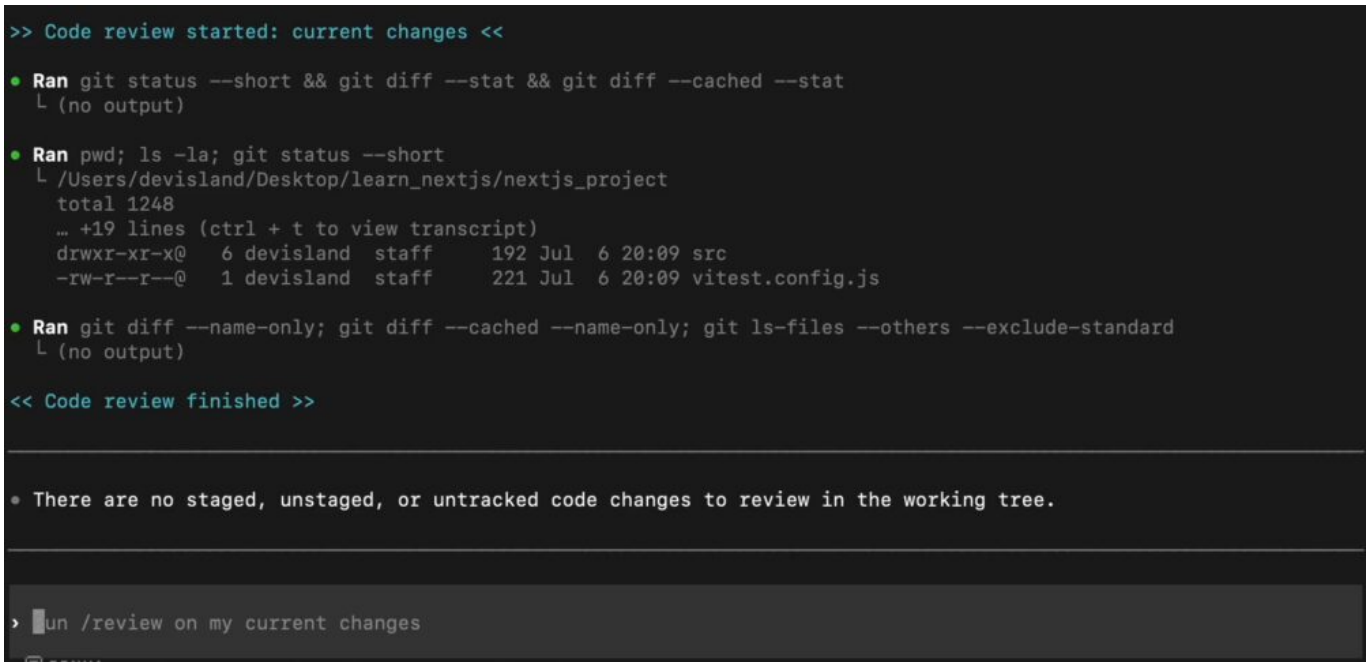
Starting from a clean baseline

We begin in the terminal with git status. A clean working directory confirms that the repository has no outstanding changes and gives us a stable baseline for the review. We then start Codex by running codex.

Inside Codex, we enter /review and choose **Review uncommitted changes**. This preset asks Codex to examine staged, unstaged, and untracked files.



Codex runs Git checks such as git status --short, git diff --stat, and git ls-files --others --exclude-standard. None of them return changes, so the review confirms that the working tree is clean and matches the state we intended.

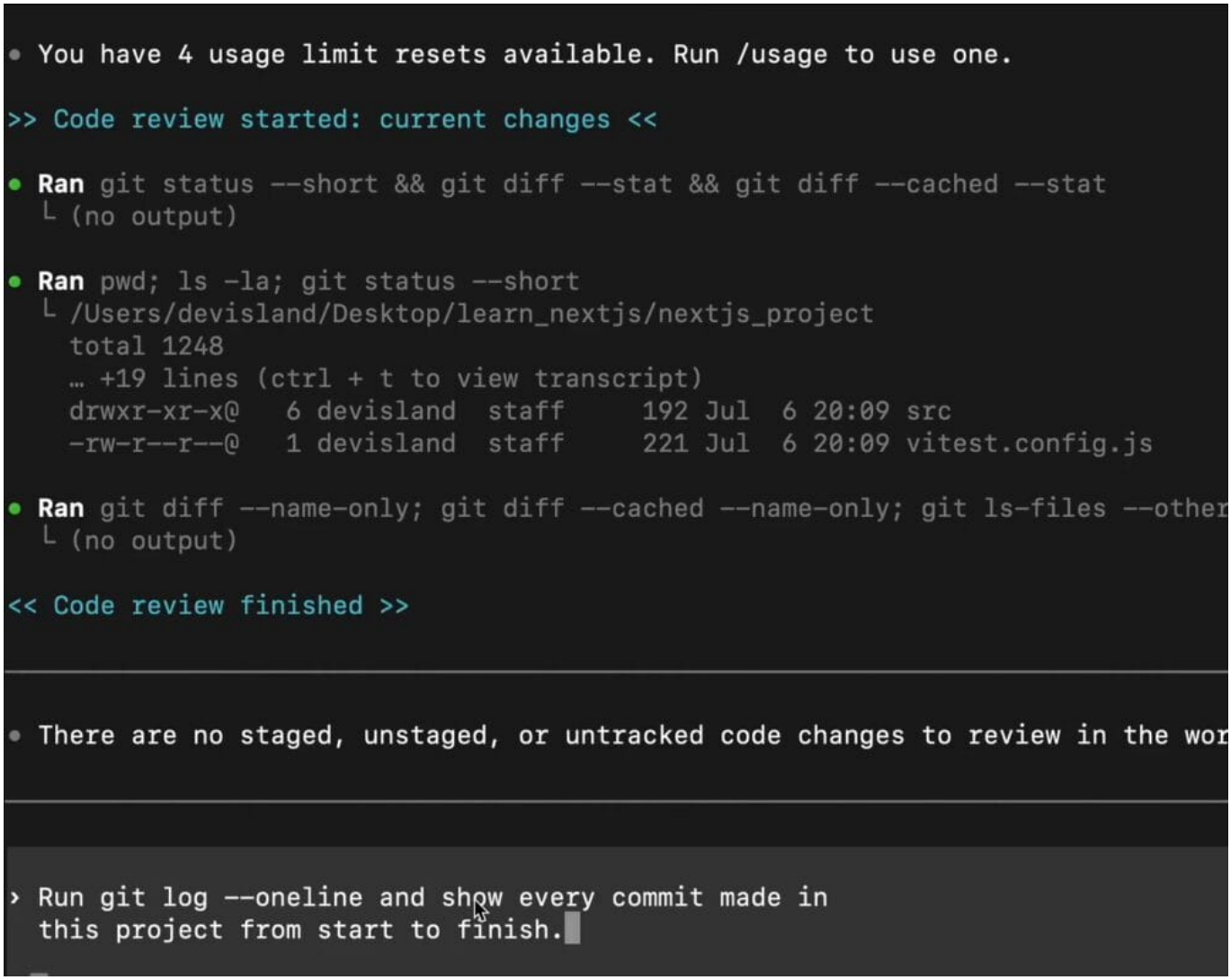


© 2026 Zenva Pty Ltd. All rights reserved.

Reviewing the complete Git history

Next, we ask Codex to retrieve every commit in chronological order. The exact prompt used is:

> Run git log –oneline and show every commit made in this project from start to finish.



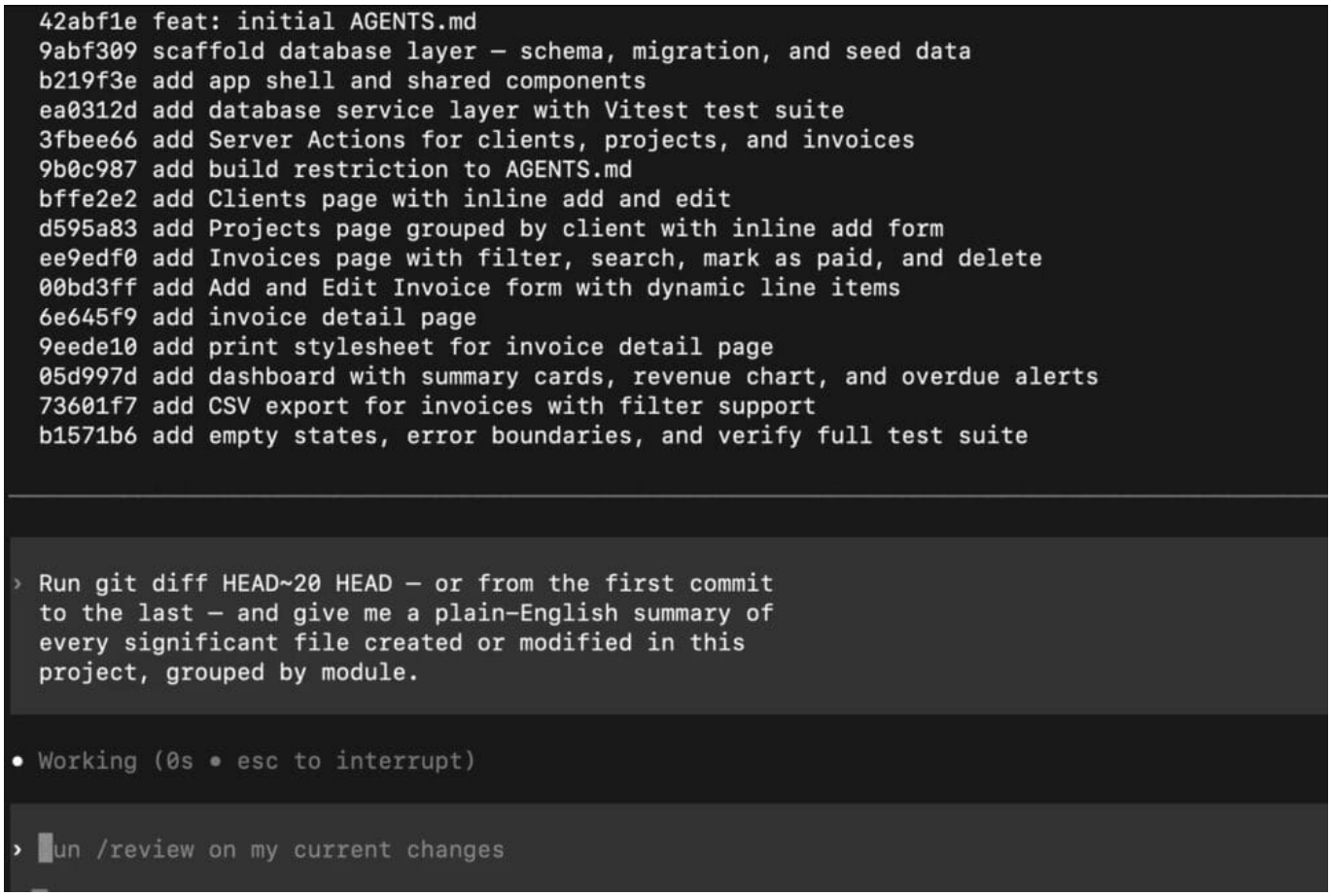
Codex runs git log --oneline --reverse, beginning with the initial Create Next App commit and continuing through the focused commits that built the application. This readable history documents how the project evolved and makes each deliberate piece of work easier to audit.

Summarizing changes by module

A list of commits explains when work happened, but a module-level summary explains what the completed system does. We generate that overview with the following exact prompt:

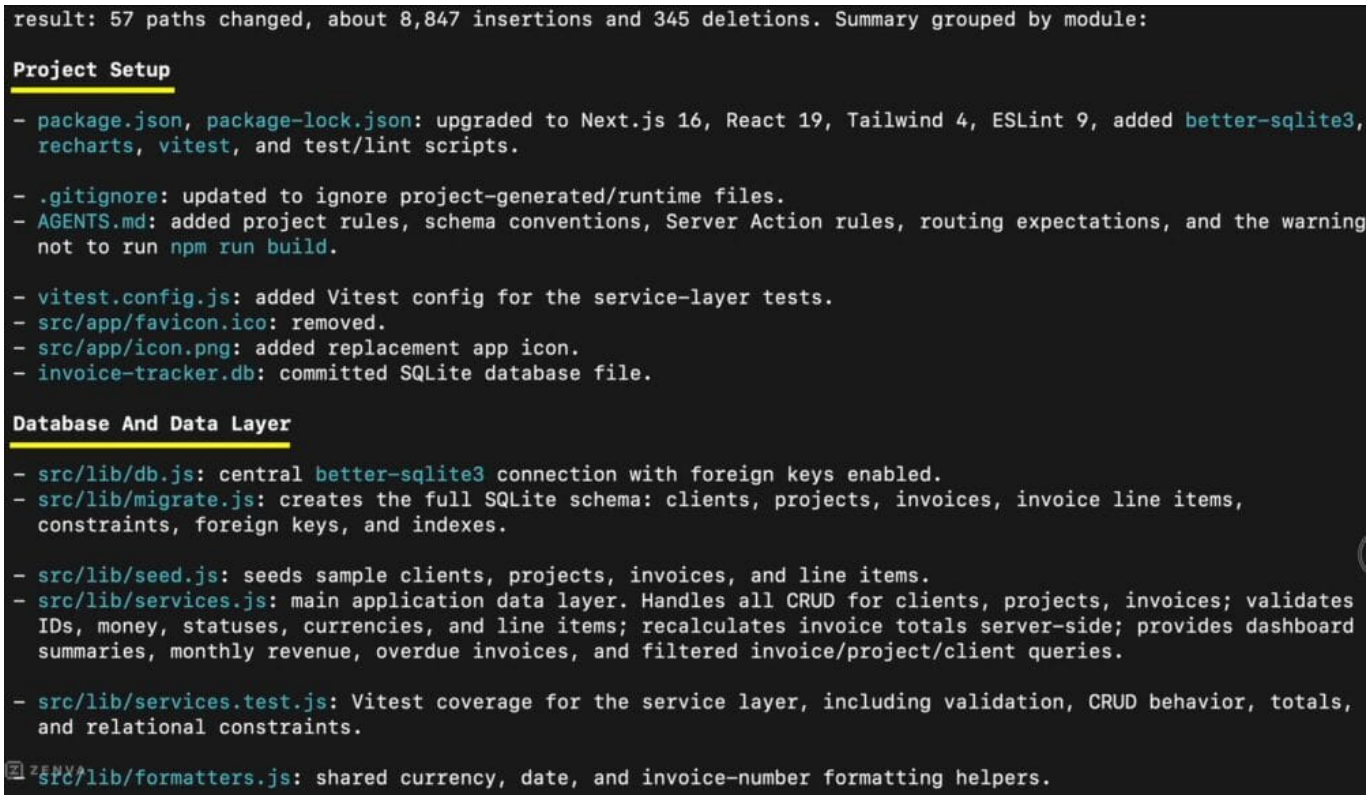
> Run git diff HEAD~20 HEAD – or from the first commit to the last – and give me a plain-English summary of every significant file created or modified in this project, grouped by module.

© 2026 Zenva Pty Ltd. All rights reserved.



Codex first determines the scope of the project history, examines the insertion and deletion statistics, and samples important source files. It then organizes the result by responsibility instead of returning a simple list of filenames.

© 2026 Zenva Pty Ltd. All rights reserved.



The resulting review covers:

- Project setup, dependency upgrades, Git exclusions, application configuration, and testing configuration.
- The SQLite database, migrations, seed data, service layer, service tests, and shared formatting helpers.
- Server actions for client, project, and invoice create, read, update, and delete operations.
- The shared application shell and reusable user interface components.
- The dashboard, client, project, and invoice modules, including their forms, tables, filters, and actions.
- The CSV export route for invoice data.

This converts a large project history into a concise explanation of the finished application.

Reflecting on the Codex workflow

The project also demonstrates several practices that make AI-assisted development more effective. Planning at the beginning helped us establish the architecture before implementation, avoiding expensive structural corrections later.

Research subagents gathered focused information about tools such as better-sqlite3 and charting libraries while keeping the main session focused. Parallel Codex sessions also accelerated development of the invoices page, reducing work that could have taken about 30 minutes sequentially to roughly 15 minutes.

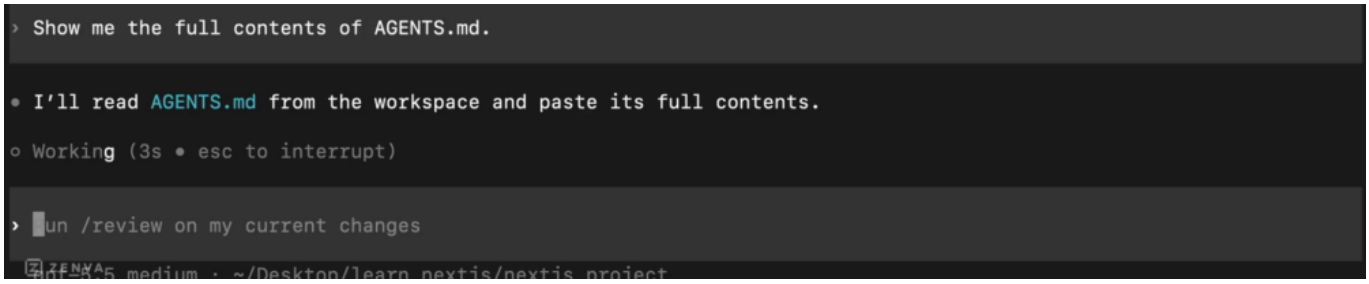
When the invoice quantity field stepped by 0.01 instead of whole units, we turned the usability issue into a permanent project rule. The final AGENTS.md requires quantity inputs in InvoiceForm to use step="1" and min="1", so future sessions inherit the correction automatically.

Using AGENTS.md as living documentation

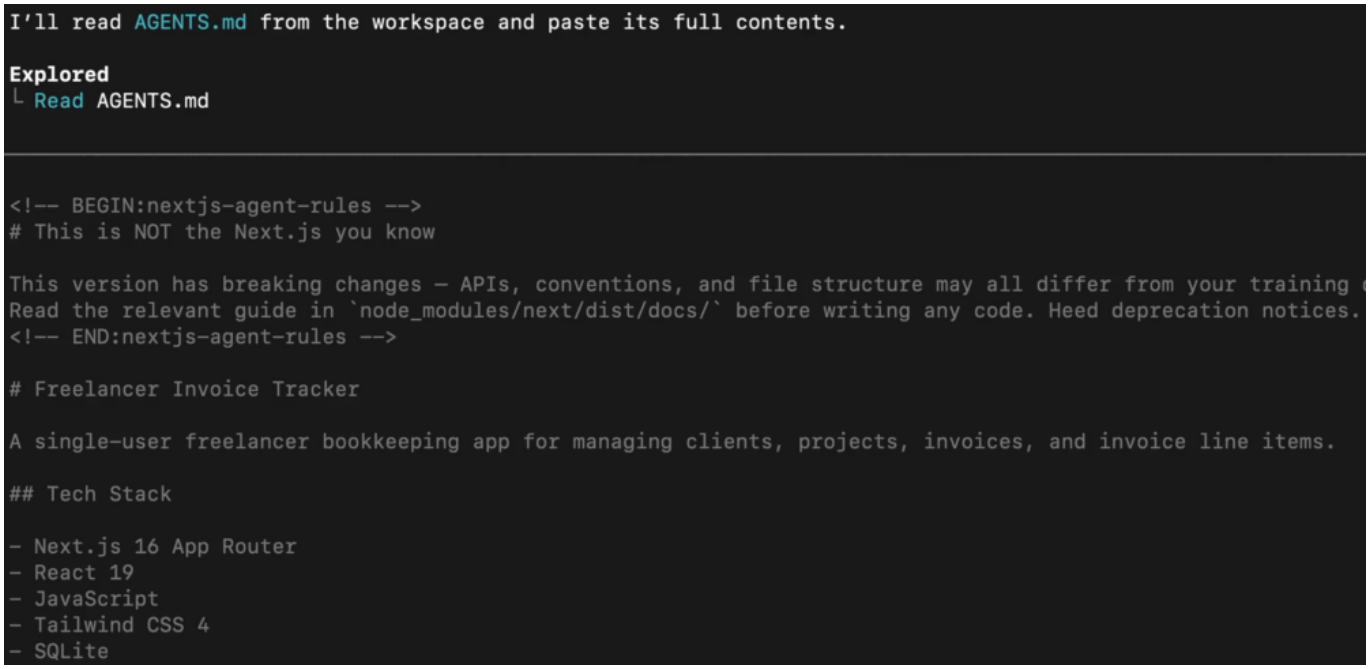
© 2026 Zenva Pty Ltd. All rights reserved.

Over the course of the project, AGENTS.md grew from a short starting document into the central reference for the codebase. We inspect its final state with this exact prompt:

> Show me the full contents of AGENTS.md.



The file identifies the Freelancer Invoice Tracker and records its Next.js 16 App Router, React 19, JavaScript, Tailwind CSS 4, SQLite, and better-sqlite3 stack. It also captures key commands, database conventions, server-action patterns, page structure, component rules, and critical implementation constraints.



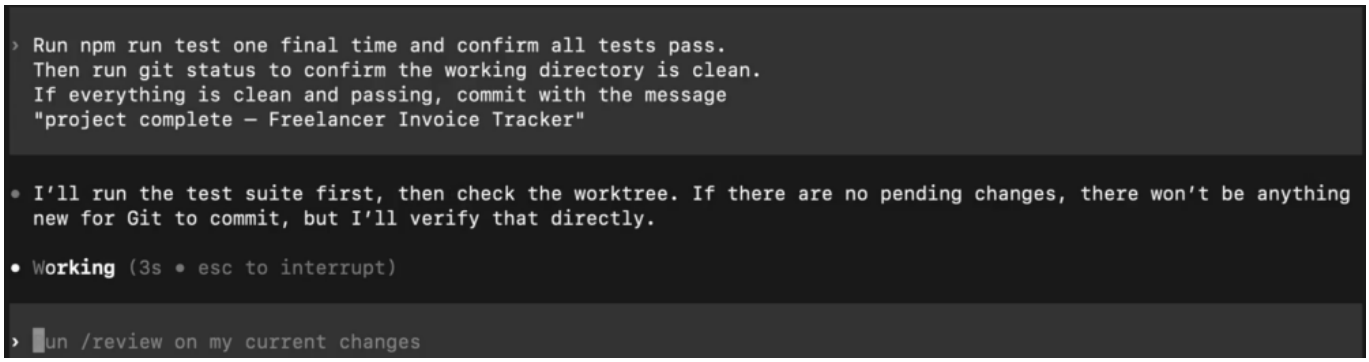
This living documentation acts as the project's institutional memory. Anyone opening the repository can inherit its architectural decisions and conventions without having to rediscover them.

Running the final checks

We finish with a multi-step request that tests the application, verifies the Git state, and marks completion. The exact prompt used is:

> Run npm run test one final time and confirm all tests pass. Then run git status to confirm the working directory is clean. If everything is clean and passing, commit with the message 'project complete – Freelancer Invoice Tracker'

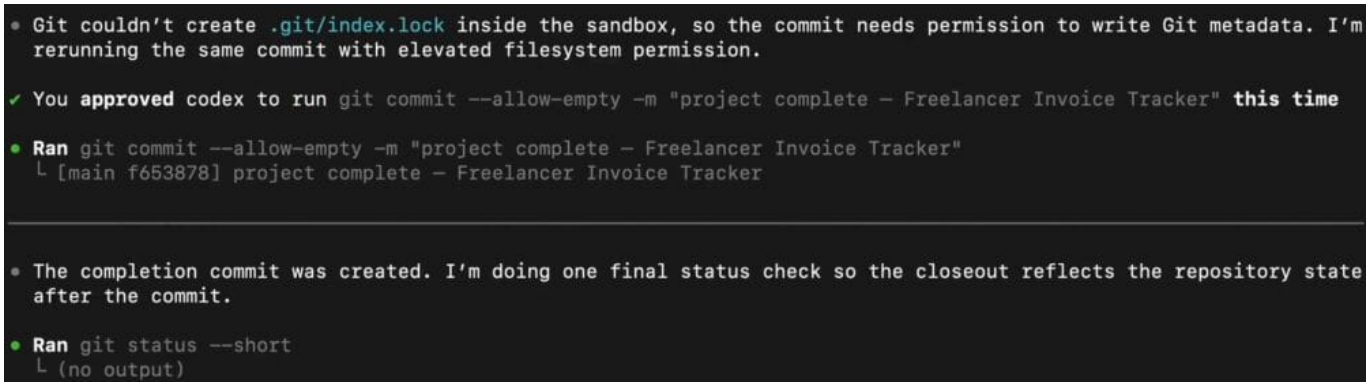
© 2026 Zenva Pty Ltd. All rights reserved.



Codex runs npm run test. The single test file and all 13 tests pass. It then runs git status --short, which produces no output and confirms that no uncommitted changes remain.



Because the worktree is already clean, Codex creates an empty marker commit with the requested completion message. A final status check confirms that the repository remains clean after the commit.



Reviewing the finished application

The completed Freelancer Invoice Tracker uses a modern Next.js 16 App Router foundation with SQLite, a tested service layer, and server actions. Its client, project, and invoice pages support complete create, read, update, and delete workflows, along with filtering, searching, sorting, and dynamic invoice line items.

The invoice detail page provides professional output through a dedicated print stylesheet for PDF exports and physical copies. Reporting features include dashboard summary cards, revenue trends, overdue alerts, and CSV export.

The application also includes useful empty states, basic error boundaries for unexpected failures, and a Vitest suite that protects the service layer from regressions. Most importantly, the evolving AGENTS.md preserves the decisions, conventions, and lessons needed to maintain and extend the project confidently.

© 2026 Zenva Pty Ltd. All rights reserved.

The Complete Codex Development Course - Review the Complete Project

1. What gives the final review a reliable starting point?

a. A clean Git working tree
b. An unverified generated summary
c. A deleted commit history
d. An active database migration

2. Why review the chronological Git history at the end of the project?

a. It shows how focused pieces of work built the application
b. It replaces the need to inspect source files
c. It automatically fixes regressions
d. It removes the need for tests

3. Which final check best confirms the project is stable and recorded?

a. Tests pass and Git status is clean
b. The dashboard opens once
c. The seed script contains sample invoices
d. The editor has no open tabs

© 2026 Zenva Pty Ltd. All rights reserved.

We have completed a significant project that demonstrates how to use Codex effectively to build a real application. The Freelancer Invoice Tracker brings together several important development habits that we can carry into future projects.

Plan before implementation

Planning discipline provided the foundation for the project. By mapping the database schema, page structure, and build order in plan mode, we identified dependencies early and prevented architectural mistakes that could have required substantial time and effort to correct later.

Delegate focused research

Subagents helped us delegate targeted research while keeping the primary Codex session focused and lean. This gave us the information needed to make confident implementation decisions without distracting from the main development workflow.

Work in parallel with clear ownership

For the invoices page, we split work across two parallel sessions with explicit ownership boundaries. One session built the UI layer while the other built the action layer. Because the responsibilities were clearly defined, both layers could be developed simultaneously without collisions and then integrated seamlessly.

Review every major change

A strong review habit helped us maintain the quality of the application. We ran Codex's /review workflow and checked Git diffs after every major change. This allowed us to inspect AI-generated code critically and apply the same professional scrutiny that we would use for any other implementation.

Maintain living project guidance

We also developed a living AGENTS.md file by documenting conventions, decisions, and constraints throughout the project. Instead of treating mistakes as isolated problems, we converted what we learned into permanent rules. The file grew from a basic skeleton into a comprehensive guide to the project's evolution and best practices.

These habits show how careful planning, focused delegation, parallel work, consistent review, and durable documentation can help us use Codex effectively on substantial real-world applications.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

AGENTS.md

Found in folder: **/**

This file provides setup instructions and conventions for a freelancer invoice tracking application built with Next.js. It outlines the database schema, data layer rules, page structure, and coding standards for the project. The document serves as a reference for developers working on the app.

```
<!-- BEGIN:nextjs-agent-rules -->
# This is NOT the Next.js you know

This version has breaking changes — APIs, conventions, and file structure may all dif
fer from your training data. Read the relevant guide in `node_modules/next/dist/docs/
` before writing any code. Heed deprecation notices.
<!-- END:nextjs-agent-rules -->

# Freelancer Invoice Tracker

A single-user freelancer bookkeeping app for managing clients, projects, invoices, an
d invoice line items.

## Tech Stack

- Next.js 16 App Router
- React 19
- JavaScript
- Tailwind CSS 4
- SQLite
- better-sqlite3
- ESLint 9 with `eslint-config-next`
- npm

## Key Commands

- Dev server: `npm run dev`
- Tests: `npm run test`
- Lint: `npm run lint`
- Database migration: `node src/lib/migrate.js`
- Database seed: `node src/lib/seed.js`

## Database Schema

Use SQLite with foreign keys enabled. Store money as integer cents and dates/timestam
ps as ISO text.

### `clients`

- Stores freelancer clients and contact details.
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
- Key columns: `id`, `name`, `email`, `phone`, `company`, `address`, `notes`, `create
d_at`, `updated_at`.
- Constraints: `name` is required.
- Index: client name.

### `projects`

- Stores projects owned by clients.
- Key columns: `id`, `client_id`, `name`, `description`, `status`, `hourly_rate_cents
`, `created_at`, `updated_at`.
- Relationship: `client_id` references `clients(id)` with restricted deletion.
- Constraints: status is one of `active`, `paused`, `completed`, `archived`; hourly r
ate must be null or non-negative.
- Indexes: client, status.

### `invoices`

- Stores invoices for projects.
- Key columns: `id`, `project_id`, `invoice_number`, `status`, `issue_date`, `due_dat
e`, `currency`, `subtotal_cents`, `total_cents`, `paid_cents`, `sent_at`, `paid_at`,
`notes`, `created_at`, `updated_at`.
- Relationship: `project_id` references `projects(id)` with restricted deletion.
- Constraints: invoice number is unique; status is one of `draft`, `sent`, `paid`, `o
verdue`, `void`; totals and paid amounts are non-
negative; paid amount cannot exceed total; currency is a 3-letter code.
- Indexes: project, status, due date, issue date.

### `invoice_line_items`

- Stores ordered line items for invoices.
- Key columns: `id`, `invoice_id`, `position`, `description`, `quantity`, `unit_price
_cents`, `line_total_cents`, `created_at`, `updated_at`.
- Relationship: `invoice_id` references `invoices(id)` with cascade deletion.
- Constraints: position is non-negative; quantity is positive; unit price and line to
tal are non-negative; each invoice has unique line item positions.
- Index: invoice.

Invoice totals are recalculated server-side whenever line items change. In v1, `subto
tal_cents` and `total_cents` are equal because tax and discounts are out of scope.

## Data Layer

- All database access goes through `src/lib/services.js`.
- Never write raw SQL queries inside components or pages.
- All money values are stored and passed as integer cents.
- Run `npm run test` before completing any task that touches the service layer.

## Server Actions

- All data mutations go through Server Actions in `src/actions/`.
- Actions import from `src/lib/services.js` only; never query the database directly.
- Every action returns `{ data, error }`; `error` is a string message or null.
- Action files must start with `'use server'`.
- For reads, use service functions directly in Server Components; no action needed.
- Never use `fetch()` to call our own data; use actions or service functions directly
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.

## Page Structure

Use `src/app` routes with shared navigation for Dashboard, Clients, Projects, and Inv
oices.

- `/`: dashboard with unpaid total, overdue invoices, paid this month, active project
s, recent invoices, upcoming due invoices, and links to create core records.
- `/clients`: client list with search/filter, project counts, unpaid totals, and link
s to detail/create pages.
- `/clients/new`: create client form.
- `/clients/[clientId]`: client detail with contact info, notes, projects, and invoic
es across the client's projects.
- `/clients/[clientId]/edit`: edit client form.
- `/projects`: project list with status/client filters, invoice counts, unpaid totals
, and links to detail/create pages.
- `/projects/new`: create project form.
- `/projects/[projectId]`: project detail with client link, invoices, and totals by s
tatus.
- `/projects/[projectId]/edit`: edit project form.
- `/invoices`: invoice list with status, client/project, due date, and invoice number
 filtering.
- `/invoices/new`: create invoice form with project selection and editable line items
.
- `/invoices/[invoiceId]`: invoice detail and print-
friendly invoice view with actions to edit, mark sent, mark paid, or void.
- `/invoices/[invoiceId]/edit`: edit invoice metadata and line items.

## Conventions

- Read the relevant guide in `node_modules/next/dist/docs/` before changing App Route
r code.
- Keep routes under `src/app`.
- Use `page.js` for route UI and `layout.js` for shared route shells.
- Use dynamic route segment folders with bracket names, such as `[clientId]`, `[proje
ctId]`, and `[invoiceId]`.
- Use lowercase kebab-case folder names for static route segments.
- Keep shared UI components outside route folders when reused across multiple section
s.
- Keep route-specific components close to the route that owns them.
- Use Server Components by default.
- Use Client Components only for interactive browser state.
- Put server-only database access behind a dedicated data layer instead of querying S
QLite directly from UI components.
- Use Server Functions for form mutations.
- Recalculate derived invoice totals on the server, not from trusted client input.
- Use integer cents for all money values.
- Use ISO text values for dates and timestamps.
- Name database columns in snake_case.
- Name JavaScript variables and functions in camelCase.
- Name React components in PascalCase.

## Shared Components
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
- Use `StatusBadge` at `src/components/StatusBadge.jsx` for all status display. Never
 hardcode status colours inline.
- Use `formatCurrency`, `formatDate`, and `formatInvoiceNumber` at `src/lib/formatter
s.js` for all currency, date, and invoice number formatting. Never format these value
s inline.

## Critical Rules

- NEVER run npm run build — it hangs in this environment. Use npm run test for the te
st suite only.
- In InvoiceForm, quantity inputs must have step="1" and min="1" — never leave number
 inputs without explicit step and min attributes.
```

RevenueChart.jsx

Found in folder: **/src/app**

This code renders a bar chart displaying revenue data over months, with currency-formatted axes and tooltips. It uses a custom formatter to abbreviate large values on the Y-axis for better readability. The chart is styled responsively within a container and includes grid lines, axis labels, and interactive tooltips.

```
"use client";

import {
  Bar,
  BarChart,
  CartesianGrid,
  ResponsiveContainer,
  Tooltip,
  XAxis,
  YAxis,
} from "recharts";

import { formatCurrency } from "@/lib/formatters";
import styles from "./Dashboard.module.css";

// Abbreviates large currency values for Y-
axis display (e.g., 100000 cents becomes "$1k")
function formatAxisCurrency(cents) {
  if (cents >= 100000) {
    return `$${Math.round(cents / 100000)}k`;
  }

  return formatCurrency(cents).replace(".00", ""); // Remove cents when zero for clea
ner labels
}

export default function RevenueChart({ data }) {
  return (
    <div className={styles.chartFrame}>
      <ResponsiveContainer width="100%" height="100%">
        <BarChart data={data} margin={{ top: 8, right: 8, left: 0, bottom: 0 }}>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
          <CartesianGrid stroke="#e2e8f0" vertical={false} />
          <XAxis
            dataKey="month" // Expects data objects with a "month" property
            axisLine={false}
            tickLine={false}
            tick={{ fill: "#64748b", fontSize: 12 }}
          />
          <YAxis
            axisLine={false}
            tickLine={false}
            tick={{ fill: "#64748b", fontSize: 12 }}
            tickFormatter={formatAxisCurrency} // Applies custom formatting to Y-
axis labels
            width={64} // Reserved width for formatted tick labels
          />
          <Tooltip
            cursor={{ fill: "#f1f5f9" }}
            formatter={(value) => [formatCurrency(value), "Revenue"]} // Formats tool
tip values as currency
            labelStyle={{ color: "#172033", fontWeight: 700 }}
            contentStyle={{
              border: "1px solid #d9e1ec",
              borderRadius: 8,
              boxShadow: "0 12px 28px rgba(15, 23, 42, 0.12)",
            }}
          />
          <Bar dataKey="totalCents" fill="#2563eb" radius={[6, 6, 0, 0]} /> // Maps "
totalCents" from data to bar heights
        </BarChart>
      </ResponsiveContainer>
    </div>
  );
}
```

ClientInlineForm.jsx

Found in folder: **/src/app/clients**

This code provides an inline form component for adding or editing client information. It conditionally renders either a button to open the form or the form itself based on state, with different behaviors for create versus edit modes. When submitted, it calls server actions to create or update client data and handles loading states and errors.

```
"use client";

import { useRouter } from "next/navigation";
import { useState, useTransition } from "react";
import {
  createClientAction,
  updateClientAction,
} from "@/actions/clientActions";
import styles from "./ClientsPage.module.css";
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
// Default empty values for new client form
const emptyFormData = {
  name: "",
  company: "",
  email: "",
  phone: "",
  address: "",
  notes: "",
};

// Convert client object to form data, handling missing properties
function clientToFormData(client) {
  return {
    name: client?.name || "",
    company: client?.company || "",
    email: client?.email || "",
    phone: client?.phone || "",
    address: client?.address || "",
    notes: client?.notes || "",
  };
}

export default function ClientInlineForm({ mode, client = null }) {
  const router = useRouter();
  const [isOpen, setIsOpen] = useState(false);
  // Initialize form with client data for edit mode or empty values for create
  const [formData, setFormData] = useState(() => (
    mode === "edit" ? clientToFormData(client) : emptyFormData
  ));
  const [error, setError] = useState(null);
  const [isPending, startTransition] = useTransition();

  const isEdit = mode === "edit";

  function openForm() {
    setFormData(isEdit ? clientToFormData(client) : emptyFormData);
    setError(null);
    setIsOpen(true);
  }

  function closeForm() {
    setFormData(isEdit ? clientToFormData(client) : emptyFormData);
    setError(null);
    setIsOpen(false);
  }

  function updateField(event) {
    const { name, value } = event.target;
    setFormData((current) => ({
      ...current,
      [name]: value,
    }));
  }

  function submitForm(event) {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  event.preventDefault();
  setError(null);

  // Use transition for loading state during async server action
  startTransition(async () => {
    const result = isEdit
      ? await updateClientAction(client.id, formData)
      : await createClientAction(formData);

    if (result.error) {
      setError(result.error);
      return;
    }

    // Reset form and refresh page data on success
    setIsOpen(false);
    setFormData(emptyFormData);
    router.refresh();
  });
}

// Show button to open form when closed
if (!isOpen) {
  return (
    <button
      className={isEdit ? styles.secondaryButton : styles.primaryButton}
      type="button"
      onClick={openForm}
    >
      {isEdit ? "Edit" : "Add Client"}
    </button>
  );
}

// Render form when open
return (
  <form
    className={`${styles.inlineForm} ${isEdit ? styles.editForm : ""}`}
    onSubmit={submitForm}
  >
    <div className={styles.formGrid}>
      <label className={styles.field}>
        <span>Name</span>
        <input
          name="name"
          required
          value={formData.name}
          onChange={updateField}
        />
      </label>
      <label className={styles.field}>
        <span>Company</span>
        <input
          name="company"
          value={formData.company}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
        onChange={updateField}
      />
    </label>
    <label className={styles.field}>
      <span>Email</span>
      <input
        name="email"
        type="email"
        value={formData.email}
        onChange={updateField}
      />
    </label>
    <label className={styles.field}>
      <span>Phone</span>
      <input
        name="phone"
        value={formData.phone}
        onChange={updateField}
      />
    </label>
    <label className={`${styles.field} ${styles.fullWidth}`}>
      <span>Address</span>
      <textarea
        name="address"
        rows="2"
        value={formData.address}
        onChange={updateField}
      />
    </label>
    <label className={`${styles.field} ${styles.fullWidth}`}>
      <span>Notes</span>
      <textarea
        name="notes"
        rows="2"
        value={formData.notes}
        onChange={updateField}
      />
    </label>
  </div>

  {error ? <p className={styles.error}>{error}</p> : null}

  <div className={styles.formActions}>
    <button
      className={styles.primaryButton}
      type="submit"
      disabled={isPending}
    >
      {isPending ? "Saving..." : isEdit ? "Save" : "Create"}
    </button>
    <button
      className={styles.secondaryButton}
      type="button"
      onClick={closeForm}
      disabled={isPending}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
        >
          Cancel
        </button>
      </div>
    </form>
  );
}
```

DeleteConfirmModal.jsx

Found in folder: **/src/app/invoices**

This component renders a modal dialog that asks the user to confirm the deletion of a specific invoice. It calls a delete action and handles the pending state and any errors that may occur. Upon successful deletion, it closes the modal and refreshes the page data.

```
"use client";

import { useRouter } from "next/navigation";
import { useState, useTransition } from "react";
import { deleteInvoiceAction } from "@/actions/invoiceActions";
import { formatInvoiceNumber } from "@/lib/formatters";
import styles from "./DeleteConfirmModal.module.css";

// Safely extracts the invoice number from various possible property names
function getInvoiceNumber(invoice) {
  return invoice?.invoice_number ?? invoice?.invoiceNumber ?? invoice?.id ?? "";
}

export default function DeleteConfirmModal({ invoice, onClose }) {
  const router = useRouter();
  const [error, setError] = useState(null);
  const [isPending, startTransition] = useTransition();
  const invoiceNumber = formatInvoiceNumber(getInvoiceNumber(invoice));

  function confirmDelete() {
    setError(null);

    startTransition(async () => {
      const result = await deleteInvoiceAction(invoice.id);

      if (result.error) {
        setError(result.error);
        return;
      }

      onClose();
      router.refresh(); // Refresh server components after successful deletion
    });
  }

  return (
    <div className={styles.backdrop} role="presentation">
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      <div
        aria-labelledby="delete-invoice-title"
        aria-modal="true"
        className={styles.modal}
        role="dialog"
      >
        <h2 className={styles.title} id="delete-invoice-title">
          Delete invoice
        </h2>
        <p className={styles.message}>
          Are you sure you want to delete invoice {invoiceNumber}?
        </p>

        {error ? <p className={styles.error}>{error}</p> : null}

        <div className={styles.actions}>
          <button
            className={styles.dangerButton}
            type="button"
            onClick={confirmDelete}
            disabled={isPending}
          >
            {isPending ? "Deleting..." : "Confirm"}
          </button>
          <button
            className={styles.secondaryButton}
            type="button"
            onClick={onClose}
            disabled={isPending}
          >
            Cancel
          </button>
        </div>
      </div>
    </div>
  );
}
```

InvoiceActions.jsx

Found in folder: **/src/app/invoices**

This component renders action buttons for an invoice, including View, Edit, Mark as Paid, and Delete. It conditionally shows the Mark as Paid button only if the invoice isn’t already paid or void. It handles marking an invoice as paid with a transition state and error handling, and opens a delete confirmation modal.

```
"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState, useTransition } from "react";
import { updateInvoiceStatusAction } from "@/actions/invoiceActions";
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
import DeleteConfirmModal from "./DeleteConfirmModal";
import styles from "./InvoiceActions.module.css";

export default function InvoiceActions({ invoice }) {
  const router = useRouter();
  const [isDeleteOpen, setIsDeleteOpen] = useState(false);
  const [error, setError] = useState(null);
  const [isPending, startTransition] = useTransition();

  if (!invoice) {
    return null;
  }

  // Only allow marking as paid if invoice is not already paid or void
  const canMarkPaid = invoice.status !== "paid" && invoice.status !== "void";

  function markAsPaid() {
    setError(null);

    startTransition(async () => {
      const result = await updateInvoiceStatusAction(invoice.id, "paid");

      if (result.error) {
        setError(result.error);
        return;
      }

      router.refresh();
    });
  }

  return (
    <>
      <div className={styles.actions}>
        <Link
          className={styles.tertiaryButton}
          href={`/invoices/${invoice.id}`}
        >
          View
        </Link>
        <Link
          className={styles.secondaryButton}
          href={`/invoices/${invoice.id}/edit`}
        >
          Edit
        </Link>
        {canMarkPaid ? (
          <button
            className={styles.primaryButton}
            type="button"
            onClick={markAsPaid}
            disabled={isPending}
          >
            {isPending ? "Updating..." : "Mark as Paid"}
          </button>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
        ) : null}
        <button
          className={styles.dangerButton}
          type="button"
          onClick={() => {
            setError(null);
            setIsDeleteOpen(true);
          }}
          disabled={isPending}
        >
          Delete
        </button>
      </div>

      {error ? <p className={styles.error}>{error}</p> : null}

      {isDeleteOpen ? (
        <DeleteConfirmModal
          invoice={invoice}
          onClose={() => setIsDeleteOpen(false)}
        />
      ) : null}
    </>
  );
}
```

InvoiceForm.jsx

Found in folder: **/src/app/invoices**

This code provides a form for creating or editing an invoice. It manages form state for invoice details and a list of line items, including calculations for totals. It validates input, builds a payload, and submits it to create or update an invoice via server actions.

```
"use client";

import { useMemo, useState, useTransition } from "react";
import { useRouter } from "next/navigation";
import {
  createInvoiceAction,
  updateInvoiceAction,
} from "@/actions/invoiceActions";
import { formatCurrency } from "@/lib/formatters";
import styles from "./InvoiceForm.module.css";

const invoiceStatuses = ["draft", "sent", "paid", "overdue", "void"];

const emptyLineItem = {
  description: "",
  quantity: "1",
  unitPriceDollars: "",
};
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
// Generates a unique key for each line item
function nextLineItemKey() {
  if (typeof crypto !== "undefined" && crypto.randomUUID) {
    return crypto.randomUUID();
  }

  return `${Date.now()}-${Math.random()}`;
}

function createLineItem(overrides = {}) {
  return {
    key: nextLineItemKey(),
    ...emptyLineItem,
    ...overrides,
  };
}

// Converts cents (stored in DB) to a dollar string for display
function centsToDollars(cents) {
  if (cents === undefined || cents === null || cents === "") {
    return "";
  }

  return (Number(cents) / 100).toFixed(2);
}

// Converts user-entered dollar string to cents for storage
function dollarsToCents(value) {
  const text = String(value || "").trim();

  if (text === "") {
    return 0;
  }

  const normalizedText = text.replace(/[$£€,\s]/g, "");

  if (!/^\d+(\.\d{1,2})?$/.test(normalizedText)) {
    return Number.NaN;
  }

  return Math.round(Number(normalizedText) * 100);
}

function getTodayDate() {
  return new Date().toISOString().slice(0, 10);
}

// Determine initial client selection from invoice data or client list
function getInitialClientId(invoice, clients) {
  if (invoice?.client?.id) {
    return String(invoice.client.id);
  }

  return clients[0]?.id ? String(clients[0].id) : "";
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
// Determine initial project selection, filtered by selected client
function getInitialProjectId(invoice, projects, clientId) {
  if (invoice?.projectId) {
    return String(invoice.projectId);
  }

  const firstProject = projects.find(
    (project) => String(project.clientId) === String(clientId),
  );

  return firstProject?.id ? String(firstProject.id) : "";
}

// Converts invoice data from server into the form's line item structure
function invoiceToLineItems(invoice) {
  if (!invoice?.lineItems?.length) {
    return [createLineItem()];
  }

  return invoice.lineItems.map((item) =>
    createLineItem({
      description: item.description || "",
      quantity: String(item.quantity ?? "1"),
      unitPriceDollars: centsToDollars(item.unitPriceCents),
    }),
  );
}

// Calculates the total cost for a single line item in cents
function getLineTotalCents(item) {
  const quantity = Number(item.quantity || 0);
  const unitPriceCents = dollarsToCents(item.unitPriceDollars);

  if (!Number.isFinite(quantity) || Number.isNaN(unitPriceCents)) {
    return 0;
  }

  return Math.round(quantity * unitPriceCents);
}

export default function InvoiceForm({
  mode,
  invoice = null,
  clients,
  projects,
}) {
  const router = useRouter();
  const isEdit = mode === "edit";
  const initialClientId = getInitialClientId(invoice, clients);
  const [selectedClientId, setSelectedClientId] = useState(initialClientId);
  const [selectedProjectId, setSelectedProjectId] = useState(() =>
    getInitialProjectId(invoice, projects, initialClientId),
  );
  const [invoiceNumber, setInvoiceNumber] = useState(
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    invoice?.invoiceNumber || "",
  );
  const [issueDate, setIssueDate] = useState(
    invoice?.issueDate || getTodayDate(),
  );
  const [dueDate, setDueDate] = useState(invoice?.dueDate || "");
  const [status, setStatus] = useState(invoice?.status || "draft");
  const [lineItems, setLineItems] = useState(() => invoiceToLineItems(invoice));
  const [error, setError] = useState(null);
  const [isPending, startTransition] = useTransition();

  // Filter projects based on the currently selected client
  const availableProjects = useMemo(
    () =>
      projects.filter(
        (project) => String(project.clientId) === String(selectedClientId),
      ),
    [projects, selectedClientId],
  );
  // Calculate the total invoice amount from all line items
  const totalCents = useMemo(
    () =>
      lineItems.reduce((sum, item) => sum + getLineTotalCents(item), 0),
    [lineItems],
  );
  const hasClients = clients.length > 0;
  const hasProjects = availableProjects.length > 0;

  // Updates client and resets project selection if needed
  function updateClient(event) {
    const nextClientId = event.target.value;
    const nextProjects = projects.filter(
      (project) => String(project.clientId) === String(nextClientId),
    );
    const selectedProjectStillValid = nextProjects.some(
      (project) => String(project.id) === String(selectedProjectId),
    );

    setSelectedClientId(nextClientId);
    setSelectedProjectId(
      selectedProjectStillValid ? selectedProjectId : String(nextProjects[0]?.id || "
"),
    );
  }

  function updateLineItem(key, field, value) {
    setLineItems((current) =>
      current.map((item) =>
        item.key === key
          ? {
              ...item,
              [field]: value,
            }
          : item,
      ),
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    );
  }

  function addLineItem() {
    setLineItems((current) => [...current, createLineItem()]);
  }

  // Ensures at least one line item remains after removal
  function removeLineItem(key) {
    setLineItems((current) => {
      if (current.length === 1) {
        return [createLineItem()];
      }

      return current.filter((item) => item.key !== key);
    });
  }

  // Builds the data payload for the server action, validating line items
  function buildPayload() {
    const parsedLineItems = lineItems.map((item, index) => {
      const unitPriceCents = dollarsToCents(item.unitPriceDollars);
      const quantity = Number(item.quantity);

      if (Number.isNaN(unitPriceCents)) {
        throw new Error(`Line item ${index + 1} unit price must be a valid dollar amo
unt.`);
      }

      return {
        description: item.description,
        quantity,
        unitPriceCents,
      };
    });

    return {
      projectId: selectedProjectId,
      invoiceNumber,
      status,
      issueDate,
      dueDate,
      currency: invoice?.currency || "USD",
      paidCents: status === "paid" ? totalCents : 0,
      lineItems: parsedLineItems,
    };
  }

  function submitForm(event) {
    event.preventDefault();
    setError(null);

    let payload;

    try {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    payload = buildPayload();
  } catch (submissionError) {
    setError(submissionError.message);
    return;
  }

  startTransition(async () => {
    const result = isEdit
      ? await updateInvoiceAction(invoice.id, payload)
      : await createInvoiceAction(payload);

    if (result.error) {
      setError(result.error);
      return;
    }

    router.push("/invoices");
    router.refresh();
  });
}

function cancelForm() {
  router.push("/invoices");
}

return (
  <form className={styles.form} onSubmit={submitForm}>
    <div className={styles.panel}>
      <div className={styles.sectionHeader}>
        <div>
          <p className={styles.eyebrow}>Invoice</p>
          <h2 className={styles.sectionTitle}>Details</h2>
        </div>
      </div>

      <div className={styles.formGrid}>
        <label className={styles.field}>
          <span>Client</span>
          <select
            value={selectedClientId}
            onChange={updateClient}
            disabled={!hasClients || isPending}
            required
          >
            {hasClients ? null : <option value="">Add a client first</option>}
            {clients.map((client) => (
              <option key={client.id} value={client.id}>
                {client.name}
              </option>
            ))}
          </select>
        </label>

        <label className={styles.field}>
          <span>Project</span>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  <select
    value={selectedProjectId}
    onChange={(event) => setSelectedProjectId(event.target.value)}
    disabled={!hasProjects || isPending}
    required
  >
    {hasProjects ? null : (
      <option value="">No projects for this client</option>
    )}
    {availableProjects.map((project) => (
      <option key={project.id} value={project.id}>
        {project.name}
      </option>
    ))}
  </select>
</label>

<label className={styles.field}>
  <span>Invoice number</span>
  <input
    value={invoiceNumber}
    onChange={(event) => setInvoiceNumber(event.target.value)}
    disabled={isPending}
    required
  />
</label>

<label className={styles.field}>
  <span>Status</span>
  <select
    value={status}
    onChange={(event) => setStatus(event.target.value)}
    disabled={isPending}
  >
    {invoiceStatuses.map((invoiceStatus) => (
      <option key={invoiceStatus} value={invoiceStatus}>
        {invoiceStatus}
      </option>
    ))}
  </select>
</label>

<label className={styles.field}>
  <span>Issue date</span>
  <input
    type="date"
    value={issueDate}
    onChange={(event) => setIssueDate(event.target.value)}
    disabled={isPending}
    required
  />
</label>

<label className={styles.field}>
  <span>Due date</span>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
        <input
          type="date"
          value={dueDate}
          onChange={(event) => setDueDate(event.target.value)}
          disabled={isPending}
        />
      </label>
    </div>
  </div>

  <div className={styles.panel}>
    <div className={styles.sectionHeader}>
      <div>
        <p className={styles.eyebrow}>Line items</p>
        <h2 className={styles.sectionTitle}>Billable work</h2>
      </div>
      <button
        className={styles.secondaryButton}
        type="button"
        onClick={addLineItem}
        disabled={isPending}
      >
        Add Line Item
      </button>
    </div>

    <div className={styles.lineItems}>
      <div className={styles.lineItemHeader} aria-hidden="true">
        <span>Description</span>
        <span>Quantity</span>
        <span>Unit price</span>
        <span>Line total</span>
        <span>Actions</span>
      </div>

      {lineItems.map((item, index) => (
        <div className={styles.lineItemRow} key={item.key}>
          <label className={styles.lineField}>
            <span>Description</span>
            <input
              value={item.description}
              onChange={(event) =>
                updateLineItem(item.key, "description", event.target.value)
              }
              placeholder={`Line item ${index + 1}`}
              disabled={isPending}
              required
            />
          </label>

          <label className={styles.lineField}>
            <span>Quantity</span>
            <input
              type="number"
              min="1"
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
              step="1"
              value={item.quantity}
              onChange={(event) =>
                updateLineItem(item.key, "quantity", event.target.value)
              }
              disabled={isPending}
              required
            />
          </label>

          <label className={styles.lineField}>
            <span>Unit price</span>
            <input
              inputMode="decimal"
              min="0"
              placeholder="125.00"
              step="0.01"
              type="number"
              value={item.unitPriceDollars}
              onChange={(event) =>
                updateLineItem(
                  item.key,
                  "unitPriceDollars",
                  event.target.value,
                )
              }
              disabled={isPending}
              required
            />
          </label>

          <output className={styles.lineTotal}>
            <span>Line total</span>
            <strong>{formatCurrency(getLineTotalCents(item), "USD")}</strong>
          </output>

          <button
            className={styles.dangerButton}
            type="button"
            onClick={() => removeLineItem(item.key)}
            disabled={isPending}
          >
            Remove
          </button>
        </div>
      ))}
    </div>

    <div className={styles.totalRow}>
      <span>Total</span>
      <strong>{formatCurrency(totalCents, "USD")}</strong>
    </div>
  </div>

  {error ? <p className={styles.error}>{error}</p> : null}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      <div className={styles.formActions}>
        <button
          className={styles.primaryButton}
          type="submit"
          disabled={isPending || !hasClients || !hasProjects}
        >
          {isPending ? "Saving..." : isEdit ? "Save Invoice" : "Create Invoice"}
        </button>
        <button
          className={styles.secondaryButton}
          type="button"
          onClick={cancelForm}
          disabled={isPending}
        >
          Cancel
        </button>
      </div>
    </form>
  );
}
```

InvoicesTable.jsx

Found in folder: **/src/app/invoices**

This code renders a table of invoices with filtering, sorting, and search capabilities. It allows users to filter by status, search by client or invoice number, and sort by due date or total amount. The component also provides options to export filtered data and clear applied filters.

```
"use client";

import Link from "next/link";
import { useMemo, useState } from "react";
import StatusBadge from "@/components/StatusBadge";
import {
  formatCurrency,
  formatDate,
  formatInvoiceNumber,
} from "@/lib/formatters";
import InvoiceActions from "./InvoiceActions";
import styles from "./InvoicesPage.module.css";

// Available status options for filtering invoices
const statusFilters = [
  { label: "All", value: "all" },
  { label: "Draft", value: "draft" },
  { label: "Sent", value: "sent" },
  { label: "Paid", value: "paid" },
  { label: "Overdue", value: "overdue" },
];

// Normalizes search input for consistent comparison
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
function normalizeSearch(value) {
  return String(value || "").trim().toLowerCase();
}

// Sorts invoices by total amount or due date in ascending/descending order
function sortInvoices(invoices, sortBy, sortDirection) {
  return [...invoices].sort((first, second) => {
    const direction = sortDirection === "asc" ? 1 : -1;

    if (sortBy === "total") {
      return (first.totalCents - second.totalCents) * direction;
    }

    const firstDue = first.dueDate || "";
    const secondDue = second.dueDate || "";

    return firstDue.localeCompare(secondDue) * direction;
  });
}

export default function InvoicesTable({
  hasInvoices,
  invoices,
  initialFilters = {},
}) {
  // Initialize status filter from server-
side props if valid, otherwise default to "all"
  const initialStatus = statusFilters.some(
    (filter) => filter.value === initialFilters.status,
  )
    ? initialFilters.status
    : "all";
  const [statusFilter, setStatusFilter] = useState(initialStatus);
  const [searchTerm, setSearchTerm] = useState("");
  const [sortBy, setSortBy] = useState("dueDate");
  const [sortDirection, setSortDirection] = useState("asc");
  const hasServerFilters = Boolean(
    initialFilters.status || initialFilters.clientId || initialFilters.projectId,
  );

  function clearClientFilters() {
    setStatusFilter("all");
    setSearchTerm("");
  }

  // Generate export URL with current filters applied
  const exportHref = useMemo(() => {
    const params = new URLSearchParams();

    if (statusFilter !== "all") {
      params.set("status", statusFilter);
    }

    if (initialFilters.clientId) {
      params.set("clientId", initialFilters.clientId);
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    }

    if (initialFilters.projectId) {
      params.set("projectId", initialFilters.projectId);
    }

    const query = params.toString();

    return query.length > 0
      ? `/api/invoices/export?${query}`
      : "/api/invoices/export";
  }, [initialFilters.clientId, initialFilters.projectId, statusFilter]);

  // Filter and sort invoices based on current search term, status filter, and sort s
ettings
  const visibleInvoices = useMemo(() => {
    const query = normalizeSearch(searchTerm);
    const filteredInvoices = invoices.filter((invoice) => {
      const matchesStatus =
        statusFilter === "all" || invoice.status === statusFilter;
      const matchesSearch =
        query.length === 0 ||
        normalizeSearch(invoice.clientName).includes(query) ||
        normalizeSearch(invoice.invoiceNumber).includes(query) ||
        normalizeSearch(formatInvoiceNumber(invoice.invoiceNumber)).includes(
          query,
        );

      return matchesStatus && matchesSearch;
    });

    return sortInvoices(filteredInvoices, sortBy, sortDirection);
  }, [invoices, searchTerm, sortBy, sortDirection, statusFilter]);

  return (
    <div className={styles.panel}>
      <div className={styles.filterBar}>
        <div className={styles.statusFilters} aria-label="Invoice status">
          {statusFilters.map((filter) => (
            <button
              className={`${styles.filterButton} ${
                statusFilter === filter.value ? styles.activeFilter : ""
              }`}
              key={filter.value}
              onClick={() => setStatusFilter(filter.value)}
              type="button"
            >
              {filter.label}
            </button>
          ))}
        </div>

        <label className={styles.searchField}>
          <span>Search</span>
          <input
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
        onChange={(event) => setSearchTerm(event.target.value)}
        placeholder="Client or invoice number"
        type="search"
        value={searchTerm}
      />
    </label>

    <div className={styles.sortControls}>
      <label className={styles.selectField}>
        <span>Sort by</span>
        <select
          onChange={(event) => setSortBy(event.target.value)}
          value={sortBy}
        >
          <option value="dueDate">Due date</option>
          <option value="total">Total amount</option>
        </select>
      </label>

      <button
        className={styles.secondaryButton}
        onClick={() =>
          setSortDirection((current) =>
            current === "asc" ? "desc" : "asc",
          )
        }
        type="button"
      >
        {sortDirection === "asc" ? "Ascending" : "Descending"}
      </button>

      <a className={styles.exportButton} href={exportHref}>
        Export CSV
      </a>
    </div>
  </div>

  {!hasInvoices ? (
    <div className={styles.emptyState}>
      <p className={styles.emptyText}>
        No invoices yet. Create your first invoice to get started.
      </p>
      <Link className={styles.primaryButton} href="/invoices/new">
        Create Invoice
      </Link>
    </div>
  ) : visibleInvoices.length === 0 ? (
    <div className={styles.emptyState}>
      <p className={styles.emptyText}>
        No invoices match the current filter.
      </p>
      {hasServerFilters ? (
        <Link className={styles.secondaryButton} href="/invoices">
          Clear Filter
        </Link>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
        ) : (
          <button
            className={styles.secondaryButton}
            onClick={clearClientFilters}
            type="button"
          >
            Clear Filter
          </button>
        )}
      </div>
    ) : (
      <div className={styles.tableWrap}>
        <table className={styles.table}>
          <thead>
            <tr>
              <th scope="col">Invoice</th>
              <th scope="col">Client</th>
              <th scope="col">Project</th>
              <th scope="col">Issue date</th>
              <th scope="col">Due date</th>
              <th scope="col">Status</th>
              <th scope="col">Total</th>
              <th scope="col">Actions</th>
            </tr>
          </thead>
          <tbody>
            {visibleInvoices.map((invoice) => (
              <tr key={invoice.id}>
                <td data-label="Invoice">
                  <strong className={styles.invoiceNumber}>
                    {formatInvoiceNumber(invoice.invoiceNumber)}
                  </strong>
                </td>
                <td data-label="Client">{invoice.clientName}</td>
                <td data-label="Project">{invoice.projectName}</td>
                <td data-label="Issue date">
                  {formatDate(invoice.issueDate)}
                </td>
                <td data-label="Due date">{formatDate(invoice.dueDate)}</td>
                <td data-label="Status">
                  <StatusBadge status={invoice.status} />
                </td>
                <td data-label="Total">
                  {formatCurrency(invoice.totalCents, invoice.currency)}
                </td>
                <td data-label="Actions">
                  <InvoiceActions invoice={invoice} />
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    )}
  </div>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  );
}
```

ProjectInlineForm.jsx

Found in folder: **/src/app/projects**

This code provides a form to create a new project inline, handling input validation and submission. It toggles between showing an “Add Project” button and the form itself when clicked. The form validates the hourly rate, submits data via a server action, and refreshes the page on success.

```
"use client";

import { useRouter } from "next/navigation";
import { useState, useTransition } from "react";
import { createProjectAction } from "@/actions/projectActions";
import styles from "./ProjectsPage.module.css";

// Default empty state for the form fields
const emptyFormData = {
  clientId: "",
  name: "",
  description: "",
  hourlyRate: "",
};

// Converts a formatted hourly rate string (like "75.50") to cents (7550) for storage
, or returns NaN if invalid
function parseHourlyRateCents(value) {
  const text = String(value || "").trim();

  if (text === "") {
    return null; // Allow empty hourly rate
  }

  const normalizedText = text.replace(/[$£€,\s]/g, ""); // Remove currency symbols an
d formatting

  if (!/^\d+(\.\d{1,2})?$/.test(normalizedText)) {
    return Number.NaN; // Invalid format
  }

  return Math.round(Number(normalizedText) * 100); // Convert to cents
}

export default function ProjectInlineForm({ clients }) {
  const router = useRouter();
  const [isOpen, setIsOpen] = useState(false); // Controls whether the form is visibl
e
  const [formData, setFormData] = useState(emptyFormData);
  const [error, setError] = useState(null);
  const [isPending, startTransition] = useTransition();
  const hasClients = clients.length > 0; // Determines if a client can be selected
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
function openForm() {
  // Pre-populate clientId with the first client if available
  setFormData({
    ...emptyFormData,
    clientId: clients[0]?.id ? String(clients[0].id) : "",
  });
  setError(null);
  setIsOpen(true);
}

function closeForm() {
  setFormData(emptyFormData);
  setError(null);
  setIsOpen(false);
}

function updateField(event) {
  const { name, value } = event.target;
  setFormData((current) => ({
    ...current,
    [name]: value,
  }));
}

function submitForm(event) {
  event.preventDefault();
  setError(null);

  const hourlyRateCents = parseHourlyRateCents(formData.hourlyRate);

  if (Number.isNaN(hourlyRateCents)) {
    setError("Hourly rate must be a valid amount, such as 75 or 75.50.");
    return;
  }

  // Use transition to keep UI responsive during submission
  startTransition(async () => {
    const result = await createProjectAction({
      clientId: formData.clientId,
      name: formData.name,
      description: formData.description,
      hourlyRateCents,
    });

    if (result.error) {
      setError(result.error);
      return;
    }

    // Reset form and refresh page data on success
    setIsOpen(false);
    setFormData(emptyFormData);
    router.refresh();
  });
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
}

// Show button to open form when closed
if (!isOpen) {
  return (
    <button className={styles.primaryButton} type="button" onClick={openForm}>
      Add Project
    </button>
  );
}

return (
  <form className={styles.inlineForm} onSubmit={submitForm}>
    <div className={styles.formGrid}>
      <label className={styles.field}>
        <span>Client</span>
        <select
          name="clientId"
          required
          value={formData.clientId}
          onChange={updateField}
          disabled={!hasClients}
        >
          {hasClients ? null : (
            <option value="">Add a client first</option>
          )}
          {clients.map((client) => (
            <option key={client.id} value={client.id}>
              {client.name}
            </option>
          ))}
        </select>
      </label>
      <label className={styles.field}>
        <span>Project name</span>
        <input
          name="name"
          required
          value={formData.name}
          onChange={updateField}
        />
      </label>
      <label className={`${styles.field} ${styles.fullWidth}`}>
        <span>Description</span>
        <textarea
          name="description"
          rows="2"
          value={formData.description}
          onChange={updateField}
        />
      </label>
      <label className={styles.field}>
        <span>Hourly rate</span>
        <input
          name="hourlyRate"
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
            inputMode="decimal"
            placeholder="75.00"
            value={formData.hourlyRate}
            onChange={updateField}
          />
        </label>
      </div>

      {error ? <p className={styles.error}>{error}</p> : null}

      <div className={styles.formActions}>
        <button
          className={styles.primaryButton}
          type="submit"
          disabled={isPending || !hasClients}
        >
          {isPending ? "Saving..." : "Create"}
        </button>
        <button
          className={styles.secondaryButton}
          type="button"
          onClick={closeForm}
          disabled={isPending}
        >
          Cancel
        </button>
      </div>
    </form>
  );
}
```

Header.jsx

Found in folder: **/src/components**

This code renders a header component that displays a page title based on the current URL path, and includes a link to create new invoices. It uses a helper function to determine the title by matching specific route patterns. The header shows a branding subtitle and conditionally updates the main heading depending on which page the user is viewing.

```
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import styles from "./Header.module.css";

// Determines the appropriate page title based on the current route
function getPageTitle(pathname) {
  if (pathname === "/") {
    return "Dashboard";
  }

  if (pathname === "/clients/new") {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    return "New Client";
  }

  if (pathname.startsWith("/clients/") && pathname.endsWith("/edit")) {
    return "Edit Client";
  }

  if (pathname.startsWith("/clients/")) {
    return "Client Details";
  }

  if (pathname === "/clients") {
    return "Clients";
  }

  if (pathname === "/projects/new") {
    return "New Project";
  }

  if (pathname.startsWith("/projects/") && pathname.endsWith("/edit")) {
    return "Edit Project";
  }

  if (pathname.startsWith("/projects/")) {
    return "Project Details";
  }

  if (pathname === "/projects") {
    return "Projects";
  }

  if (pathname === "/invoices/new") {
    return "New Invoice";
  }

  if (pathname.startsWith("/invoices/") && pathname.endsWith("/edit")) {
    return "Edit Invoice";
  }

  if (pathname.startsWith("/invoices/")) {
    return "Invoice Details";
  }

  if (pathname === "/invoices") {
    return "Invoices";
  }

  // Fallback title for any unmatched routes
  return "Invoice Tracker";
}

export default function Header() {
  const pathname = usePathname();

  return (
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    <header className={styles.header}>
      <div>
        <p className={styles.eyebrow}>Freelancer Bookkeeping</p>
        <h1 className={styles.title}>{getPageTitle(pathname)}</h1>
      </div>

      {/* Always shows a link to create a new invoice */}
      <Link href="/invoices/new" className={styles.action}>
        New Invoice
      </Link>
    </header>
  );
}
```

Sidebar.jsx

Found in folder: **/src/components**

This code renders a sidebar navigation component for a web application. It defines a list of navigation links and highlights the active route based on the current URL path. The sidebar includes a branded header and accessible navigation menu.

```
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import styles from "./Sidebar.module.css";

// Navigation items for the sidebar
const navigationItems = [
  { href: "/", label: "Dashboard" },
  { href: "/clients", label: "Clients" },
  { href: "/projects", label: "Projects" },
  { href: "/invoices", label: "Invoices" },
];

// Determines if a navigation link corresponds to the current route
function isActiveRoute(pathname, href) {
  if (href === "/") {
    return pathname === "/";
  }

  // Active for exact match or nested routes under the href
  return pathname === href || pathname.startsWith(`${href}/`);
}

export default function Sidebar() {
  const pathname = usePathname();

  return (
    <aside className={styles.sidebar}>
      {/* Brand/logo link that navigates to the homepage */}
      <Link href="/" className={styles.brand} aria-label="Invoice Tracker home">
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
        <span className={styles.brandMark}>IT</span>
        <span className={styles.brandText}>Invoice Tracker</span>
      </Link>

      <nav className={styles.nav} aria-label="Primary navigation">
        {navigationItems.map((item) => {
          const isActive = isActiveRoute(pathname, item.href);

          return (
            <Link
              key={item.href}
              href={item.href}
              className={`${styles.navLink} ${isActive ? styles.active : ""}`}
              aria-current={isActive ? "page" : undefined}
            >
              {item.label}
            </Link>
          );
        })}
      </nav>
    </aside>
  );
}
```

StatusBadge.jsx

Found in folder: **/src/components**

This code maps status strings to color tones and displays them as styled badges with formatted labels. It normalizes input statuses to lowercase keys for consistent lookups and applies CSS classes for styling. The badge text is formatted by replacing hyphens or underscores with spaces and capitalizing each word.

```
import styles from "./StatusBadge.module.css";

// Maps status values to color tone names used in CSS classes
const statusToneMap = {
  draft: "grey",
  sent: "blue",
  paid: "green",
  overdue: "red",
  void: "darkGrey",
  active: "green",
  paused: "yellow",
  completed: "blue",
  archived: "grey",
};

// Formats status strings for display by replacing separators and capitalizing words
function formatStatusLabel(status) {
  return String(status)
    .replace(/[-_]/g, " ")
    .replace(/\b\w/g, (letter) => letter.toUpperCase());
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
}

export default function StatusBadge({ status }) {
  // Normalize status to lowercase for consistent mapping, default to empty string
  const normalizedStatus = String(status || "").toLowerCase();
  // Get tone from map, default to "grey" if status not found
  const tone = statusToneMap[normalizedStatus] || "grey";

  return (
    <span className={`${styles.badge} ${styles[tone]}`}>
      {formatStatusLabel(normalizedStatus)}
    </span>
  );
}
```

Dashboard.module.css

Found in folder: **/src/app**

This CSS file defines styles for a dashboard layout, including a summary grid, cards, panels, and a responsive list for overdue items. It uses flexbox and grid for layout, with specific typography, colors, and spacing for elements like headers, titles, and data values. The code includes media queries to adjust the layout for smaller screens, changing grid columns and font sizes.

```
.page {
  display: flex;
  flex-direction: column;
  gap: 24px;
}

.header,
.panelHeader {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 24px;
}

.eyebrow {
  margin: 0 0 8px;
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
}

.title {
  margin: 0;
  color: #172033;
  font-size: 2rem;
  line-height: 1.15;
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.sectionTitle {
  margin: 0;
  color: #172033;
  font-size: 1.25rem;
  line-height: 1.2;
}

.summaryGrid {
  display: grid;
  grid-template-
columns: repeat(4, minmax(0, 1fr)); /* Four equal columns by default */
  gap: 16px;
}

.summaryCard,
.panel {
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
}

.summaryCard {
  min-width: 0; /* Prevents flex/grid items from overflowing */
  padding: 18px;
}

.cardLabel {
  margin: 0 0 12px;
  color: #64748b;
  font-size: 0.82rem;
  font-weight: 700;
}

.cardValue {
  margin: 0;
  overflow-wrap: anywhere; /* Allows text to break anywhere to prevent overflow */
  color: #172033;
  font-size: 1.55rem;
  font-weight: 800;
  line-height: 1.15;
}

.dangerValue {
  color: #b91c1c; /* Red color for negative or critical values */
}

.panel {
  overflow: hidden; /* Contains child elements within rounded borders */
}

.panelHeader {
  border-bottom: 1px solid #e2e8f0;
  background: #f8fafc;
  padding: 18px;
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.chartFrame {
  height: 320px;
  padding: 18px 18px 12px;
}

.overdueList {
  display: flex;
  flex-direction: column;
}

.overdueRow {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto; /* Client info takes available space, m
eta aligns to end */
  gap: 16px;
  align-items: center;
  border-bottom: 1px solid #e2e8f0;
  color: inherit;
  padding: 16px 18px;
  text-decoration: none;
}

.overdueRow:last-child {
  border-bottom: 0; /* Removes border from the last row */
}

.overdueRow:hover {
  background: #f8fafc; /* Highlight row on hover */
}

.invoiceIdentity {
  display: flex;
  min-width: 0;
  flex-direction: column;
  gap: 4px;
}

.clientName {
  overflow: hidden;
  color: #172033;
  font-weight: 800;
  text-overflow: ellipsis;
  white-space: nowrap; /* Ensures single line with ellipsis for overflow */
}

.invoiceNumber {
  color: #64748b;
  font-size: 0.88rem;
  font-weight: 700;
}

.invoiceMeta {
  display: flex;
  align-items: center;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  gap: 18px;
  color: #64748b;
  font-size: 0.92rem;
  white-space: nowrap;
}

.invoiceMeta strong {
  color: #b91c1c; /* Highlights important numbers, like overdue amounts */
}

.emptyState {
  padding: 32px;
}

.emptyTitle {
  margin: 0 0 8px;
  color: #172033;
  font-size: 1.25rem;
}

.emptyText {
  margin: 0;
  color: #64748b;
}

@media (max-width: 1100px) {
  .summaryGrid {
    grid-template-
columns: repeat(2, minmax(0, 1fr)); /* Two columns on medium screens */
  }
}

@media (max-width: 700px) {
  .title {
    font-size: 1.65rem; /* Smaller title on mobile */
  }

  .summaryGrid {
    grid-template-columns: 1fr; /* Single column on small screens */
  }

  .overdueRow {
    grid-template-columns: 1fr; /* Stack invoice identity and meta vertically */
  }

  .invoiceMeta {
    justify-content: space-between; /* Spread out meta items on mobile */
  }
}
```

Error.module.css

Found in folder: **/src/app**

© 2026 Zenva Pty Ltd. All rights reserved.

This CSS defines styles for an error state component, including a container, message text, and a button. The container has a bordered card layout with padding and spacing, while the button includes hover effects for interactivity. These styles ensure visual consistency and user feedback within the interface.

```
.errorState {
  display: flex;
  max-width: 640px;
  flex-direction: column;
  align-items: flex-start;
  gap: 16px;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
  padding: 32px;
}

.message {
  margin: 0;
  color: #334155;
  font-size: 1rem;
  line-height: 1.5;
}

.button {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #334155;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
}

.button:hover {
  background: #f8fafc; /* Light background on hover for visual feedback */
}
```

globals.css

Found in folder: **/src/app**

This CSS file sets global styles for the project, ensuring consistent box-sizing and establishing a full-height layout. It defines a base color scheme and font family for the body, and resets margins for a clean start. Additionally, it ensures form elements inherit the font settings from the body for visual consistency.

```
/* Apply border-box sizing to all elements for predictable layout calculations */
* {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  box-sizing: border-box;
}

/* Ensure the html element always spans at least the full viewport height */
html {
  min-height: 100%;
}

/* Set base styles for the body: full height, no margin, background/foreground colors
, and default font stack */
body {
  min-height: 100%;
  margin: 0;
  background: #f1f5f9;
  color: #172033;
  font-family:
    Arial,
    Helvetica,
    sans-serif;
}

/* Make form elements inherit the font family and size from the body for consistency
*/
button,
input,
select,
textarea {
  font: inherit;
}
```

layout.module.css

Found in folder: **/src/app**

This CSS defines a responsive layout with a sidebar and main content area, adjusting for mobile screens and print media. It uses CSS Grid for the overall structure and Flexbox for inner content alignment. The print styles hide non-essential elements and optimize the page for printing.

```
.shell {
  display: grid;
  min-height: 100vh; /* Full viewport height */
  grid-template-columns: 260px minmax(0, 1fr); /* Sidebar and flexible main column */
  background: #f1f5f9;
}

.content {
  display: flex;
  min-width: 0; /* Prevents flex overflow issues */
  flex-direction: column;
}

.main {
  flex: 1; /* Takes remaining vertical space */
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  padding: 32px;
}

@media (max-width: 760px) {
  .shell {
    grid-template-columns: 1fr; /* Collapses to single column on mobile */
  }

  .main {
    padding: 20px; /* Reduced padding for smaller screens */
  }
}

@media print {
  .shell {
    display: block;
    min-height: auto;
    background: #ffffff; /* White background for printing */
  }

  .shell > aside,
  .content > header {
    display: none; /* Hide sidebar and header when printing */
  }

  .content {
    display: block;
  }

  .main {
    padding: 0; /* Remove padding for print layout */
  }
}
```

ClientsPage.module.css

Found in folder: **/src/app/clients**

This CSS file defines styles for a responsive web interface, including layout containers, a data table, form elements, and interactive buttons. It uses media queries to adapt the layout for mobile and tablet screens, converting the table into a stacked list view. The styles also manage visual states like hover effects, disabled buttons, and form validation errors.

```
.page {
  display: flex;
  flex-direction: column;
  gap: 24px;
}

.header {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  gap: 24px;
}

.eyebrow {
  margin: 0 0 8px;
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
}

.title {
  margin: 0;
  color: #172033;
  font-size: 2rem;
  line-height: 1.15;
}

.panel {
  overflow: hidden;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
}

.tableWrap {
  overflow-x: auto; /* Allows horizontal scrolling for wide tables */
}

.table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.95rem;
}

.table th,
.table td {
  border-bottom: 1px solid #e2e8f0;
  padding: 16px;
  text-align: left;
  vertical-align: top;
}

.table th {
  background: #f8fafc;
  color: #526071;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
  white-space: nowrap;
}

.table tr:last-child td {
  border-bottom: 0;
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.clientName {
  color: #172033;
}

.projectCount {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  white-space: nowrap;
}

.primaryButton,
.secondaryButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid transparent;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
  text-decoration: none;
  white-space: nowrap;
}

.primaryButton {
  background: #1d4ed8;
  color: #ffffff;
}

.primaryButton:hover {
  background: #1e40af;
}

.secondaryButton {
  border-color: #cbd5e1;
  background: #ffffff;
  color: #334155;
}

.secondaryButton:hover {
  background: #f8fafc;
}

.primaryButton:disabled,
.secondaryButton:disabled {
  cursor: not-allowed;
  opacity: 0.65;
}

.inlineForm {
  min-width: min(640px, calc(100vw - 64px));
  border: 1px solid #d9e1ec;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  border-radius: 8px;
  background: #f8fafc;
  padding: 16px;
}

.editForm {
  min-width: min(520px, calc(100vw - 96px));
}

.formGrid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 14px;
}

.field {
  display: flex;
  min-width: 0;
  flex-direction: column;
  gap: 6px;
  color: #526071;
  font-size: 0.82rem;
  font-weight: 700;
}

.field input,
.field textarea {
  width: 100%;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #172033;
  padding: 10px 12px;
}

.field textarea {
  resize: vertical;
}

.field input:focus,
.field textarea:focus {
  border-color: #1d4ed8;
  outline: 2px solid #bfdbfe;
  outline-offset: 1px;
}

.fullWidth {
  grid-column: 1 / -1; /* Spans across all columns in the form grid */
}

.formActions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin-top: 14px;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
}

.error {
  margin: 12px 0 0;
  border-radius: 8px;
  background: #fee2e2;
  color: #991b1b;
  font-size: 0.9rem;
  font-weight: 700;
  padding: 10px 12px;
}

.emptyState {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 16px;
  padding: 32px;
}

.emptyText {
  margin: 0;
  color: #64748b;
}

/* Responsive styles for tablet screens */
@media (max-width: 860px) {
  .header {
    flex-direction: column;
  }

  /* Converts the table into a block layout for mobile */
  .table,
  .table thead,
  .table tbody,
  .table th,
  .table td,
  .table tr {
    display: block;
  }

  .table thead {
    display: none;
  }

  .table tr {
    border-bottom: 1px solid #e2e8f0;
    padding: 14px 16px;
  }

  .table tr:last-child {
    border-bottom: 0;
  }

  .table td {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    display: grid;
    grid-template-columns: minmax(120px, 42%) minmax(0, 1fr);
    gap: 12px;
    border-bottom: 0;
    padding: 8px 0;
  }

  /* Uses data-label attribute to show column headers */
  .table td::before {
    content: attr(data-label);
    color: #64748b;
    font-size: 0.78rem;
    font-weight: 700;
    text-transform: uppercase;
  }

  .inlineForm,
  .editForm {
    min-width: 0;
    width: 100%;
  }
}

/* Responsive styles for mobile screens */
@media (max-width: 560px) {
  .title {
    font-size: 1.65rem;
  }

  .formGrid {
    grid-template-columns: 1fr; /* Single column layout */
  }

  .table td {
    grid-template-columns: 1fr;
    gap: 6px;
  }

  .primaryButton,
  .secondaryButton {
    width: 100%; /* Full-width buttons on small screens */
  }
}
```

Error.module.css

Found in folder: **/src/app/clients**

This CSS defines styles for an error state container, its message text, and a button. The error state is a bordered box with internal spacing, while the button has interactive hover effects. The styles focus on layout, color, and visual feedback.

```
.errorState {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 16px;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
  padding: 32px;
}

.message {
  margin: 0;
  color: #334155;
  font-size: 1rem;
  line-height: 1.5;
}

.button {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #334155;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
}

.button:hover {
  background: #f8fafc; /* Light background on hover for visual feedback */
}
```

DeleteConfirmModal.module.css

Found in folder: **/src/app/invoices**

This CSS styles a modal component with a backdrop, content area, title, message, action buttons, and error display. It defines visual properties like colors, spacing, and hover states for interactive elements. The styles ensure the modal is centered with a semi-transparent overlay and includes responsive width constraints.

```
.backdrop {
  position: fixed;
  z-index: 50;
  inset: 0; /* Covers entire viewport */
  display: flex;
  align-items: center;
  justify-content: center;
  background: rgb(15 23 42 / 0.52); /* Semi-transparent overlay */
  padding: 24px;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
}

.modal {
  width: min(420px, 100%); /* Responsive width, max 420px */
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
  box-shadow: 0 24px 48px rgb(15 23 42 / 0.22);
  padding: 20px;
}

.title {
  margin: 0;
  color: #172033;
  font-size: 1.2rem;
  line-height: 1.3;
}

.message {
  margin: 12px 0 0;
  color: #334155;
  line-height: 1.5;
}

.actions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  justify-content: flex-end; /* Aligns buttons to the right */
  margin-top: 20px;
}

.dangerButton,
.secondaryButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid transparent;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
  white-space: nowrap; /* Prevents text wrapping */
}

.dangerButton {
  background: #b91c1c;
  color: #ffffff;
}

.dangerButton:hover {
  background: #991b1b;
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.secondaryButton {
  border-color: #cbd5e1;
  background: #ffffff;
  color: #334155;
}

.secondaryButton:hover {
  background: #f8fafc;
}

.dangerButton:disabled,
.secondaryButton:disabled {
  cursor: not-allowed;
  opacity: 0.65;
}

.error {
  margin: 12px 0 0;
  border-radius: 8px;
  background: #fee2e2;
  color: #991b1b;
  font-size: 0.9rem;
  font-weight: 700;
  padding: 10px 12px;
}
```

Error.module.css

Found in folder: **/src/app/invoices**

This CSS defines styles for an error state container, its message text, and a button. The `.errorState` class creates a bordered box with internal spacing, while `.button` styles an interactive element with hover effects. The `.message` class sets typography for text within the error container.

```
.errorState {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 16px;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
  padding: 32px;
}

.message {
  margin: 0;
  color: #334155;
  font-size: 1rem;
  line-height: 1.5;
}

.button {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #334155;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
}

.button:hover {
  background: #f8fafc; /* Light background on hover for visual feedback */
}
```

InvoiceActions.module.css

Found in folder: **/src/app/invoices**

This CSS file defines styles for action buttons and error messages. It sets up a flex container for button groups and creates four button types with distinct colors and hover effects. It also includes a disabled state for all buttons and a styled error message box.

```
.actions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px; /* Spacing between buttons in the container */
}

.primaryButton,
.secondaryButton,
.tertiaryButton,
.dangerButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid transparent;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
  text-decoration: none;
  white-space: nowrap;
}

.primaryButton {
  background: #1d4ed8;
  color: #ffffff;
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.primaryButton:hover {
  background: #1e40af; /* Darker blue on hover */
}

.secondaryButton {
  border-color: #cbd5e1;
  background: #ffffff;
  color: #334155;
}

.secondaryButton:hover {
  background: #eef2f7; /* Light gray background on hover */
}

.tertiaryButton {
  border-color: #e2e8f0;
  background: #ffffff;
  color: #64748b;
}

.tertiaryButton:hover {
  background: #f8fafc;
  color: #334155; /* Darker text color on hover */
}

.dangerButton {
  background: #b91c1c;
  color: #ffffff;
}

.dangerButton:hover {
  background: #991b1b; /* Darker red on hover */
}

.primaryButton:disabled,
.secondaryButton:disabled,
.tertiaryButton:disabled,
.dangerButton:disabled {
  cursor: not-allowed;
  opacity: 0.65; /* Visual indication for disabled state */
}

.error {
  margin: 10px 0 0;
  border-radius: 8px;
  background: #fee2e2; /* Light red background */
  color: #991b1b;
  font-size: 0.9rem;
  font-weight: 700;
  padding: 10px 12px;
}
```

InvoiceForm.module.css

© 2026 Zenva Pty Ltd. All rights reserved.

Found in folder: **/src/app/invoices**

This CSS defines the visual layout and styling for a form interface, including sections, input fields, line items, and buttons. It styles form elements with specific colors, borders, and spacing, and includes responsive design rules for different screen sizes. The code ensures the form is accessible with focus states and disabled states for interactive elements.

```
.form {
  display: flex;
  max-width: 1120px;
  flex-direction: column;
  gap: 20px;
}

.panel {
  overflow: hidden;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
}

.sectionHeader {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  border-bottom: 1px solid #e2e8f0;
  background: #f8fafc;
  padding: 18px 20px;
}

.eyebrow {
  margin: 0 0 6px;
  color: #64748b;
  font-size: 0.75rem;
  font-weight: 700;
  text-transform: uppercase;
}

.sectionTitle {
  margin: 0;
  color: #172033;
  font-size: 1.2rem;
  line-height: 1.2;
}

.formGrid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr)); /* Two-column grid */
  gap: 18px;
  padding: 20px;
}

.field,
.lineField {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  display: flex;
  min-width: 0;
  flex-direction: column;
  gap: 7px;
  color: #526071;
  font-size: 0.82rem;
  font-weight: 700;
}

/* Styling for input and select elements */
.field input,
.field select,
.lineField input {
  width: 100%;
  min-height: 42px;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #172033;
  padding: 8px 12px;
}

.field input:focus,
.field select:focus,
.lineField input:focus {
  border-color: #1d4ed8;
  outline: 2px solid #bfdbfe;
  outline-offset: 1px;
}

.field input:disabled,
.field select:disabled,
.lineField input:disabled {
  background: #f1f5f9;
  color: #64748b;
  cursor: not-allowed;
}

.lineItems {
  display: flex;
  flex-direction: column;
}

/* Grid layout for line item headers and rows */
.lineItemHeader,
.lineItemRow {
  display: grid;
  grid-template-columns:
    minmax(220px, 1.6fr)
    minmax(110px, 0.55fr)
    minmax(140px, 0.65fr)
    minmax(130px, 0.55fr)
    minmax(96px, auto);
  gap: 12px;
  align-items: end;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
}

.lineItemHeader {
  border-bottom: 1px solid #e2e8f0;
  color: #64748b;
  font-size: 0.76rem;
  font-weight: 700;
  padding: 12px 20px;
  text-transform: uppercase;
}

.lineItemRow {
  border-bottom: 1px solid #e2e8f0;
  padding: 16px 20px;
}

.lineItemRow:last-child {
  border-bottom: 0;
}

.lineField span,
.lineTotal span {
  display: none; /* Hidden labels for desktop view */
}

.lineTotal {
  display: flex;
  min-height: 42px;
  align-items: center;
  color: #172033;
  font-size: 0.95rem;
}

.totalRow {
  display: flex;
  align-items: center;
  justify-content: flex-end;
  gap: 24px;
  border-top: 1px solid #d9e1ec;
  background: #f8fafc;
  color: #172033;
  font-size: 1.05rem;
  padding: 18px 20px;
}

.totalRow span {
  color: #526071;
  font-size: 0.82rem;
  font-weight: 700;
  text-transform: uppercase;
}

/* Base button styles */
.primaryButton,
.secondaryButton,
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.dangerButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
  text-decoration: none;
  white-space: nowrap;
}

.primaryButton {
  border-color: #1d4ed8;
  background: #1d4ed8;
  color: #ffffff;
}

.primaryButton:hover {
  background: #1e40af;
}

.secondaryButton {
  background: #ffffff;
  color: #334155;
}

.secondaryButton:hover {
  background: #eef2f7;
}

.dangerButton {
  border-color: #fecaca;
  background: #ffffff;
  color: #991b1b;
}

.dangerButton:hover {
  background: #fef2f2;
}

.primaryButton:disabled,
.secondaryButton:disabled,
.dangerButton:disabled {
  cursor: not-allowed;
  opacity: 0.65;
}

.error {
  margin: 0;
  border: 1px solid #fecaca;
  border-radius: 8px;
  background: #fef2f2;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  color: #991b1b;
  font-weight: 700;
  padding: 12px 14px;
}

.formActions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}

/* Responsive design for tablets and smaller screens */
@media (max-width: 980px) {
  .lineItemHeader {
    display: none; /* Hide header on smaller screens */
  }

  .lineItemRow {
    grid-template-columns: repeat(2, minmax(0, 1fr)); /* Switch to two-
column layout */
    align-items: stretch;
  }

  .lineField span,
  .lineTotal span {
    display: inline; /* Show labels for mobile view */
  }

  .lineTotal {
    flex-direction: column;
    align-items: flex-start;
    gap: 7px;
    color: #526071;
    font-size: 0.82rem;
    font-weight: 700;
  }

  .lineTotal strong {
    color: #172033;
    font-size: 0.95rem;
  }
}

/* Responsive design for mobile screens */
@media (max-width: 680px) {
  .formGrid,
  .lineItemRow {
    grid-template-columns: 1fr; /* Single column layout */
  }

  .sectionHeader,
  .formActions {
    align-items: stretch;
    flex-direction: column;
  }
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  .totalRow {
    justify-content: space-between;
  }
}
```

InvoicesPage.module.css

Found in folder: **/src/app/invoices**

This CSS styles a responsive dashboard interface with a filter bar, data table, and action buttons. It defines layout, colors, and interactive states for components like buttons, search fields, and a table that adapts for mobile. Media queries adjust the grid and table into stacked, single-column layouts on smaller screens.

```
.page {
  display: flex;
  flex-direction: column;
  gap: 24px;
}

.header {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 24px;
}

.eyebrow {
  margin: 0 0 8px;
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
}

.title {
  margin: 0;
  color: #172033;
  font-size: 2rem;
  line-height: 1.15;
}

.panel {
  overflow: hidden;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
}

.filterBar {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(220px, 300px) auto;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  gap: 16px;
  align-items: end;
  border-bottom: 1px solid #e2e8f0;
  background: #f8fafc;
  padding: 16px;
}

.statusFilters {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

/* Base styles for all button variants */
.filterButton,
.primaryButton,
.secondaryButton,
.exportButton,
.dangerButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #334155;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
  text-decoration: none;
  white-space: nowrap;
}

.primaryButton {
  border-color: #1d4ed8;
  background: #1d4ed8;
  color: #ffffff;
}

.primaryButton:hover {
  background: #1e40af;
}

.filterButton:hover,
.secondaryButton:hover,
.exportButton:hover {
  background: #eef2f7;
}

/* Style for an active filter button */
.activeFilter {
  border-color: #1d4ed8;
  background: #1d4ed8;
  color: #ffffff;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
}

.activeFilter:hover {
  background: #1e40af;
}

.dangerButton {
  border-color: #fecaca;
  color: #991b1b;
}

.dangerButton:hover {
  background: #fef2f2;
}

.secondaryButton:disabled,
.dangerButton:disabled {
  cursor: not-allowed;
  opacity: 0.65;
}

.searchField,
.selectField {
  display: flex;
  min-width: 0;
  flex-direction: column;
  gap: 6px;
  color: #526071;
  font-size: 0.82rem;
  font-weight: 700;
}

.searchField input,
.selectField select {
  width: 100%;
  min-height: 40px;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #172033;
  padding: 8px 12px;
}

.searchField input:focus,
.selectField select:focus {
  border-color: #1d4ed8;
  outline: 2px solid #bfdbfe;
  outline-offset: 1px;
}

.sortControls {
  display: flex;
  gap: 10px;
  align-items: end;
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.tableWrap {
  overflow-x: auto; /* Allows horizontal scrolling on small screens */
}

.table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.95rem;
}

.table th,
.table td {
  border-bottom: 1px solid #e2e8f0;
  padding: 16px;
  text-align: left;
  vertical-align: top;
}

.table th {
  color: #526071;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
  white-space: nowrap;
}

.table tr:last-child td {
  border-bottom: 0;
}

.invoiceNumber {
  color: #172033;
  white-space: nowrap;
}

.rowActions {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

.emptyState {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 16px;
  padding: 32px;
}

.emptyText {
  margin: 0;
  color: #64748b;
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
/* Responsive layout adjustments */
@media (max-width: 980px) {
  .filterBar {
    grid-template-columns: 1fr; /* Stack filter bar sections */
  }

  .sortControls {
    flex-wrap: wrap;
  }
}

@media (max-width: 860px) {
  .header {
    flex-direction: column;
  }

  /* Convert table to a block layout for mobile */
  .table,
  .table thead,
  .table tbody,
  .table th,
  .table td,
  .table tr {
    display: block;
  }

  .table thead {
    display: none; /* Hide table headers */
  }

  .table tr {
    border-bottom: 1px solid #e2e8f0;
    padding: 14px 16px;
  }

  .table tr:last-child {
    border-bottom: 0;
  }

  .table td {
    display: grid;
    grid-template-columns: minmax(120px, 42%) minmax(0, 1fr);
    gap: 12px;
    border-bottom: 0;
    padding: 8px 0;
  }

  /* Use data-label attribute to show column headers */
  .table td::before {
    content: attr(data-label);
    color: #64748b;
    font-size: 0.78rem;
    font-weight: 700;
    text-transform: uppercase;
  }
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
}

@media (max-width: 560px) {
  .title {
    font-size: 1.65rem;
  }

  /* Stack button groups vertically on very small screens */
  .statusFilters,
  .sortControls,
  .rowActions {
    align-items: stretch;
    flex-direction: column;
  }
}
```

InvoiceDetailPage.module.css

Found in folder: **/src/app/invoices/[invoiceId]**

This CSS styles an invoice document with a responsive layout, defining visual styles for elements like buttons, headers, client information, line items, and totals. It includes interactive hover states for buttons and links, and adjusts the layout for different screen sizes using media queries. For printing, it hides interactive elements and optimizes the document’s appearance with black-and-white styling and proper page breaks.

```
.page {
  display: flex;
  flex-direction: column;
  gap: 18px;
}

.actions {
  display: flex;
  flex-wrap: wrap;
  justify-content: flex-end;
  gap: 10px;
}

/* Base styles for all button and link variants */
.primaryLink,
.secondaryLink,
.primaryButton,
.secondaryButton,
.dangerButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  font-weight: 700;
  padding: 0 14px;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  text-decoration: none;
  white-space: nowrap;
}

/* Primary button/link styling */
.primaryButton,
.primaryLink {
  border-color: #1d4ed8;
  background: #1d4ed8;
  color: #ffffff;
}

.primaryButton:hover,
.primaryLink:hover {
  background: #1e40af;
}

/* Secondary button/link styling */
.secondaryButton,
.secondaryLink {
  background: #ffffff;
  color: #334155;
}

.secondaryButton:hover,
.secondaryLink:hover {
  background: #eef2f7;
}

/* Danger button styling */
.dangerButton {
  border-color: #fecaca;
  background: #ffffff;
  color: #991b1b;
}

.dangerButton:hover {
  background: #fef2f2;
}

/* Ensure buttons show pointer cursor */
.primaryButton,
.secondaryButton,
.dangerButton {
  cursor: pointer;
}

/* Main invoice document container */
.document {
  width: min(100%, 980px);
  margin: 0 auto;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
  color: #172033;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  padding: 56px;
  box-shadow: 0 24px 60px rgba(15, 23, 42, 0.08);
}

/* Header section with brand and invoice metadata */
.documentHeader {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  gap: 48px;
  align-items: start;
  border-bottom: 2px solid #172033;
  padding-bottom: 36px;
}

.brandLabel,
.invoiceNumber,
.clientName,
.projectName,
.amountDue strong {
  color: #111827;
}

.brandLabel {
  margin: 0;
  font-size: 2.25rem;
  font-weight: 800;
  line-height: 1.05;
}

.tagline {
  margin: 12px 0 0;
  color: #64748b;
  font-size: 1rem;
}

.invoiceMeta {
  min-width: 250px;
  text-align: right;
}

.invoiceLabel {
  margin: 0 0 6px;
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 800;
  letter-spacing: 0;
  text-transform: uppercase;
}

.invoiceNumber {
  margin: 0 0 22px;
  font-size: 2rem;
  line-height: 1.1;
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.metaList {
  display: grid;
  gap: 10px;
  margin: 0;
}

.metaList div {
  display: grid;
  grid-template-columns: 95px minmax(0, 1fr);
  gap: 16px;
  align-items: center;
}

.metaList dt {
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 800;
  text-transform: uppercase;
}

.metaList dd {
  margin: 0;
  font-weight: 700;
}

/* Client and project information sections */
.parties {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(280px, 0.8fr);
  gap: 48px;
  padding: 44px 0;
}

.billTo,
.project,
.payment,
.notes {
  min-width: 0;
}

.billTo h2,
.project h2,
.payment h2,
.notes h2,
.sectionTitle {
  margin: 0 0 14px;
  color: #64748b;
  font-size: 0.82rem;
  font-weight: 800;
  letter-spacing: 0;
  text-transform: uppercase;
}

.clientBlock {
  color: #475569;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  line-height: 1.6;
}

.clientName,
.projectName {
  margin: 0 0 6px;
  font-size: 1.2rem;
  font-weight: 800;
  line-height: 1.3;
}

.contactLine,
.address,
.projectNote {
  margin: 0;
}

.address {
  max-width: 34rem;
  white-space: pre-line;
}

.projectNote {
  color: #475569;
  line-height: 1.6;
}

/* Line items table section */
.lineItems {
  border-top: 1px solid #e2e8f0;
  padding-top: 34px;
}

.tableWrap {
  overflow-x: auto;
}

.table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.96rem;
}

.table th,
.table td {
  border-bottom: 1px solid #e2e8f0;
  padding: 17px 0;
  text-align: left;
  vertical-align: top;
}

.table thead th {
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 800;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  text-transform: uppercase;
}

/* Align numeric columns to the right */
.table th:nth-child(n + 2),
.table td:nth-child(n + 2),
.table tfoot th,
.table tfoot td {
  text-align: right;
}

.table tbody td:first-child {
  width: 52%;
  color: #172033;
  font-weight: 600;
}

.table tfoot th,
.table tfoot td {
  border-bottom: 0;
  padding: 14px 0 0;
}

.table tfoot th {
  color: #475569;
  font-weight: 700;
}

.totalRow th,
.totalRow td {
  color: #111827;
  font-size: 1.1rem;
  font-weight: 800;
}

/* Payment details section */
.payment {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  gap: 32px;
  align-items: end;
  border-top: 2px solid #172033;
  margin-top: 34px;
  padding-top: 30px;
}

.paymentDueDate {
  margin: 0;
  color: #475569;
}

.amountDue {
  min-width: 260px;
  text-align: right;
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.amountDue p {
  margin: 0 0 8px;
  color: #64748b;
  font-size: 0.82rem;
  font-weight: 800;
  text-transform: uppercase;
}

.amountDue strong {
  display: block;
  font-size: 2rem;
  line-height: 1.1;
}

/* Notes section */
.notes {
  border-top: 1px solid #e2e8f0;
  margin-top: 32px;
  padding-top: 28px;
}

.notes p {
  margin: 0;
  color: #475569;
  line-height: 1.65;
  white-space: pre-line;
}

/* Responsive adjustments for medium screens */
@media (max-width: 820px) {
  .document {
    padding: 32px;
  }

  /* Switch grid layouts to single column */
  .documentHeader,
  .parties,
  .payment {
    grid-template-columns: 1fr;
  }

  .invoiceMeta,
  .amountDue {
    min-width: 0;
    text-align: left;
  }

  .metaList div {
    grid-template-columns: 105px minmax(0, 1fr);
  }
}

/* Responsive adjustments for small screens */
@media (max-width: 560px) {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  .actions {
    flex-direction: column-reverse;
  }

  .primaryLink,
  .secondaryLink {
    width: 100%;
  }

  .document {
    padding: 24px;
  }

  .brandLabel,
  .invoiceNumber,
  .amountDue strong {
    font-size: 1.65rem;
  }

  .documentHeader {
    gap: 28px;
  }

  .parties {
    gap: 30px;
    padding: 34px 0;
  }

  /* Ensure table remains scrollable on very small screens */
  .table {
    min-width: 620px;
  }
}

/* Print-specific styles */
@media print {
  @page {
    margin: 15mm;
  }

  .actions {
    display: none;
  }

  .page {
    display: block;
  }

  .document {
    width: 100%;
    max-width: none;
    border: 0;
    border-radius: 0;
    background: #ffffff;
    color: #000000;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    box-shadow: none;
    padding: 0;
  }

  /* Force all elements to print in black and white */
  .document *,
  .document *::before,
  .document *::after {
    background: #ffffff;
    color: #000000;
    box-shadow: none;
  }

  .documentHeader,
  .lineItems,
  .payment,
  .notes {
    border-color: #000000;
  }

  .tableWrap {
    overflow: visible;
  }

  .table {
    border: 1px solid #000000;
    page-break-inside: auto;
  }

  .table th,
  .table td {
    border: 1px solid #000000;
    padding: 10px;
  }

  /* Prevent page breaks inside key elements */
  .table tr,
  .table tbody,
  .table tfoot {
    break-inside: avoid;
    page-break-inside: avoid;
  }

  .payment {
    break-inside: avoid;
    page-break-inside: avoid;
  }
}
```

Error.module.css

Found in folder: **/src/app/projects**

© 2026 Zenva Pty Ltd. All rights reserved.

This CSS defines styles for an error state component, including a container, a message, and a button. The container is a white bordered box with internal spacing, while the button changes background color on hover.

```
.errorState {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 16px;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
  padding: 32px;
}

.message {
  margin: 0;
  color: #334155;
  font-size: 1rem;
  line-height: 1.5;
}

.button {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #334155;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
}

.button:hover {
  background: #f8fafc; /* Subtle hover effect for interactivity */
}
```

ProjectsPage.module.css

Found in folder: **/src/app/projects**

This CSS styles a client dashboard page with a responsive layout, including a header, client sections with tables, and form components. It defines visual styles for elements like buttons, tables, and form fields, with hover states and disabled states. The code includes responsive breakpoints that adjust the layout for tablets and mobile screens.

```
.page {
  display: flex;
  flex-direction: column;
  gap: 24px;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
}

.header {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 24px;
}

.eyebrow {
  margin: 0 0 8px;
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
}

.title {
  margin: 0;
  color: #172033;
  font-size: 2rem;
  line-height: 1.15;
}

.panel {
  overflow: hidden;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
}

.clientSections {
  display: flex;
  flex-direction: column;
}

.clientSection {
  border-bottom: 1px solid #d9e1ec;
}

.clientSection:last-child {
  border-bottom: 0; /* Removes border from the last section */
}

.clientHeader {
  border-bottom: 1px solid #e2e8f0;
  background: #f8fafc;
  padding: 16px;
}

.clientName {
  margin: 0;
  color: #172033;
  font-size: 1rem;
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.tableWrap {
  overflow-x: auto; /* Allows horizontal scrolling on small screens */
}

.table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.95rem;
}

.table th,
.table td {
  border-bottom: 1px solid #e2e8f0;
  padding: 16px;
  text-align: left;
  vertical-align: top;
}

.table th {
  color: #526071;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
  white-space: nowrap;
}

.table tr:last-child td {
  border-bottom: 0; /* Removes border from the last row */
}

.projectSummary {
  display: flex;
  min-width: 0;
  flex-direction: column;
  gap: 4px;
}

.projectName {
  color: #172033;
}

.projectDescription {
  color: #64748b;
  font-size: 0.86rem;
  line-height: 1.45;
}

.invoiceLink {
  color: #1d4ed8;
  font-weight: 700;
  text-decoration: none;
  white-space: nowrap;
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.invoiceLink:hover {
  color: #1e40af;
  text-decoration: underline;
}

.primaryButton,
.secondaryButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid transparent;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
  text-decoration: none;
  white-space: nowrap;
}

.primaryButton {
  background: #1d4ed8;
  color: #ffffff;
}

.primaryButton:hover {
  background: #1e40af;
}

.secondaryButton {
  border-color: #cbd5e1;
  background: #ffffff;
  color: #334155;
}

.secondaryButton:hover {
  background: #f8fafc;
}

.primaryButton:disabled,
.secondaryButton:disabled {
  cursor: not-allowed;
  opacity: 0.65;
}

.inlineForm {
  min-width: min(640px, calc(100vw - 64px)); /* Responsive width with a max */
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #f8fafc;
  padding: 16px;
}

.formGrid {
  display: grid;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 14px;
}

.field {
  display: flex;
  min-width: 0;
  flex-direction: column;
  gap: 6px;
  color: #526071;
  font-size: 0.82rem;
  font-weight: 700;
}

.field input,
.field select,
.field textarea {
  width: 100%;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #172033;
  padding: 10px 12px;
}

.field textarea {
  resize: vertical;
}

.field input:focus,
.field select:focus,
.field textarea:focus {
  border-color: #1d4ed8;
  outline: 2px solid #bfdbfe;
  outline-offset: 1px;
}

.fullWidth {
  grid-column: 1 / -1; /* Spans across all columns in the grid */
}

.formActions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin-top: 14px;
}

.error {
  margin: 12px 0 0;
  border-radius: 8px;
  background: #fee2e2;
  color: #991b1b;
  font-size: 0.9rem;
  font-weight: 700;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  padding: 10px 12px;
}

.emptyState {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 16px;
  padding: 32px;
}

.emptyText {
  margin: 0;
  color: #64748b;
}

@media (max-width: 860px) {
  .header {
    flex-direction: column;
  }

  /* Converts table to a block layout for mobile */
  .table,
  .table thead,
  .table tbody,
  .table th,
  .table td,
  .table tr {
    display: block;
  }

  .table thead {
    display: none;
  }

  .table tr {
    border-bottom: 1px solid #e2e8f0;
    padding: 14px 16px;
  }

  .table tr:last-child {
    border-bottom: 0;
  }

  .table td {
    display: grid;
    grid-template-columns: minmax(120px, 42%) minmax(0, 1fr);
    gap: 12px;
    border-bottom: 0;
    padding: 8px 0;
  }

  .table td::before {
    content: attr(data-label); /* Uses data-label attribute for column headers */
    color: #64748b;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    font-size: 0.78rem;
    font-weight: 700;
    text-transform: uppercase;
  }

  .inlineForm {
    min-width: 0;
    width: 100%;
  }
}

@media (max-width: 560px) {
  .title {
    font-size: 1.65rem;
  }

  .formGrid {
    grid-template-columns: 1fr; /* Single column layout on very small screens */
  }
}
```

Header.module.css

Found in folder: **/src/components**

This CSS styles a header section with a title, subtitle, and a call-to-action button, including responsive behavior for mobile screens. It defines visual styles for layout, typography, colors, and interactive states. The media query adjusts the layout and button width on smaller screens.

```
.header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 20px;
  border-bottom: 1px solid #d9e1ec;
  background: #ffffff;
  padding: 22px 32px;
}

.eyebrow {
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 700;
  margin: 0 0 6px;
  text-transform: uppercase;
}

.title {
  color: #172033;
  font-size: 1.65rem;
  line-height: 1.2;
  margin: 0;
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.action {
  display: inline-flex;
  min-height: 42px;
  align-items: center;
  justify-content: center;
  border-radius: 8px;
  background: #1d4ed8;
  color: #ffffff;
  font-weight: 700;
  padding: 0 16px;
  text-decoration: none;
  white-space: nowrap;
}

.action:hover {
  background: #1e40af; /* Darker shade on hover for visual feedback */
}

@media (max-width: 640px) {
  .header {
    align-items: flex-start;
    flex-direction: column; /* Stack header elements vertically on small screens */
    padding: 20px;
  }

  .action {
    width: 100%; /* Button spans full width on mobile */
  }
}
```

Sidebar.module.css

Found in folder: **/src/components**

This CSS styles a sidebar component with a brand logo and navigation links. It defines layout, colors, and interactive states like hover and active. On smaller screens, it adapts to a horizontal bottom-aligned navigation bar.

```
.sidebar {
  display: flex;
  min-height: 100vh; /* Full viewport height */
  flex-direction: column;
  gap: 32px; /* Spacing between child elements */
  border-right: 1px solid #d9e1ec;
  background: #f8fafc;
  padding: 24px 18px;
}

.brand {
  display: flex;
  align-items: center;
  gap: 12px;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  color: #172033;
  font-weight: 700;
  text-decoration: none;
}

.brandMark {
  display: grid;
  width: 38px;
  height: 38px;
  place-items: center; /* Centers content both horizontally and vertically */
  border-radius: 8px;
  background: #1d4ed8;
  color: #ffffff;
  font-size: 0.85rem;
  letter-spacing: 0;
}

.brandText {
  line-height: 1.2;
}

.nav {
  display: flex;
  flex-direction: column;
  gap: 6px; /* Spacing between navigation links */
}

.navLink {
  border-radius: 8px;
  color: #526071;
  font-size: 0.95rem;
  font-weight: 600;
  padding: 11px 12px;
  text-decoration: none;
  transition:
    background-color 160ms ease,
    color 160ms ease; /* Smooth hover/active state transitions */
}

.navLink:hover {
  background: #eef4ff;
  color: #1d4ed8;
}

.active {
  background: #dbeafe;
  color: #1d4ed8;
}

@media (max-width: 760px) {
  .sidebar {
    min-height: auto; /* Remove full height on mobile */
    border-right: 0;
    border-bottom: 1px solid #d9e1ec;
    padding: 18px;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  }

  .nav {
    display: grid;
    grid-template-columns: repeat(4, minmax(0, 1fr)); /* Four-
column grid for links */
  }

  .navLink {
    text-align: center; /* Center text in mobile layout */
  }
}
```

StatusBadge.module.css

Found in folder: **/src/components**

This CSS code defines styles for badge components, including a base class for layout and spacing, and several color variant classes for different visual states. Each color variant sets a background and text color to create distinct appearances. The badges are designed as small, rounded, inline elements with bold text and no wrapping.

```
.badge {
  display: inline-flex;          /* Makes badge an inline flex container */
  min-height: 24px;              /* Ensures a consistent minimum height */
  align-items: center;           /* Vertically centers content */
  border-radius: 999px;          /* Creates a pill-shaped rounded border */
  font-size: 0.78rem;            /* Sets a small font size */
  font-weight: 700;              /* Makes text bold */
  line-height: 1;                /* Tight line spacing */
  padding: 5px 9px;              /* Internal spacing */
  white-space: nowrap;           /* Prevents text from wrapping */
}

/* Color variants */
.grey {
  background: #e5e7eb;
  color: #374151;
}

.blue {
  background: #dbeafe;
  color: #1d4ed8;
}

.green {
  background: #dcfce7;
  color: #15803d;
}

.red {
  background: #fee2e2;
  color: #b91c1c;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
}

.yellow {
  background: #fef3c7;
  color: #a16207;
}

.darkGrey {
  background: #374151;
  color: #f9fafb;
}
```

clientActions.js

Found in folder: **/src/actions**

This code defines server actions for creating, updating, and deleting client records. It validates input data, converts FormData to objects, and handles errors with user-friendly messages. The actions wrap service functions and return standardized result objects.

```
'use server'

import {
  createClient,
  deleteClient,
  updateClient,
} from '../lib/services.js'

// Helper function to return standardized result objects
function result(data = null, error = null) {
  return { data, error }
}

// Extract meaningful error message from error objects
function errorMessage(error) {
  return error instanceof Error ? error.message : 'Something went wrong'
}

// Convert FormData to plain object if input is FormData
function formDataToObject(input) {
  if (!(input instanceof FormData)) {
    return input
  }

  return Object.fromEntries(input.entries())
}

export async function createClientAction(input) {
  const data = formDataToObject(input)

  // Validate required client name field
  if (!data?.name || String(data.name).trim() === '') {
    return result(null, 'Client name is required')
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  }

  try {
    return result(createClient(data), null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
}

export async function updateClientAction(id, data) {
  try {
    return result(updateClient(id, formDataToObject(data)), null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
}

export async function deleteClientAction(id) {
  try {
    deleteClient(id)
    return result(true, null)
  } catch (error) {
    const message = errorMessage(error)

    // Handle specific constraint error for clients with projects
    if (message.includes('because it has projects')) {
      return result(
        null,
        'Cannot delete this client because projects are still assigned to it.',
      )
    }

    return result(null, message)
  }
}
```

invoiceActions.js

Found in folder: **/src/actions**

This code defines server-side actions for managing invoices, including creating, updating, deleting, and updating invoice statuses. It validates input data for creating invoices and ensures status updates are valid before applying changes. Each action returns a standardized result object containing either data or an error message.

```
'use server'

import {
  createInvoice,
  deleteInvoice,
  getInvoiceById,
  updateInvoice,
} from '../lib/services.js'
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
// Valid invoice statuses for status update validation
const INVOICE_STATUSES = new Set(['draft', 'sent', 'paid', 'overdue', 'void'])

// Helper function to standardize response format
function result(data = null, error = null) {
  return { data, error }
}

// Helper to extract error message from caught errors
function errorMessage(error) {
  return error instanceof Error ? error.message : 'Something went wrong'
}

export async function createInvoiceAction(data) {
  // Validate required fields for invoice creation
  if (!data?.projectId || String(data.projectId).trim() === '') {
    return result(null, 'Project is required')
  }

  if (!data?.invoiceNumber || String(data.invoiceNumber).trim() === '') {
    return result(null, 'Invoice number is required')
  }

  if (!data?.issueDate || String(data.issueDate).trim() === '') {
    return result(null, 'Issue date is required')
  }

  if (!Array.isArray(data.lineItems) || data.lineItems.length === 0) {
    return result(null, 'At least one invoice line item is required')
  }

  try {
    return result(createInvoice(data), null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
}

export async function updateInvoiceAction(id, data) {
  try {
    return result(updateInvoice(id, data), null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
}

export async function deleteInvoiceAction(id) {
  try {
    deleteInvoice(id)
    return result(true, null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
export async function updateInvoiceStatusAction(id, status) {
  // Validate that the provided status is allowed
  if (!INVOICE_STATUSES.has(status)) {
    return result(null, `Invalid invoice status: ${status}`)
  }

  try {
    const invoice = getInvoiceById(id)

    if (!invoice) {
      return result(null, `Invoice ${id} not found`)
    }

    // Update paid amount and date only when status changes to 'paid'
    const nextInvoice = {
      ...invoice,
      status,
      paidCents: status === 'paid' ? invoice.totalCents : invoice.paidCents,
      paidAt: status === 'paid' ? (invoice.paidAt || new Date().toISOString()) : invo
ice.paidAt,
    }

    return result(updateInvoice(id, nextInvoice), null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
}
```

projectActions.js

Found in folder: **/src/actions**

This code defines server actions for creating, updating, and deleting projects. It validates input for the create action and handles errors by returning user-friendly messages. The delete action includes a specific check for projects that cannot be deleted due to existing invoices.

```
'use server'

import {
  createProject,
  deleteProject,
  updateProject,
} from '../lib/services.js'

// Helper function to standardize the response format for all actions
function result(data = null, error = null) {
  return { data, error }
}

// Helper to extract a safe error message from caught errors
function errorMessage(error) {
  return error instanceof Error ? error.message : 'Something went wrong'
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
}

export async function createProjectAction(data) {
  // Validate required fields for project creation
  if (!data?.clientId || String(data.clientId).trim() === '') {
    return result(null, 'Client is required')
  }

  if (!data?.name || String(data.name).trim() === '') {
    return result(null, 'Project name is required')
  }

  try {
    return result(createProject(data), null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
}

export async function updateProjectAction(id, data) {
  try {
    return result(updateProject(id, data), null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
}

export async function deleteProjectAction(id) {
  try {
    deleteProject(id)
    return result(true, null)
  } catch (error) {
    const message = errorMessage(error)

    // Provide a specific error message if deletion fails due to existing invoices
    if (message.includes('because it has invoices')) {
      return result(
        null,
        'Cannot delete this project because invoices are still assigned to it.',
      )
    }

    return result(null, message)
  }
}
```

error.js

Found in folder: **/src/app**

This code defines a React component that displays an error message with a refresh button. When clicked, the button triggers a page refresh using Next.js's router. The component is styled with CSS modules and includes ARIA roles for accessibility.

© 2026 Zenva Pty Ltd. All rights reserved.

```
"use client";

import { useRouter } from "next/navigation";
import styles from "./Error.module.css";

export default function Error() {
  const router = useRouter();

  return (
    <section className={styles.errorState} role="alert">
      <p className={styles.message}>
        Something went wrong. Please try refreshing the page.
      </p>
      <button
        className={styles.button}
        onClick={() => router.refresh()} // Refreshes the current page
        type="button"
      >
        Refresh
      </button>
    </section>
  );
}
```

layout.js

Found in folder: **/src/app**

This code defines the root layout for the application, setting global metadata and structuring the page with a sidebar, header, and main content area. It imports and applies CSS styles to arrange these components. It also injects an external script from a local development server.

```
import "./globals.css";
import Header from "@/components/Header";
import Sidebar from "@/components/Sidebar";

import styles from "./layout.module.css";

// Metadata for the application
export const metadata = {
  title: "Invoice Tracker",
  description: "Freelancer invoice tracker",
};

// Root layout component defining the overall page structure
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        {/* Container for the sidebar and main content area */}
        <div className={styles.shell}>
          <Sidebar />
          <div className={styles.content}>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
            <Header />
            {/* Main area where page content is rendered */}
            <main className={styles.main}>{children}</main>
          </div>
        </div>
        {/* Script injected for development purposes (e.g., live reload or iframe com
munication) */}
        <Script src="http://localhost:5000/iframe-script" />
      </body>
    </html>
  );
}
```

page.js

Found in folder: **/src/app**

This code renders a dashboard page that displays business summary cards, a revenue chart for the last six months, and a list of overdue invoices. It fetches summary data, monthly revenue, and overdue invoices, then formats and presents them using styled components. If there are no overdue invoices, it shows a message instead of a list.

```
import Link from "next/link";

import { formatCurrency, formatDate, formatInvoiceNumber } from "@/lib/formatters";
import {
  getDashboardSummary,
  getMonthlyRevenue,
  getOverdueInvoices,
} from "@/lib/services";
import RevenueChart from "./RevenueChart";
import styles from "./Dashboard.module.css";

export default function DashboardPage() {
  // Fetch data for dashboard display
  const summary = getDashboardSummary();
  const monthlyRevenue = getMonthlyRevenue();
  const overdueInvoices = getOverdueInvoices();

  return (
    <section className={styles.page}>
      <div className={styles.header}>
        <div>
          <p className={styles.eyebrow}>Dashboard</p>
          <h1 className={styles.title}>Business overview</h1>
        </div>
      </div>

      {/* Summary cards showing key business metrics */}
      <div className={styles.summaryGrid}>
        <article className={styles.summaryCard}>
          <p className={styles.cardLabel}>Total Outstanding</p>
          <p className={styles.cardValue}>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
        {formatCurrency(summary.totalOutstandingCents)}
      </p>
    </article>

    <article className={styles.summaryCard}>
      <p className={styles.cardLabel}>Paid This Month</p>
      <p className={styles.cardValue}>
        {formatCurrency(summary.totalPaidThisMonthCents)}
      </p>
    </article>

    <article className={styles.summaryCard}>
      <p className={styles.cardLabel}>Overdue Invoices</p>
      <p
        className={`${styles.cardValue} ${
          summary.overdueCount > 0 ? styles.dangerValue : ""
        }`}
      >
        {summary.overdueCount}
      </p>
    </article>

    <article className={styles.summaryCard}>
      <p className={styles.cardLabel}>Top Client</p>
      <p className={styles.cardValue}>
        {summary.topClientByRevenue || "No paid invoices"}
      </p>
    </article>
  </div>

  {/* Revenue chart section for the last six months */}
  <section className={styles.panel} aria-labelledby="revenue-heading">
    <div className={styles.panelHeader}>
      <div>
        <p className={styles.eyebrow}>Revenue</p>
        <h2 id="revenue-heading" className={styles.sectionTitle}>
          Last 6 months
        </h2>
      </div>
    </div>
    <RevenueChart data={monthlyRevenue} />
  </section>

  {/* Overdue invoices list or empty state message */}
  <section className={styles.panel} aria-labelledby="overdue-heading">
    <div className={styles.panelHeader}>
      <div>
        <p className={styles.eyebrow}>Invoices</p>
        <h2 id="overdue-heading" className={styles.sectionTitle}>
          Overdue invoices
        </h2>
      </div>
    </div>

    {overdueInvoices.length === 0 ? (
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
          <div className={styles.emptyState}>
            <h3 className={styles.emptyTitle}>No overdue invoices</h3>
            <p className={styles.emptyText}>
              Everything is current. There are no overdue balances to chase.
            </p>
          </div>
        ) : (
          <div className={styles.overdueList}>
            {overdueInvoices.map((invoice) => {
              const amountDueCents = invoice.totalCents - invoice.paidCents;

              return (
                <Link
                  key={invoice.id}
                  href={`/invoices/${invoice.id}`}
                  className={styles.overdueRow}
                >
                  <div className={styles.invoiceIdentity}>
                    <span className={styles.clientName}>{invoice.clientName}</span>
                    <span className={styles.invoiceNumber}>
                      {formatInvoiceNumber(invoice.invoiceNumber)}
                    </span>
                  </div>
                  <div className={styles.invoiceMeta}>
                    <span>{formatDate(invoice.dueDate)}</span>
                    <strong>{formatCurrency(amountDueCents, invoice.currency)}</stron
g>
                  </div>
                </Link>
              );
            })}
          </div>
        )}
      </section>
    </section>
  );
}
```

route.js

Found in folder: **/src/app/api/invoices/export**

This code defines an API endpoint that exports invoice data as a CSV file. It filters invoices based on URL query parameters, formats the data appropriately, and returns the CSV as a downloadable file with proper headers.

```
import { getInvoices } from "@/lib/services";

// Column headers for the CSV output
const csvColumns = [
  "Invoice Number",
  "Client",
  "Project",
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  "Issue Date",
  "Due Date",
  "Status",
  "Total",
];

// Extracts and validates filter values from URL search parameters
function getFilter(searchParams, name) {
  const value = searchParams.get(name);

  if (value === null || value === "" || value === "all") {
    return undefined;
  }

  return value;
}

// Formats date values to YYYY-MM-DD format for CSV
function formatCsvDate(value) {
  if (!value) {
    return "";
  }

  return String(value).slice(0, 10);
}

// Converts cents to dollars with two decimal places
function formatCsvTotal(cents) {
  return (Number(cents || 0) / 100).toFixed(2);
}

// Escapes CSV special characters and wraps fields in quotes when needed
function escapeCsvField(value) {
  const text = String(value ?? "");

  if (/[",\r\n]/.test(text)) {
    return `"${text.replaceAll('"', '""')}"`;
  }

  return text;
}

// Converts 2D array of data into CSV string format
function serializeCsv(rows) {
  return rows
    .map((row) => row.map(escapeCsvField).join(","))
    .join("\r\n");
}

// API endpoint handler for CSV export
export async function GET(request) {
  const { searchParams } = new URL(request.url);
  const invoices = getInvoices({
    status: getFilter(searchParams, "status"),
    clientId: getFilter(searchParams, "clientId"),
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    projectId: getFilter(searchParams, "projectId"),
  });

  // Build CSV rows: headers first, then invoice data
  const rows = [
    csvColumns,
    ...invoices.map((invoice) => [
      invoice.invoiceNumber,
      invoice.clientName,
      invoice.projectName,
      formatCsvDate(invoice.issueDate),
      formatCsvDate(invoice.dueDate),
      invoice.status,
      formatCsvTotal(invoice.totalCents),
    ]),
  ];

  // Return CSV response with download headers
  return new Response(`${serializeCsv(rows)}\r\n`, {
    headers: {
      "Content-Type": "text/csv",
      "Content-Disposition": 'attachment; filename="invoices.csv"',
    },
  });
}
```

error.js

Found in folder: **/src/app/clients**

This component renders an error state UI when client data fails to load, displaying a message and a retry button. The button triggers a retry function passed as a prop to attempt reloading the data. It uses CSS modules for styling and includes ARIA roles for accessibility.

```
"use client";

import styles from "./Error.module.css";

export default function ClientsError({ unstable_retry }) {
  return (
    <section className={styles.errorState} role="alert">
      <p className={styles.message}>
        Unable to load clients. Please try again.
      </p>
      <button
        className={styles.button}
        onClick={() => unstable_retry()} // Calls the retry function passed via props
        type="button"
      >
        Try Again
      </button>
    </section>
  );
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
}
```

page.js

Found in folder: **/src/app/clients**

This component displays a list of clients. It shows a header with a form to create a new client and a main panel. The panel either shows an empty state message with a creation form if there are no clients, or a table listing all clients with their details and an edit form.

```
import StatusBadge from "@/components/StatusBadge";
import { formatCurrency } from "@/lib/formatters";
import { getClients } from "@/lib/services";
import ClientInlineForm from "./ClientInlineForm";
import styles from "./ClientsPage.module.css";

export default function ClientsPage() {
  // Fetch the list of clients from the service layer
  const clients = getClients();

  return (
    <section className={styles.page}>
      <div className={styles.header}>
        <div>
          <p className={styles.eyebrow}>Clients</p>
          <h1 className={styles.title}>Client accounts</h1>
        </div>
        {/* Form for creating a new client directly in the header */}
        <ClientInlineForm mode="create" />
      </div>

      <div className={styles.panel}>
        {/* Conditional rendering: empty state vs. client table */}
        {clients.length === 0 ? (
          <div className={styles.emptyState}>
            <p className={styles.emptyText}>
              No clients yet. Add your first client to get started.
            </p>
            <ClientInlineForm mode="create" />
          </div>
        ) : (
          <div className={styles.tableWrap}>
            <table className={styles.table}>
              <thead>
                <tr>
                  <th scope="col">Name</th>
                  <th scope="col">Company</th>
                  <th scope="col">Email</th>
                  <th scope="col">Active projects</th>
                  <th scope="col">Unpaid</th>
                  <th scope="col">Actions</th>
                </tr>
              </thead>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
            <tbody>
              {/* Map over each client to create a table row */}
              {clients.map((client) => (
                <tr key={client.id}>
                  <td data-label="Name">
                    <strong className={styles.clientName}>{client.name}</strong>
                  </td>
                  <td data-label="Company">{client.company || "—"}</td>
                  <td data-label="Email">{client.email || "—"}</td>
                  <td data-label="Active projects">
                    <span className={styles.projectCount}>
                      <span>{client.activeProjectCount}</span>
                      <StatusBadge status="active" />
                    </span>
                  </td>
                  <td data-label="Unpaid">
                    {/* Format currency from cents to a readable string */}
                    {formatCurrency(client.totalUnpaidCents)}
                  </td>
                  <td data-label="Actions">
                    {/* Inline form for editing this specific client */}
                    <ClientInlineForm mode="edit" client={client} />
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </div>
  </section>
 );
}
```

error.js

Found in folder: **/src/app/invoices**

This code defines a React error boundary component that displays a user-friendly error message and a retry button when invoices fail to load. It provides visual feedback through CSS modules and includes accessibility features like role=”alert”. The component accepts a retry function prop that gets called when the user clicks the button to attempt reloading the data.

```
"use client";

import styles from "./Error.module.css";

// Error boundary component for invoice loading failures
export default function InvoicesError({ unstable_retry }) {
  return (
    <section className={styles.errorState} role="alert">
      <p className={styles.message}>
        Unable to load invoices. Please try again.
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      </p>
      <button
        className={styles.button}
        onClick={() => unstable_retry()}  // Retry function passed as prop
        type="button"
      >
        Try Again
      </button>
    </section>
  );
}
```

page.js

Found in folder: **/src/app/invoices**

This code renders a page that displays invoices, applying optional filters from URL search parameters. It fetches invoices using these filters and passes them to a table component for display. The page also shows a header with a title and checks if any invoices exist.

```
import { getInvoices } from "@/lib/services";
import InvoicesTable from "./InvoicesTable";
import styles from "./InvoicesPage.module.css";

// Extracts and validates a single search parameter value
function getSearchParam(searchParams, name) {
  const value = searchParams?.[name];
  const stringValue = Array.isArray(value) ? value[0] : value;

  // Return undefined for empty values or the "all" placeholder
  if (!stringValue || stringValue === "all") {
    return undefined;
  }

  return stringValue;
}

export default async function InvoicesPage({ searchParams }) {
  const resolvedSearchParams = await searchParams;
  // Build filter object from URL parameters
  const filters = {
    status: getSearchParam(resolvedSearchParams, "status"),
    clientId: getSearchParam(resolvedSearchParams, "clientId"),
    projectId: getSearchParam(resolvedSearchParams, "projectId"),
  };
  const invoices = getInvoices(filters);
  // Check if any invoices exist (without filters)
  const hasInvoices = getInvoices().length > 0;

  return (
    <section className={styles.page}>
      <div className={styles.header}>
        <div>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
          <p className={styles.eyebrow}>Invoices</p>
          <h1 className={styles.title}>Invoice ledger</h1>
        </div>
      </div>

      <InvoicesTable
        hasInvoices={hasInvoices}
        invoices={invoices}
        initialFilters={filters}
      />
    </section>
  );
}
```

page.js

Found in folder: **/src/app/invoices/new**

This code renders a page for creating a new invoice. It fetches client and project data, then passes it to an invoice form component set to creation mode. The page includes a styled header section with a title.

```
import { getClients, getProjects } from "@/lib/services";
import InvoiceForm from "../InvoiceForm";
import styles from "../InvoicesPage.module.css";

export default function NewInvoicePage() {
  // Fetch data needed for the invoice form
  const clients = getClients();
  const projects = getProjects();

  return (
    <section className={styles.page}>
      <div className={styles.header}>
        <div>
          <p className={styles.eyebrow}>Invoices</p>
          <h1 className={styles.title}>Create invoice</h1>
        </div>
      </div>

      {/* Form component pre-populated with available clients and projects */}
      <InvoiceForm mode="create" clients={clients} projects={projects} />
    </section>
  );
}
```

page.js

Found in folder: **/src/app/invoices/[invoiceId]**

This code renders an invoice detail page with actions to update its status, displays invoice

© 2026 Zenva Pty Ltd. All rights reserved.

information like client details and line items, and includes validation for the invoice ID and status transitions. It formats quantities and currency values appropriately and conditionally shows action buttons based on the current invoice status. The page also revalidates cache paths after status updates to ensure data freshness.

```
import Link from "next/link";
import { revalidatePath } from "next/cache";
import { notFound } from "next/navigation";
import { updateInvoiceStatusAction } from "@/actions/invoiceActions";
import StatusBadge from "@/components/StatusBadge";
import {
  formatCurrency,
  formatDate,
  formatInvoiceNumber,
} from "@/lib/formatters";
import { getInvoiceById } from "@/lib/services";
import styles from "./InvoiceDetailPage.module.css";

// Formats quantity values, preserving non-
numeric strings and showing two decimals for non-integers
function formatQuantity(quantity) {
  const value = Number(quantity);

  if (!Number.isFinite(value)) {
    return quantity;
  }

  return Number.isInteger(value) ? String(value) : value.toFixed(2);
}

// Conditionally renders a contact line only if children content exists
function ContactLine({ children }) {
  if (!children) {
    return null;
  }

  return <p className={styles.contactLine}>{children}</p>;
}

// Validates that the value is a positive integer (no leading zeros)
function isPositiveInteger(value) {
  return /^[1-9]\d*$/.test(String(value));
}

// Creates a server action to update invoice status and revalidate relevant paths
function createStatusAction(invoiceId, status) {
  return async function updateStatus() {
    "use server";

    const result = await updateInvoiceStatusAction(invoiceId, status);

    if (result.error) {
      throw new Error(result.error);
    }
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    revalidatePath("/invoices");
    revalidatePath(`/invoices/${invoiceId}`);
  };
}

export default async function InvoiceDetailPage({ params }) {
  const { invoiceId } = await params;

  // Validate invoice ID format before fetching
  if (!isPositiveInteger(invoiceId)) {
    notFound();
  }

  const invoice = getInvoiceById(invoiceId);

  if (!invoice) {
    notFound();
  }

  const invoiceNumber = formatInvoiceNumber(invoice.invoiceNumber);
  const markSentAction = createStatusAction(invoice.id, "sent");
  const markPaidAction = createStatusAction(invoice.id, "paid");
  const voidAction = createStatusAction(invoice.id, "void");
  const canMarkSent = invoice.status === "draft";
  const canMarkPaid = invoice.status === "sent" || invoice.status === "overdue";
  const canVoid = invoice.status !== "void" && invoice.status !== "paid";
  const balanceDueCents = Math.max(
    invoice.totalCents - invoice.paidCents,
    0,
  );

  return (
    <section className={styles.page}>
      <div className={styles.actions} aria-label="Invoice actions">
        <Link className={styles.secondaryLink} href="/invoices">
          Back to Invoices
        </Link>
        {canMarkSent ? (
          <form action={markSentAction}>
            <button className={styles.secondaryButton} type="submit">
              Mark as Sent
            </button>
          </form>
        ) : null}
        {canMarkPaid ? (
          <form action={markPaidAction}>
            <button className={styles.primaryButton} type="submit">
              Mark as Paid
            </button>
          </form>
        ) : null}
        {canVoid ? (
          <form action={voidAction}>
            <button className={styles.dangerButton} type="submit">
              Void
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
        </button>
      </form>
    ) : null}
    <Link
      className={styles.primaryLink}
      href={`/invoices/${invoice.id}/edit`}
    >
      Edit
    </Link>
  </div>

  <article className={styles.document}>
    <header className={styles.documentHeader}>
      <div>
        <p className={styles.brandLabel}>Freelancer Invoice</p>
        <p className={styles.tagline}>Professional Services</p>
      </div>

      <div className={styles.invoiceMeta}>
        <p className={styles.invoiceLabel}>Invoice</p>
        <h1 className={styles.invoiceNumber}>{invoiceNumber}</h1>
        <dl className={styles.metaList}>
          <div>
            <dt>Issue date</dt>
            <dd>{formatDate(invoice.issueDate)}</dd>
          </div>
          <div>
            <dt>Due date</dt>
            <dd>{formatDate(invoice.dueDate)}</dd>
          </div>
          <div>
            <dt>Status</dt>
            <dd>
              <StatusBadge status={invoice.status} />
            </dd>
          </div>
        </dl>
      </div>
    </header>

    <div className={styles.parties}>
      <section className={styles.billTo} aria-labelledby="bill-to-heading">
        <h2 id="bill-to-heading">Bill To:</h2>
        <div className={styles.clientBlock}>
          <p className={styles.clientName}>{invoice.client.name}</p>
          <ContactLine>{invoice.client.company}</ContactLine>
          <ContactLine>{invoice.client.email}</ContactLine>
          {invoice.client.address ? (
            <p className={styles.address}>{invoice.client.address}</p>
          ) : null}
        </div>
      </section>

      <section className={styles.project} aria-labelledby="project-heading">
        <h2 id="project-heading">Project</h2>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      <p className={styles.projectName}>{invoice.project.name}</p>
      <p className={styles.projectNote}>
        This invoice covers work completed for {invoice.project.name}.
      </p>
    </section>
  </div>

  <section className={styles.lineItems} aria-labelledby="items-heading">
    <h2 id="items-heading" className={styles.sectionTitle}>
      Services
    </h2>
    <div className={styles.tableWrap}>
      <table className={styles.table}>
        <thead>
          <tr>
            <th scope="col">Description</th>
            <th scope="col">Quantity</th>
            <th scope="col">Unit Price</th>
            <th scope="col">Total</th>
          </tr>
        </thead>
        <tbody>
          {invoice.lineItems.map((item) => (
            <tr key={item.id}>
              <td>{item.description}</td>
              <td>{formatQuantity(item.quantity)}</td>
              <td>
                {formatCurrency(item.unitPriceCents, invoice.currency)}
              </td>
              <td>
                {formatCurrency(item.lineTotalCents, invoice.currency)}
              </td>
            </tr>
          ))}
        </tbody>
        <tfoot>
          <tr>
            <th scope="row" colSpan="3">
              Subtotal
            </th>
            <td>{formatCurrency(invoice.subtotalCents, invoice.currency)}</td>
          </tr>
          <tr className={styles.totalRow}>
            <th scope="row" colSpan="3">
              Total
            </th>
            <td>{formatCurrency(invoice.totalCents, invoice.currency)}</td>
          </tr>
        </tfoot>
      </table>
    </div>
  </section>

  <section className={styles.payment} aria-labelledby="payment-heading">
    <div>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
            <h2 id="payment-heading">Payment</h2>
            <p className={styles.paymentDueDate}>
              Payment due by {formatDate(invoice.dueDate)}
            </p>
          </div>
          <div className={styles.amountDue}>
            <p>Total amount due</p>
            <strong>{formatCurrency(balanceDueCents, invoice.currency)}</strong>
          </div>
        </section>

        {invoice.notes ? (
          <section className={styles.notes} aria-labelledby="notes-heading">
            <h2 id="notes-heading">Notes</h2>
            <p>{invoice.notes}</p>
          </section>
        ) : null}
      </article>
    </section>
  );
}
```

page.js

Found in folder: **/src/app/invoices/[invoiceId]/edit**

This code loads an invoice by ID and renders a form to edit it, fetching related clients and projects for the form. If the invoice doesn’t exist, it shows a 404 page. It then passes all this data to the InvoiceForm component in edit mode.

```
import { notFound } from "next/navigation";
import { getClients, getInvoiceById, getProjects } from "@/lib/services";
import InvoiceForm from "../../InvoiceForm";
import styles from "../../InvoicesPage.module.css";

export default async function EditInvoicePage({ params }) {
  // Extract invoice ID from dynamic route parameters
  const { invoiceId } = await params;
  const invoice = getInvoiceById(invoiceId);

  // Show 404 page if invoice doesn't exist
  if (!invoice) {
    notFound();
  }

  // Fetch all clients and projects for form dropdowns
  const clients = getClients();
  const projects = getProjects();

  return (
    <section className={styles.page}>
      <div className={styles.header}>
        <div>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
          <p className={styles.eyebrow}>Invoices</p>
          <h1 className={styles.title}>Edit invoice</h1>
        </div>
      </div>

      {/* Pass invoice data and related entities to form component */}
      <InvoiceForm
        mode="edit"
        invoice={invoice}
        clients={clients}
        projects={projects}
      />
    </section>
  );
}
```

error.js

Found in folder: **/src/app/projects**

This code defines a React error boundary component that displays a user-friendly error message and a retry button when projects fail to load. It renders a styled alert section containing an error message and a button that triggers the provided retry function when clicked. The component uses CSS modules for styling and is marked as a client component for Next.js.

```
"use client";

import styles from "./Error.module.css";

export default function ProjectsError({ unstable_retry }) {
  return (
    <section className={styles.errorState} role="alert">
      <p className={styles.message}>
        Unable to load projects. Please try again.
      </p>
      <button
        className={styles.button}
        onClick={() => unstable_retry()} // Triggers retry mechanism when clicked
        type="button"
      >
        Try Again
      </button>
    </section>
  );
}
```

page.js

Found in folder: **/src/app/projects**

This code displays a list of projects grouped by their client, showing each project’s name, status,

© 2026 Zenva Pty Ltd. All rights reserved.

total invoiced amount, and a link to its invoices. It provides an inline form to add new projects and shows an empty state message when no projects exist. The projects are sorted alphabetically by client name.

```
import Link from "next/link";
import StatusBadge from "@/components/StatusBadge";
import { formatCurrency } from "@/lib/formatters";
import { getClients, getProjects } from "@/lib/services";
import ProjectInlineForm from "./ProjectInlineForm";
import styles from "./ProjectsPage.module.css";

// Groups an array of projects by their client, returning sorted groups
function groupProjectsByClient(projects) {
  const groups = new Map();

  for (const project of projects) {
    const group = groups.get(project.clientId) || {
      clientId: project.clientId,
      clientName: project.clientName,
      projects: [],
    };

    group.projects.push(project);
    groups.set(project.clientId, group);
  }

  // Convert map to array and sort groups alphabetically by client name
  return Array.from(groups.values()).sort((first, second) => (
    first.clientName.localeCompare(second.clientName)
  ));
}

export default function ProjectsPage() {
  const projects = getProjects();
  const clients = getClients();
  const projectGroups = groupProjectsByClient(projects);

  return (
    <section className={styles.page}>
      <div className={styles.header}>
        <div>
          <p className={styles.eyebrow}>Projects</p>
          <h1 className={styles.title}>Project ledger</h1>
        </div>
        <ProjectInlineForm clients={clients} />
      </div>

      <div className={styles.panel}>
        {projectGroups.length === 0 ? (
          // Show empty state with form to add first project
          <div className={styles.emptyState}>
            <p className={styles.emptyText}>
              No projects yet. Add your first project to get started.
            </p>
            <ProjectInlineForm clients={clients} />
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      </div>
    ) : (
      // Render project groups, each with a table of projects
      <div className={styles.clientSections}>
        {projectGroups.map((group) => (
          <section className={styles.clientSection} key={group.clientId}>
            <header className={styles.clientHeader}>
              <h2 className={styles.clientName}>{group.clientName}</h2>
            </header>

            <div className={styles.tableWrap}>
              <table className={styles.table}>
                <thead>
                  <tr>
                    <th scope="col">Project</th>
                    <th scope="col">Status</th>
                    <th scope="col">Total invoiced</th>
                    <th scope="col">Invoices</th>
                  </tr>
                </thead>
                <tbody>
                  {group.projects.map((project) => (
                    <tr key={project.id}>
                      <td data-label="Project">
                        <div className={styles.projectSummary}>
                          <strong className={styles.projectName}>
                            {project.name}
                          </strong>
                          {project.description ? (
                            <span className={styles.projectDescription}>
                              {project.description}
                            </span>
                          ) : null}
                        </div>
                      </td>
                      <td data-label="Status">
                        <StatusBadge status={project.status} />
                      </td>
                      <td data-label="Total invoiced">
                        {formatCurrency(project.totalInvoicedCents)}
                      </td>
                      <td data-label="Invoices">
                        <Link
                          className={styles.invoiceLink}
                          href={`/invoices?projectId=${project.id}`}
                        >
                          View invoices
                        </Link>
                      </td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          </section>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
            ))}
          </div>
        )}
      </div>
    </section>
  );
}
```

db.js

Found in folder: **/src/lib**

This code initializes a SQLite database connection, setting the database file path based on the environment (in-memory for testing, file-based otherwise). It configures the database to use Write-Ahead Logging (WAL) mode and enables foreign key constraints. Finally, it exports the database connection instance for use elsewhere.

```
import Database from 'better-sqlite3'
import path from 'node:path'

// Determine database path: in-memory for tests, file-based for other environments
const dbPath = process.env.NODE_ENV === 'test'
  ? ':memory:'
  : path.join(process.cwd(), 'invoice-tracker.db')
const db = new Database(dbPath)

// Configure database for better performance and enforce referential integrity
db.pragma('journal_mode = WAL')
db.pragma('foreign_keys = ON')

export default db
```

formatters.js

Found in folder: **/src/lib**

This code provides three utility functions: one converts cents to a localized currency string, another formats an ISO date string into a UK-style date, and a third pads numeric invoice numbers with leading zeros.

```
export function formatCurrency(cents, currency = "USD") {
  // Convert cents to a decimal amount (e.g., 1999 cents -> 19.99)
  const amount = Number(cents || 0) / 100;

  // Format the amount as currency using the browser's Intl API
  return new Intl.NumberFormat("en-US", {
    style: "currency",
    currency,
  }).format(amount);
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
export function formatDate(isoString) {
  // Return empty string for falsy input
  if (!isoString) {
    return "";
  }

  // Format date in UK style (DD Mon YYYY) using UTC timezone
  return new Intl.DateTimeFormat("en-GB", {
    day: "2-digit",
    month: "short",
    year: "numeric",
    timeZone: "UTC",
  }).format(new Date(isoString));
}

export function formatInvoiceNumber(number) {
  const value = String(number);

  // Pad numeric strings with leading zeros to 4 digits
  if (/^\d+$/.test(value)) {
    return value.padStart(4, "0");
  }

  // Return non-numeric strings unchanged
  return value;
}
```

migrate.js

Found in folder: **/src/lib**

This code defines a database migration that creates tables for clients, projects, invoices, and invoice line items with their respective indexes and constraints. It ensures the tables only exist if they haven't been created already, using SQLite's transaction and exec methods. The migration runs immediately when the module is loaded.

```
import db from './db.js'

// Define and execute a database migration transaction
const migrate = db.transaction(() => {
  db.exec(`
    -- Create clients table with contact information and timestamps
    CREATE TABLE IF NOT EXISTS clients (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      name TEXT NOT NULL,
      email TEXT,
      phone TEXT,
      company TEXT,
      address TEXT,
      notes TEXT,
      created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
      updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
    );
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
-- Index for faster client name lookups
CREATE INDEX IF NOT EXISTS idx_clients_name
  ON clients (name);

-- Create projects table linked to clients with status and rate constraints
CREATE TABLE IF NOT EXISTS projects (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  client_id INTEGER NOT NULL,
  name TEXT NOT NULL,
  description TEXT,
  status TEXT NOT NULL DEFAULT 'active',
  hourly_rate_cents INTEGER,
  created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  FOREIGN KEY (client_id) REFERENCES clients (id) ON DELETE RESTRICT,
  CHECK (status IN ('active', 'paused', 'completed', 'archived')),
  CHECK (hourly_rate_cents IS NULL OR hourly_rate_cents >= 0)
);

-- Indexes for project filtering by client and status
CREATE INDEX IF NOT EXISTS idx_projects_client_id
  ON projects (client_id);

CREATE INDEX IF NOT EXISTS idx_projects_status
  ON projects (status);

-- Create invoices table with financial tracking and validation rules
CREATE TABLE IF NOT EXISTS invoices (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  project_id INTEGER NOT NULL,
  invoice_number TEXT NOT NULL UNIQUE,
  status TEXT NOT NULL DEFAULT 'draft',
  issue_date TEXT NOT NULL,
  due_date TEXT,
  currency TEXT NOT NULL DEFAULT 'USD',
  subtotal_cents INTEGER NOT NULL DEFAULT 0,
  total_cents INTEGER NOT NULL DEFAULT 0,
  paid_cents INTEGER NOT NULL DEFAULT 0,
  sent_at TEXT,
  paid_at TEXT,
  notes TEXT,
  created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  FOREIGN KEY (project_id) REFERENCES projects (id) ON DELETE RESTRICT,
  CHECK (status IN ('draft', 'sent', 'paid', 'overdue', 'void')),
  CHECK (length(currency) = 3 AND currency = upper(currency)),
  CHECK (subtotal_cents >= 0),
  CHECK (total_cents >= 0),
  CHECK (paid_cents >= 0),
  CHECK (paid_cents <= total_cents)
);

-- Indexes for invoice filtering and date-based queries
CREATE INDEX IF NOT EXISTS idx_invoices_project_id
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      ON invoices (project_id);

    CREATE INDEX IF NOT EXISTS idx_invoices_status
      ON invoices (status);

    CREATE INDEX IF NOT EXISTS idx_invoices_due_date
      ON invoices (due_date);

    CREATE INDEX IF NOT EXISTS idx_invoices_issue_date
      ON invoices (issue_date);

    -- Create line items table for invoice details with position ordering
    CREATE TABLE IF NOT EXISTS invoice_line_items (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      invoice_id INTEGER NOT NULL,
      position INTEGER NOT NULL,
      description TEXT NOT NULL,
      quantity REAL NOT NULL,
      unit_price_cents INTEGER NOT NULL,
      line_total_cents INTEGER NOT NULL,
      created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
      updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
      FOREIGN KEY (invoice_id) REFERENCES invoices (id) ON DELETE CASCADE,
      UNIQUE (invoice_id, position),
      CHECK (position >= 0),
      CHECK (quantity > 0),
      CHECK (unit_price_cents >= 0),
      CHECK (line_total_cents >= 0)
    );

    -- Index for faster line item retrieval by invoice
    CREATE INDEX IF NOT EXISTS idx_invoice_line_items_invoice_id
      ON invoice_line_items (invoice_id);
  `)
})

// Execute the migration immediately
migrate()
```

seed.js

Found in folder: **/src/lib**

This code seeds a database with sample client, project, invoice, and line item data. It first clears existing records from related tables, then inserts the client and project data while mapping project names to their IDs. Finally, it processes invoice data by calculating totals and inserting invoices along with their corresponding line items.

```
import db from './db.js'

// Sample data for clients and their projects
const clients = [
  {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    name: 'Maya Chen',
    email: 'maya.chen@northstar-studio.com',
    phone: '+1 415 555 0194',
    company: 'Northstar Studio',
    address: '210 Market Street, Suite 804, San Francisco, CA 94105',
    notes: 'Prefers concise monthly status summaries.',
    projects: [
      {
        name: 'Brand Identity Refresh',
        description: 'Updated visual identity, design system foundations, and launch
collateral.',
        status: 'completed',
        hourlyRateCents: 12500,
      },
      {
        name: 'Marketing Website Optimization',
        description: 'Landing page experiments, analytics cleanup, and conversion imp
rovements.',
        status: 'active',
        hourlyRateCents: 11500,
      },
    ],
  },
  {
    name: 'Jordan Patel',
    email: 'jordan.patel@harborlegal.com',
    phone: '+1 212 555 0138',
    company: 'Harbor Legal Group',
    address: '88 Broad Street, Floor 12, New York, NY 10004',
    notes: 'Invoices route through operations before payment.',
    projects: [
      {
        name: 'Client Portal Planning',
        description: 'Discovery, technical planning, and implementation roadmap for a
 secure client portal.',
        status: 'completed',
        hourlyRateCents: 14000,
      },
      {
        name: 'Document Workflow Automation',
        description: 'Automation of intake forms, document generation, and internal r
eview queues.',
        status: 'active',
        hourlyRateCents: 15000,
      },
    ],
  },
  {
    name: 'Elena Rodriguez',
    email: 'elena@brightpathfitness.co',
    phone: '+1 512 555 0177',
    company: 'BrightPath Fitness',
    address: '450 South Congress Avenue, Austin, TX 78704',
    notes: 'Seasonal campaigns align with class launches.',
    projects: [
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      {
        name: 'Membership Dashboard',
        description: 'Member retention dashboard and reporting views for studio manag
ers.',
        status: 'active',
        hourlyRateCents: 10500,
      },
      {
        name: 'Spring Campaign Assets',
        description: 'Email templates, ad creative, and performance reporting for spr
ing promotions.',
        status: 'completed',
        hourlyRateCents: 9500,
      },
    ],
  },
  {
    name: 'Thomas Wright',
    email: 'thomas.wright@greenridgehomes.com',
    phone: '+1 303 555 0122',
    company: 'Greenridge Homes',
    address: '1720 Wazee Street, Denver, CO 80202',
    notes: 'Requests itemized invoices by project phase.',
    projects: [
      {
        name: 'Property Listing Platform',
        description: 'Custom listing pages, lead capture, and broker administration t
ools.',
        status: 'active',
        hourlyRateCents: 13000,
      },
      {
        name: 'Sales Collateral System',
        description: 'Reusable templates and content workflow for neighborhood sales
materials.',
        status: 'completed',
        hourlyRateCents: 11000,
      },
    ],
  },
]

// Sample invoice data, each linked to a project by name
const invoiceData = [
  {
    projectName: 'Brand Identity Refresh',
    invoiceNumber: 'INV-2026-001',
    status: 'paid',
    issueDate: '2026-01-08',
    dueDate: '2026-02-07',
    sentAt: '2026-01-08T16:15:00.000Z',
    paidAt: '2026-01-24T18:40:00.000Z',
    notes: 'Final identity package and handoff.',
    items: [
      ['Discovery workshop and stakeholder interviews', 6, 12500],
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      ['Logo refinement and visual identity exploration', 14, 12500],
      ['Brand guideline documentation', 8, 12500],
    ],
  },
  {
    projectName: 'Brand Identity Refresh',
    invoiceNumber: 'INV-2026-002',
    status: 'void',
    issueDate: '2026-02-03',
    dueDate: '2026-03-05',
    sentAt: '2026-02-03T15:30:00.000Z',
    paidAt: null,
    notes: 'Voided and replaced after scope correction.',
    items: [
      ['Unused rush production estimate', 5, 12500],
      ['Draft collateral resizing', 3, 12500],
    ],
  },
  {
    projectName: 'Marketing Website Optimization',
    invoiceNumber: 'INV-2026-003',
    status: 'sent',
    issueDate: '2026-06-14',
    dueDate: '2026-07-14',
    sentAt: '2026-06-14T13:05:00.000Z',
    paidAt: null,
    notes: 'June optimization sprint.',
    items: [
      ['Landing page copy and layout testing', 9, 11500],
      ['Analytics event cleanup', 4, 11500],
      ['Conversion report preparation', 2, 11500],
    ],
  },
  {
    projectName: 'Client Portal Planning',
    invoiceNumber: 'INV-2026-004',
    status: 'paid',
    issueDate: '2026-01-22',
    dueDate: '2026-02-21',
    sentAt: '2026-01-22T14:00:00.000Z',
    paidAt: '2026-02-12T20:10:00.000Z',
    notes: 'Discovery phase completed.',
    items: [
      ['Security requirements workshop', 5, 14000],
      ['Architecture planning and estimates', 10, 14000],
      ['Roadmap presentation', 3, 14000],
    ],
  },
  {
    projectName: 'Document Workflow Automation',
    invoiceNumber: 'INV-2026-005',
    status: 'overdue',
    issueDate: '2026-03-17',
    dueDate: '2026-04-16',
    sentAt: '2026-03-17T17:20:00.000Z',
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    paidAt: null,
    notes: 'Awaiting operations approval.',
    items: [
      ['Form intake mapping', 7, 15000],
      ['Automation prototype build', 12, 15000],
      ['Review queue configuration', 6, 15000],
      ['Internal demo and revisions', 2, 15000],
    ],
  },
  {
    projectName: 'Document Workflow Automation',
    invoiceNumber: 'INV-2026-006',
    status: 'draft',
    issueDate: '2026-06-26',
    dueDate: '2026-07-26',
    sentAt: null,
    paidAt: null,
    notes: 'Draft for current implementation work.',
    items: [
      ['Template generation rules', 8, 15000],
      ['Reviewer notification workflow', 5, 15000],
      ['QA pass and bug fixes', 4, 15000],
    ],
  },
  {
    projectName: 'Membership Dashboard',
    invoiceNumber: 'INV-2026-007',
    status: 'sent',
    issueDate: '2026-05-29',
    dueDate: '2026-06-28',
    sentAt: '2026-05-29T12:45:00.000Z',
    paidAt: null,
    notes: 'Dashboard build milestone.',
    items: [
      ['Retention metric modeling', 6, 10500],
      ['Manager dashboard implementation', 15, 10500],
      ['Data export view', 4, 10500],
    ],
  },
  {
    projectName: 'Membership Dashboard',
    invoiceNumber: 'INV-2026-008',
    status: 'draft',
    issueDate: '2026-06-30',
    dueDate: '2026-07-30',
    sentAt: null,
    paidAt: null,
    notes: 'Draft for reporting refinements.',
    items: [
      ['Weekly cohort chart refinements', 5, 10500],
      ['Studio comparison filters', 6, 10500],
    ],
  },
  {
    projectName: 'Spring Campaign Assets',
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    invoiceNumber: 'INV-2026-009',
    status: 'paid',
    issueDate: '2026-02-19',
    dueDate: '2026-03-21',
    sentAt: '2026-02-19T18:00:00.000Z',
    paidAt: '2026-03-08T16:35:00.000Z',
    notes: 'Campaign asset package.',
    items: [
      ['Email template design', 6, 9500],
      ['Paid social ad variants', 8, 9500],
      ['Campaign performance summary', 3, 9500],
    ],
  },
  {
    projectName: 'Property Listing Platform',
    invoiceNumber: 'INV-2026-010',
    status: 'overdue',
    issueDate: '2026-04-10',
    dueDate: '2026-05-10',
    sentAt: '2026-04-10T11:15:00.000Z',
    paidAt: null,
    notes: 'Listing platform implementation milestone.',
    items: [
      ['Listing detail page build', 11, 13000],
      ['Lead capture integration', 7, 13000],
      ['Broker admin permissions', 5, 13000],
    ],
  },
  {
    projectName: 'Property Listing Platform',
    invoiceNumber: 'INV-2026-011',
    status: 'sent',
    issueDate: '2026-06-05',
    dueDate: '2026-07-05',
    sentAt: '2026-06-05T14:25:00.000Z',
    paidAt: null,
    notes: 'Map and search improvements.',
    items: [
      ['Search filter improvements', 8, 13000],
      ['Map marker clustering', 6, 13000],
      ['Mobile listing QA', 4, 13000],
    ],
  },
  {
    projectName: 'Sales Collateral System',
    invoiceNumber: 'INV-2026-012',
    status: 'paid',
    issueDate: '2026-03-04',
    dueDate: '2026-04-03',
    sentAt: '2026-03-04T10:50:00.000Z',
    paidAt: '2026-03-29T19:05:00.000Z',
    notes: 'Template system delivery.',
    items: [
      ['Template component library', 9, 11000],
      ['Neighborhood flyer layouts', 7, 11000],
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      ['Content publishing documentation', 3, 11000],
    ],
  },
]

// Prepared statements for database insertion
const insertClient = db.prepare(`
  INSERT INTO clients (name, email, phone, company, address, notes)
  VALUES (@name, @email, @phone, @company, @address, @notes)
`)

const insertProject = db.prepare(`
  INSERT INTO projects (client_id, name, description, status, hourly_rate_cents)
  VALUES (@clientId, @name, @description, @status, @hourlyRateCents)
`)

const insertInvoice = db.prepare(`
  INSERT INTO invoices (
    project_id,
    invoice_number,
    status,
    issue_date,
    due_date,
    currency,
    subtotal_cents,
    total_cents,
    paid_cents,
    sent_at,
    paid_at,
    notes
  )
  VALUES (
    @projectId,
    @invoiceNumber,
    @status,
    @issueDate,
    @dueDate,
    @currency,
    @subtotalCents,
    @totalCents,
    @paidCents,
    @sentAt,
    @paidAt,
    @notes
  )
`)

const insertLineItem = db.prepare(`
  INSERT INTO invoice_line_items (
    invoice_id,
    position,
    description,
    quantity,
    unit_price_cents,
    line_total_cents
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  )
  VALUES (
    @invoiceId,
    @position,
    @description,
    @quantity,
    @unitPriceCents,
    @lineTotalCents
  )
`)

// Transaction ensures all inserts succeed or fail together
const seed = db.transaction(() => {
  // Clear existing data in reverse order of foreign key dependencies
  db.exec(`
    DELETE FROM invoice_line_items;
    DELETE FROM invoices;
    DELETE FROM projects;
    DELETE FROM clients;
  `)

  const projectIdsByName = new Map()

  // Insert clients and their projects, storing project IDs by name for invoice linki
ng
  for (const client of clients) {
    const clientId = insertClient.run(client).lastInsertRowid

    for (const project of client.projects) {
      const projectId = insertProject.run({
        clientId,
        name: project.name,
        description: project.description,
        status: project.status,
        hourlyRateCents: project.hourlyRateCents,
      }).lastInsertRowid

      projectIdsByName.set(project.name, projectId)
    }
  }

  // Process each invoice, calculate totals, and insert invoice with line items
  for (const invoice of invoiceData) {
    const projectId = projectIdsByName.get(invoice.projectName)

    if (!projectId) {
      throw new Error(`Missing project for invoice ${invoice.invoiceNumber}`)
    }

    // Calculate line item totals and overall invoice total
    const lineItems = invoice.items.map(([description, quantity, unitPriceCents], ind
ex) => ({
      position: index,
      description,
      quantity,
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      unitPriceCents,
      lineTotalCents: Math.round(quantity * unitPriceCents),
    }))
    const totalCents = lineItems.reduce((total, item) => total + item.lineTotalCents,
 0)
    const paidCents = invoice.status === 'paid' ? totalCents : 0
    const invoiceId = insertInvoice.run({
      projectId,
      invoiceNumber: invoice.invoiceNumber,
      status: invoice.status,
      issueDate: invoice.issueDate,
      dueDate: invoice.dueDate,
      currency: 'USD',
      subtotalCents: totalCents,
      totalCents,
      paidCents,
      sentAt: invoice.sentAt,
      paidAt: invoice.paidAt,
      notes: invoice.notes,
    }).lastInsertRowid

    // Insert each line item for this invoice
    for (const item of lineItems) {
      insertLineItem.run({
        invoiceId,
        ...item,
      })
    }
  }
})

seed()
```

services.js

Found in folder: **/src/lib**

This code provides utility functions and database operations for managing clients, projects, and invoices. It includes validation and normalization functions for data inputs, as well as CRUD operations that interact with a SQLite database. The functions handle business logic like calculating totals, ensuring data integrity, and generating summary reports.

```
import 'server-only'

import db from './db.js'

// Valid status values for projects and invoices
const PROJECT_STATUSES = new Set(['active', 'paused', 'completed', 'archived'])
const INVOICE_STATUSES = new Set(['draft', 'sent', 'paid', 'overdue', 'void'])

// Returns current date/time in ISO format
function nowIso() {
  return new Date().toISOString()
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
}

// Returns first day of the month for a given date
function monthStart(date) {
  return new Date(Date.UTC(date.getUTCFullYear(), date.getUTCMonth(), 1))
}

// Adds specified number of months to a date
function addMonths(date, count) {
  return new Date(Date.UTC(date.getUTCFullYear(), date.getUTCMonth() + count, 1))
}

// Formats date as YYYY-MM-DD string
function formatDateOnly(date) {
  return date.toISOString().slice(0, 10)
}

// Formats date as "MMM YYYY" (e.g., "Jan 2024")
function formatMonthLabel(date) {
  return new Intl.DateTimeFormat('en-US', {
    month: 'short',
    year: 'numeric',
    timeZone: 'UTC',
  }).format(date)
}

// Converts ID to positive integer, throws if invalid
function normalizeId(id, label = 'id') {
  const value = Number(id)

  if (!Number.isInteger(value) || value <= 0) {
    throw new Error(`Invalid ${label}: expected a positive integer`)
  }

  return value
}

// Converts value to trimmed string or null if empty/undefined
function normalizeOptionalText(value) {
  if (value === undefined || value === null) {
    return null
  }

  const text = String(value).trim()
  return text === '' ? null : text
}

// Requires non-empty text value
function requireText(value, label) {
  const text = normalizeOptionalText(value)

  if (!text) {
    throw new Error(`${label} is required`)
  }
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  return text
}

// Validates money amount in cents
function normalizeMoney(value, label, { allowNull = false } = {}) {
  if (value === undefined || value === null || value === '') {
    if (allowNull) {
      return null
    }

    throw new Error(`${label} is required`)
  }

  const cents = Number(value)

  if (!Number.isInteger(cents) || cents < 0) {
    throw new Error(`${label} must be a non-negative integer number of cents`)
  }

  return cents
}

// Validates quantity is a positive finite number
function normalizeQuantity(value, label = 'Quantity') {
  const quantity = Number(value)

  if (!Number.isFinite(quantity) || quantity <= 0) {
    throw new Error(`${label} must be greater than zero`)
  }

  return quantity
}

// Validates currency code (defaults to USD)
function normalizeCurrency(value) {
  const currency = value === undefined || value === null || value === ''
    ? 'USD'
    : String(value).trim().toUpperCase()

  if (!/^[A-Z]{3}$/.test(currency)) {
    throw new Error('Currency must be a 3-letter ISO code')
  }

  return currency
}

// Validates project status (defaults to 'active')
function normalizeProjectStatus(value) {
  const status = value === undefined || value === null || value === ''
    ? 'active'
    : String(value).trim()

  if (!PROJECT_STATUSES.has(status)) {
    throw new Error(`Invalid project status: ${status}`)
  }
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  return status
}

// Validates invoice status (defaults to 'draft')
function normalizeInvoiceStatus(value) {
  const status = value === undefined || value === null || value === ''
    ? 'draft'
    : String(value).trim()

  if (!INVOICE_STATUSES.has(status)) {
    throw new Error(`Invalid invoice status: ${status}`)
  }

  return status
}

// Validates and normalizes invoice line items array
function normalizeLineItems(lineItems) {
  if (!Array.isArray(lineItems) || lineItems.length === 0) {
    throw new Error('At least one invoice line item is required')
  }

  return lineItems.map((item, index) => {
    const quantity = normalizeQuantity(item.quantity, `Line item ${index + 1} quantit
y`)
    const unitPriceCents = normalizeMoney(
      item.unitPriceCents,
      `Line item ${index + 1} unit price`,
    )

    return {
      position: index,
      description: requireText(item.description, `Line item ${index + 1} description`
),
      quantity,
      unitPriceCents,
      lineTotalCents: Math.round(quantity * unitPriceCents),
    }
  })
}

// Validates and normalizes complete invoice data
function normalizeInvoiceData(data) {
  if (!data || typeof data !== 'object') {
    throw new Error('Invoice data is required')
  }

  const lineItems = normalizeLineItems(data.lineItems)
  const totalCents = lineItems.reduce((sum, item) => sum + item.lineTotalCents, 0)
  const paidCents = normalizeMoney(data.paidCents ?? 0, 'Paid amount')
  const status = normalizeInvoiceStatus(data.status)

  // Business rule validation
  if (paidCents > totalCents) {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    throw new Error('Paid amount cannot exceed invoice total')
  }

  if (status === 'paid' && paidCents !== totalCents) {
    throw new Error('Paid invoices must have paid amount equal to total')
  }

  if (status !== 'paid' && data.paidAt) {
    throw new Error('Only paid invoices can have a paid date')
  }

  return {
    projectId: normalizeId(data.projectId, 'project id'),
    invoiceNumber: requireText(data.invoiceNumber, 'Invoice number'),
    status,
    issueDate: requireText(data.issueDate, 'Issue date'),
    dueDate: normalizeOptionalText(data.dueDate),
    currency: normalizeCurrency(data.currency),
    subtotalCents: totalCents,
    totalCents,
    paidCents,
    sentAt: normalizeOptionalText(data.sentAt),
    paidAt: normalizeOptionalText(data.paidAt),
    notes: normalizeOptionalText(data.notes),
    lineItems,
  }
}

// Maps database row to client object
function mapClient(row) {
  if (!row) {
    return null
  }

  return {
    id: row.id,
    name: row.name,
    email: row.email,
    phone: row.phone,
    company: row.company,
    address: row.address,
    notes: row.notes,
    createdAt: row.created_at,
    updatedAt: row.updated_at,
  }
}

// Maps database row to project object
function mapProject(row) {
  if (!row) {
    return null
  }

  return {
    id: row.id,
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    clientId: row.client_id,
    name: row.name,
    description: row.description,
    status: row.status,
    hourlyRateCents: row.hourly_rate_cents,
    createdAt: row.created_at,
    updatedAt: row.updated_at,
  }
}

// Maps database row to invoice object
function mapInvoice(row) {
  if (!row) {
    return null
  }

  return {
    id: row.id,
    projectId: row.project_id,
    invoiceNumber: row.invoice_number,
    status: row.status,
    issueDate: row.issue_date,
    dueDate: row.due_date,
    currency: row.currency,
    subtotalCents: row.subtotal_cents,
    totalCents: row.total_cents,
    paidCents: row.paid_cents,
    sentAt: row.sent_at,
    paidAt: row.paid_at,
    notes: row.notes,
    createdAt: row.created_at,
    updatedAt: row.updated_at,
  }
}

// Maps database row to line item object
function mapLineItem(row) {
  return {
    id: row.id,
    invoiceId: row.invoice_id,
    position: row.position,
    description: row.description,
    quantity: row.quantity,
    unitPriceCents: row.unit_price_cents,
    lineTotalCents: row.line_total_cents,
    createdAt: row.created_at,
    updatedAt: row.updated_at,
  }
}

// Throws error if client doesn't exist
function ensureClientExists(id) {
  const client = db.prepare('SELECT id FROM clients WHERE id = ?').get(id)

  if (!client) {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    throw new Error(`Client ${id} does not exist`)
  }
}

// Throws error if project doesn't exist
function ensureProjectExists(id) {
  const project = db.prepare('SELECT id FROM projects WHERE id = ?').get(id)

  if (!project) {
    throw new Error(`Project ${id} does not exist`)
  }
}

// Throws error if invoice doesn't exist
function ensureInvoiceExists(id) {
  const invoice = db.prepare('SELECT id FROM invoices WHERE id = ?').get(id)

  if (!invoice) {
    throw new Error(`Invoice ${id} does not exist`)
  }
}

// Returns all clients with active project count and unpaid amount
export function getClients() {
  const rows = db.prepare(`
    SELECT
      c.*,
      COUNT(DISTINCT CASE WHEN p.status = 'active' THEN p.id END) AS active_project_c
ount,
      COALESCE(SUM(
        CASE
          WHEN i.status != 'void' AND i.paid_cents < i.total_cents
            THEN i.total_cents - i.paid_cents
          ELSE 0
        END
      ), 0) AS total_unpaid_cents
    FROM clients c
    LEFT JOIN projects p ON p.client_id = c.id
    LEFT JOIN invoices i ON i.project_id = p.id
    GROUP BY c.id
    ORDER BY c.name COLLATE NOCASE ASC
  `).all()

  return rows.map((row) => ({
    ...mapClient(row),
    activeProjectCount: row.active_project_count,
    totalUnpaidCents: row.total_unpaid_cents,
  }))
}

// Returns single client by ID with project count and unpaid amount
export function getClientById(id) {
  const clientId = normalizeId(id, 'client id')
  const row = db.prepare(`
    SELECT
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      c.*,
      COUNT(DISTINCT p.id) AS project_count,
      COALESCE(SUM(
        CASE
          WHEN i.status != 'void' AND i.paid_cents < i.total_cents
            THEN i.total_cents - i.paid_cents
          ELSE 0
        END
      ), 0) AS total_unpaid_cents
    FROM clients c
    LEFT JOIN projects p ON p.client_id = c.id
    LEFT JOIN invoices i ON i.project_id = p.id
    WHERE c.id = ?
    GROUP BY c.id
  `).get(clientId)

  if (!row) {
    return null
  }

  return {
    ...mapClient(row),
    projectCount: row.project_count,
    totalUnpaidCents: row.total_unpaid_cents,
  }
}

// Creates new client record
export function createClient(data) {
  if (!data || typeof data !== 'object') {
    throw new Error('Client data is required')
  }

  const result = db.prepare(`
    INSERT INTO clients (name, email, phone, company, address, notes)
    VALUES (@name, @email, @phone, @company, @address, @notes)
  `).run({
    name: requireText(data.name, 'Client name'),
    email: normalizeOptionalText(data.email),
    phone: normalizeOptionalText(data.phone),
    company: normalizeOptionalText(data.company),
    address: normalizeOptionalText(data.address),
    notes: normalizeOptionalText(data.notes),
  })

  return getClientById(result.lastInsertRowid)
}

// Updates existing client record
export function updateClient(id, data) {
  const clientId = normalizeId(id, 'client id')
  const existing = mapClient(db.prepare('SELECT * FROM clients WHERE id = ?').get(cli
entId))

  if (!existing) {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    throw new Error(`Client ${clientId} does not exist`)
  }

  db.prepare(`
    UPDATE clients
    SET
      name = @name,
      email = @email,
      phone = @phone,
      company = @company,
      address = @address,
      notes = @notes,
      updated_at = @updatedAt
    WHERE id = @id
  `).run({
    id: clientId,
    name: data?.name === undefined ? existing.name : requireText(data.name, 'Client n
ame'),
    email: data?.email === undefined ? existing.email : normalizeOptionalText(data.em
ail),
    phone: data?.phone === undefined ? existing.phone : normalizeOptionalText(data.ph
one),
    company: data?.company === undefined ? existing.company : normalizeOptionalText(d
ata.company),
    address: data?.address === undefined ? existing.address : normalizeOptionalText(d
ata.address),
    notes: data?.notes === undefined ? existing.notes : normalizeOptionalText(data.no
tes),
    updatedAt: nowIso(),
  })

  return getClientById(clientId)
}

// Deletes client if they have no projects
export function deleteClient(id) {
  const clientId = normalizeId(id, 'client id')
  ensureClientExists(clientId)

  const projectCount = db.prepare('SELECT COUNT(*) AS count FROM projects WHERE clien
t_id = ?')
    .get(clientId).count

  if (projectCount > 0) {
    throw new Error(`Cannot delete client ${clientId} because it has projects`)
  }

  db.prepare('DELETE FROM clients WHERE id = ?').run(clientId)
}

// Returns projects with optional filtering
export function getProjects(filters = {}) {
  const where = []
  const params = {}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  if (filters.clientId !== undefined && filters.clientId !== null && filters.clientId
 !== '') {
    params.clientId = normalizeId(filters.clientId, 'client id')
    where.push('p.client_id = @clientId')
  }

  if (filters.status !== undefined && filters.status !== null && filters.status !== '
') {
    params.status = normalizeProjectStatus(filters.status)
    where.push('p.status = @status')
  }

  const rows = db.prepare(`
    SELECT
      p.*,
      c.name AS client_name,
      COALESCE(SUM(
        CASE
          WHEN i.status != 'void' THEN i.total_cents
          ELSE 0
        END
      ), 0) AS total_invoiced_cents
    FROM projects p
    INNER JOIN clients c ON c.id = p.client_id
    LEFT JOIN invoices i ON i.project_id = p.id
    ${where.length > 0 ? `WHERE ${where.join(' AND ')}` : ''}
    GROUP BY p.id
    ORDER BY p.name COLLATE NOCASE ASC
  `).all(params)

  return rows.map((row) => ({
    ...mapProject(row),
    clientName: row.client_name,
    totalInvoicedCents: row.total_invoiced_cents,
  }))
}

// Returns single project by ID with client details and invoice summary
export function getProjectById(id) {
  const projectId = normalizeId(id, 'project id')
  const row = db.prepare(`
    SELECT
      p.*,
      c.name AS client_name,
      c.email AS client_email,
      c.phone AS client_phone,
      c.company AS client_company,
      c.address AS client_address,
      c.notes AS client_notes,
      COUNT(i.id) AS invoice_count,
      COALESCE(SUM(i.total_cents), 0) AS invoice_total_cents,
      COALESCE(SUM(i.paid_cents), 0) AS invoice_paid_cents,
      COALESCE(SUM(
        CASE
          WHEN i.status != 'void' AND i.paid_cents < i.total_cents
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
            THEN i.total_cents - i.paid_cents
          ELSE 0
        END
      ), 0) AS invoice_unpaid_cents
    FROM projects p
    INNER JOIN clients c ON c.id = p.client_id
    LEFT JOIN invoices i ON i.project_id = p.id
    WHERE p.id = ?
    GROUP BY p.id
  `).get(projectId)

  if (!row) {
    return null
  }

  return {
    ...mapProject(row),
    client: {
      id: row.client_id,
      name: row.client_name,
      email: row.client_email,
      phone: row.client_phone,
      company: row.client_company,
      address: row.client_address,
      notes: row.client_notes,
    },
    invoiceSummary: {
      count: row.invoice_count,
      totalCents: row.invoice_total_cents,
      paidCents: row.invoice_paid_cents,
      unpaidCents: row.invoice_unpaid_cents,
    },
  }
}

// Creates new project record
export function createProject(data) {
  if (!data || typeof data !== 'object') {
    throw new Error('Project data is required')
  }

  const clientId = normalizeId(data.clientId, 'client id')
  ensureClientExists(clientId)

  const result = db.prepare(`
    INSERT INTO projects (client_id, name, description, status, hourly_rate_cents)
    VALUES (@clientId, @name, @description, @status, @hourlyRateCents)
  `).run({
    clientId,
    name: requireText(data.name, 'Project name'),
    description: normalizeOptionalText(data.description),
    status: normalizeProjectStatus(data.status),
    hourlyRateCents: normalizeMoney(data.hourlyRateCents, 'Hourly rate', { allowNull:
 true }),
  })
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  return getProjectById(result.lastInsertRowid)
}

// Updates existing project record
export function updateProject(id, data) {
  const projectId = normalizeId(id, 'project id')
  const existing = mapProject(db.prepare('SELECT * FROM projects WHERE id = ?').get(p
rojectId))

  if (!existing) {
    throw new Error(`Project ${projectId} does not exist`)
  }

  const clientId = data?.clientId === undefined
    ? existing.clientId
    : normalizeId(data.clientId, 'client id')

  ensureClientExists(clientId)

  db.prepare(`
    UPDATE projects
    SET
      client_id = @clientId,
      name = @name,
      description = @description,
      status = @status,
      hourly_rate_cents = @hourlyRateCents,
      updated_at = @updatedAt
    WHERE id = @id
  `).run({
    id: projectId,
    clientId,
    name: data?.name === undefined ? existing.name : requireText(data.name, 'Project
name'),
    description: data?.description === undefined
      ? existing.description
      : normalizeOptionalText(data.description),
    status: data?.status === undefined ? existing.status : normalizeProjectStatus(dat
a.status),
    hourlyRateCents: data?.hourlyRateCents === undefined
      ? existing.hourlyRateCents
      : normalizeMoney(data.hourlyRateCents, 'Hourly rate', { allowNull: true }),
    updatedAt: nowIso(),
  })

  return getProjectById(projectId)
}

// Deletes project if it has no invoices
export function deleteProject(id) {
  const projectId = normalizeId(id, 'project id')
  ensureProjectExists(projectId)

  const invoiceCount = db.prepare('SELECT COUNT(*) AS count FROM invoices WHERE proje
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
ct_id = ?')
    .get(projectId).count

  if (invoiceCount > 0) {
    throw new Error(`Cannot delete project ${projectId} because it has invoices`)
  }

  db.prepare('DELETE FROM projects WHERE id = ?').run(projectId)
}

// Returns dashboard summary statistics
export function getDashboardSummary() {
  const currentMonthStart = monthStart(new Date())
  const nextMonthStart = addMonths(currentMonthStart, 1)

  const row = db.prepare(`
    SELECT
      COALESCE(SUM(
        CASE
          WHEN i.status IN ('sent', 'overdue') THEN i.total_cents - i.paid_cents
          ELSE 0
        END
      ), 0) AS total_outstanding_cents,
      COALESCE(SUM(
        CASE
          WHEN i.status = 'paid'
            AND i.paid_at >= @currentMonthStart
            AND i.paid_at < @nextMonthStart
            THEN i.paid_cents
          ELSE 0
        END
      ), 0) AS total_paid_this_month_cents,
      COUNT(CASE WHEN i.status = 'overdue' THEN 1 END) AS overdue_count
    FROM invoices i
  `).get({
    currentMonthStart: currentMonthStart.toISOString(),
    nextMonthStart: nextMonthStart.toISOString(),
  })

  const topClient = db.prepare(`
    SELECT c.name
    FROM clients c
    INNER JOIN projects p ON p.client_id = c.id
    INNER JOIN invoices i ON i.project_id = p.id
    GROUP BY c.id
    HAVING SUM(i.paid_cents) > 0
    ORDER BY SUM(i.paid_cents) DESC, c.name COLLATE NOCASE ASC
    LIMIT 1
  `).get()

  return {
    totalOutstandingCents: row.total_outstanding_cents,
    totalPaidThisMonthCents: row.total_paid_this_month_cents,
    overdueCount: row.overdue_count,
    topClientByRevenue: topClient?.name ?? null,
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  }
}

// Returns revenue data for last 6 months
export function getMonthlyRevenue() {
  const currentMonthStart = monthStart(new Date())
  const months = Array.from({ length: 6 }, (_, index) => {
    const start = addMonths(currentMonthStart, index - 5)
    const end = addMonths(start, 1)

    return {
      month: formatMonthLabel(start),
      start: formatDateOnly(start),
      end: formatDateOnly(end),
    }
  })

  const totalsByMonth = new Map(db.prepare(`
    SELECT
      strftime('%Y-%m', issue_date) AS month_key,
      COALESCE(SUM(total_cents), 0) AS total_cents
    FROM invoices
    WHERE issue_date >= @startDate
      AND issue_date < @endDate
    GROUP BY month_key
  `).all({
    startDate: months[0].start,
    endDate: months[months.length - 1].end,
  }).map((row) => [row.month_key, row.total_cents]))

  return months.map((month) => ({
    month: month.month,
    totalCents: totalsByMonth.get(month.start.slice(0, 7)) ?? 0,
  }))
}

// Returns all overdue invoices with client/project details
export function getOverdueInvoices() {
  const rows = db.prepare(`
    SELECT
      i.*,
      p.name AS project_name,
      c.id AS client_id,
      c.name AS client_name
    FROM invoices i
    INNER JOIN projects p ON p.id = i.project_id
    INNER JOIN clients c ON c.id = p.client_id
    WHERE i.status = 'overdue'
    ORDER BY i.due_date ASC, i.id ASC
  `).all()

  return rows.map((row) => ({
    ...mapInvoice(row),
    clientId: row.client_id,
    clientName: row.client_name,
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    projectName: row.project_name,
  }))
}

// Returns invoices with optional filtering
export function getInvoices(filters = {}) {
  const where = []
  const params = {}

  if (filters.status !== undefined && filters.status !== null && filters.status !== '
') {
    params.status = normalizeInvoiceStatus(filters.status)
    where.push('i.status = @status')
  }

  if (filters.clientId !== undefined && filters.clientId !== null && filters.clientId
 !== '') {
    params.clientId = normalizeId(filters.clientId, 'client id')
    where.push('p.client_id = @clientId')
  }

  if (filters.projectId !== undefined && filters.projectId !== null && filters.projec
tId !== '') {
    params.projectId = normalizeId(filters.projectId, 'project id')
    where.push('i.project_id = @projectId')
  }

  const rows = db.prepare(`
    SELECT
      i.*,
      p.name AS project_name,
      c.id AS client_id,
      c.name AS client_name
    FROM invoices i
    INNER JOIN projects p ON p.id = i.project_id
    INNER JOIN clients c ON c.id = p.client_id
    ${where.length > 0 ? `WHERE ${where.join(' AND ')}` : ''}
    ORDER BY i.issue_date DESC, i.id DESC
  `).all(params)

  return rows.map((row) => ({
    ...mapInvoice(row),
    projectName: row.project_name,
    clientId: row.client_id,
    clientName: row.client_name,
  }))
}

// Returns single invoice by ID with line items and related details
export function getInvoiceById(id) {
  const invoiceId = normalizeId(id, 'invoice id')
  const row = db.prepare(`
    SELECT
      i.*,
      p.name AS project_name,
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      p.description AS project_description,
      p.status AS project_status,
      p.hourly_rate_cents AS project_hourly_rate_cents,
      c.id AS client_id,
      c.name AS client_name,
      c.email AS client_email,
      c.phone AS client_phone,
      c.company AS client_company,
      c.address AS client_address,
      c.notes AS client_notes
    FROM invoices i
    INNER JOIN projects p ON p.id = i.project_id
    INNER JOIN clients c ON c.id = p.client_id
    WHERE i.id = ?
  `).get(invoiceId)

  if (!row) {
    return null
  }

  const lineItems = db.prepare(`
    SELECT *
    FROM invoice_line_items
    WHERE invoice_id = ?
    ORDER BY position ASC
  `).all(invoiceId).map(mapLineItem)

  return {
    ...mapInvoice(row),
    lineItems,
    project: {
      id: row.project_id,
      name: row.project_name,
      description: row.project_description,
      status: row.project_status,
      hourlyRateCents: row.project_hourly_rate_cents,
    },
    client: {
      id: row.client_id,
      name: row.client_name,
      email: row.client_email,
      phone: row.client_phone,
      company: row.client_company,
      address: row.client_address,
      notes: row.client_notes,
    },
  }
}

// Transaction to create invoice with line items atomically
const insertInvoiceWithItems = db.transaction((data) => {
  ensureProjectExists(data.projectId)

  const result = db.prepare(`
    INSERT INTO invoices (
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    project_id,
    invoice_number,
    status,
    issue_date,
    due_date,
    currency,
    subtotal_cents,
    total_cents,
    paid_cents,
    sent_at,
    paid_at,
    notes
  )
  VALUES (
    @projectId,
    @invoiceNumber,
    @status,
    @issueDate,
    @dueDate,
    @currency,
    @subtotalCents,
    @totalCents,
    @paidCents,
    @sentAt,
    @paidAt,
    @notes
  )
`).run(data)

const invoiceId = result.lastInsertRowid
const insertLineItem = db.prepare(`
  INSERT INTO invoice_line_items (
    invoice_id,
    position,
    description,
    quantity,
    unit_price_cents,
    line_total_cents
  )
  VALUES (
    @invoiceId,
    @position,
    @description,
    @quantity,
    @unitPriceCents,
    @lineTotalCents
  )
`)

for (const item of data.lineItems) {
  insertLineItem.run({
    invoiceId,
    ...item,
  })
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  return invoiceId
})

// Creates new invoice with validation
export function createInvoice(data) {
  const invoice = normalizeInvoiceData(data)

  try {
    const invoiceId = insertInvoiceWithItems(invoice)
    return getInvoiceById(invoiceId)
  } catch (error) {
    if (error.code === 'SQLITE_CONSTRAINT_UNIQUE') {
      throw new Error(`Invoice number ${invoice.invoiceNumber} already exists`)
    }

    throw error
  }
}

// Transaction to update invoice and replace line items atomically
const updateInvoiceWithItems = db.transaction((id, data) => {
  ensureInvoiceExists(id)
  ensureProjectExists(data.projectId)

  db.prepare(`
    UPDATE invoices
    SET
      project_id = @projectId,
      invoice_number = @invoiceNumber,
      status = @status,
      issue_date = @issueDate,
      due_date = @dueDate,
      currency = @currency,
      subtotal_cents = @subtotalCents,
      total_cents = @totalCents,
      paid_cents = @paidCents,
      sent_at = @sentAt,
      paid_at = @paidAt,
      notes = @notes,
      updated_at = @updatedAt
    WHERE id = @id
  `).run({
    id,
    updatedAt: nowIso(),
    ...data,
  })

  db.prepare('DELETE FROM invoice_line_items WHERE invoice_id = ?').run(id)

  const insertLineItem = db.prepare(`
    INSERT INTO invoice_line_items (
      invoice_id,
      position,
      description,
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      quantity,
      unit_price_cents,
      line_total_cents
    )
    VALUES (
      @invoiceId,
      @position,
      @description,
      @quantity,
      @unitPriceCents,
      @lineTotalCents
    )
  `)

  for (const item of data.lineItems) {
    insertLineItem.run({
      invoiceId: id,
      ...item,
    })
  }
})

// Updates existing invoice with validation
export function updateInvoice(id, data) {
  const invoiceId = normalizeId(id, 'invoice id')
  const invoice = normalizeInvoiceData(data)

  try {
    updateInvoiceWithItems(invoiceId, invoice)
    return getInvoiceById(invoiceId)
  } catch (error) {
    if (error.code === 'SQLITE_CONSTRAINT_UNIQUE') {
      throw new Error(`Invoice number ${invoice.invoiceNumber} already exists`)
    }

    throw error
  }
}

// Deletes invoice record
export function deleteInvoice(id) {
  const invoiceId = normalizeId(id, 'invoice id')
  ensureInvoiceExists(invoiceId)
  db.prepare('DELETE FROM invoices WHERE id = ?').run(invoiceId)
}
```

services.test.js

Found in folder: **/src/lib**

This code sets up a test suite for client, invoice, project, and dashboard services. It includes helper functions for date manipulation and seeding a test database with sample data. Each test case verifies that the service functions correctly handle data operations like creation, retrieval, and

© 2026 Zenva Pty Ltd. All rights reserved.

deletion.

```
import { beforeEach, describe, expect, it } from 'vitest'

import db from './db.js'
import './migrate.js'
import {
  createClient,
  createInvoice,
  deleteClient,
  deleteInvoice,
  getDashboardSummary,
  getClientById,
  getClients,
  getInvoiceById,
  getInvoices,
  getMonthlyRevenue,
  getOverdueInvoices,
  getProjects,
} from './services.js'

// Helper functions for date calculations
function monthStart(date) {
  return new Date(Date.UTC(date.getUTCFullYear(), date.getUTCMonth(), 1))
}

function addMonths(date, count) {
  return new Date(Date.UTC(date.getUTCFullYear(), date.getUTCMonth() + count, 1))
}

function dateOnly(date) {
  return date.toISOString().slice(0, 10)
}

function monthLabel(date) {
  return new Intl.DateTimeFormat('en-US', {
    month: 'short',
    year: 'numeric',
    timeZone: 'UTC',
  }).format(date)
}

// Populates the database with test data before each test
function seedTestData() {
  // Clear existing data
  db.exec(`
    DELETE FROM invoice_line_items;
    DELETE FROM invoices;
    DELETE FROM projects;
    DELETE FROM clients;
  `)

  // Prepared statements for inserting test data
  const insertClient = db.prepare(`
    INSERT INTO clients (name, email, phone, company, address, notes)
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  VALUES (@name, @email, @phone, @company, @address, @notes)
`)
const insertProject = db.prepare(`
  INSERT INTO projects (client_id, name, description, status, hourly_rate_cents)
  VALUES (@clientId, @name, @description, @status, @hourlyRateCents)
`)
const insertInvoice = db.prepare(`
  INSERT INTO invoices (
    project_id,
    invoice_number,
    status,
    issue_date,
    due_date,
    currency,
    subtotal_cents,
    total_cents,
    paid_cents,
    sent_at,
    paid_at,
    notes
  )
  VALUES (
    @projectId,
    @invoiceNumber,
    @status,
    @issueDate,
    @dueDate,
    @currency,
    @subtotalCents,
    @totalCents,
    @paidCents,
    @sentAt,
    @paidAt,
    @notes
  )
`)
const insertLineItem = db.prepare(`
  INSERT INTO invoice_line_items (
    invoice_id,
    position,
    description,
    quantity,
    unit_price_cents,
    line_total_cents
  )
  VALUES (
    @invoiceId,
    @position,
    @description,
    @quantity,
    @unitPriceCents,
    @lineTotalCents
  )
`)
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
// Insert two test clients
const acmeClientId = insertClient.run({
  name: 'Acme Studio',
  email: 'billing@acme.test',
  phone: null,
  company: 'Acme',
  address: null,
  notes: 'Primary test client',
}).lastInsertRowid

const betaClientId = insertClient.run({
  name: 'Beta Labs',
  email: 'ops@beta.test',
  phone: null,
  company: 'Beta',
  address: null,
  notes: null,
}).lastInsertRowid

// Insert a project for Acme
const projectId = insertProject.run({
  clientId: acmeClientId,
  name: 'Website Build',
  description: 'Marketing site',
  status: 'active',
  hourlyRateCents: 10000,
}).lastInsertRowid

// Insert an invoice for the project
const invoiceId = insertInvoice.run({
  projectId,
  invoiceNumber: 'INV-TEST-001',
  status: 'sent',
  issueDate: '2026-06-01',
  dueDate: '2026-07-01',
  currency: 'USD',
  subtotalCents: 25000,
  totalCents: 25000,
  paidCents: 0,
  sentAt: '2026-06-01T12:00:00.000Z',
  paidAt: null,
  notes: 'Seed invoice',
}).lastInsertRowid

// Add line items to the invoice
insertLineItem.run({
  invoiceId,
  position: 0,
  description: 'Design',
  quantity: 1,
  unitPriceCents: 10000,
  lineTotalCents: 10000,
})
insertLineItem.run({
  invoiceId,
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    position: 1,
    description: 'Development',
    quantity: 3,
    unitPriceCents: 5000,
    lineTotalCents: 15000,
  })

  return {
    acmeClientId,
    betaClientId,
    projectId,
    invoiceId,
  }
}

// Helper to insert a project for testing
function insertProject(clientId, name) {
  return db.prepare(`
    INSERT INTO projects (client_id, name, description, status, hourly_rate_cents)
    VALUES (?, ?, ?, ?, ?)
  `).run(clientId, name, null, 'active', null).lastInsertRowid
}

// Helper to insert an invoice for testing
function insertInvoice({
  projectId,
  invoiceNumber,
  status,
  issueDate,
  dueDate = null,
  totalCents,
  paidCents = 0,
  paidAt = null,
}) {
  return db.prepare(`
    INSERT INTO invoices (
      project_id,
      invoice_number,
      status,
      issue_date,
      due_date,
      currency,
      subtotal_cents,
      total_cents,
      paid_cents,
      sent_at,
      paid_at,
      notes
    )
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
  `).run(
    projectId,
    invoiceNumber,
    status,
    issueDate,
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    dueDate,
    'USD',
    totalCents,
    totalCents,
    paidCents,
    null,
    paidAt,
    null,
  ).lastInsertRowid
}

let fixture

// Reset test data before each test
beforeEach(() => {
  fixture = seedTestData()
})

describe('client services', () => {
  it('getClients() returns all clients', () => {
    const clients = getClients()

    expect(clients).toHaveLength(2)
    expect(clients.map((client) => client.name)).toEqual(['Acme Studio', 'Beta Labs']
)
    expect(clients[0]).toMatchObject({
      activeProjectCount: 1,
      totalUnpaidCents: 25000,
    })
    expect(clients[1]).toMatchObject({
      activeProjectCount: 0,
      totalUnpaidCents: 0,
    })
  })

  it('getClients() counts active projects and unpaid balances', () => {
    // Add a paused project and additional invoices to test aggregation
    const pausedProjectId = db.prepare(`
      INSERT INTO projects (client_id, name, description, status, hourly_rate_cents)
      VALUES (?, ?, ?, ?, ?)
    `).run(
      fixture.acmeClientId,
      'Retainer',
      'Paused work',
      'paused',
      12000,
    ).lastInsertRowid

    db.prepare(`
      INSERT INTO invoices (
        project_id,
        invoice_number,
        status,
        issue_date,
        due_date,
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    currency,
    subtotal_cents,
    total_cents,
    paid_cents,
    sent_at,
    paid_at,
    notes
  )
  VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
`).run(
  pausedProjectId,
  'INV-TEST-002',
  'sent',
  '2026-06-10',
  '2026-07-10',
  'USD',
  10000,
  10000,
  2500,
  '2026-06-10T12:00:00.000Z',
  null,
  null,
)

db.prepare(`
  INSERT INTO invoices (
    project_id,
    invoice_number,
    status,
    issue_date,
    due_date,
    currency,
    subtotal_cents,
    total_cents,
    paid_cents,
    sent_at,
    paid_at,
    notes
  )
  VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
`).run(
  fixture.projectId,
  'INV-TEST-003',
  'void',
  '2026-06-11',
  '2026-07-11',
  'USD',
  50000,
  50000,
  0,
  null,
  null,
  null,
)
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    const [client] = getClients()

    expect(client).toMatchObject({
      id: fixture.acmeClientId,
      activeProjectCount: 1,
      totalUnpaidCents: 32500,
    })
  })

  it('getClientById() returns the correct client', () => {
    const client = getClientById(fixture.acmeClientId)

    expect(client).toMatchObject({
      id: fixture.acmeClientId,
      name: 'Acme Studio',
      projectCount: 1,
      totalUnpaidCents: 25000,
    })
  })

  it('createClient() inserts and returns a new client', () => {
    const client = createClient({
      name: 'Gamma Co',
      email: 'hello@gamma.test',
      phone: '555-0100',
      company: 'Gamma',
      address: '1 Test Way',
      notes: 'New client',
    })

    expect(client).toMatchObject({
      name: 'Gamma Co',
      email: 'hello@gamma.test',
      projectCount: 0,
      totalUnpaidCents: 0,
    })
    expect(getClients()).toHaveLength(3)
  })

  it('deleteClient() throws when projects exist', () => {
    expect(() => deleteClient(fixture.acmeClientId))
      .toThrow(`Cannot delete client ${fixture.acmeClientId} because it has projects`
)
  })
})

describe('invoice services', () => {
  it('getInvoices() returns all invoices', () => {
    const invoices = getInvoices()

    expect(invoices).toHaveLength(1)
    expect(invoices[0]).toMatchObject({
      id: fixture.invoiceId,
      invoiceNumber: 'INV-TEST-001',
      clientName: 'Acme Studio',
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      projectName: 'Website Build',
    })
  })

  it('getInvoiceById() returns the invoice with line items', () => {
    const invoice = getInvoiceById(fixture.invoiceId)

    expect(invoice).toMatchObject({
      id: fixture.invoiceId,
      invoiceNumber: 'INV-TEST-001',
      totalCents: 25000,
      client: {
        id: fixture.acmeClientId,
        name: 'Acme Studio',
      },
      project: {
        id: fixture.projectId,
        name: 'Website Build',
      },
    })
    expect(invoice.lineItems).toHaveLength(2)
    expect(invoice.lineItems.map((item) => item.lineTotalCents)).toEqual([10000, 1500
0])
  })

  it('createInvoice() calculates totals correctly from line items', () => {
    const invoice = createInvoice({
      projectId: fixture.projectId,
      invoiceNumber: 'INV-TEST-002',
      status: 'draft',
      issueDate: '2026-06-15',
      dueDate: '2026-07-15',
      currency: 'USD',
      lineItems: [
        {
          description: 'Strategy',
          quantity: 2,
          unitPriceCents: 12500,
        },
        {
          description: 'Implementation',
          quantity: 1.5,
          unitPriceCents: 20000,
        },
      ],
    })

    expect(invoice.subtotalCents).toBe(55000)
    expect(invoice.totalCents).toBe(55000)
    expect(invoice.paidCents).toBe(0)
    expect(invoice.lineItems.map((item) => item.lineTotalCents)).toEqual([25000, 3000
0])
  })

  it('deleteInvoice() cascades to line items', () => {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    deleteInvoice(fixture.invoiceId)

    const invoiceCount = db.prepare('SELECT COUNT(*) AS count FROM invoices WHERE id
= ?')
      .get(fixture.invoiceId).count
    const lineItemCount = db.prepare(`
      SELECT COUNT(*) AS count
      FROM invoice_line_items
      WHERE invoice_id = ?
    `).get(fixture.invoiceId).count

    expect(invoiceCount).toBe(0)
    expect(lineItemCount).toBe(0)
  })
})

describe('project services', () => {
  it('getProjects() returns projects with non-void invoiced totals', () => {
    // Add a void invoice to ensure it's excluded from totals
    db.prepare(`
      INSERT INTO invoices (
        project_id,
        invoice_number,
        status,
        issue_date,
        due_date,
        currency,
        subtotal_cents,
        total_cents,
        paid_cents,
        sent_at,
        paid_at,
        notes
      )
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    `).run(
      fixture.projectId,
      'INV-TEST-VOID',
      'void',
      '2026-06-20',
      '2026-07-20',
      'USD',
      50000,
      50000,
      0,
      null,
      null,
      null,
    )

    const projects = getProjects()

    expect(projects).toHaveLength(1)
    expect(projects[0]).toMatchObject({
      id: fixture.projectId,
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      clientId: fixture.acmeClientId,
      clientName: 'Acme Studio',
      name: 'Website Build',
      totalInvoicedCents: 25000,
    })
  })
})

describe('dashboard services', () => {
  it('getDashboardSummary() returns outstanding totals, paid month total, overdue cou
nt, and top client', () => {
    const currentMonth = monthStart(new Date())
    const previousMonth = addMonths(currentMonth, -1)
    const betaProjectId = insertProject(fixture.betaClientId, 'Analytics Cleanup')

    // Insert test invoices for different scenarios
    insertInvoice({
      projectId: fixture.projectId,
      invoiceNumber: 'INV-DASH-OVERDUE-ACME',
      status: 'overdue',
      issueDate: dateOnly(previousMonth),
      dueDate: dateOnly(addMonths(currentMonth, -1)),
      totalCents: 20000,
      paidCents: 5000,
    })
    insertInvoice({
      projectId: betaProjectId,
      invoiceNumber: 'INV-DASH-PAID-BETA',
      status: 'paid',
      issueDate: dateOnly(currentMonth),
      dueDate: dateOnly(currentMonth),
      totalCents: 40000,
      paidCents: 40000,
      paidAt: currentMonth.toISOString(),
    })
    insertInvoice({
      projectId: fixture.projectId,
      invoiceNumber: 'INV-DASH-PAID-ACME-OLD',
      status: 'paid',
      issueDate: dateOnly(previousMonth),
      dueDate: dateOnly(previousMonth),
      totalCents: 50000,
      paidCents: 50000,
      paidAt: previousMonth.toISOString(),
    })

    expect(getDashboardSummary()).toEqual({
      totalOutstandingCents: 40000,
      totalPaidThisMonthCents: 40000,
      overdueCount: 1,
      topClientByRevenue: 'Acme Studio',
    })
  })

  it('getMonthlyRevenue() returns the last 6 months with zero-filled totals', () => {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  const currentMonth = monthStart(new Date())
  const fiveMonthsAgo = addMonths(currentMonth, -5)
  const twoMonthsAgo = addMonths(currentMonth, -2)
  const outsideWindow = addMonths(currentMonth, -6)

  // Insert invoices within and outside the 6-month window
  insertInvoice({
    projectId: fixture.projectId,
    invoiceNumber: 'INV-REV-FIVE-MONTHS-AGO',
    status: 'paid',
    issueDate: dateOnly(fiveMonthsAgo),
    dueDate: dateOnly(fiveMonthsAgo),
    totalCents: 30000,
    paidCents: 30000,
    paidAt: fiveMonthsAgo.toISOString(),
  })
  insertInvoice({
    projectId: fixture.projectId,
    invoiceNumber: 'INV-REV-TWO-MONTHS-AGO',
    status: 'sent',
    issueDate: dateOnly(twoMonthsAgo),
    dueDate: dateOnly(twoMonthsAgo),
    totalCents: 45000,
  })
  insertInvoice({
    projectId: fixture.projectId,
    invoiceNumber: 'INV-REV-OUTSIDE-WINDOW',
    status: 'paid',
    issueDate: dateOnly(outsideWindow),
    dueDate: dateOnly(outsideWindow),
    totalCents: 99000,
    paidCents: 99000,
    paidAt: outsideWindow.toISOString(),
  })

  // Build expected totals map including the seed invoice
  const expectedTotals = new Map([
    [dateOnly(fiveMonthsAgo).slice(0, 7), 30000],
    [dateOnly(twoMonthsAgo).slice(0, 7), 45000],
  ])
  const seedMonthKey = '2026-06'
  expectedTotals.set(seedMonthKey, (expectedTotals.get(seedMonthKey) ?? 0) + 25000)

  expect(getMonthlyRevenue()).toEqual(
    Array.from({ length: 6 }, (_, index) => {
      const month = addMonths(currentMonth, index - 5)
      const monthKey = dateOnly(month).slice(0, 7)

      return {
        month: monthLabel(month),
        totalCents: expectedTotals.get(monthKey) ?? 0,
      }
    }),
  )
})
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  it('getOverdueInvoices() returns overdue invoices ordered by due date', () => {
    const betaProjectId = insertProject(fixture.betaClientId, 'Reporting Setup')
    const laterInvoiceId = insertInvoice({
      projectId: fixture.projectId,
      invoiceNumber: 'INV-OVERDUE-LATER',
      status: 'overdue',
      issueDate: '2026-03-15',
      dueDate: '2026-04-15',
      totalCents: 18000,
    })
    const earlierInvoiceId = insertInvoice({
      projectId: betaProjectId,
      invoiceNumber: 'INV-OVERDUE-EARLIER',
      status: 'overdue',
      issueDate: '2026-02-01',
      dueDate: '2026-03-01',
      totalCents: 27000,
    })

    // Non-overdue invoice should be excluded
    insertInvoice({
      projectId: fixture.projectId,
      invoiceNumber: 'INV-NOT-OVERDUE',
      status: 'sent',
      issueDate: '2026-03-01',
      dueDate: '2026-03-31',
      totalCents: 12000,
    })

    expect(getOverdueInvoices()).toMatchObject([
      {
        id: earlierInvoiceId,
        clientName: 'Beta Labs',
        projectName: 'Reporting Setup',
        dueDate: '2026-03-01',
        totalCents: 27000,
      },
      {
        id: laterInvoiceId,
        clientName: 'Acme Studio',
        projectName: 'Website Build',
        dueDate: '2026-04-15',
        totalCents: 18000,
      },
    ])
  })
})
```

© 2026 Zenva Pty Ltd. All rights reserved.